

Git احترف

Scott Chacon

Ben Straub

Version 2.1.449, 2025-10-24

فهرس المحتويات

احترف جت

توطئة

تمهيد المترجم

تمهيد Scott Chacon

تمهيد Ben Straub

إهداء

الرخصة

المساهمون

مقدمة

الباب الأول: البدء

١، ١، عن إدارة النسخ

١، ٢، تاريخ جت بإيجاز

١، ٣، ما هو جت؟

١، ٤، سطر الأوامر

١، ٥، تثبيت جت

١، ٦، إعداد جت أول مرة

١، ٧، الحصول على المساعدة

١، ٨، الخلاصة

الباب الثاني: أسس جت

٢، ١، الحصول على مستودع جت

٢، ٢، تسجيل التعديلات في المستودع

٢، ٣، رؤية تاريخ الإيداعات

٢، ٤، التراجع عن الأفعال

٢، ٥، التعامل مع المستودعات البعيدة

٢، ٦، الوسوم

٢، ٧، الكُنَيَات في جت

٢، ٨، الخلاصة

الباب الثالث: التفريع في جت

٣، ١، الفروع بإيجاز

٣، ٢، أسس التفرع والدمج

٣، ٣، إدارة الفروع

٣، ٤، أساليب التطوير التفرعية

٣، ٥، الفروع البعيدة

٣، ٦، إعادة التأسيس

٣، ٧، الخلاصة

الباب الرابع: جت على الخادوم

٤، ١، الموافيق (البروتوكولات)

٤، ٢، تثبيت جت على خادوم

٤، ٣، توليد مفتاح SSH عمومي لك

٤، ٤، إعداد الخادوم

٤، ٥، عفريت جت

٤، ٦، ميفاق HTTP الذكي

٤، ٧، GitWeb

٤، ٨، GitLab

٤، ٩، خيارات الاستضافة الخارجية

٤، ١٠، الخلاصة

الباب الخامس: جت المتوزع

٥، ١، أساليب التطوير المتوزع

٥، ٢، المساهمة في مشروع — مقدمة

٥، ٣، المساهمة في مشروع — إرشادات الإيداع

٥، ٤، المساهمة في مشروع — فريق خصوصي صغير

٥، ٥، المساهمة في مشروع — فريق خصوصي بمدير دمج

٥، ٦، المساهمة في مشروع — مشروع عمومي باشتقاقات

٥، ٧، المساهمة في مشروع — مشروع عمومي عبر البريد الشابيكي

٥، ٨، المساهمة في مشروع — الخلاصة

٥، ٩، رعاية مشروع — مقدمة

٥، ١٠، رعاية مشروع — تطبيق الرقع من البريد

٥، ١١، رعاية مشروع — سحب فروع بعيدة

٥، ١٢، رعاية مشروع — تحديد التغييرات الجديدة

٥، ١٣، رعاية مشروع — دمج المساهمات

٥، ١٤، رعاية مشروع — إصدار مشروعك

٥، ١٥، الخلاصة

الباب السادس: GitHub

٦، ١، إعداد الحساب وتهيئته

~~٦، ٤، المساهمة في مشروع~~

~~٦، ٣، رعاية مشروع~~

~~٦، ٤، إدارة منظمة~~

~~٦، ٥، برمجة جت~~

٦، ٦، الخلاصة

~~الباب السابع: أدوات جت~~

الباب الثامن: تخصيص جت

~~٦، ١، تهيئة جت~~

~~٦، ٤، مسات الملفات في جت~~

٨، ٣، خطاطيف جت

~~٦، ٤، مثال لغرض السياسات في جت~~

٨، ٥، الخلاصة

~~الباب التاسع: جت والأنظمة الأخرى~~

~~الباب العاشر: دواخل جت~~

الملحق الأول: جت في البيئات الأخرى

~~٦، ١، الواجهات الرسومية~~

أ، ٢، جت في Visual Studio

أ، ٣، جت في Visual Studio Code

أ، ٤، جت في IntelliJ / PyCharm / WebStorm / PhpStorm / RubyMine

أ، ٥، جت في Sublime Text

أ، ٦، جت في Bash

أ، ٧، جت في Zsh

~~٦، ٨، جت في PowerShell~~

أ، ٩، الخلاصة

الملحق الثاني: تضمين جت في تطبيقاتك

ب، ١، جت في سطر الأوامر

ب، ٤، مكتبة Libgit2

~~ب.٣، جت في جافا: JGit~~

~~ب.٤، جت في لغة غو: go-git~~

ب، ٥، جت ببيثون: Dulwich

~~الملحق الثالث: أوامر جت~~

الملحق الرابع: دليل المصطلحات

الملحق الخامس: المصطلحات والمفاهيم باللغتين

هـ ١، الإيداع والسحب

هـ ٢، الدفع والجذب والاستحضار

هـ ٣، الإرجاع والاستعادة والنقض

هـ ٤، أسماء أوامر جت

هـ ٥، التعديل والتحرير

هـ ٦، إشارة الرأس وإشارات الكائنات

هـ ٧، التعقب والمتابعة: tracking

هـ ٨، الفروع البعيدة والفروع المتعقّبة لبعيد والفروع المتعقّبة

هـ ٩، إننا القراءة والتحرير

هـ ١٠، تعريب كلمة كود

هـ ١١، المركز والمركزي، الرئيس والرئيسي

هـ ١٢، إصلاحات لغوية عامة ونصائح أسلوبية

توطئة

تمهيد المترجم

كتاب احترفت جت من «أمهات الكتب» في صناعة البرمجيات. فهو يعلم نظاماً من أشهر أنظمة إدارة النسخ على الإطلاق، هو جت، ويعلم فن التطوير باستخدام نظام إدارة نسخ موزع. فليس يقتصر نفعه على من يريد استخدام Git و GitHub، بل يمتد إلى من أراد استخدام GitLab أو حتى نظام آخر مثل Mercurial.

دراسة هذا الكتاب ضرورية لأي مبرمج.

وقد حرصت على تقليل الإنجليزية في الكتاب، إلا مما يلزم (مثل الأوامر ونواتجها) فترجمت ما نحتاجه منها مع إبقاء أصله كله، حتى يفهم الكتاب العامة والخاصة فلا نجعل إتقان الإنجليزية شرطاً لفهم كتاب عربي. مع أن معرفة الإنجليزية ضرورة للمبرمجين ولمستخدمي جت لا محالة.

ودراسة العلم بلغة الإنسان الأولى تساعد على الفهم العميق لما يتعلم. ولا سبيل للنهضة إلا بنقل العلوم واستيعابها بلغتنا. ألمانيا، وفرنسا، وإيطاليا، واليابان التي سُحقت في منتصف القرن الماضي، والصين الشعبية، وكوريا الجنوبية، وإسرائيل — جميعهم يدرسون العلوم بلغاتهم حتى إتمام الشهادة الجامعية، وجميعهم في أعلى دول العالم في البحث العلمي وفي الصناعة. وتلك الأخيرة ماتت لغتها ألفي عام فبعثوها من موتها وتعلموها وعلموها أبناءهم بل وأخذوا مصطلحاتهم العلمية والطبية من اللغة العربية! لأن وقتئذٍ كانت اللغة العربية ما تزال لغة العلم والتدريس في بلدانها.

ولا يعني هذا إهمال اللغات الأجنبية؛ فتعلمها ضرورة. والإنجليزية بالأخص.

أما عن الألفاظ: فلغة أي فن من الفنون هي دائماً غريبة على مسامع من لم يعتادوها. حتى الإنجليزية؛ اسأل شخصاً وُلد وعاش وشاخ في بلد لا يتكلم غيرها، ولم يسمع أو يقرأ في حياته بأي لغة أخرى، عن عبارات مثل “fork a process” أو “clone a repository”...

والأمثلة على ذلك كثيرة، منها: «لا يدخل الزحاف في شيء من الأوتاد، وإنما يدخل في الأسباب خاصة». وهذا من كتاب في علم العروض. الذي أتى بمصطلحاته، مثل الأسباب والأوتاد والزحاف، هو الخليل ابن أحمد، واضع أسس علم النحو، وجامع معجم العين، أول معجم عربي.

ومن درس علم التجويد مثلاً فقد وجد أن كل لفظ يُعرّف أولاً بمعناه في اللغة ثم بمعناه في العلم، فيقولون مثلاً «الإخفاء لغةٌ هو كذا، واصطلاحاً هو كذا».

فلكل علم لغته الخاصة داخل اللغة العامة. ومثال آخر على ذلك لغة الحساب الحديث، ففيها الاسم «زائد» يُستعمل حرف عطف، والفعل المنصرف «يساوي» يُستعمل جامداً ولا يتغير شكله أبداً، فنقول «اثنين زائد ثلاثة يساوي خمسة». أفهذه الجملة غير عربية؟

ولن يجد العربي غير المتخصص عبارةً مثل «استنسخ مستودعاً» أو «أودع التعديلات المؤهلة» أغرب مما يجد الإنجليزي العامي عبارةً مثل “clone a repository” أو “commit the staged changes”. فكل فن لغته الخاصة داخل اللغة العامة.

ولسنا اليوم بصدد تغيير قواعد اللغة هنا كما فعل أهل الحساب، إنما نحتاج فقط إلى توسيع معاني بعض الكلمات كما فعل أهل التجويد. ولن نأت بمصطلحات كثيرة كما فعل الخليل رحمه الله في علمي العروض والقافية.

وأرجو من القارئ الكريم التماس العذر لي إن بدا مني خطأ، وأن ينبهني إليه على صفحة [مسائل ترجمة الكتاب على جت هب](https://github.com/noureddin/progit2/issues) (الرابط: <https://github.com/noureddin/progit2/issues>).

تمهيد Scott Chacon

مرحباً بكم في الإصدار الثانية من كتاب احتراف جت. نُشرت الإصدار الأولى منذ ما يزيد الآن على أربعة أعوام. وتغيّرت أمور كثيرة منذ ذلك الحين، إلا أن الكثير من الأشياء المهمة لم تتغير. ومع أن أكثر الأوامر والمفاهيم الأساسية لا تزال سارية اليوم—لأن الفريق الأساسي القائم على جت يقوم بمجهود خيالي للحفاظ على التوافقية مع الإصدارات السابقة (backward compatibility)—فقد ظهرت بعض الإضافات والتغييرات البارزة في المجتمع الذي حول جت. المراد من الإصدار الثانية من هذا الكتاب هو تناول تلك التغييرات، وتحديثه ليكون أكثر إفادة للمستخدم الجديد.

عندما كتبت الإصدار الأولى، كان جت صعب الاستخدام نسبياً، وبالكاد يستخدمه الخارقون (hackers) الأشداء. كان بدأ ينتشر في بعض المجتمعات، لكن لم تقترب حاله مما صار عليه اليوم من الوجود المطلق في كل مكان. ثم بدأت تستخدمه أكثر مجتمعات المصادر المفتوحة. ولقد تقدم جت تقدماً مذهلاً: في عمله على ويندوز، وفي انفجار أعداد الواجهات الرسومية له على جميع المنصات، وفي دعم بيئات التطوير له، وفي الاستخدام التجاري. وكتاب احتراف جت الذي جاء منذ أربعة أعوام لم يكن يعلم شيئاً من هذا كله. فكان ذكر جميع تلك الآفاق الجديدة في مجتمع جت من أكبر همومنا في هذه الإصدار الجديدة.

وأيضاً تضاعف بسرعة مجتمع المصادر المفتوحة الذي يستخدم جت. فعندما جلست أول مرة أكتب هذا الكتاب منذ قرابة خمسة أعوام (فقد استغرق الأمر مني وقتاً لإطلاق الإصدار الأولى)، كنت وقتئذٍ قد بدأت العمل في شركة مغمورة جداً تطور موقع استضافة جت يسمى جت هب (GitHub). وعند نشره، لم يكن للموقع سوى نحو بضعة آلاف مستخدم، ولم تكن إلا أربعة موظفين نعمل عليه. وبينما أنا أكتب هذه المقدمة الآن، إذ أعلن جت هب استضافتنا للمشروع رقم عشرة ملايين، مع قرابة خمسة ملايين حساب مطوّر مسجّل، وأكثر من ٢٣٠ موظفاً. سواء أحببت أم كرهت، لقد غيّر جت هب كثيراً في جماعات ضخمة من مجتمع المصادر المفتوحة بطريقة لم يكد يتخيلها أحد عند كتابتي الإصدار الأولى.

كتبت فصلاً قصيراً في الإصدار الأولى من كتاب احتراف جت عن جت هب ليكون مثلاً لاستضافة جت، ولم أشعر قط بالارتياح الكامل إلى هذا الفصل؛ لم يعجبني أنني أكتب ما أشعر أنه في الأصل نُحراً للمجتمع، ثم أتحدث فيه عن شركتي. ومع أنني ما زلت لا أحب تضارب المصالح، فإن أهمية جت هب في مجتمع جت لا يمكن التغاضي عنها. وبدلاً من كون هذا الجزء من الكتاب مثلاً على استضافة جت، فقد رأيت تحويله إلى شرح عميق لماهية جت هب وكيفية استخدامه بشكل فعال. فإذا كنت تنوي تعلم استخدام جت، فستساعدك معرفة استخدام جت هب في المشاركة في مجتمع مترامي الأطراف، فهي قيمة بغض النظر عن استضافة جت التي ستقرر استخدامها لمشروعاتك البرمجية.

أما التغيير الكبير الآخر منذ النشر السابق فكان تطوير وبروغ بروتوكول HTTP لمعاملات جت الشبكية. فغَيَّرنا معظم أمثلة الكتاب من SSH إلى HTTP لأنه أسهل كثيراً.

لقد كان مذهلاً مشاهدة جت ينمو عبر الأعوام من نظام إدارة نسخ مغمور نسبياً إلى نظام إدارة النسخ المهيمن في المشروعات التجارية والمفتوحة. أنا سعيد بكون كتاب احترف جت قد أبلى بلاءً حسناً وكان قادراً أن يكون من الكتب التقنية، القليلة في السوق، الناجحة والمفتوحة بالكامل معاً.

أرجو أن تنتفع بهذه الإصدار المحدث من احترف جت.

تمهيد Ben Straub

الإصدار الأولى من هذا الكتاب هي ما جعلني مولعاً بجت. كانت هي ما عرّفتني أسلوب صناعة برمجيات وجدته طبيعياً أكثر من كل ما رأيت سابقاً. لقد كنت بالفعل مطوّراً منذ عدة أعوام وقتئذٍ، لكن هذا كان المنعطف الصحيح الذي ساقني إلى طريق أمتع كثيراً من الطريق الذي كنت عليه.

واليوم وبعد أعوام، أنا مساهم في تطبيق جت رائد، وعملت في أكبر شركة استضافة جت، وطُفّت العالم معلماً الناس جت. وعندما سألتني Scott إذا ما كنت أرغب في العمل على الإصدار الثانية من الكتاب، لم أحتجّ حتى للتفكير.

لقد كان العمل على هذا الكتاب شرف عظيم وسعادة كبيرة لي. وأرجو أن يساعدك بقدر ما ساعدني.

إهداء

إلى زوجتي Becky، التي بغير وجودها لم تكن هذه المغامرة لتبدأ قط. — Ben

أهدي هذه الإصدار إلى فتيتاتي: إلى زوجتي Jessica التي ساندتني طوال السنين، وإلى ابنتي Josephine التي ستساندني عندما أكون شيخاً مسناً لا يدرك ما يدور حوله. — Scott

الرخصة

هذا العمل متاح برخصة المشاع الإبداعي الإصدار الثالثة غير الموطنة بشروط النسبة للمصنّف والاستخدام غير التجاري والترخيص بالمثل (CC BY-NC-SA 3.0 unported). لرؤية نسخة من هذه الرخصة، برجاء زيارة <https://creativecommons.org/licenses/by-nc-sa/3.0/deed.ar> أو إرسال بريد إلى

Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

المساهمون

لأن هذا الكتاب مفتوح المصدر، فقد تبرع لنا الناس بالعديد من التصحيحات وتعديلات المحتوى. إليكم جميع من ساهم في النسخة الإنجليزية من المشروع مفتوح المصدر Pro Git. شكراً لكم جميعاً على مساعدتنا في جعل هذا الكتاب أفضل للناس جميعاً.

4wk-	HairyFotr	Pessimist
Adam Laflamme	Hamid Nazari	Peter Kokot
Adrien Ollier	Hamidreza MahdaviPanah	peterwillis
agkhal	haripetrov	Petr Bodnar
ajax333221	Haruo Nakayama	Petr Janeček
Akrom K	Helmut K. C. Tessarek	Petr Kajzar
Alan D. Salewski	Hemant Kumar Meena	petsuter
Alba Mendez	Hidde de Vries	Philippe Blain
Aleh Suprunovich	HonkingGoose	Philippe Miossec
Alexander Bezzubov	Howard	Phil Mitchell
Alexandre Garnier	i-give-up	Pratik Nadagouda
alex-koziell	Ignacy	Rafi
Alex Povel	Igor	rahrh
Alfred Myers	Ilker Cat	Raphael R
allen joslin	iprok	Ray Chen
Amanda Dillon	Jan Groenewald	Rex Kerr
andreas	Jannick Kremer	Reza Ahmadi
Andreas Bjørnstad	Jaswinder Singh	Richard Hoyle
Andrei Dascalu	Jean-Noël Avila	Ricky Senft
Andrei Korshikov	Jeroen Oortwijn	Rintze M. Zelle
Andrew Blommestyn	Jim Hill	rmzelle
Andrew Kreimer	jingsam	Rob Blanco
Andrew Layman	Jin Park	Robert P. Goldman
Andrew MacFie	jliljekrantz	Robert P. J. Day
Andrew Metcalf	Joel Davies	Robert Theis
Andrew Murphy	Johannes Dewender	Rohan D'Souza
AndyGee	Johannes Schindelin	Roman Kosenko
AnneTheAgile	johnhar	Ronald Wampler
Anthony Loiseau	John Lin	root
Antonello Piemonte	Jonathan	Rory
Antonino Ingargiola	Jon Forrest	Rüdiger Herrmann
Anton Trunov	Jon Freed	Ryan Cavicchioni
applecuckoo	Jordan Hayashi	Sam Ford
Ardavast Dayleryan	Joris Valette	Sam Joseph
Artem Leshchev	Josh Byster	Sanders Kleinfeld
atalakam	Joshua Webb	sanders@oreilly.com
Atul Varma	Junjie Yuan	Sarah Schneider
axmbo	Junyeong Yim	SATO Yusuke
Bagas Sanjaya	Justin Clift	Saurav Sachidanand
Benjamin Dopplinger	Kaartic Sivaraam	Scott Bronson
Ben Sima	Károly Ozsvárt	Scott Jones
bermudi	KatDwo	Sean Head
Billy Griffin	Katrin Leinweber	Sean Jacobs
Bob Kline	Kausar Mehmood	Sebastian Krause
Bohdan Pylypenko	Keith Hill	Sergey Kuznetsov
Borek Bernard	Kenneth Kin Lum	Severino Lorilla Jr
Brett Cannon	Kenneth Lum	sharpire
bripmccann	Klaus Frank	Shengbin Meng
brotherben	Kristijan "Fremen" Velkovski	Sherry Hietala

Buzut	Krzysztof Szumny	Shi Yan
Cadel Watson	Kyrylo Yatsenko	Siarhei Bobryk
Carlos Martín Nieto	Lars Vogel	Siarhei Krukau
Carlos Tafur	Laxman	Skyper
Chaitanya Gurrapu	Lazar95	slavos1
Changwoo Park	leerg	Smaug123
Christian Decker	Leonard Laszlo	Snehal Shekatkar
Christoph Bachhuber	Linus Heckemann	Solt Budavári
Christopher Wilson	Logan Hasson	Song Li
Christoph Prokop	Louise Corrigan	spacewander
C Nguyen	Luc Morin	Stephan van Maris
CodingSpiderFox	Lukas Röllin	Steven Roddis
Cory Donnelly	maks	Stuart P. Bentley
Cullen Rhodes	Marat Radchenko	SudarsanGP
Cyril	Marcin Sędkak-Jakubowski	Suhaib Mujahid
Damien Tournoud	Marie-Helene Burle	Susan Stevens
Daniele Tricoli	Marius Žilėnas	Sven Selberg
Daniel Hollas	Markus KARG	td2014
Daniel Knittl-Frank	Marti Bolivar	Thanix
Daniel Shahaf	Mashrur Mia (Sa'ad)	Thomas Ackermann
Daniel Sturm	Masood Fallahpoor	Thomas Hartmann
Daniil Larionov	Mathieu Dubreuilh	Tiffany
Danny Lin	Matt Cooper	Tomas Fiers
Dan Schmidt	Matthew Miner	Tomoki Aonuma
Davide Angelocola	Matthieu Moy	Tom Schady
David Rogers	Matt Trzcinski	Trevor Jobling
delta4d	Mavaddat Javid	Tvirus
Denis Savitskiy	Max Coplan	twekberg
devwebcl	Máximo Cuadros	Tyler Cipriani
Dexter	Michael MacAskill	Ud Yzr
Dexter Morganov	Michael Sheaver	uerdogan
Diamondex	Michael Welch	UgmaDevelopment
Dieter Ziller	Michiel van der Wulp	ugultopu
Dino Karic	Miguel Bernabeu	universal
Dmitri Tikhonov	Mike Charles	Vadim Markovtsev
Dmitriy Smirnov	Mike Pennisi	Vangelis Katsikaros
Doug Richardson	Mike Thibodeau	Vegar Vikan
dualsky	Mikhail Menshikov	Victor Ma
Duncan Dean	Mitsuru Kariya	Vipul Kumar
Dustin Frank	mmikeww	Vitaly Kuznetsov
Eden Hochbaum	mosdalsvsocld	Volker-Weissmann
Ed Flanagan	nicktime	Volker Weißmann
Eduard Bardají Puig	Niels Widger	Wesley Gonçalves
Eric Henziger	Niko Stotz	William Gathoye
evanderiel	Nils Reuße	William Turrell
Explorare	Noelle Leigh	Wlodek Bzyl
eyherabh	noureddin	Xavier Bonaventura
Ezra Buehler	OliverSieweke	xJom
Fabien-jrt	Olleg Samoylov	xtreak
Fady Nagh	Osman Khwaja	yakirwin
Felix Nehrke	Otto Kekäläinen	Yann Soubeyrand
Filip Kucharczyk	Owen	Y. E

flip111	Pablo Schläpfer	Your Name
flyingzumwalt	Pascal Berger	Yue Lin Ho
Fornost461	Pascal Borreli	Yuhang Guo
franjozen	Patrice Krakow	Yunhai Luo
Frank	patrick96	Yusuke SATO
Frederico Mazzone	Patrick Steinhardt	z-hed
Frej Drejhammar	paveljanik	zwPapEr
goekboet	Pavel Janík	VELQCE7
grgbnc	Paweł Krupiński	狄卢
Guthrie McAfee Armstrong	pedrorijo91	

مقدمة

أنت على وشك قضاء عدة ساعات من عمرك في القراءة عن جت (Git)، فلنأخذ منها دقيقة لنشرح ما سنقدمه لك؛ هذا ملخص سريع لأبواب الكتاب العشرة وملاحقه الثلاثة.

في **الباب الأول**، نتناول أنظمة إدارة النسخ وأسس جت — لا شيء تقني، وإنما نعرف ما هو جت ولماذا أتى في أرض ممتلئة بأنظمة إدارة النسخ، وما يجعله مختلفاً، ولماذا يستخدمه الكثيرون. بعدئذٍ نشرح كيفية تنزيل جت وإعداده للمرة الأولى إن لم يكن لديك بالفعل على نظامك.

في **الباب الثاني**، نمر على مبادئ استخدام جت: كيف تستخدمه في ٨٠٪ من الحالات التي ستقابلها معظم الوقت. بعد قراءة هذا الفصل، ستكون قادراً على استنساخ مستودع، ورؤية ما قد حدث في تاريخ المشروع، وتعديل ملفات، والمساهمة بتعديلات. لو اشتعل الكتاب ذاتياً وقتئذٍ، فسيكون جت طبعاً في متناولك وتنتفع به، إلى أن تحصل على نسخة أخرى من الكتاب.

أما **الباب الثالث** فغن نموذج التفرع في جت، الذي غالباً ما يوصف بأنه ميزته القاتلة للمنافسة. نتعلم هنا ما الذي يميز جت حقاً عن الآخرين. وعندما تنتهيه، سترغب في التوقف لحظات للتفكير كيف عشت حياتك سابقاً بغير أن يكون تفرع جت جزءاً منها.

يتناول **الباب الرابع** جت على الخادوم. وهو لمن يريدون إعداد جت داخل منظماتهم أو على خادومهم الخاص للتعاون. ونستكشف أيضاً الخيارات المستضافة إذا كنت تفضل أن يديره لك شخص آخر.

يشرح **الباب الخامس** بالتفصيل الممل أساليب التطوير المتوزع المختلفة وكيفية تحقيقها مع جت. عندما تفرغ من هذا الباب، ستكون قادراً على العمل ببراعة مع العديد من المستودعات البعيدة، واستخدام جت عبر البريد الشبكي، واللعب برشاقة بالكثير من الفروع البعيدة والرُّقْع المساهم بها.

يتناول **الباب السادس** بعمق خدمة استضافة جتْهَب (GitHub) وأدواتها. فنورد إنشاء حساب وإدارته، وإنشاء مستودعات جت واستعمالها، وأساليب التطوير الشائعة للمساهمة في مشروعات الآخرين وقبول المساهمات في مشروعاتك، وواجهة جتْهَب البرمجية، وبحراً من النصائح الصغيرة لجعل حياتك أسهل عموماً.

الباب السابع عن أوامر جت المتقدمة. ستتعلم هنا عن أمورٍ مثل إتقان أمر الإرجاع المرعب (reset)، واستعمال طريقة البحث الثنائي لتحديد العلل، وتحرير التاريخ، واختيار المراجعات بالتفصيل، والمزيد المزيد. يسعى هذا الباب لإتمام معرفتك بجت حتى تكون أستاذاً بحق.

يتناول **الباب الثامن** تهيئة بيئة جت الخاصة بك. هذا يشمل إعداد بُرِيمِجات الخطاطيف (hook scripts) لفرض أو تشجيع السياسات المفضلة واستعمال إعدادات تهيئة البيئة لكي تعمل بالطريقة التي تريدها، وكذلك بناء مجموعتك الخاصة من البُرِيمِجات (scripts) لفرض سياسة إيداع مخصصة.

يناقش **الباب التاسع** جت والأنظمة الأخرى لإدارة النسخ. هذا يشمل استخدام جت داخل عالم Subversion (SVN)، وتحويل المشروعات من الأنظمة الأخرى إلى جت. فلا تزال منظمات كثيرة تستخدم SVN ولا تنوي التغيير، لكنك في هذه المرحلة ستكون قد تعلمت قوة جت الخيالية؛ فإليك هذا الفصل كيف تدبر أمورك إن كنت ما زلت مضطرا إلى استخدام خادم SVN. نذكر أيضا كيفية استيراد مشروعات من الأنظمة الأخرى لإدارة النسخ، إذا أقنعت الجميع بالغطس في جت.

يغوص **الباب العاشر** في أعماق جت المظلمة لكن الجميلة. لأنك عندئذ تعلم كل شيء عن جت وتستطيع استخدامه ببراعة ورشاقة، يمكننا الانتقال إلى مناقشة كيف يخزن جت كائناته، وما هو نموذج الكائنات، وما تفاصيل الملفات المُعلبة (packfiles) وموافق (بروتوكولات) الخواديم، والمزيد. سنشير خلال هذا الكتاب إلى فصول هذا الباب، إن رغبت في الغوص عميقا في تلك المرحلة، لكن إذا كنت مثلنا وتريد الغوص في التفاصيل التقنية، فقد تحب قراءة الباب العاشر أولا. نترك هذا الخيار لك.

في **الملحق الأول**، نرى عددا من أمثلة استخدام جت في بيئات معينة مختلفة. نذكر عددا من الواجهات الرسومية وبيئات التطوير المختلفة التي ربما تريد استخدام جت فيها وما المتاح لك. إذا كنت مهتما بنظرة عامة على استخدام جت في الطرفية أو بيئة التطوير أو محرر النصوص، ألق نظرة هنا.

في **الملحق الثاني**، نستكشف برمجة جت وتوسيعه عبر أدوات مثل libgit2 وJGit. إذا كنت مهتما بكتابة أدوات مخصصة سريعة ومعقدة وتحتاج وصولا إلى المستويات الدنيا من جت، هنا مكانك لمعرفة كيف يبدو المنظر العام لهذه المنطقة.

وأخيرا، في **الملحق الثالث**، نمر على جميع أوامر جت المهمة واحدا تلو الآخر ونتذكر أين شرحناه في الكتاب وماذا فعلنا به. إذا أردت أن تعرف أين في الكتاب استخدمنا أمر جت معين، يمكنك البحث عنه هنا.

هيا بنا نبدأ.

الباب الأول:

البداية

هذا الباب عن البدء مع جيت (Git). نبدأ بشرح خلفية عن أدوات إدارة النسخ، ثم ننتقل إلى كيفية تشغيل جيت على نظامك، وأخيراً كيفية إعداد البدء للعمل معه. في نهاية الفصل ستكون قد فهمت سبب وجود جيت، ولماذا عليك استخدامه، وأن تكون قد أعددتته للاستخدام.

أ، أ، عن إدارة النسخ

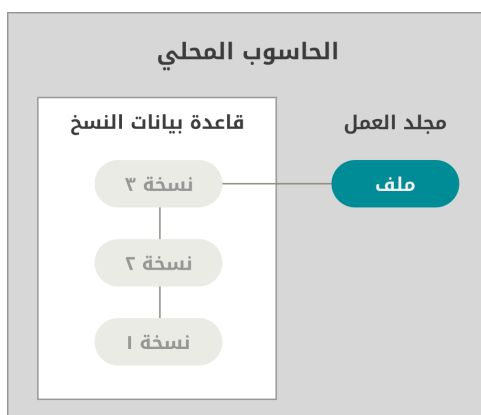
ما هي «إدارة النسخ»، ولماذا عليك أن تهتم؟ إدارة النسخ هي نظام يسجل التعديلات على ملف أو مجموعة من الملفات عبر الزمان، حتى يمكنك استدعاء نسخ معينة منها فيما بعد. نستعمل في أمثلة هذا الكتاب ملفات مصادر برمجية لإدارة نُسخها، ولو أن في الحقيقة يمكنك فعل ذلك مع أكثر أنواع الملفات الحاسوبية.

إذا كنت مصمم رسومات أو وب، وتريد الإبقاء على جميع نسخ صورة أو تخطيط (وهذا شيء يُفترض أنك تريد فعله يقيناً)، فإن استخدام نظام إدارة نسخ (VCS) لهو قرار حكيم حقاً. فهو يسمح لك بإرجاع ملف محدد إلى حالة سابقة، أو إرجاع المشروع كله إلى حالة سابقة، أو مقارنة التعديلات عبر الزمان، أو معرفة مَنْ آخر من غيّر شيئاً قد يكون سبب مشكلة، أو مَنْ تسبب في إحداث علة، وغير ذلك. استخدام نظام إدارة نسخ أيضاً يعني في العموم أنك إذا دمّرت أشياء أو فقدت ملفات، فإنك تستطيع استرداد الأمور بسهولة. وكل هذا تحصل عليه مقابل عبء ضئيل جداً.

أ، أ، الأنظمة المحلية لإدارة النسخ

طريقة إدارة النسخ عند الكثيرين هي نسخ الملفات إلى مجلد آخر (ربما بختم زمني، إذا كان المستخدم بارعاً). هذه الطريقة شائعة جداً لأنها سهلة جداً، لكنها أيضاً خطأً جداً. فسهل جداً أن تنسى أي مجلد أنت فيه وتحرر ملفاً خطأً أو تنسخ على ملفات لم ترد إبدالها.

لحل هذه القضية، طوّر المبرمجون منذ زمن بعيد «أنظمة محلية لإدارة النسخ» ذات قاعدة بيانات بسيطة تحفظ جميع التعديلات على الملفات المراد التحكم في نُسخها.

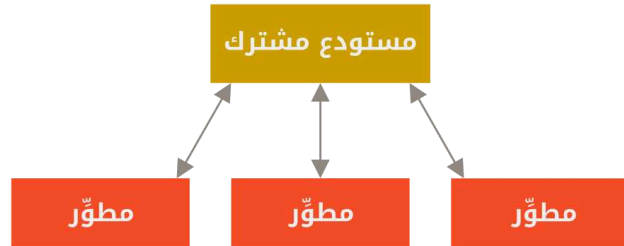


شكل ١. رسم توضيحي للإدارة المحلية للنسخ

كان من أشهر أنظمة إدارة النسخ نظام يسمى RCS، وهو ما زال موزعاً مع حواسيب كثيرة اليوم. يعمل RCS (<https://www.gnu.org/software/rcs/>) بالاحتفاظ بمجموعات الرقع (أي الفروقات بين الملفات) بصيغة مخصصة على القرص؛ فيمكنه إذاً إحياء أي ملف من أي حقبة زمنية بمجرد جمع الرقع.

١، ٢، الأنظمة المركزية لإدارة النسخ

المشكلة الكبرى الأخرى التي واجهت الناس هي احتياجهم إلى التعاون مع مطوّرين على أجهزة أخرى. فحلوها بإنشاء «الأنظمة المركزية لإدارة النسخ» (CVCS). لهذه الأنظمة (مثل CVS و Subversion و Perforce) خادم وحيد به جميع الملفات المراقبة، وعملاء يسحبون الملفات من ذلك المركز الوحيد. كان هذا هو المعيار المتبع لإدارة النسخ لأعوام عديدة.



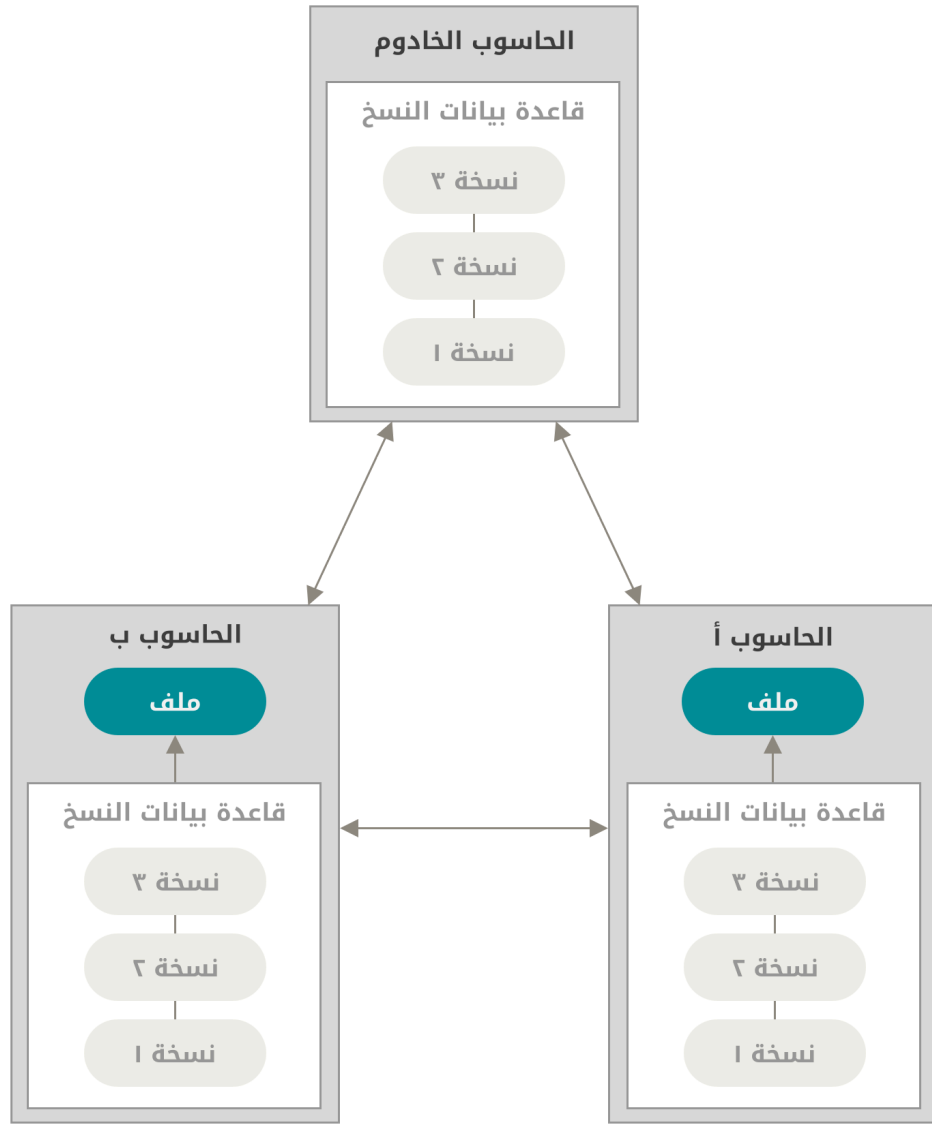
شكل ٢. رسم توضيحي للإدارة المركزية للنسخ

لهذا الترتيب مزايا كثيرة، لا سيما على الإدارة المحلية للنسخ. مثلاً، الجميع يعلم، إلى حدٍ ما، كل ما يفعله الآخرون في المشروع نفسه. ولدى المديرين تحكم مفصّل في تحديد من يستطيع فعل ماذا، وإنه لأسهل كثيراً إدارة نظام مركزي مقارنةً بالتعامل مع قواعد بيانات محلية عند كل عميل.

ولكن لهذا الترتيب بعض العيوب الجسيمة. أوضحها هو نقطة الانهيار الحاسمة الذي يمثلها الخادم المركز. فإذا تعطل ساعة، ففي تلك الساعة لن يستطيع أحدٌ قَطُّ التعاون أو حتى حفظ تعديلاتهم على ما يعملون عليه. وإذا تلف قرص قاعدة بيانات المركز، ولم تحفظ أي نسخ احتياطية، ستفقد كل شيء تماماً: تاريخ المشروع برُمته، ما عدا أي لقطات فردية صادف أن يُبقيها الناس على أجهزتهم المحلية. تعاني الأنظمة المحلية لإدارة النسخ كذلك من هذه العلة نفسها؛ وقتما جعلت كل تاريخ المشروع في مكان وحيد، عرّضت نفسك لفقد كل شيء.

١، ٣، الأنظمة المتوزعة لإدارة النسخ

الآن تتدخل الأنظمة المتوزعة لإدارة النسخ (DVCS). في نظام متوزّع (مثل جت و Mercurial و Darcs)، لا يستنسخ العملاء اللقطة الأخيرة فقط من الملفات، بل يستنسخون المستودع برُمته، بتاريخه بالكامل. لذا فإن انهيار أحد الخواديم فجأة، وكانت هذه الأنظمة تتعاون عبر هذا الخادم، فيمكن نسخ مستودع أي عميل إلى الخادم مجدداً لإعادته للعمل. كل نسخة هي في الحقيقة نسخة احتياطية كاملة لجميع البيانات.



شكل ٣. رسم توضيحي للإدارة الموزعة للنسخ

علاوة على ذلك، الكثير من هذه الأنظمة تتعامل جيداً مع وجود العديد من المستودعات البعيدة التي يمكنها العمل معها، لذا يمكنك التعاون مع مجموعات مختلفة من الناس بأساليب متعددة في الوقت نفسه داخل المشروع الواحد. هذا يسمح لك باتباع أساليب تطوير مختلفة لم تكن ممكنة في الأنظمة المركزية، كالأشكال الشجرية.

١، ٢، تاريخ جت بايجاز

مثل الكثير من الأشياء العظيمة، بدأ جت بشيء من التدمير الإبداعي والخلافات المتفجرة.

نواة لينكس هي مشروع برمجي مفتوح المصدر ذو امتداد شاسع إلى حد ما. في السنوات الأولى من تطوير نواة لينكس (١٩٩١-٢٠٠٢)، كانت التعديلات البرمجية تتناقل في صورة رقع وملفات مضغوطة. وفي عام ٢٠٠٢، بدأ مشروع نواة لينكس باستخدام نظام إدارة نسخ متوزع احتكاري يسمى BitKeeper.

ولكن في عام ٢٠٠٥، تدهورت العلاقة بين المجتمع الذي يطور نواة لينكس والشركة التجارية التي تتطور BitKeeper، وأسقطت صفة المجانية عن الأداة. دفع هذا مجتمع تطوير لينكس (وبالأخص Linus Torvalds، مؤسس لينكس) إلى صناعة أدواتهم الخاصة بناءً على بعض ما تعلموه خلال استخدامهم BitKeeper. وكان من أهداف النظام الجديد ما يلي:

- السرعة
- التصميم البسيط
- دعم وثيق للتطوير اللاخطي (آلاف الفروع المتوازية)
- متوزع بالكامل
- التعامل مع مشروعات ضخمة كنواة لينكس بكفاءة (من ناحية السرعة وحجم البيانات)

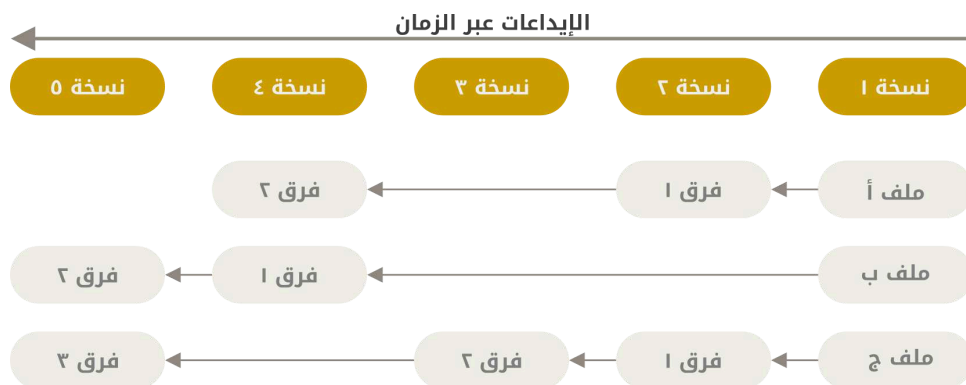
منذ ولادة جت في عام ٢٠٠٥، وقد نما ونضج حتى صار سهل الاستخدام، ومع هذا فقد احتفظ بهذه الصفات الأولية. إنه سريع لدرجة مذهلة. إنه كفاء جدا مع المشروعات الضخمة. إن له نظام تفريع خيالي للتطوير اللاخطي (انظر باب [التفريع في جت](#)).

١، ٣، ما هو جت؟

إنّ، ما هو جت باختصار؟ هذا فصل مهم ويجب استيعابه جيدا، لأنك إذا فهمت ماهية جت وأصول طريقة عمله، فسيكون سهل جدا عليك استخدام جت بفعالية. وخلال تعلمك جت، عليك تصفية ذهنك من كل ما تعمله عن أنظمة إدارة النسخ الأخرى، مثل CVS أو Subversion أو Perforce؛ يساعدك هذا على تجنب أي التباسات خفية عندما تستخدمه. ومع أن واجهة جت قريبة الشبه بالأنظمة الأخرى، إلا أن جت يخزن المعلومات بطريقة وينظر إليها نظرة مختلفة أشد الاختلاف، ويساعدك فهم هذه الفروق على تجنب الالتباس عند استخدامه.

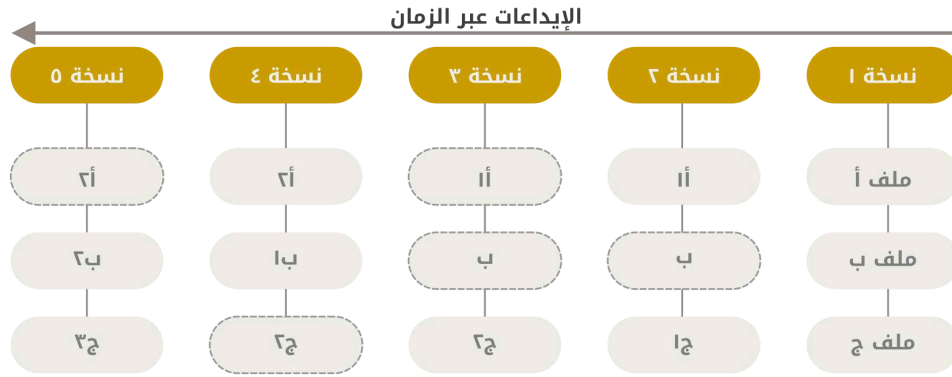
١، ٣، ١، لقطات، وليس فروقات

الفرق الأكبر بين جت وأي نظام آخر (بما في ذلك Subversion وأشباهه)، هو نظرة جت إلى بياناته. من حيث المفهوم، تخزن معظم الأنظمة الأخرى المعلومات في صورة سلسلة تعديلات على الملفات. هذه الأنظمة الأخرى (CVS و Subversion و Perforce إلخ) تنظر إلى المعلومات التي تخزنها على أنها مجموعة من الملفات والتعديلات التي تتم على كل ملف عبر الزمان (يوصف هذا غالبا بأنه إدارة نسخ بناءً على الفروقات).



شكل ٤. تخزين البيانات في صورة تعديلات على نسخة أساسية من كل ملف

لا ينظر جت إلى بياناته ولا يخزنها هكذا. بل يعتبرها أشبه بلقطات من نظام ملفات مصغر. في جت، كل مرة تصنع إيداعا (commit)، أي تحفظ حالة مشروعك، يلتقط جت صورة لما تبدو عليه ملفاتك جميعاً في هذه اللحظة، ويخزن إشارة لهذه اللقطة. وحتى يُحسن استغلال الموارد، فإن الملفات التي لم تتغير لا يخزنها جت مجدداً، بل يخزن فقط إشارة إلى الملف السابق المطابق الذي خزّنه سابقاً. فإن جت يعتبر أن بياناته **سيل من اللقطات**.



شكل ٥. تخزين البيانات في صورة لقطات من المشروع على مر الزمان

هذا تمييز مهم بين جت وأكثر الأنظمة الأخرى. إنه يجعل جت يعيد التفكير في أغلب جوانب إدارة النسخ التي نسختها معظم الأنظمة الأخرى من الأجيال السابقة. إنه يجعل جت أشبه بنظام ملفات مصغر ذي أدوات خارقة مبنية عليه، بدلا من مجرد نظام إدارة نسخ. عندما نتناول التفرع في جت في باب [التفرع في جت](#)، سنستكشف بعضًا من المنافع التي تحصل عليها عندما تنتظر إلى بياناتك هذه النظرة.

١، ٣، ٢، أغلب العمليات محلية

أكثر العمليات في جت لا تحتاج إلا إلى ملفات وموارد محلية لكي تعمل؛ فعموما لا حاجة إلى أي معلومات من حواسيب أخرى على الشبكة. إذا كنت معتادًا على نظام إدارة نسخ مركزي، حيث معظم العمليات مثقلة بعبء زمن الانتقال في الشبكة، فإن هذا الجانب من جت سيجعلك تظن أنه صنع عفريت من الجن حتى يكون سريعا هكذا. فلأن لديك التاريخ الكامل للمشروع بين يديك على قرصك المحلي، فإن معظم العمليات تبدو آنية.

مثلا، لتصفح تاريخ المشروع، لا يحتاج جت إلى السفر إلى الخادوم ليعود إليك حاملا التاريخ ليعرضه لك؛ إنما يقرؤه من قاعدة بياناتك المحلية. هذا يعني أنك ترى تاريخ المشروع أسرع من طرفة عين. وإذا أردت أن ترى التعديلات التي حدثت على ملف بين نسخته الآن ومنذ شهر، فيستطيع جت أن يأتي بهذا الملف منذ شهر ويحسب الفرق على حاسوبك، بدلا من الاضطرار إلى طلب هذا الفرق من خادوم بعيد أو طلب النسخة القديمة منه وحساب الفرق محليا.

هذا أيضا يعني أنك ما زلت تستطيع فعل كل شيء، إلا القليل النادر، إذا كنت غير متصل بالشبكة (الإنترنت)، أو بشبكته الوهمية الخصوصية (VPN). فإذا كنت في طائرة أو قطار، وتريد العمل قليلا، تستطيع الإيداع بكل سرور (إلى نسختك المحلية، أنتنكر؟) حتى تجد اتصالا شبكيا للرفع. وإذا عدت إلى المنزل ولم تجد عميل شبكتك الوهمية يستطيع العمل، فإنك ما زلت تستطيع العمل. أما في الكثير من الأنظمة الأخرى، فالعمل من غير اتصال إما أليم جدا وإما مستحيل أصلا. في Perforce مثلا، لا يمكنك فعل الكثير إن لم تكن متصلا بالخادوم. في Subversion و CVS تستطيع تعديل الملفات، لكن لا تستطيع إيداع أي تعديلات في قاعدة بياناتك (لأن قاعدة بياناتك غير متصلة). ربما تظن أن هذا ليس بالأمر العظيم، لكنك إذا ستنتفاجأ بضخامة الفرق الذي يصنعه.

١، ٣، ٣، في جت السلامة

يضمن جت سلامة البيانات دائماً، فهو يحسب قيم البصمات لكل شيء قبل أن يخزنه، وبعدئذ يشير إلى الأشياء ببصماتها. هذا يعني أن تعديل محتويات أي ملف أو مجلد بغير علم جت مستحيل. هذا مبني في أساس جت ومن أركان فلسفته. فمستحيل فقد معلومات أثناء النقل أو حتى فساد ملفات من غير أن يكتشف جت ذلك.

الآلية التي يستخدمها جت لحساب البصمة معروفة باسم بصمة SHA-1. وهي تنتج سلسلة نصية من أربعين رقماً ستعشرياً (0-9 و a-f) محسوبين من محتوى الملف أو بنية المجلد في جت. تشبه بصمة SHA-1 هذا:

```
24b9da6552252987aa493b52f8696cd6d3b00373
```

ترى قيم البصمات هذه في كل مكان في جت لأنه يستخدمها كثيراً. في الحقيقة، يخزن جت كل شيء في قاعدة بياناته، ليس بأسماء الملفات، بل بقيم بصمة محتواها.

١، ٣، ٤، يضيف جت بيانات فقط عموماً

أكثر الإجراءات في جت لا تفعل شيئاً سوى أن تضيف بيانات إلى قاعدة بيانات جت. ومن الصعب أن تجعله يفعل شيئاً لا يمكن التراجع عنه أو أن تجعله يمسح بيانات بأي طريقة. مثلما الحال مع أي نظام إدارة نسخ، يمكن أن تفقد أو تدمر التعديلات التي لم تودعها بعد، لكن ما إن تودعها في جت، فمن العسير جداً أن تفقدها، خصوصاً إذا كنت تدفع (push) قاعدة بياناتك بانتظام إلى مستودع آخر.

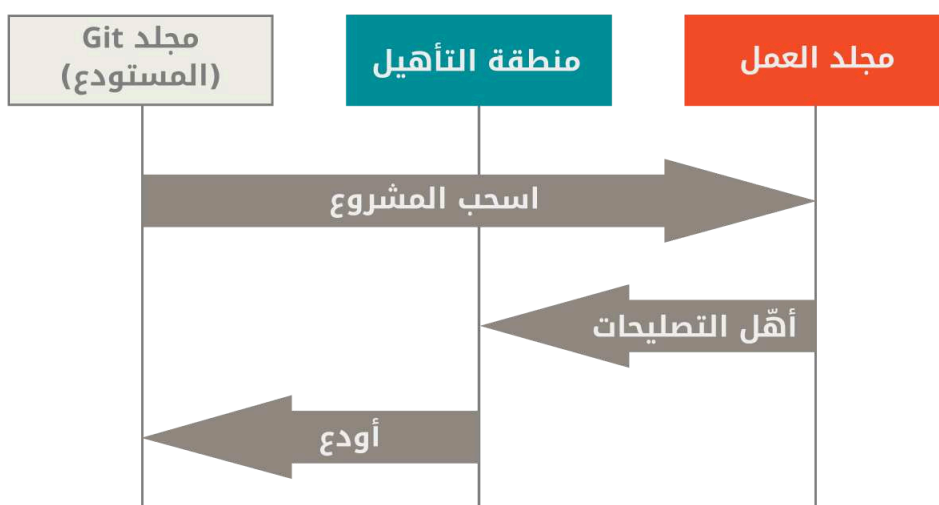
هذا يجعل استخدام جت مبهجاً لعلماً أننا نستطيع التجريب بغير خطر التخریب. للتعلم في كيفية تخزين جت لبياناته وكيفية استعادة ما يبدو أنه قد فُقد، انظر فصل [التراجع عن الأفعال](#).

١، ٣، ٥، المراحل الثلاثة

انتبه الآن وركز؛ هذا أهم شيء عليك تذكره دوماً عن جت إذا أردت أن تمضي رحلة تعلمك بسلاسة. في جت ثلاث مراحل يمكن أن تكون ملفاتك فيها: معدّل، ومؤهل، ومودّع.

- معدّل يعني أنك عدلت الملف لكنك لم تُودع التعديلات بعد في قاعدة بياناتك.
- مؤهل يعني أنك حددت ملفاً معدّلاً في نسخته الحالية ليكون ضمن لقطة الإيداع التالية.
- مودّع يعني أن البيانات صارت مخزنة بأمان في قاعدة بياناتك.

يقودنا هذا إلى الأقسام الرئيسية الثلاثة في أي مشروع جت: شجرة العمل، ومنطقة التأهيل، ومجلد جت.



شكل ٦. شجرة العمل، ومنطقة التأهيل، ومجلد جت

شجرة العمل (أو مجلد العمل) هي نسخة واحدة مسحوبة من المشروع. تجلب لك هذه الملفات من قاعدة البيانات المضغوطة في مجلد جت وتوضع لك على القرص لتستخدمها أو تعديلها.

منطقة التأهيل هي ملف يخزن معلومات عما سيكون في إيداعك التالي، وعادةً يكون ملف منطقة التأهيل في مجلد جت لمشروعك. المصطلح التقني في لغة جت هو «الفهرس» (index)، لكن العبارة «منطقة التأهيل» (staging area) مناسبة ومستخدمة أيضا.

مجلد جت هو المكان الذي يخزن فيه جت البيانات الوصفية وقاعدة بيانات الكائنات لمشروعك. هذا هو أهم جزء في جت، وهم الذي يُنسخ عندما تلتلسخ (clone) مستودعا من حاسوب آخر.

يبدو أسلوب التطوير الأساسي في جت مثل هذا:

١. تعدّل ملفات في شجرة عملك.

٢. تنتقي من تلك التعديلات ما تؤهله ليكون جزءًا من إيداعك التالي، وهذا لا يضيف إلا هذه التعديلات إلى منطقة التأهيل.

٣. تصنع إيداعا، وهذا يلتقط صورة للملفات كما هي من منطقة التأهيل ويخزن هذه اللقطة في مجلد جت لمشروعك إلى الأبد.

إذا كانت نسخة معينة من أحد الملفات موجودة داخل مجلد جت، فإنها تعتبر مودّعة. وإذا كانت معدّلة وقد أضيفت إلى منطقة التأهيل، فإنها مؤهّلة. وإذا كانت معدّلة بعد آخر مرة سُحبت فيها لكنها لم تؤهل بعد، فإنها معدّلة. ستتعلم المزيد في باب [أسس جت](#) عن هذه الحالات وكيف يمكنك استغلالها أو تخطي مرحلة التأهيل برمتها.

٤، ٤، سطر الأوامر

لاستخدام جت طرائق عديدة مختلفة. فلدينا أدوات سطر الأوامر الأصلية، وأيضا الكثير من الواجهات الرسومية ذات القدرات المتفاوتة. نستخدم في هذا الكتاب جت من سطر الأوامر. فسطر الأوامر هو المكان الوحيد الذي يمكنك فيه تنفيذ جميع أوامر جت؛ فمعظم الواجهات الرسومية لا تتيح إلا جزءًا من وظائف جت للتسهيل. وإذا كنت تعرف كيف تستخدم نسخة سطر الأوامر، ففي الغالب أنك أيضا ستعرف كيف تستخدم النسخة الرسومية، لكن العكس ليس بالضرورة صحيح. وأيضا اختيارك لعمل رسومي يخضع لذوقك الشخصي، لكن جميع المستخدمين لديهم أدوات سطر الأوامر مثبتة ومتاحة.

لذلك فإننا نتوقع منك معرفة كيف تفتح الطرفية (Terminal) في الأنظمة اليونكسية أو موجه سطر الأوامر (Command Prompt) أو PowerShell في ويندوز. إن لم تكن تعلم عما نتحدث، فعليك التوقف الآن وبحث هذا سريعا حتى تستطيع السير مع الأمثلة والأوصاف التي في الكتاب.

٥، ٥، تثبيت جت

قبل الشروع في استخدام جت، عليك جعله متاحًا على حاسوبك. حتى لو كان مثبتًا بالفعل، فغالبا من الأفضل تحديثه إلى آخر نسخة. يمكنك إما تثبيته من حزمة أو عبر مثبت آخر وإما تنزيل المصدر البرمجي وبناءه بنفسك.



كُتِبَ هذا الكتاب عن جت نسخة ٢.٠. لكن لأن جت متميز في الحفاظ على التوافقية مع الإصدارات السابقة، فأني نسخة حديثة يُتوقع أن تعمل كما ينبغي. ومع أن معظم الأوامر يتوقع أن تعمل حتى في نسخ جت الأثرية، فقد لا يعمل بعضها أو يعمل باختلاف طفيف.

١، ٥، ١، التثبيت على لينكس

إذا أردت تثبيت أدوات جت الأساسية على لينكس عبر مثبت برمجيات مبنية، فيمكنك غالباً فعل ذلك عبر أداة إدارة الحزم التي في توزيعتك. فإذا كنت على فيدورا (أو أي توزيعة قريبة منها تستخدم حزم RPM، مثل ردهات (RHEL) أو CentOS)، فيمكنك استخدام `dnf` :

```
$ sudo dnf install git-all
```

CONSOLE

وإذا كنت على توزيعة دبيانية مثل أوبنتو، جرب `apt` :

```
$ sudo apt install git-all
```

CONSOLE

لخيارات أخرى، توجد على موقع جت تعليمات لتثبيته على توزيعات لينكسية ويونكسية عديدة، في <https://git-scm.com/download/linux>.

١، ٥، ٢، التثبيت على ماك أو إس

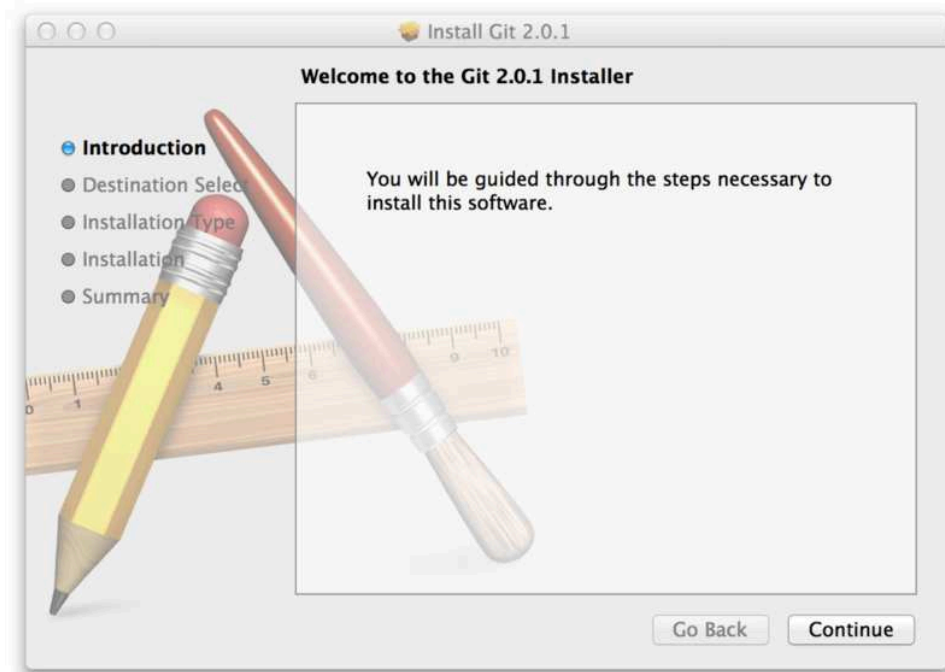
توجد طرائق عديدة لتثبيت جت على ماك. ربما أسهلها هو تثبيت أدوات سطر أوامر إكسكود (Xcode Command Line Tools). وعلى ماك مافريكس (Mavericks, 10.9) أو أحدث، يمكنك فعل هذا بمجرد محاولة تنفيذ `git` في الطرفية أول مرة إطلاقاً.

```
$ git --version
```

CONSOLE

فإذا لم يكن مثبتاً لديك بالفعل، فسيحتك على تثبيته.

أما إذا أردت نسخة أحدث، فيمكنك أيضاً تثبيته عبر مثبت برمجيات مبنية. يوجد مثبت جت لماك على موقع جت، في <https://git-scm.com/download/mac>.



شكل ٧. مثبت جت على ماك أو إس

١، ٥، ٣، التثبيت على ويندوز

لتثبيت جت على ويندوز عدة طرائق أيضا. النسخة المبنية الأكثر رسميةً متاحة على موقع جت. عليك فقط الذهاب إلى <https://git-scm.com/download/win> وسيبدأ التنزيل تلقائياً. لاحظ أن هذا مشروع يسمى «جت لويندوز» (Git for Windows)، وهو منفصل عن جت نفسه؛ للمزيد من المعلومات عنه، اذهب إلى <https://gitforwindows.org>.

أما إذا أردت مثبتاً آلياً فيمكنك استخدام [حزمة Git على Chocolatey](https://community.chocolatey.org/packages/git) (الرابط: <https://community.chocolatey.org/packages/git>). لاحظ أن المجتمع هو من يرفع حزمة Chocolatey.

١، ٥، ٤، التثبيت من المصدر البرمجي

ربما يفيد بعض الناس تثبيت جت من المصدر البرمجي بدلاً من ذلك، لأنك عندئذٍ ستحصل على أحدث نسخة قَط. مثبتات البرمجيات المبنية تميل إلى التأخر قليلاً، لكن لأن جت قد نضج في الأعوام الأخيرة، فلم يعد هذا يشكل فارقاً كما كان.

إذا أردت تثبيت جت من المصدر البرمجي، فستحتاج إلى المكتبات التالية التي يعتمد عليها جت: `curl` و `autotools` و `zlib` و `openssl` و `expat` و `libiconv`. مثلاً، إذا كنت على نظام يستخدم `dnf` (مثل فيدورا) أو `apt-get` (مثل الأنظمة الديبانية)، فيمكنك استخدام أحد هذين الأمرين لتثبيت أقل اعتماديات مطلوبة لبناء جت وتثبيته:

```
$ sudo dnf install dh-autoreconf curl-devel expat-devel gettext-devel \
openssl-devel perl-devel zlib-devel
$ sudo apt-get install dh-autoreconf libcurl4-gnutls-dev libexpat1-dev \
gettext libz-dev libssl-dev
```

CONSOLE

وتحتاج هذه الاعتماديات حتى تضيف التوثيق بصيغته المختلفة (`doc` و `html` و `info`):

```
$ sudo dnf install asciidoc xmlto docbook2X
$ sudo apt-get install asciidoc xmlto docbook2x
```

يحتاج مستخدمو ردهات والتوزيعات الردهاتية مثل CentOS و Scientific Linux إلى [تفعيل مستودع EPEL \(الشرح بالإنجليزية\)](#) [الرابط: https://docs.fedoraproject.org/en-US/epel/](https://docs.fedoraproject.org/en-US/epel/) حتى يستطيعوا تثبيت حزمة docbook2X. [#how_can_i_use_these_extra_packages](#)



إذا كنت تستخدم توزيعاً دبيانية (دبيان أو أوبنتو أو إحدى مشتقاتهما)، فحتاج أيضاً حزمة `install-info`:

```
$ sudo apt-get install install-info
```

إذا كنت تستخدم توزيعاً تستخدم RPM (فيدورا أو ردهات أو إحدى مشتقاتهما)، فحتاج أيضاً حزمة `getopt` (المثبتة مبدئياً في التوزيعات الدبانية):

```
$ sudo dnf install getopt
```

إضافة إلى ذلك، إذا كنت تستخدم فيدورا أو ردهات أو إحدى مشتقاتهما، فحتاج أن تفعل هذا أيضاً:

```
$ sudo ln -s /usr/bin/db2x_docbook2texi /usr/bin/docbook2x-texi
```

بسبب اختلافات في أسماء الأوامر.

عندما يكون لديك جميع الاعتماديات المطلوبة، يمكنك تنزيل أحدث ملف مضغوط موسوم برقم إصدار، من عدة أماكن. يمكنك تنزيله من موقع نواة لينكس، من <https://www.kernel.org/pub/software/scm/git>، أو من النسخة المقابلة على موقع جت هب، من <https://github.com/git/git/tags>. غالباً يكون أوضح قليلاً على جت هب ما هي النسخة الأحدث، ولكن في صفحة موقع نواة لينكس ستجد بصمات الإصدارات، إذا أحببت تفقد صحة الملفات التي نزلتها.

بعدئذٍ يمكنك البناء والتنصيب:

```
$ tar -zxf git-2.8.0.tar.gz
$ cd git-2.8.0
$ make configure
$ ./configure --prefix=/usr
$ make all doc info
$ sudo make install install-doc install-html install-info
```

بعد إتمام هذا، يمكنك الحصول على جت عبر جت نفسه للتحديثات:

```
$ git clone https://git.kernel.org/pub/scm/git/git.git
```

١، ٢ إعدادات جت أول مرة

الآن وقد صار جت على نظامك، ستودّ عمل بعض الأمور لتخصيص بيئته لك. تحتاج عملها مرة واحدة فقط على أي حاسوب؛ فإنها تبقى عندما تحدّث جت. يمكنك أيضا تعديلها في أي وقت بالمرور على الأوامر مرة أخرى.

في جت أداة «تهيئة» `git config`، لتعرض أو تعيّن متغيرات التهيئة التي تتحكم في جميع مناحي مظهر وسلوك جت. ونُخزّن هذه المتغيرات في ثلاثة أماكن مختلفة:

١. ملف `[path]/etc/gitconfig`: يحتوي القيم التي تُطبّق على جميع المستخدمين ومستودعاتهم. إذا أعطيت الخيار `--system` («نظام») إلى أمر التهيئة `git config`، فإنه يقرأ ويكتب في هذا الملف تحديداً. طبعاً تحتاج صلاحيات إدارية لتعديل هذا الملف لأنه ملف إعدادات خاص بالنظام.

٢. ملف `~/.gitconfig` أو `~/.config/git/config`: القيم الخاصة بك أنت تحديداً. يمكنك جعل جت يقرأ ويكتب في هذا الملف تحديداً بالخيار `--global` («عام»), وهذا يؤثر في جميع مستودعاتك على هذا النظام.

٣. ملف `config` في مجلد جت (أي `.git/config`) في أي مستودع أنت فيه الآن: القيم الخاصة بهذا المستودع وحده. يمكنك إجبار جت على القراءة والكتابة في هذا الملف بالخيار `--local` («محلي»), ولكن في الحقيقة هذا هو المفترض. بالطبع تحتاج إلى التواجد في مكان ما في مستودع جت حتى يمكنك استخدام هذا الخيار.

قيم كل مستوى تغطي على قيم المستوى السابق، لذا فقيم `.git/config` تتفوق على التي في `[path]/etc/gitconfig`.

في أنظمة ويندوز، يبحث جت عن ملف `.gitconfig` في مجلد المنزل، `$HOME` (الذي غالبا يكون `C:\Users\%USER`). ويبحث كذلك عن `[path]/etc/gitconfig`، ولكن بالنسبة إلى جذر `MSys`، وهو أينما قررت تثبيت جت فيه على نظامك عندما شغلت المثبت. وإذا كنت تستخدم `Git for Windows` النسخة 2.x أو أحدث، فستجد أيضا ملف إعداد على مستوى النظام، في `C:\Documents and Settings\All Users\Application Data\Git\config` على ويندوز إكس بي، وفي `C:\ProgramData\Git\config` على ويندوز فيستا والأحدث. لا يمكن تغيير هذا الملف إلا بتنفيذ الأمر «ملف» `git config -f` بحساب المدير.

يمكنك رؤية جميع إعداداتك ومن أين أتت باستخدام:

```
$ git config --list --show-origin
```

١، ٢، ٣ هويتك

أول ما تحتاج فعله بعد تثبيت جت هو تعيين اسمك وعنوان بريدك الشبكي. هذا مهم لأن كل إيداع جت يحتاج معرفتهما، وبصيران جزءاً ثابتاً في الإيداعات التي ستبدأ في صنعها.

```
$ git config --global user.name "Mohammad Ali"
$ git config --global user.email mohammadali@example.com
```

نكرر: لن تحتاج إلى فعل هذا إلا مرة واحدة إذا استخدمت الخيار `--global` («عام»)، لأن جت عندئذٍ يستعمل هاتين المعلوماتين لما تفعله مستخدمًا حسابك على هذا النظام. وإن احتجت إلى تجاوز إحدى هاتين القيمتين في مشروعات محددة، يمكنك تنفيذ الأمر في ذلك المشروع بغير الخيار `--global`.

تساعدك الكثير من الواجهات الرسومية في فعل هذا عند تشغيلها أول مرة.

١، ٢، محررك

الآن وقد أعدنا هويتك، يمكنك تعيين محرر النصوص الذي يشغله جت عندما يريد منك أن تكتب رسالة. فإذا لم يكن معينًا، فسيشغل جت المحرر المبدئي لنظامك.

إذا أردت محررًا آخر، مثل Emacs، فافعل الآتي:

```
$ git config --global core.editor emacs
```

على ويندوز، إذا أردت تعيين محرر آخر، فعليك تحديد المسار الكامل لملفه التنفيذي. الذي يختلف باختلاف طريقة تحريم محررك.

في حالة Notepad++، وهو محرر برمجيات مشهور، ستريد غالبًا أن تستخدم نسخة ٣٢-بت منه، لأن حتى وقت كتابة هذا، لا تدعم نسخة ٦٤-بت جميع الإضافات. إذا كنت على ويندوز ٣٢-بت أو تستخدم محرر ٦٤-بت على ويندوز ٦٤-بت، فإنك ستنفذ شيئًا مثل هذا:

```
$ git config --global core.editor '"C:/Program Files/Notepad++/notepad++.exe' -multiInst -notabbar -nosession -noPlugin"
```

Vim و Emacs و Notepad++ هي محررات نصوص شهيرة يستخدمها المبرمجون على ويندوز والأنظمة اليونكسية مثل لينكس وماك. إذا كنت تستخدم محررًا آخر، أو نسخة ٣٢-بت، فرجاءً انظر شرح تعيين محررك المراد في جت في فصل [git config core.editor commands](#).



إذا لم تعين محررك هكذا، فإنك قد تجد نفسك في حالة محيرة جدًا، عندما يحاول جت فتحه. مثال ذلك على ويندوز أن يحاول جت فتح المحرر فلا يستطيع فيغلق مبكرًا.



١، ٢، ٣، اسم الفرع المبدئي

عندما تنشئ مستودعًا جديدًا بالأمر `git init`، فإن جت سيُنشئ فيه فرعًا، ويسميه مبدئيًا `master`. يمكنك تعيين اسم آخر للفرع المبدئي، بدءًا من النسخة 2.28 من جت.

لجعل اسم الفرع المبدئي هو main، نَفِّذْ:

```
$ git config --global init.defaultBranch main
```

CONSOLE

١، ٢، ٤، تفقّد إعداداتك

إذا أردت تفقّد إعدادات تهيئتك، نَفِّذْ أمر التهيئة مع خيار السرد — `git config --list` — ليسرد لك جميع الإعدادات التي يراها جت وقتئذٍ:

```
$ git config --list
user.name=Mohammad Ali
user.email=mohammadali@example.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto
...
```

CONSOLE

ربما ترى بعض الأسماء مكررة، هذا لأن جت قد وجد الاسم نفسه في أكثر من ملف (`[path]/etc/gitconfig` و `~/.gitconfig` مثلاً). في مثل هذه الحالة يعتمد جت القيمة الأخيرة لكل اسم يراه.

يمكنك أيضاً سؤال جت عن القيمة التي يراها لاسم معين، بالأمر «اسم» `git config`:

```
$ git config user.name
Mohammad Ali
```

CONSOLE

قد يقرأ جت متغير تهيئة معين من أكثر من ملف، فقد تجد بعض القيم مثيرة للدهشة ولا تعرف من أين أنت. يمكنك في مثل هذه الحالة سؤال جت: من أين لك هذا؟—أي بخيار إظهار المصدر `--show-origin`، الذي يخبرك بالملف الذي غلب على أمرهم في قيمة هذا المتغير:

```
$ git config --show-origin rerere.autoUpdate
file:/home/moali/.gitconfig false
```

CONSOLE



١، ٧، الحصول على المساعدة

إذا احتجت يوماً إلى المساعدة في جت، فعندك ثلاث طرائق متكافئة للحصول على صفحة الدليل الشامل (`manpage`) لأي أمر من أوامر جت:

```
$ git help «أمر»
$ git --help «أمر»
```

CONSOLE

```
$ man git-«أمر»
```

مثلا، للحصول على صفحة مساعدة الأمر `git config` ، نفذ هذا:

```
$ git help config
```

CONSOLE

هذه الأوامر جميلة لأنك تستطيع استعمالها في أي مكان، حتى عندما تكون غير متصل بالشبكة. إن لم تكن صفحات المساعدة وهذا الكتاب كافيين واحتجت مساعدة شخصية، يمكنك تجربة إحدى قنوات IRC مثل `#git` أو `#github` أو `#gitlab` على خادم `Libera Chat`، الذي تجده على <https://libera.chat>. هذه القنوات مليئة باستمرار بمئات الخبراء في جت والذين أغلب الأوقات يولّدون المساعدة.

وإن لم تحتج إلى صفحة الدليل الكبيرة الكاملة، ولكن احتجت فقط إلى تذكير بالخيارات المتاحة لأحد أوامر جت، فيمكنك طلب المساعدة الموجزة بالخيار `-h` ، مثل:

```
$ git add -h
```

```
usage: git add [<options>] [--] <pathspec>...
```

CONSOLE

<code>-n, --dry-run</code>	dry run
<code>-v, --verbose</code>	be verbose
<code>-i, --interactive</code>	interactive picking
<code>-p, --patch</code>	select hunks interactively
<code>-e, --edit</code>	edit current diff and apply
<code>-f, --force</code>	allow adding otherwise ignored files
<code>-u, --update</code>	update tracked files
<code>--renormalize</code>	renormalize EOL of tracked files (implies -u)
<code>-N, --intent-to-add</code>	record only the fact that the path will be added later
<code>-A, --all</code>	add changes from all tracked and untracked files
<code>--ignore-removal</code>	ignore paths removed in the working tree (same as --no-all)
<code>--refresh</code>	don't add, only refresh the index
<code>--ignore-errors</code>	just skip files which cannot be added because of errors
<code>--ignore-missing</code>	check if - even missing - files are ignored in dry run
<code>--sparse</code>	allow updating entries outside of the sparse-checkout cone
<code>--chmod (+ -)x</code>	override the executable bit of the listed files
<code>--pathspec-from-file <file></code>	read pathspec from file
<code>--pathspec-file-nul</code>	with --pathspec-from-file, pathspec elements are separated with NUL character

٨، الخلاصة

أنت الآن مُلم بماهية جت وكيف هو مختلف عن أي نظام إدارة نسخ مركزي ربما تكون استخدمته سابقا. وكذلك الآن لديك جت مثبتا على نظامك ومضبوطا بهويتك الشخصية. حان الآن موعد تعلم بعض أسس جت.

الباب الثاني:

أسس جت

لو لم تكن ستقرأ إلا باباً واحداً لتنطلق مع جت، فهذا هو. يشرح هذا الباب جميع الأوامر الأساسية التي تحتاجها لعمل الغالبية العظمى من الأمور التي ستقضي أغلب وقتك مع جت تفعلها بعد ذلك. مع نهاية هذا الباب ستكون قادراً على تهيئة مستودع وابتدائه، وبدء تعقب ملفات وإيقافه، وتأهيل تعديلات وإيداعها. سنريك أيضاً كيف تجعل جت يتجاهل ملفات معينة أو أنماطاً معينة من الملفات، وكيف تتراجع عن الأخطاء بسرعة وسهولة، وكيف تتصفح تاريخ مشروعك، وكيف ترى التعديلات بين الإيداعات، وكيف تدفع إلى المستودعات البعيدة وتجذب منها.

٢، ١، الحصول على مستودع جت

في المعتاد تحصل على مستودع جت بإحدى طريقتين:

١. تأتي مجلدًا محليًا ليس تحت إدارة نُسخ، وتحوله إلى مستودع جت،

٢. أو تلتلسخ مستودع جت موجودًا بالفعل.

في كلتا الحالتين، سيصير معك مستودع جت على حاسوبك المحلي وجاهز للعمل.

٢، ١، ١، ابتداء مستودع في مجلد موجود

إذا كان لديك مجلد مشروع ليس تحت إدارة نسخ الآن، وتريد أن تبدأ في إدارته باستخدام جت، تحتاج أولاً إلى الذهاب إلى ذلك المجلد. إن لم تفعل هذا من قبل، فهذا قد يختلف قليلاً حسب نظامك:

للينكس:

```
$ cd /home/user/my_project
```

CONSOLE

لماك أو إس:

```
$ cd /Users/user/my_project
```

CONSOLE

لويندوز:

```
$ cd C:/Users/user/my_project
```

CONSOLE

ثم اكتب:

```
$ git init
```

CONSOLE

هذا ينشئ لك مجلدًا فرعيًا جديدًا يُسمى `git`. (يبدأ اسمه بنقطة، ليكون مجلدًا مخفيًا) ويحوي كل الملفات الضرورية لمستودعك؛ أي هيكل مستودع جت. حتى الآن، لا شيء في مشروعك متعقب بعد. انظر باب [دواخل جت](#) للمزيد من المعلومات عن تفاصيل الملفات التي في مجلد `git` الذي أنشأته للتو.

إذا أردت أن تبدأ في إدارة نسخ ملفات موجودة (وليس مجلدًا فارغًا)، فعليك بدء تعقب هذه الملفات وصنع إيداع أولي. يمكنك فعل هذا ببعض أوامر الإضافة `git add`، التي تحدد الملفات التي تريد تعقبها، ثم بأمر الإيداع `git commit`:

```
$ git add *.c
$ git add LICENSE
$ git commit -m 'Initial project version' # إيداع «أول نسخة من المشروع»
```

CONSOLE

سنعرف ماذا تفعل هذه الأوامر خلال لحظات. لكن الآن، لديك مستودع جت به ملفات متعقبة وإيداع أول.

٢، ١، ٢، استنساخ مستودع موجود

إذا كنت تريد الحصول على نسخة من مستودع جت موجود—مثلا مشروع تحب المساهمة فيه—فإن الأمر الذي تريده هو أمر الاستنساخ `git clone`. إذا كنت تعرف أنظمة أخرى لإدارة النسخ مثل Subversion، ستلاحظ أن الأمر هو `clone` (استنساخ) وليس `checkout` (سحب). هذا فرق مهم؛ فبدلاً من جلب مجرد نسخة للعمل عليها، فإن جت يحضر لك تقريباً كل شيء لدى الخادوم؛ كل نسخة من كل ملف عبر تاريخ المشروع، يجذبها جت إليك عندما تنفذ `git clone`. في الحقيقة، إذا تلف قرص الخادوم، يمكنك في الغالب استعمال أي استنساخ عند أي عميل لإرجاع الخادوم إلى حالته عندما أُستنسخ (قد تفقد بعض الخطاطيف الخاصة بالخادوم وأشياء من هذا القبيل، لكن جميع البيانات التي تحت إدارة النسخ ستكون موجودة؛ انظر فصل [تنشيط جت على خادم](#) للمزيد من التفاصيل).

استنسخ مستودعاً بالأمر «رابط» `git clone`. مثلاً إذا أردت استنساخ مكتبة جت القابلة للربط المسماة `libgit2`، يمكنك فعل ذلك هكذا:

```
$ git clone https://github.com/libgit2/libgit2
```

CONSOLE

هذا ينشئ مجلدًا اسمه `libgit2`، ويبتدئ مجلد `git`. فيه، ويجذب جميع بيانات هذا المستودع، ويسحب لك من النسخة الأخيرة منه نسخة للعمل. فإذا دخلت مجلد `libgit2` الجديد الذي أنشئ آنفًا، فستجد فيه ملفات المشروع تنتظرك للعمل عليها أو استعمالها.

وإذا أردت استنساخ المستودع إلى مجلد باسم غير `libgit2`، يمكنك تعيين هذا الاسم الجديد بإضافته إلى معاملات الأمر:

```
$ git clone https://github.com/libgit2/libgit2 mylibgit
```

هذا الأمر يفعل الشيء نفسه الذي يفعله الأمر السابق، لكن يختلف المجلد الهدف فصار `mylibgit`.

يستطيع جت التعامل مع عدد من موافيق (بروتوكولات) النقل المختلفة. استخدم المثال السابق ميفاق `https://`، ولكنك قد ترى أيضا `git://`، أو `user@server:path/to/repo.git` الذي يستخدم ميفاق SSH. يخبرك فصل [تثبيت جت على خادم](#) بجميع الخيارات التي يستطيع الخادوم إعدادها حتى يمكنك الوصول إلى مستودع جت الخاص بك، ومزايا وعيوب كل منها.

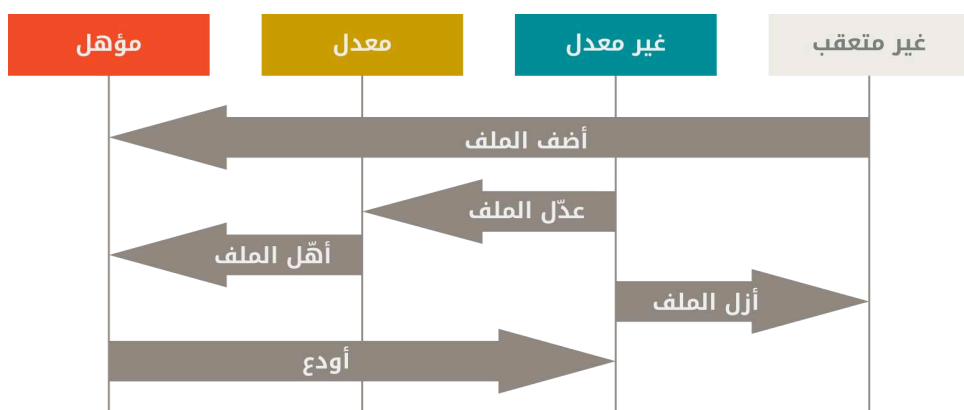
٢، ٢، تسجيل التعديلات في المستودع

الآن لديك مستودع جت أصيل على حاسوبك، وأمامك نسخة مسحوبة من جميع ملفاته، أي «نسخة عمل». غالبا ستود البدء بعمل تعديلات وإيداع لقطات من هذه التعديلات في مستودعك كل مرة يصل مشروعك إلى مرحلة تريد تسجيلها.

تذكر أن كل ملف في مجلد العمل لديك يمكن أن يكون في حالة من اثنتين: **متعقب** أو **غير متعقب**. الملفات المتعقبة هي الملفات التي كانت في اللقطة الأخيرة أو أي ملف **أهل** حديثاً. ويمكن أن تكون غير معدلة، أو معدلة، أو مؤهلة. باختصار، الملفات المتعقبة هي الملفات التي يعرفها جت.

الملفات غير المتعقبة هي كل شيء آخر: أي ملفات في مجلد عملك لم تكن ضمن لقطتك الأخيرة وليست في منطقة التأهيل. عندما تستنسخ مستودعاً أول مرة، تكون جميع ملفاتك متعقبة وغير معدلة، لأن جت سحبتها لك للتو ولم تعدل فيها شيئاً بعد.

وعندما تبدأ في تعديل ملفات، سيراهما جت معدلة، لأنك غيرتها عما كانت عليه في إيداعك الأخير. وعندما تشرع في العمل، سنتتقي من هذه الملفات ما تؤهله ثم تودع هذه التعديلات المؤهلة، ثم تعيد الكرة.



شكل ٨. دورة حياة حالة ملفاتك

٢، ٢، ١، فحص حالة ملفاتك

الأداة الأساسية التي تستعملها لمعرفة أي ملف في أي حالة، هي أمر الحالة `git status`. إذا نفذت هذا الأمر مباشرة بعد استنساخ مستودع جديد، سترى شيئاً مثل هذا:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
nothing to commit, working tree clean
```

هذا يعني أن لديك مجلد عمل نظيف؛ أي أن لا ملف من ملفاتك المتعقبة معدل. وأيضا لا يرى جت أي ملفات غير متعقبة، وإلا لسرّكها هنا. وأخيرا، يخبرك هذا الأمر أي فرع أنت فيه، ويعلمك أنه لم يختلف عن أخيه الفرع الذي في المستودع البعيد. ذلك الفرع حتى الآن هو دائما master، وهو المبدئي؛ لا تحتاج أن تقلق بشأنه هنا. سيناقدش باب [التفرع في جت](#) الفروع والإشارات بالتفصيل.

غيّرت شركة جت هب (GitHub) اسم الفرع المبدئي من master إلى main في منتصف عام ٢٠٢٠، ثم تبعته خدمات استضافة جت الأخرى. لذلك قد تجد أن اسم الفرع المبدئي هو main في المستودعات التي أنشئت حديثاً، وليس master. وأيضا يمكنك تغيير اسم الفرع المبدئي (كما رأيت في فصل [اسم الفرع المبدئي](#))، فربما ترى اسماً آخر للفرع المبدئي.



ولكن ما زال جت نفسه يسمي الفرع المبدئي master، لذا فهذا ما سنستعمل خلال الكتاب.

لنقل إنك أضفت ملفاً جديداً إلى مشروعك، مثلاً ملف README («أقراني») صغير. إذا لم يكن هذا الملف موجوداً من قبل، ونفذت أمر الحالة git status، فسترى ملفك غير المتعقب هكذا:

```
$ echo 'My Project' > README
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Untracked files:
  (use "git add <file>..." to include in what will be committed)

  README

nothing added to commit but untracked files present (use "git add" to track)
```

نرى أن ملفك الجديد غير متعقب، لأنه تحت عنوان "Untracked files" («ملفات غير متعقبة») في ناتج الحالة. «غير متعقب» لا يعني إلا أن جت يرى ملفاً لم يكن في اللقطة السابقة (الإيداع الأخير)، ولم تؤهله بعد. ولن يبدأ جت في ضمه إلى لقطات الإيداعات إلا بعد أن تخبره بذلك بأمر صريح. إنه لا يفعل ذلك لكيلا تضم بالخطأ ملفات رقمية مولدة أو ملفات أخرى لم تنشأ ضمها أصلاً. ولكنك تريد ضم README، فهيا نبدأ تعقب هذا الملف.

١، ٢، ٣، تعقب ملفات جديدة

لبدء تعقب ملف جديد، استخدم أمر الإضافة git add. فمثلاً لبدء تعقب ملف README، نفذ هذا:

```
$ git add README
```

إذا كررت أمر الحالة، فسترى ملف README قد صار متعقباً ومؤهلاً للإيداع:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)

    new file:   README
```

نعرف أنه مؤهل لأنه تحت عنوان "Changes to be committed" («تغييرات ستودع»). إذا أودعت الآن، فإن نسخة الملف وقت تنفيذ أمر الإضافة `git add` هي التي ستكون في اللقطة التاريخية التالية. تذكر أنك عندما نفذت أمر الابتداء `git init` سابقاً، أتبعته بأمر الإضافة «ملفات» `git add` لبدأ تعقب ملفات مستودعك. أمر الإضافة `git add` يُعطى مسار ملف أو مجلد. فإذا كان مجلدًا فإنه يضيف جميع الملفات التي فيه وفي أي مجلد فرعي فيه.

٢، ٣، تأهيل ملفات معدلة

لنعدّل ملفاً قد جعلناه متعقباً. إذا عدّلت الملف المتعقب `CONTRIBUTING.md` وكررت أمر الحالة، فسترى ما يشبه هذا:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

يظهر اسم الملف `CONTRIBUTING.md` تحت عنوان "Changes not staged for commit" («تغييرات غير مؤهلة للإيداع») —الذي يعني أن ملفاً متعقباً قد تغيّر في مجلد العمل، ولكنه لم يؤهل بعد. لتأهيله، نفذ أمر الإضافة `git add`. يُستعمل أمر الإضافة لأغراض عديدة: لبدء تعقب ملفات جديدة، ولتأهيل الملفات المعدلة، ولأفعال أخرى مثل إعلان حل نزاع دمج في ملف. ربما من المفيد أن تعتبره بمعنى «أضف تحديداً هذا المحتوى إلى الإيداع التالي» بدلا من «أضف هذا الملف إلى المستودع». لتنفيذ `git add` الآن لتأهيل ملف `CONTRIBUTING.md` ثم نكرر `git status`:

```
$ git add CONTRIBUTING.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)
```

```
new file:   README
modified:   CONTRIBUTING.md
```

كلا الملفين مؤهلان وسيكونان في إيداعك التالي. لنقل إنك تذكرت الآن تعديلًا طفيفًا أردته في ملف CONTRIBUTING.md قبل إيداعه. ستفتح الملف مجددًا، وتعدل فيه، ثم تحفظه وتغلقه. الآن أنت جاهز للإيداع. ولكن، لنرى الحالة git status مرة أخرى:

```
CONSOLE
$ vim CONTRIBUTING.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README
    modified:   CONTRIBUTING.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

يا للهول! لقد صار CONTRIBUTING.md مسرودًا أنه مؤهل وكذلك غير مؤهل. كيف يُعقل هذا؟ يتضح أن جت يؤهل الملف تمامًا كما هو عندما تنفذ git add. فإذا أودعت الآن، فإن ما سيودع هو نسخة CONTRIBUTING.md التي كانت موجودة عندما نفذت أمر الإضافة git add آخر مرة، وليس نسخة الملف الظاهرة لديك في مجلد العمل عندما تنفذ أمر الإيداع git commit. فإن عدلت ملفًا بعد إضافته، فعليك إضافته مرة أخرى لتأهيل النسخة الأخيرة منه:

```
CONSOLE
$ git add CONTRIBUTING.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README
    modified:   CONTRIBUTING.md
```

٢، ٤، الحالة الموجزة

مع كون ناتج أمر الحالة git status شاملاً، إلا أنه كثير الكلام. يتيح جت أيضاً خياراً للحالة الموجزة، لترى تعديلاتك بإيجاز: إذا نفذت git status -s أو git status --short، فسيعطيك الأمر ناتجاً أقصر كثيراً:

```
CONSOLE
$ git status -s
M README
MM Rakefile
A lib/git.rb
```

```
M lib/simplegit.rb
?? LICENSE.txt
```

أمام الملفات الجديدة التي لم تُتَعَب علامتا `??`. والملفات الجديدة المؤهلة أمامها `A` (اختصار «أضيف»). والملفات المعدلة أمامها `M` (اختصار «معدل»). وهكذا. ويوجد عمودان في الناتج أمام أسماء الملفات: العمود الأيسر يوضح حالته في منطقة التأهيل، والأيمن يوضح حالته في شجرة العمل. لذا ففي ناتج مثالنا هذا، ملف `README` معدل في مجلد العمل ولكنه ليس مؤهلاً بعد، ولكن ملف `lib/simplegit.rb` معدل ومؤهل. وملف `Rakefile` معدل ومؤهل ثم معدل مرة أخرى، ففيه تعديلات مؤهلة وتعديلات غير مؤهلة.

٢.٥، تجاهل ملفات

سيكون لديك غالباً فئة من الملفات التي لا تريد من جت أن يضيفها آلياً ولا حتى أن يخبرك أنها غير متعقبة. أشهر أمثلتها الملفات المولدة آلياً مثل ملفات السجلات أو الملفات المبنية. يمكنك عندئذٍ إنشاء ملف يسمى `.gitignore` (يبدأ بنقطة، لجعله مخفياً) ليسرد أنماط أسماء تلك الملفات ليتجاهلها جت. هذا مثال على ملف `.gitignore`:

```
$ cat .gitignore
*.oa
*~
```

CONSOLE

يطلب السطر الأول من جت أن يتجاهل أي ملفات ينتهي اسمها بـ `"o"` أو `"a"`—ملفات الكائنات وملفات المكتبات المضغوطة التي قد تُنتج أثناء بناء مصدر البرمجي. ويطلب السطر الثاني من جت أن يتجاهل جميع الملفات التي ينتهي اسمها بعلامة التلدة (`~`)، التي تستعملها محررات نصوص عديدة مثل Emacs لتميز الملفات المؤقتة. يمكنك كذلك إضافة مجلد `log` أو `tmp` أو `pid`، أو الوثائق المولدة آلياً، إلخ. إعداد ملف التجاهل `.gitignore` لمستودعك الجديد قبل الانطلاق في المشروع هو تفكير حسن عموماً، لكيلا تودع بالخطأ ملفات يقيماً لا تريدها في مستودعك.

إليك قواعد الأنماط التي تستطيع استعمالها في ملف التجاهل:

- تهمل الأسطر الفارغة أو الأسطر البائدة بعلامة `#`.
- يمكن استعمال أنماط توسيع المسارات (`glob`) المعتادة (ستُوضح بالتفصيل)، وستُطبق في جميع مجلدات شجرة العمل.
- يمكنك بدء الأنماط بفاصلة مائلة أمامية (`/`) لمطابقة الملفات أو المجلدات في المجلد الحالي فقط، وليس أي مجلد فرعي.
- يمكنك إنهاء الأنماط بفاصلة مائلة أمامية (`/`) لتحديد مجلد.
- يمكنك نفي نمط ببداية بعلامة تعجب (`!`).

تشبه أنماط `glob` نسخة مُيسَّرة من «التعابير النمطية»، وتستعملها الصدفات. فتُطابق النجمة (`*`) صفر أو أكثر من المحارف؛ ويُطابق `[abc]` أي حرف داخل القوسين المربعين (أي `a` أو `b` أو `c` في هذه الحالة)؛ وتُطابق علامة الاستفهام الغريبة (`?`) محرّفاً واحداً؛ ولمطابقة مدًى من المحارف، نكتب أول محرّف وآخر محرّف (بترتيبهما في Unicode)

داخل قوسين مربعين وبينهما شرطة، فمثلاً لمطابقة رقمًا من الأرقام المغربية (من 0 إلى 9) نكتب [0-9]. كذلك النجمتان يطابقان أي عدد من المجلدات الفرعية، فمثلاً يطابق النمط a/**/z كلاً من a/z و a/b/z و a/b/c/z وهكذا.

إليك مثال آخر على ملف .gitignore :

```
# تجاهل كل الملفات ذات الامتداد a
*.a

# لكن تعقب lib.a، حتى لو كنت تتجاهل جميع ملفات a بالأعلى
!lib.a

# تجاهل فقط TODO في المجلد الحالي، وليس subdir/TODO مثلًا
/TODO

# تجاهل أي مجلد اسمه build وكل شيء داخله
build/

# تجاهل doc/notes.txt ولكن ليس doc/server/arch.txt مثلًا
doc/*.txt

# تجاهل جميع pdf في مجلد doc أو أي مجلد فرعي فيه
doc/**/*.pdf
```

يرعى جتهد قائمةً شاملةً نسبياً من أمثلة ملفات التجاهل الحسنة لعشرات المشروعات واللغات في <https://github.com/github/gitignore>، إن احتجت شيئاً تبدأ منه لمشروعك.



قد يكون لدى المستودع في الحالات البسيطة ملف تجاهل واحد في مجلد الجذر، والذي يطبق على المستودع بجميع مجلداته الفرعية. ولكن ممكن كذلك وجود ملفات تجاهل أخرى في مجلدات فرعية. وملفات التجاهل الداخلية هذه لا تطبق قواعدها إلا على الملفات التي في مجلداتها. ومثلاً لدى مستودع نواة لينكس ٢٠٦ ملف تجاهل.



يخرج عن نطاق الكتاب الغوص في تفاصيل ملفات التجاهل المتعددة؛ انظر `man gitignore` للتفاصيل.

٢، ١، رؤية تعديلاتك المؤهلة وغير المؤهلة

إذا كنت تجد ناتج أمر الحالة `git status` شديد الغموض—تريد معرفة ما الذي عدّله تحديدًا، وليس مجرد أسماء الملفات التي تعدّلت—فعليك بأمر الفرق `git diff`. نتناوله بالتفصيل فيما بعد، لكنك في الغالب تستخدمه لإجابة أحد التساؤلين: ما الذي عدّله ولم تؤهله بعد؟ وما الذي أهّلته وعلى وشك إيداعه؟ وبالرغم من أن أمر الحالة `git status` يجيبهما إجابةً عامةً جداً بسرد أسماء الملفات، إلا أن أمر الفرق `git diff` يُظهر لك بالتحديد السطور المضافة والمزالة: الرقعة، إن جاز التعبير.

لننقل إِنْكَ عَدَلْتِ ملف README مجددا وأَهْلَتِهِ، ثم عَدَلْتِ ملف CONTRIBUTING.md من غير تَأْهِيلِهِ. إِذَا نَفَذْتَ أَمْرَ الْحَالَةِ، سَتَرَى مِنْ جَدِيدٍ شَيْئاً مِثْلَ هَذَا:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

لرؤية ما الذي عَدَلْتِهِ ولم تَوَهْلِهِ بعد، اكتب `git diff` من غير أي معاملات أخرى:

```
$ git diff
diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md
index 8ebb991..643e24f 100644
--- a/CONTRIBUTING.md
+++ b/CONTRIBUTING.md
@@ -65,7 +65,8 @@ branch directly, things can get messy.
Please include a nice description of your changes when you submit your PR;
if we have to read the whole diff to figure out why you're contributing
in the first place, you're less likely to get feedback and have your change
-merged in.
+merged in. Also, split your changes into comprehensive chunks if your patch is
+longer than a dozen lines.

If you are starting to work on a particular area, feel free to submit a PR
that highlights your work in progress (and note in the PR title that it's
```

يقارن هذا الأمر بين محتويات مجلد العمل ومنطقة التَأْهِيل، وَيُخْبِرُكَ بما عَدَلْتِهِ ولم تَوَهْلِهِ بعد.

إِذَا أَرَدْتَ رؤية ما الذي أَهْلَتِهِ لِيَكُونَ فِي الإِيدَاعِ التَّالِي، يُمْكِنُكَ استخدام `git diff --staged`. يقارن هذا الأمر بين تعديلاتك المؤهلة وإيداعك الأخير:

```
$ git diff --staged
diff --git a/README b/README
new file mode 100644
index 0000000..03902a1
--- /dev/null
+++ b/README
@@ -0,0 +1 @@
+My Project
```


مهمّ ملاحظة أن `git diff` وحده لا يُظهر جميع التعديلات التي تمت بعد الإيداع الأخير؛ إنما يُظهر التعديلات غير المؤهلة فقط. فإذا أهّلت جميع تعديلاتك، فلن ترَ من `git diff` أي ناتج.

مثالٌ آخر: إذا أهّلت ملف `CONTRIBUTING.md` ثم عدّلته، يمكنك بأمر الفرق معرفة تعديلات الملف التي أهّلت والتعديلات التي لم تؤهل. فإذا كانت بيئتنا تبدو هكذا:

```
$ git add CONTRIBUTING.md
$ echo '# test line' >> CONTRIBUTING.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   CONTRIBUTING.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

يمكننا إذاً بأمر الفرق `git diff` أن نرى ما لم يؤهل بعد:

```
$ git diff
diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md
index 643e24f..87f08c8 100644
--- a/CONTRIBUTING.md
+++ b/CONTRIBUTING.md
@@ -119,3 +119,4 @@ at the
 ## Starter Projects

 See our [projects list](https://github.com/libgit2/libgit2/blob/development/PROJECTS.md).
+# test line
```

ومع خيار «المؤهل»: `git diff --cached`، نرى ما الذي أهّلته حتى الآن (الخياران `--staged` و `--cached` مترادفان):

```
$ git diff --cached
diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md
index 8ebb991..643e24f 100644
--- a/CONTRIBUTING.md
+++ b/CONTRIBUTING.md
@@ -65,7 +65,8 @@ branch directly, things can get messy.
 Please include a nice description of your changes when you submit your PR;
 if we have to read the whole diff to figure out why you're contributing
 in the first place, you're less likely to get feedback and have your change
-merged in.
+merged in. Also, split your changes into comprehensive chunks if your patch is
```

If you are starting to work on a particular area, feel free to submit a PR that highlights your work in progress (and note in the PR title that it's

٢، ٧، ايداع تعديلاتك

CONSOLE



```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
#
# Your branch is up-to-date with 'origin/master'.
#
# Changes to be committed:
#   new file:   README
#   modified:   CONTRIBUTING.md
#
~
~
~
".git/COMMIT_EDITMSG" 9L, 283C
```

ويترجم أولها إلى: «أدخل رسالة الإيداع لتعديلاتك. الأسطر البادئة بعلامة # ستُهمل، ورسالة فارغة ستُلغى الإيداع.»

نرى أن رسالة الإيداع المبدئية تشمل ناتج أمر الحالة الأحدث في صورة تعليق، وأن بها سطر فارغ في أولها. يمكنك إزالة هذه التعليقات وكتابة رسالة إيداعك، أو تركها في مكانها لتتذكر ماذا تودع.

إذا احتجت تذكيراً أشد تفصيلاً بما عدلت، يمكنك إمرار خيار الإطناب -v لأمر الإيداع `git commit`. هذا يضع فروقات اللقطة في المحرر، كي ترى بالتحديد ما التعديلات التي ستودع.



عندما تحفظ وتغلق المحرر، سيصنع جت إيداعك بالرسالة التي كتبتها (ما عدا الفروقات والتعليقات، أي الأسطر البادئة بعلامة #).

يمكنك عوضاً عن ذلك كتابة رسالة إيداعك في أمر الإيداع نفسه، بالخيار -m، مثل هذا:

```
$ git commit -m "Story 182: fix benchmarks for speed"
[master 463dc4f] Story 182: fix benchmarks for speed
2 files changed, 2 insertions(+)
create mode 100644 README
```

CONSOLE

الآن قد صنعت إيداعك الأول! نرى أن الإيداع قد أخبرك شيئاً عن نفسه، مثل الفرع الذي أودعت فيه (master)، وبصمة الإيداع (463dc4f)، وعدد الملفات المعدلة (2 files changed)، وإحصاءات عن السطور المضافة والمزالة في هذا الإيداع (2 insertion(+)).

تذكر أن هذا الإيداع يسجل اللقطة التي أعدتها في منطقة تأهيلك. أي ملف عدلته ولم تؤهله سيظل جالساً في مجلد العمل وهو معدّل؛ يمكنك صنع إيداع آخر لإضافته إلى تاريخ مشروعك. في كل مرة تصنع إيداعاً، تسجل من مشروعك لقطة يمكنك إرجاع مشروعك إليها أو المقارنة معها فيما بعد.

٢، ١، ٨، تخطي منطقة التأهيل

مع أن منطقة التأهيل مفيدة لدرجة مدهشة في صياغة الإيداعات كما تشاء بالضبط، إلا أنها أحياناً أعقد مما تحتاج في عملك. يتيح لك جت اختصاراً سهلاً متى أردت تخطي منطقة التأهيل: إضافة الخيار -a إلى أمر `git commit` تجعل جت يؤهل من نفسه كل ملف متعقب قبل هذا الإيداع، لتتخطى مرحلة الإضافة:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   CONTRIBUTING.md

no changes added to commit (use "git add" and/or "git commit -a")
$ git commit -a -m 'Add new benchmarks'
```

CONSOLE

```
[master 83e38c7] Add new benchmarks
1 file changed, 5 insertions(+), 0 deletions(-)
```

لاحظ أنك لم تحتجِ إلى تنفيذ `git add` على ملف `CONTRIBUTING.md` في هذه الحالة قبل الإيداع، لأن خيار `-a` يضم جميع الملفات المعدلة. هذا مريح، لكن احذر: قد يضم هذا الخيار تعديلات غير مرغوب فيها.

٢، ٩، إزالة ملفات

لإزالة ملف من جت، عليك أن تزيله من ملفاتك المتعقبة (أو بتعبير أدق، من منطقة تأهيلك)، ثم تودع. يفعل أمر الإزالة `git rm` هذا، وأيضا يزيل الملف من مجلد عملك حتى لا تراه ملفاً غير متعقب في المرة القادمة.

إذا أزلت الملف من مجلد عملك فقط، سيظهر تحت عنوان "Changes not staged for commit" («تعديلات غير مؤهلة للإيداع») في ناتج أمر الحالة:

```
CONSOLE
$ rm PROJECTS.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        deleted:    PROJECTS.md

no changes added to commit (use "git add" and/or "git commit -a")
```

عندئذٍ تنفيذك أمر `git rm` يؤهل إزالة الملف:

```
CONSOLE
$ git rm PROJECTS.md
rm 'PROJECTS.md'
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

        deleted:    PROJECTS.md
```

عندما تودع في المرة القادمة ستجد أن الملف قد ذهب ولم يعد متعقباً. وإذا كنت قد عدلت الملف أو كنت قد أضفته بالفعل إلى منطقة التأهيل، فعليك إزالته عنوةً بالخيار `-f`. هذه ميزة أمان لكيلا تزيل بالخطأ بيانات لم تسجلها بعد في لقطة ولا يمكن استردادها من جت.

شيء آخر مفيد قد تود فعله هو إبقاء الملف في شجرة عملك لكن إزالته من منطقة تأهيلك. بلفظ آخر، تريد أن ينسى جت وجوده ولا يتعقبه ولكن يبقيه لك على قرصك. هذا مفيد خصوصاً إن نسيت إضافة شيء إلى ملف التجاهل `.gitignore`. ثم أهلتته بالخطأ، مثل ملف سجل كبير أو مجموعة من الملفات المبنية. استعمل خيار `--cached`

(«مؤهل») مع أمر الإزالة لفعل هذا (الخياران --staged و --cached مترادفان):

```
$ git rm --cached README
```

CONSOLE

يمكنك إعطاءه أسماء ملفات أو مجلدات أو أنماط توسيع المسارات (glob). يعني هذا أن بإمكانك فعل أشياء مثل:

```
$ git rm log/*.log
```

CONSOLE

لاحظ الشرطة المائلة الخلفية (\) قبل النجمة * ؛ هذا ضروري، لأن جت يوسع بنفسه أسماء الملفات بعد أن توسعها صدفتك. فبغير الشرطة المائلة الخلفية، ستوسع الصدفية أسماء الملفات قبل أن يراها جت. يحذف هذا الأمر جميع الملفات ذات الامتداد log. في مجلد log/. أو يمكنك فعل شيء مثل هذا:

```
$ git rm \**~
```

CONSOLE

يحذف هذا الأمر جميع الملفات المنتهي اسمها بعلامة التلدة (~).

٢، ١٠، نقل ملفات

لا يتعقب جت حركة الملفات تعقباً صريحاً، خلافاً لكثير من أنظمة إدارة النسخ الأخرى. فإذا غيّرت اسم ملف في جت، لا يخزن جت بيانات وصفية تخبره أنك غيرته. لكن جت ذكي جداً في تخمين ذلك وهو أمام الأمر الواقع — سنتعامل مع اكتشاف نقل الملفات بعد قليل.

لذا فقد تجد أنه من المحير وجود أمر «نقل» (mv) في جت. فإذا أردت تغيير اسم ملف في جت، يمكنك طلبه هكذا:

```
$ git mv file_from file_to
```

CONSOLE

وسيعمل كما ينبغي. وفي الحقيقة، إذا نفذت أمراً مثل هذا، ونظرت إلى الحالة، ستري أن جت يعتبره تغيير اسم ملف:

```
$ git mv README.md README
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:   README.md -> README
```

CONSOLE

ولكن هذا مكافئ لتنفيذ شيء مثل هذا:

```
$ mv README.md README
$ git rm README.md
```

CONSOLE

```
$ git add README
```

يُخمن جت في سرّه أن الاسم قد تغير، لذا فلا يهم إن غيّرت اسمه بهذه الطريقة أو عبر نظام التشغيل أو مدير الملفات (مثلاً بأمر النظام mv). الفرق الحقيقي الوحيد هو أن `git mv` أمر واحد وليس ثلاثة؛ إنه وسيلة راحة. والأهم أنك تستطيع استخدام أي أداة تريدها لتغيير أسماء الملفات، ثم تتعامل مع الإضافة والإزالة في جت فيما بعد، قبل الإيداع.

٢، ٣، رؤية تاريخ الإيداعات

بعدما صنعت عدداً من الإيداعات، أو استنسخت مستودعاً ذا تاريخ من الإيداعات بالفعل، قد تود الالتفات إلى الماضي ورؤية ماذا حدث. أسهل وأقوى أداة لفعل هذا هي أمر السجل `git log`:

تستعمل هذه الأمثلة مشروعاً صغيراً جداً يسمى "simplegit". للحصول على المشروع، نفذ:

```
$ git clone https://github.com/schacon/simplegit-progit
```

CONSOLE

عندما تنفذ `git log` داخل هذا المشروع، ترى شيئاً مثل هذا:

```
$ git log
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date: Mon Mar 17 21:52:11 2008 -0700

    Change version number

commit 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
Author: Scott Chacon <schacon@gee-mail.com>
Date: Sat Mar 15 16:40:33 2008 -0700

    Remove unnecessary test

commit a11bef06a3f659402fe7563abf99ad00de2209e6
Author: Scott Chacon <schacon@gee-mail.com>
Date: Sat Mar 15 10:31:28 2008 -0700

    Initial commit
```

CONSOLE

عندما تنادي أمر السجل بلا مُعاملات، أي `git log` فقط، فإنه بطبيعته يسرد لك الإيداعات التي في هذا المستودع بترتيب زمني عكسي؛ أي أن الإيداع الأحدث يظهر أولاً. كما ترى، يسرد هذا الأمر كل إيداع مع بصمته واسم مؤلفه وعنوان بريده الشبكي وتاريخ الإيداع ورسالته.

يتيح أمر السجل عدداً عظيماً متنوعاً من الخيارات لتظهر بالضبط ما تريد. سنعرض لك هنا بعضاً من أشهرها.

واحد من أكثر الخيارات إفادةً هو `-p` أو `--patch` («رُقعة»)، الذي يظهر لك الفرق (أي الرقعة) الذي أتى به كل إيداع. يمكنك كذلك تقييد عدد السجلات المعروضة، مثلاً بالخيار `-2` لإظهار آخر بيانين فقط.

```
$ git log -p -2
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date: Mon Mar 17 21:52:11 2008 -0700

    Change version number

diff --git a/Rakefile b/Rakefile
index a874b73..8f94139 100644
--- a/Rakefile
+++ b/Rakefile
@@ -5,7 +5,7 @@ require 'rake/gempackagetask'
spec = Gem::Specification.new do |s|
  s.platform = Gem::Platform::RUBY
  s.name = "simplegit"
- s.version = "0.1.0"
+ s.version = "0.1.1"
  s.author = "Scott Chacon"
  s.email = "schacon@gee-mail.com"
  s.summary = "A simple gem for using Git in Ruby code."

commit 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
Author: Scott Chacon <schacon@gee-mail.com>
Date: Sat Mar 15 16:40:33 2008 -0700
```

Remove unnecessary test

```
diff --git a/lib/simplegit.rb b/lib/simplegit.rb
index a0a60ae..47c6340 100644
--- a/lib/simplegit.rb
+++ b/lib/simplegit.rb
@@ -18,8 +18,3 @@ class SimpleGit
  end

end
-
- if $0 == __FILE__
-   git = SimpleGit.new
-   puts git.show
- end
```

يعرض هذا الخيار المعلومات نفسها أيضا ولكن مع إتباع كل بيان بالفروقات. هذا مفيد جدا لمراجعة الرُّقْع (code review) أو للنظر السريع فيما حدث في سلسلة من الإيداعات التي أضافها زميل. ولدى أمر السجل كذلك عدداً من خيارات التلخيص. فمثلاً إذا أردت رؤية إحصاءات مختصرة عن كل إيداع، جرّب خيار الإحصاء `--stat`:

```
$ git log --stat
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date: Mon Mar 17 21:52:11 2008 -0700
```

Change version number

Rakefile | 2 +-

```
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
commit 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
Author: Scott Chacon <schacon@gee-mail.com>
Date: Sat Mar 15 16:40:33 2008 -0700
```

```
Remove unnecessary test
```

```
lib/simplegit.rb | 5 -----
1 file changed, 5 deletions(-)
```

```
commit a11bef06a3f659402fe7563abf99ad00de2209e6
Author: Scott Chacon <schacon@gee-mail.com>
Date: Sat Mar 15 10:31:28 2008 -0700
```

```
Initial commit
```

```
README           | 6 ++++++
Rakefile          | 23 ++++++++++++++++++++++
lib/simplegit.rb  | 25 ++++++++++++++++++++++
3 files changed, 54 insertions(+)
```

فكما ترى، يطبع لك خيار الإحصاء `--stat` تحت بيان كل إيداع، قائمةً بالملفات المعدلة وعددها وعدد السطور المضافة والمزالة في هذه الملفات. ثم يلخّص هذه المعلومات في النهاية.

وخيار آخر عظيم النفع هو `--pretty` («منسّق»)، الذي يعرض ناتج السجل بصيغ أخرى غير الصيغة المبدئية. يعرف جت أسماءً لصيغ جاهزة يمكن استعمالها مع هذا الخيار. قيمة `oneline` («سطر واحد») تطبع كل إيداع على سطر وحيد، ما يفيد عندما تنتظر إلى إيداعات كثيرة. وكذلك، القيم `short` («قصير») و `full` («كامل») و `fuller` («أكمل») تُظهر لك ناتجًا مثل المبدئي مع زيادة أو نقصان في بعض المعلومات.

CONSOLE

```
$ git log --pretty=oneline
ca82a6dff817ec66f44342007202690a93763949 Change version number
085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7 Remove unnecessary test
a11bef06a3f659402fe7563abf99ad00de2209e6 Initial commit
```

القيمة الأكثر إمتاعًا هي `format` («صياغة»)، التي تتيح لك تعيين صيغة ناتج السجل بدقة. هذا مفيد خصوصًا عندما تولّد ناتجًا لكي يقرؤه ويحلله برنامج أو بُريمج (script) — فلأنك تعيّن الصيغة بصراحة ووضوح، فإنك تطمئن أنها لن تتغيّر مع تحديثات جت.

CONSOLE

```
$ git log --pretty=format:"%h - %an, %ar : %s"
ca82a6d - Scott Chacon, 6 years ago : Change version number
085bb3b - Scott Chacon, 6 years ago : Remove unnecessary test
a11bef0 - Scott Chacon, 6 years ago : Initial commit
```

يسرد جدول [متغيرات مفيدة لصياغة السجلات باستخدام `git log --pretty=format`](#) بعض المتغيرات المفيدة التي تفهمها `format`:

جدول ١. متغيرات مفيدة لصياغة السجلات باستخدام `git log --pretty=format`

المتغير	وصف الناتج
%H	بصمة الإيداع
%h	بصمة الإيداع المختصرة
%T	بصمة الشجرة
%t	بصمة الشجرة المختصرة
%P	بصمات الآباء
%p	بصمات الآباء المختصرة
%an	اسم المؤلف
%ae	بريد المؤلف
%ad	تاريخ التأليف (الصيغة تتبع <code>--date=option</code>)
%ar	تاريخ التأليف، نسبي
%cn	اسم المودع
%ce	بريد المودع
%cd	تاريخ الإيداع
%cr	تاريخ الإيداع، نسبي
%s	الموضوع (عنوان رسالة الإيداع)

ربما تتساءل عن الفرق بين المؤلف والمودع. المؤلف هو من كتب العمل في الأصل، والمودع هو من طبق العمل في النهاية. فمثلاً إذا أرسلت رقعة إلى مشروع، وطبقها أحد الأعضاء الأساسيين، فيجب الاعتراف بالفضل لكليهما: أنت مؤلفاً، والعضو الأساسي مودعاً. سنتناول هذا التمييز بالتفصيل في باب [جيت المنزوع](#).

القيمتان `oneline` و `format` مفيدتان خصوصاً مع خيار آخر لأمر السجل يسمى «رسم» `--graph`. يضيف هذا الخيار رسماً لطيفاً بالمحارف لإظهار تاريخ التفريع والدمج.

```
$ git log --pretty=format:"%h %s" --graph
* 2d3acf9 Ignore errors from SIGCHLD on trap
```

CONSOLE

```
* 5e3ee11 Merge branch 'master' of https://github.com/dustin/grit.git
|\
| * 420eac9 Add method for getting the current branch
* | 30e367c Timeout code and tests
* | 5a09431 Add timeout protection to grit
* | e1193f8 Support for heads with slashes in them
|/
* d6016bc Require time for xmlschema
* 11d191e Merge branch 'defunkt' into local
```

سيصير هذا النوع من الناتج ممتعاً أكثر خلال تناولنا التفريع والدمج في الباب التالي.

هذه فقط بعض خيارات تنسيق ناتج `git log` اليسيرة؛ متاح عدد أكبر من ذلك كثيراً. يسرد جدول [خيارات شائعة لأمر السجل](#) الخيارات التي تناولناها حتى الآن، وكذلك بعض خيارات التنسيق الشائعة الأخرى التي قد تفيد، إضافةً إلى كيفية تغيير ناتج أمر السجل.

جدول ٢. خيارات شائعة لأمر السجل

الخيار	الوصف
-p	أظهر الرقعة التي أتى بها كل إيداع.
--stat	أظهر إحصاءات الملفات المعدلة في كل إيداع.
--shortstat	اعرض فقط سطر التعديلات/الإضافات/الإزالات من أمر --stat.
--name-only	اسرد أسماء الملفات المعدلة بعد كل إيداع.
--name-status	اسرد أسماء الملفات مرفقة بحالتها: معدل/مضاف/مزال.
--abbrev-commit	أظهر فقط الحروف القليلة الأولى من البصمة، بدلاً من الأربعين جميعاً.
--relative-date	اعرض التاريخ بصيغة نسبية (مثلاً "2 weeks ago") بدلاً من صيغة التاريخ الكاملة.
--graph	اعرض رسماً بالمحارف لتاريخ التفريع والدمج بجانب ناتج السجل.
--pretty	اعرض الإيداعات بصيغة أخرى. قيم الخيار المتاحة تشمل oneline و short و full و fuller و format (التي تتيح لك تحديد صياغتك المخصصة).
--oneline	اختصار الخيارين --abbrev-commit --pretty=oneline معاً.

٢، ٣، ١، تقييد ناتج السجل

إضافةً إلى خيارات صياغة الناتج، يتيح أمر السجل عدداً من خيارات تقييد الناتج؛ أي إظهار جزء من الإيداعات فقط. لقد رأيت أحد هذه الخيارات بالفعل: خيار 2- الذي يُظهر آخر إيداعين فقط. الحقيقة أن استخدام «ن»-، حيث ن هو أي عدد صحيح موجب، يُظهر لك آخر ن إيداعاً. لن تحتاج هذا كثيراً في الواقع، لأن جت بطبيعته يمرر الناتج كله إلى برنامج عرض ("pager" مثل less) حتى ترى ناتج السجل صفحةً صفحةً.

لكن خيارات التقييد بالزمن مثل --since («منذ») و --until («حتى») مفيدة جداً. مثلاً، هذا الأمر يسرد الإيداعات التي تمت خلال الأسبوعين السابقين:

```
$ git log --since=2.weeks
```

CONSOLE

يعمل هذا الأمر مع العديد من الصيغ؛ يمكنك تحديد تاريخ محدد مثل "2008-01-15" أو تاريخ نسبي مثل "2 years" "1 day 3 minutes ago".

يمكنك كذلك سرد الإيداعات المطابقة لمعايير بحث معينة. مثلاً خيار --author يتيح لك سرد إيداعات مؤلف معين فقط، و --grep يتيح لك البحث عن كلمات معينة في رسائل الإيداعات.

يمكنك استعمال --author أو --grep أكثر من مرة في الأمر الواحد، لتسرد الإيداعات التي توافق أي نمط --author معطى وتوافق أي نمط --grep معطى؛ ولكن إضافة خيار --all-match يقيّد الناتج إلى الإيداعات الموافقة لجميع أنماط --grep.



مصفة مفيدة جداً أخرى هي خيار -S (المعروف بالاسم الدارج: خيار «فأس» جت)، الذي يُعطى سلسلة نصية ولا يظهر إلا الإيداعات التي عدّلت عدد مرات وجودها. فمثلاً إذا أردت إظهار آخر إيداع أضاف أو أزال إشارة إلى دالة معينة، يمكنك تنفيذ:

```
$ git log -S function_name
```

CONSOLE

آخر خيار تصفية مفيد حقاً لأمر السجل هو إعطاؤه مسار. فإذا أعطيته مجلداً أو ملفاً، فإنه يقيّد ناتج السجل إلى الإيداعات التي عدّلت هذه الملفات. يكون هذا دائماً آخر خيار وفي الغالب يُسبق بشرطتين (--) لفصل المسارات عن الخيارات:

```
$ git log -- path/to/file
```

CONSOLE

نسرد في جدول [خيارات تقييد ناتج أمر السجل](#) هذه الخيارات وبعض الخيارات الأخرى حتى تكون مرجعاً لك.

جدول ٣. خيارات تقييد ناتج أمر السجل

الخيار	الوصف
--n	أظهر فقط آخر ن إيداعًا.
--after أو --since	قيّد الناتج إلى الإيداعات التي تمت بعد التاريخ المعطى.
--before أو --until	قيّد الناتج إلى الإيداعات التي تمت قبل التاريخ المعطى.
--author	لا تظهر إلا الإيداعات التي يطابق اسم مؤلفها السلسلة النصية المعطاة.
--committer	لا تظهر إلا الإيداعات التي يطابق اسم مودعها السلسلة النصية المعطاة.
--grep	لا تظهر إلا الإيداعات التي تشتمل رسالتها على السلسلة النصية المعطاة.
-S	لا تظهر إلا الإيداعات التي أضافت أو أزلت سطورًا برمجية فيها السلسلة النصية المعطاة.

مثلاً، إذا أردت رؤية أيّ الإيداعات التي عدّلت ملفات الاختبارات في مصدر جت، وقد أودعها Junio Hamano في شهر أكتوبر عام ٢٠٠٨، وليست إيداعات دمج، يمكنك فعل شيء مثل هذا:

```
$ git log --pretty="%h - %s" --author='Junio C Hamano' --since="2008-10-01" \
--before="2008-11-01" --no-merges -- t/
5610e3b - Fix testcase failure when extended attributes are in use
acd3b9e - Enhance hold_lock_file_for_{update,append}() API
f563754 - demonstrate breakage of detached checkout with symbolic link HEAD
d1a43f2 - reset --hard/read-tree --reset -u: remove unmerged new paths
51a94af - Fix "checkout --track -b newbranch" on detached HEAD
b0ad11e - pull: allow "git pull origin $something:$current_branch" into an unborn branch
```

CONSOLE

حوالي أربعين ألف إيداع في تاريخ مصدر جت، وهذا الأمر لا يُظهر منها إلا الإيداعات الستة المطابقة لتلك المعايير.

منع عرض إيداعات الدمج

حسب أسلوب التطوير في مستودعك، قد يكون عدد ضخ من الإيداعات في تاريخ سجلك مجرد إيداعات دمج، وهي لا تفيد كثيراً. لمنع عرضها وإزاحتها تاريخ سجلك، أضف إلى أمر السجل خيار منع الدمج -no-merges.



٢، ٤، التراجع عن الأفعال

قد تحتاج في أي مرحلة إلى التراجع عن فعلٍ ما. سنرى الآن بعض الأدوات الأساسية للتراجع عن تعديلاتك. كن حذراً، لأن بعض هذه التراجعات لا يمكن التراجع عنها فيما بعد. هذه من المناطق القليلة في جت التي يمكنك أن تفقد فيها شيئاً من عملك إذا فعلت شيئاً خطأ.

واحد من أشهر التراجعات هو عندما تودع قبل الأوان وتنسى إضافة ملفات أو تخطئ في رسالة إيداعك. إذا أردت إعادة هذا الإيداع، فقم بالتعديلات التي نسيته، وأهله، ثم أودع مجدداً مع خيار التصحيح `--amend` :

```
$ git commit --amend
```

CONSOLE

يأخذ هذا الأمر منطقة تأهيلك ويستعملها للإيداع. وإذا لم تعدل ملفاً منذ إيداعك الأخير (مثلاً نفذت هذا الأمر مباشرة بعد إيداعك السابق)، فإن لقطتك ستتطابق تماماً بلا اختلاف، ولن تغير سوى رسالة الإيداع.

سيظهر لك محرر رسالة الإيداع، لكن ستجد فيه رسالة الإيداع السابقة في انتظارك لتعديلها إن شئت أو تغيّرها تماماً.

مثلاً، إذا أودعت ثم أدركت أنك نسييت تأهيل تعديلات على ملف تريدها في هذا الإيداع، يمكنك فعل شيء مثل هذا:

```
$ git commit -m 'Initial commit'
$ git add forgotten_file
$ git commit --amend
```

CONSOLE

ستجد في النهاية إيداعاً واحداً؛ فالإيداع الثاني يحل محل الأول.

مهمّ فهم أنك عندما تصحح إيداعك الأخير، فإنك لا تصلحه ولكن تبدّله برُمّته وتضع مكانه إيداعاً جديداً محسّناً وتزيح القديم عن الطريق. في الحقيقة، هذا كأن الإيداع السابق لم يحدث من الأصل، ولن يظهر في تاريخ مستودعك.



الفائدة الواضحة لتصحيح الإيداعات هو التحسينات الطفيفة للإيداع الأخير، بغير إزحام تاريخ مستودعك برسائل إيداعات من نوعية «عذراً، نسييت إضافة ملف» أو «سحقاً، خطأ مطبعي في الإيداع السابق؛ أصلحته».

لا تصحح إلا الإيداعات التي لا تزال على جهازك ولم تدفعها بعد إلى أي مكان آخر. فتصحيح إيداع قد دُفع بالفعل ثم الدفع عنوة (`git push --force`) سيسبب مشاكل للمتعاونين معك. لمعرفة ما سيحدث إن فعلت هذا وكيف تتعافي إذا كنت الطرف المتلقي، اقرأ فصل [محذورات إعادة التأسيس](#).



٢، ٤، ١، إلغاء تأهيل ملف مؤهل

سيوضح الفصلان التاليان كيف تتعامل مع التعديلات في منطقة تأهيلك ومجلد عملك. الجميل أن الأمر الذي تستخدمه لمعرفة حالة إحدى هاتين المنطقتين يذكرك أيضا بكيفية التراجع عما فيهما من تعديلات. إن كنت مثلاً قد عدلت ملفين وأردت إيداع كلٍ منهما في إيداع منفصل، ولكنك كتبت خطأً `git add *` فأهلت كليهما. كيف يمكنك إلغاء تأهيل أحدهما؟ أمر الحالة يذكرك:

```
$ git add *
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README.md -> README
    modified:   CONTRIBUTING.md
```

مباشرةً تحت "Changes to be committed" («تعديلات ستودع») تجده يقول «استخدم «ملفات» `git reset HEAD` لإلغاء التأهيل». فلنعمل بهذه النصيحة إننا، لإلغاء تأهيل ملف `CONTRIBUTING.md`:

```
$ git reset HEAD CONTRIBUTING.md
Unstaged changes after reset:
M   CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README.md -> README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

هذا الأمر غريب قليلاً، لكنه يعمل كما توقعنا. ملف `CONTRIBUTING.md` معدّل لكنه عاد من جديد غير مؤهل.

صدقاً إن `git reset` أمر خطير، خصوصاً مع الخيار `--hard`. مع ذلك، فإن الملف الذي في مجلد عملك لم يُمس في الموقف الموضح بالأعلى، لذا فهذا الأمر آمن نسبياً في مثل هذا الموقف.



هذا الأمر السحري هو كل ما تحتاج معرفته الآن عن أمر الإرجاع `git reset`. سنغوص في فصل [تغيير النصوص عن أمر الإرجاع](#) `reset` في تفاصيل أعمق كثيراً عن أمر الإرجاع وماذا يفعل وكيف تتقنه لتفعل أفعالاً شائعة وممتعة جداً.

٢، ٤، ٢، إعادة ملف معدل إلى حالته قبل التعديل

ماذا لو أدركت أنك لم تعد تريد تعديل ملف CONTRIBUTING.md من الأساس؟ كيف يمكنك إرجاعه إلى حالته عند الإيداع الأخير (أو الاستنساخ الأول، أو كيفما حصلت عليه في مجلد عملك)؟ لحسن الحظ، يخبرك أمر الحالة بهذا أيضاً. في ناتج المثال الأخير، كان جزء التعديلات غير المؤهلة هكذا:

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md

CONSOLE

فيخبرك أن تستخدم الأمر «ملفات» git checkout -- لتجاهل التعديلات التي في مجلد عملك. لنفعل ما يخبرنا به:

```
$ git checkout -- CONTRIBUTING.md
```

```
$ git status
```

On branch master

Changes to be committed:

(use "git reset HEAD <file>..." to unstage)

renamed: README.md -> README

CONSOLE

كما ترى، أُلغيت التعديلات.

من المهم جداً فهم أن «ملفات» git checkout -- أمر خطير؛ أي تعديلات غير مودعة، على هذا الملف، قد ضاعت، فقد أزال جت للتو هذا الملف ووضع مكانه آخر نسخة مؤهلة أو مودعة منه. إياك أبدا أن تستعمل هذا الأمر، إلا أن تكون واعياً أشد الوعي أنك لا تريد هذه التعديلات غير المحفوظة.



إذا أردت الإبقاء على تعديلاتك على هذا الملف لكنك لا تزال تريد إزاحته جانباً الآن، فسنشرح التفرع في [باب التفرع في جت](#) والتخبيّة بعد ذلك؛ هاتان الطريقتان أفضل عمومًا.

تذكر أن أي شيء تودعه في جت يمكن شبيه دائماً استعادته. حتى الإيداعات في الفروع المحذوفة أو الإيداعات المبدلة بخيار التصحيح (--amend) يمكن استعادتها (انظر فصل [استرجاع البيانات لاستعادة البيانات](#)). مع ذلك، أي شيء تفقده لم يكن مودعاً، صعب أن تراه مرة أخرى.

٢، ٤، ٣، التراجع بأمر الاستعادة git restore

أضافت النسخة 2.23.0 من جت أمراً جديداً: git restore. هذا في الأصل بديل لأمر الإرجاع git reset الذي ناقشناه للتو. بدءاً من النسخة 2.23.0 من جت، سيستخدم جت أمر الاستعادة git restore بدلاً من أمر الإرجاع git reset في الكثير من عمليات التراجع.

لنرند على آثارنا قصصاً ونعيد الكرة ونراجع بأمر الاستعادة git restore بدلاً من أمر الإرجاع git reset.

إلغاء تأهيل ملف مؤهل بأمر الاستعادة

سيوضح الفصلان التاليان كيف نتعامل مع التعديلات في منطقة تأهيلك ومجلد عملك بأمر الاستعادة `git restore`. الجميل أن الأمر الذي تستخدمه لمعرفة حالة إحدى هاتين المنطقتين يذكرك أيضا بكيفية التراجع عما فيهما من تعديلات. إن كنت مثلاً قد عدلت ملفين وأردت إيداع كلٍ منهما في إيداع منفصل، ولكنك كتبت خطأً `git add *` فأهّلت كليهما. كيف يمكنك إلغاء تأهيل أحدهما؟ أمر الحالة يذكرك:

```
$ git add *
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   CONTRIBUTING.md
    renamed:    README.md -> README
```

مباشرةً تحت "Changes to be committed" («تعديلات ستودع») تجده يقول «استخدم `git restore --staged` لإلغاء التأهيل». فلنعمل بهذه النصيحة إننا، لإلغاء تأهيل ملف `CONTRIBUTING.md`:

```
$ git restore --staged CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:    README.md -> README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   CONTRIBUTING.md
```

ملف `CONTRIBUTING.md` معدّل لكنه عاد من جديد غير مؤهل.

إعادة ملف معدّل إلى حالته قبل التعديل بأمر الاستعادة

ماذا لو أدركت أنك لم تعد تريد تعديل ملف `CONTRIBUTING.md` من الأساس؟ كيف يمكنك إرجاعه إلى حالته عند الإيداع الأخير (أو الاستنساخ الأول، أو كيفما حصلت عليه في مجلد عملك)؟ لحسن الحظ، يخبرك أمر الحالة بهذا أيضاً. في ناتج المثال الأخير، كان جزء التعديلات غير المؤهلة هكذا:

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   CONTRIBUTING.md
```

فيخبرك أن تستخدم الأمر «ملفات» `git restore` لتجاهل التعديلات التي في مجلد عملك. لنفعل ما يخبرنا به:


```
$ git restore CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:    README.md -> README
```

من المهم جداً فهم أن «ملفات» `git restore` أمر خطير؛ أي تعديلات غير مودعة، على هذا الملف، قد ضاعت، فقد أزال جت للتو هذا الملف ووضع مكانه آخر نسخة مؤهلة أو مودعة منه. إياك أبداً أن تستعمل هذا الأمر، إلا أن تكون واعياً أشد الوعي أنك لا تريد هذه التعديلات غير المحفوظة.



٢، ٥، التعامل مع المستودعات البعيدة

حتى تستطيع التعاون في أي مشروع يستخدم جت، تحتاج معرفة كيف تدير مستودعاتك البعيدة. المستودعات البعيدة هي نسخ من مشروعك، وهذه النسخ مستضافة على الشبكة (الإنترنت) أو على شبكة داخلية. يمكن أن يكون لديك عدد منها، وكل واحد منها غالباً يسمح لك إما بالقراءة فحسب («القراءة فقط»)، وإما بالتحريك كذلك («القراءة والكتابة»). والتعاون مع الآخرين يشمل إدارة هذه المستودعات البعيدة ودفع البيانات إليها وجذبها منها عندما تحتاج إلى مشاركة العمل. وإدارة المستودعات البعيدة تشمل معرفة كيف تضيفها في مستودعك وكيف تزيلها وكيف تدير العديد من الفروع البعيدة وكيف تجعل الفروع البعيدة متعقبة أو غير متعقبة، وغير ذلك. سنتناول في هذا الفصل بعضاً من مهارات الإدارة هذه.

المستودعات البعيدة قد تكون على جهازك المحلي

من الممكن جداً أن تعمل مع مستودع «بعيد» («remote»)، ولكنه في الحقيقة على جهازك الذي تستعمله نفسه. كلمة «بعيد» لا تعني بالضرورة أن المستودع في مكان آخر على الشبكة المحلية أو على الشبكة (الإنترنت)، ولكنها تعني فقط أنه في مكان آخر. فالعمل مع مستودع بعيد مثل هذا ما زال يحتاج جميع عمليات الدفع والجذب والاستحضار المعتادة مثل أي مستودع بعيد آخر.



٢، ٥، ١، سرد مستودعاتك البعيدة

لسرد المستودعات البعيدة التي هيأتها، استعمل أمر `git remote`. فإنه يُظهر لك الاسم المختصر لكل بعيد في مستودعك. وإذا استنسخت مستودعاً، فإنك على الأقل ستري `origin` («الأصل»)، وهو الاسم الذي يعطيه جت للمستودع الذي استنسخته منه:

```
$ git clone https://github.com/schacon/ticgit
Cloning into 'ticgit'...
remote: Reusing existing pack: 1857, done.
remote: Total 1857 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (1857/1857), 374.35 KiB | 268.00 KiB/s, done.
Resolving deltas: 100% (772/772), done.
Checking connectivity... done.
$ cd ticgit
```

```
$ git remote
origin
```

يمكنك أيضا استعمال الخيار `-v` («إطناّب»)، ليُظهر لك الروابط التي خزنها جت للأسماء المختصرة للمستودعات البعيدة ليستعملها لقراءة ذلك المستودع البعيد ولتحريره:

```
$ git remote -v
origin https://github.com/schacon/ticgit (fetch)
origin https://github.com/schacon/ticgit (push)
```

CONSOLE

إذا كان لديك أكثر من مستودع بعيد واحد، فإن هذا الأمر سيسردهم جميعاً. مثلاً، قد يبدو مستودع مرتبط بعدد من المستودعات البعيدة للعمل مع رهط من المتعاونين هكذا:

```
$ cd grit
$ git remote -v
bakkdoor https://github.com/bakkdoor/grit (fetch)
bakkdoor https://github.com/bakkdoor/grit (push)
cho45 https://github.com/cho45/grit (fetch)
cho45 https://github.com/cho45/grit (push)
defunkt https://github.com/defunkt/grit (fetch)
defunkt https://github.com/defunkt/grit (push)
koke git://github.com/koke/grit.git (fetch)
koke git://github.com/koke/grit.git (push)
origin git@github.com:mojombo/grit.git (fetch)
origin git@github.com:mojombo/grit.git (push)
```

CONSOLE

هذا يعني أننا نستطيع جذب المساهمات من أيٍّ من هؤلاء المستخدمين بسهولة. وقد يكون لدينا إذن الدفع إلى واحد أو أكثر منهم، ولكن لا نستطيع معرفة هذا من هنا.

لاحظ أن هذه المستودعات البعيدة تستعمل موافيق (بروتوكولات) متنوعة؛ سنتحدث عن هذا في فصل [تنشيت جت على خادم](#).

٢، ٥، ٢، إضافة مستودعات بعيدة

ذكرنا أن أمر الاستنساخ `git clone` يضيف لك الأصل البعيد `origin` آلياً، ورأيت مثالين على ذلك. لنعرف الآن كيف نضيف مستودعاً بعيداً بأمر صريح. لإضافة مستودع جت بعيد جديد وتسميته باسم مختصر للإشارة إليه به بسهولة فيما بعد، نفذ `git remote add <shortname> <url>`، أي الاسم المختصر ثم الرابط:

```
$ git remote
origin
$ git remote add pb https://github.com/paulboone/ticgit
$ git remote -v
origin https://github.com/schacon/ticgit (fetch)
origin https://github.com/schacon/ticgit (push)
```

CONSOLE

```
pb https://github.com/paulboone/ticgit (fetch)
pb https://github.com/paulboone/ticgit (push)
```

يمكنك الآن استعمال الاسم pb في سطر الأوامر، بدلاً من الرابط بكامله. مثلاً إذا أردت استحضار جميع المعلومات التي لدي پول ولكن ليست لديك في مستودعك بعد، استعمل أمر الاستحضار معه، أي git fetch pb :

```
$ git fetch pb
remote: Counting objects: 43, done.
remote: Compressing objects: 100% (36/36), done.
remote: Total 43 (delta 10), reused 31 (delta 5)
Unpacking objects: 100% (43/43), done.
From https://github.com/paulboone/ticgit
* [new branch]      master      -> pb/master
* [new branch]      ticgit      -> pb/ticgit
```

CONSOLE

الآن صار فرع master من مستودع پول متاحاً محلياً (في مستودعك على جهازك) بالاسم pb/master ؛ يمكنك دمجه في أحد فروعك، أو سحب إيداعه الأخير إلى فرع محلي إذا أردت تفقّده. سنشرح بالتفصيل ما هي الفروع وكيف نستعملها في [باب التفرع في جت](#).

٢، ٥، ٣، الاستحضار والجذب من مستودعاتك البعيدة

كما رأيت للتو، للحصول على بيانات من مستودعاتك البعيدة، يمكنك تنفيذ:

```
$ git fetch «البعيد»
```

CONSOLE

يذهب هذا الأمر إلى المستودع البعيد ويجذب منه كل البيانات التي لديه وليست لديك بعد. بعد أن تفعل هذا، ستجد لديك إشارات لجميع الفروع التي لدى هذا البعيد، فيمكنك دمجها أو النظر فيها وقتما شئت.

إذا استنسخت مستودعاً، فإن أمر الاستنساخ يضيف آلياً هذا المستودع البعيد بالاسم "origin". لذا فإن git fetch origin يستحضر أي عمل قد دُفع إلى هذا المستودع بعدما استنسخته (أو استحضرت منه) آخر مرة. مهمٌ ملاحظة أن أمر الاستحضار ينزل فقط البيانات إلى مستودعك المحلي؛ لكنه لا يدمجها مع عملك ولا يغير في أي شيء تعمل عليه. عليك دمجها يدوياً مع عملك عندما تكون مستعداً لذلك.

إذا كان الفرع الحالي مُعدّاً ليتعقّب فرعاً بعيداً (انظر الفصل الفرعي التالي وباب [التفرع في جت](#) للمزيد من المعلومات)، فأمر الجذب git pull يستحضر آلياً هذا الفرع البعيد ثم يدمجه في الفرع الحالي. هذا قد يكون أسهل أو أريح لك. وأمر الاستنساخ بطبيعته يجعل لك الفرع المبدئي المحلي يتعقّب الفرع المبدئي البعيد (master أو main أو أيّاً كان اسمه) في المستودع الذي استنسخت منه. يستحضر أمر الجذب البيانات من المستودع الذي استنسخت منه في الأصل عادةً ثم يحاول دمجها آلياً في الفرع الذي تعمل فيه حالياً.

بدءًا من النسخة 2.27 من جت، سيحذرَك أمر الجذب `git pull` إن لم يكن متغير `pull.rebase` مهياً. وسيبقى يحذرَك حتى تعيّن له قيمة.

إن أردت سلوك جت المبدئي (التسريع متى أمكن، وإلا فإينشاء إيداع دمج)، فننفيذ:

```
git config --global pull.rebase "false"
```

وإذا أردت إعادة التأسيس عند الجذب، فننفيذ:

```
git config --global pull.rebase "true"
```



٢، ٥، ٤، الدفع إلى مستودعاتك البعيدة

عندما يكون مشروعك في مرحلة تود مشاركتها، عليك دفعه إلى المنبع. الأمر الذي يفعل هذا يسير: `git push` `<branch> <remote>`، أي اسم المستودع البعيد ثم الفرع: فإذا أردت دفع فرع `master` الخاص بك إلى المستودع الأصل `origin` (غالبا يعيّن لك الاستنساخ هذين الاسمين آلياً)، فتنفيذ هذا الأمر يدفع أي إيداعات صنعتها إلى الخادوم:

```
$ git push origin master
```

CONSOLE

يعمل هذا الأمر فقط إذا استنسخت من مستودع لديك إذن تحريره، ولم يدفع أي أحد آخر إليه في هذه الأثناء. أما إذا استنسخت أنت وشخص آخر في وقت واحد، ودفع هو إلى المنبع، ثم أردت أنت الدفع إلى المنبع، فإن دفعك سيرفض عن حق. وسيتوجب عليك عندئذٍ استحضار عمله أولاً وضمه إلى عملك قبل أن يُسمح لك بالدفع. انظر باب [التفريع في جت](#) لمعلومات أشد تفصيلاً عن الدفع إلى مستودعات بعيدة.

٢، ٥، ٥، فحص مستودع بعيد

إذا أردت رؤية معلومات أكثر عن بعيد معين، فعليك بالأمر «البعيد» `git remote show`. إذا نفذت هذا الأمر مع اسم مختصر معين، مثل `origin`، فسترى شيئاً مثل هذا:

```
$ git remote show origin
* remote origin
Fetch URL: https://github.com/schacon/ticgit
Push URL: https://github.com/schacon/ticgit
HEAD branch: master
Remote branches:
  master                tracked
  dev-branch            tracked
Local branch configured for 'git pull':
  master merges with remote master
Local ref configured for 'git push':
  master pushes to master (up to date)
```

CONSOLE

إنه يسرد لك رابط المستودع البعيد إضافةً إلى معلومات تعقب الفروع. وللإفادة يخبرك كذلك بأنك إذا كنت في فرع master واستعملت أمر الجذب git pull فإنه تلقائيًا سيدمج فرع master البعيد في الفرع المحلي بعد استحضاره. ويسرد لك كذلك جميع الإشارات البعيدة التي جذبها إليك.

هذا مثال صغير غالبًا ستقابله. ولكن عندما تستخدم جت بكثرة، فسيعطيك git remote show معلومات أكثر كثيرًا:

```
$ git remote show origin
* remote origin
URL: https://github.com/my-org/complex-project
Fetch URL: https://github.com/my-org/complex-project
Push URL: https://github.com/my-org/complex-project
HEAD branch: master
Remote branches:
  master                tracked
  dev-branch            tracked
  markdown-strip        tracked
  issue-43              new (next fetch will store in remotes/origin)
  issue-45              new (next fetch will store in remotes/origin)
  refs/remotes/origin/issue-11  stale (use 'git remote prune' to remove)
Local branches configured for 'git pull':
  dev-branch merges with remote dev-branch
  master      merges with remote master
Local refs configured for 'git push':
  dev-branch      pushes to dev-branch      (up to date)
  markdown-strip  pushes to markdown-strip  (up to date)
  master          pushes to master          (up to date)
```

يُظهر لك هذا الأمر ما الفرع الذي يجذب جت إليه تلقائيًا عندما تنفذ git push وأنت في فرع معين. ويُظهر لك كذلك ما فروع المستودع البعيد التي ليست لديك بعد، وما الفروع البعيدة التي لديك وأُزيلت من البعيد، وما الفروع المحلية التي تقبل الدمج التلقائي من فروعها المتعقبة للبعيد عندما تنفذ git pull.

٢، ٥، ٦، تغيير اسم بعيد أو حذفه

يمكنك استعمال git remote rename لتغيير الاسم المختصر لمستودع بعيد. مثلاً، لتغيير اسم pb إلى paul، نفذ:

```
$ git remote rename pb paul
$ git remote
origin
paul
```

مهم ملاحظة أن هذا يغيّر أسماء فروعك المتعقبة للبعيد أيضاً. فالذي كان يسمى pb/master صار paul/master.

وإذا أردت إزالة إشارة لمستودع بعيد لسبب ما — نقلت المستودع، أو لم تعد تستخدم خادم مرآة معين، أو مساهم لم يعد يساهم — استعمل أمر إزالة البعيد git remote remove أو اختصاره git remote rm:

```
$ git remote remove paul
$ git remote
```

وما إن تحذف الإشارة إلى بعيدٍ هكذا، فإن جميع الفروع المتعقبة له وجميع إعدادات التهيئة المرتبطة به ستُحذف كذلك.

٢، ٦، الوسوم

مثل معظم أنظمة إدارة النسخ، يستطيع جت وسم المراحل المهمة في تاريخ المشروع. يستعمل الناس هذه الآلية في الغالب لتمييز الإصدارات (v1.0 و v2.0 وهكذا). سنتعلم في هذا الفصل كيف نسرد الوسوم الموجودة وكيف ننشئ وسومًا ونحذفها وما النوعان المختلفان للوسوم.

٢، ٦، ١، سرد وسوم مشروعك

سرد الوسوم الموجودة في مستودع جت سهل ومباشر؛ فقط اكتب `git tag` (اختياريًا مع `-l` أو `--list`):

```
$ git tag
v1.0
v2.0
```

CONSOLE

يسرد لك هذا الأمر الوسوم بترتيب أبجدي؛ أي أن ترتيب عرضها ليس له أهمية حقيقية.

يمكنك أيضا البحث عن الوسوم التي تطابق نمطًا معينًا. يحتوي مستودع مصدر جت مثلاً على أكثر من خمسمئة وسم. فإذا كنت مهتمًا برؤية سلسلة 1.8.5 فقط، فننفذ هذا:

```
$ git tag -l "v1.8.5*"
v1.8.5
v1.8.5-rc0
v1.8.5-rc1
v1.8.5-rc2
v1.8.5-rc3
v1.8.5.1
v1.8.5.2
v1.8.5.3
v1.8.5.4
v1.8.5.5
```

CONSOLE

سرد الوسوم بأنماط يحتاج الخيار `-l` أو `--list`

إذا لم تُردِ إلا قائمة الوسوم بكاملها، فتنفذ `git tag` يفترض أنك تريد سرد الوسوم فيفعل ذلك؛ فإلحاق `-l` أو `--list` اختياريًا عندئذٍ.



لكن إذا أعطيته نمطًا لمطابقة وسوم عديدة، فحتاج خيار السرد: `-l` أو `--list`.

٢، ٦، ٢، إنشاء وسوم

يتيح جت نوعين من الوسوم: خفيفة، ومعنونة.

الوسم الخفيف كأنه فرع لا يتغيّر: مجرد إشارة إلى إيداع معين.

لكن على النقيض، الوسوم المعنونة هي كائنات كاملة في قاعدة بيانات جت؛ يحسب جت بصمتها، ويسجل معها اسم الواسم، وبريده، وتاريخ الوسم، ورسالته، ويمكن توقيعها وتوثيقها باستعمال GNU Privacy Guard (GPG). من الأفضل في العموم إنشاء وسوم معنونة حتى تنتفع بكل هذه المعلومات، لكن إذا أردت وسماً مؤقتاً أو لسبب ما لم تنشأ الاحتفاظ بكل هذه المعلومات، فالوسوم الخفيفة متاحة.

٢، ٦، ٣، الوسوم المعنونة

إنشاء الوسوم المعنونة في جت سهل. الطريقة الأيسر هي إضافة `-a` (اختصار `--annotate` «عنونة») إلى أمر الوسم:

```
$ git tag -a v1.4 -m "my version 1.4"
$ git tag
v0.1
v1.3
v1.4
```

CONSOLE

والخيار `-m` (اختصار `--message`) يعبّن رسالة الوسم التي تخزّن معه. وإذا لم تعيّن لها لوسم معنون، فسيفتح لك جت محررك حتى تكتبها فيه.

يمكنك أيضاً رؤية تاريخ الوسم مع الإيداع الموسوم بأمر الإظهار `git show`:

```
$ git show v1.4
tag v1.4
Tagger: Ben Straub <ben@straub.cc>
Date: Sat May 3 20:19:12 2014 -0700

my version 1.4

commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date: Mon Mar 17 21:52:11 2008 -0700

Change version number
```

CONSOLE

يُظهر لك هذا معلومات الواسم وتاريخ وسم الإيداع ورسالة الوسم، ثم معلومات الإيداع نفسه.

٢، ٦، ٤، الوسوم الخفيفة

طريقة أخرى لوسم الإيداعات هي بالوسوم الخفيفة. هذا لا يعني إلا تخزين بصمة الإيداع في ملف؛ فلا معلومات أخرى تُخزّن. لإنشاء وسم خفيف، لا تعطِ أمر الوسم أيّاً من الخيارات -a أو -s أو -m (اختصارات --annotate و --sign و --message= على الترتيب)؛ أعطه فقط اسم الوسم:

```
$ git tag v1.4-lw
$ git tag
v0.1
v1.3
v1.4
v1.4-lw
v1.5
```

CONSOLE

تنفيذ `git show` على مثل هذا الوسم لن يعطيك معلومات الوسم الإضافية، بل يُظهر فقط معلومات الإيداع:

```
$ git show v1.4-lw
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date: Mon Mar 17 21:52:11 2008 -0700
```

Change version number

CONSOLE

٢، ٦، ٥، الوسم فيما بعد

يمكنك أيضاً وسم إيداعات قديمة تخطيتها. لنفترض مثلاً أن تاريخ إيداعاتك يبدو هكذا:

```
$ git log --pretty=oneline
15027957951b64cf874c3557a0f3547bd83b3ff6 Merge branch 'experiment'
a6b4c97498bd301d84096da251c98a07c7723e65 Create write support
0d52aaab4479697da7686c15f77a3d64d9165190 One more thing
6d52a271eda8725415634dd79daabbc4d9b6008e Merge branch 'experiment'
0b7434d86859cc7b8c3d5e1dddfed66ff742fcbc Add commit function
4682c3261057305bdd616e23b64b0857d832627b Add todo file
166ae0c4d3f420721acbb115cc33848dfcc2121a Create write support
9fceb02d0ae598e95dc970b74767f19372d61af8 Update rakefile
964f16d36dfccde844893cac5b347e7b3d44abbc Commit the todo
8a5cbc430f1a9c3d00faaeffd07798508422908a Update readme
```

CONSOLE

لنفترض الآن أنك نسيت وسم المشروع عند `v1.2`، التي كانت عند إيداع "Update rakefile". يمكنك فعل هذا بعدما حدث ما حدث. لوسم ذلك الإيداع، اكتب في نهاية أمر الوسم بصمة الإيداع (أو جزءاً من أولها):

```
$ git tag -a v1.2 9fceb02
```

CONSOLE

والآن ستجد أنك قد وسمت الإيداع:


```
$ git tag
v0.1
v1.2
v1.3
v1.4
v1.4-lw
v1.5

$ git show v1.2
tag v1.2
Tagger: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Feb 9 15:32:16 2009 -0800

version 1.2
commit 9fceb02d0ae598e95dc970b74767f19372d61af8
Author: Magnus Chacon <mchacon@gee-mail.com>
Date:   Sun Apr 27 20:43:35 2008 -0700

    Update rakefile
...
```

٢، ١، ١، مشاركة الوسوم

أمر الدفع لا ينقل ألياً الوسوم إلى المستودعات البعيدة. فعليك دفعها بأمر صريح بعد إنشائها. هذه العملية كبيرة الشبه بمشاركة الفروع البعيدة: نَفِّذ «اسم-الوسم» `git push origin`.

```
$ git push origin v1.5
Counting objects: 14, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 2.05 KiB | 0 bytes/s, done.
Total 14 (delta 3), reused 0 (delta 0)
To git@github.com:schacon/simplegit.git
 * [new tag]          v1.5 -> v1.5
```

وإذا كانت لديك العديد من الوسوم التي تريد دفعها جملةً واحدة، فيمكنك إضافة خيار الوسوم `--tags` إلى أمر الدفع `git push`، لينقل إلى المستودع البعيد جميع وسومك التي ليست هناك بالفعل.

```
$ git push origin --tags
Counting objects: 1, done.
Writing objects: 100% (1/1), 160 bytes | 0 bytes/s, done.
Total 1 (delta 0), reused 0 (delta 0)
To git@github.com:schacon/simplegit.git
 * [new tag]          v1.4 -> v1.4
 * [new tag]          v1.4-lw -> v1.4-lw
```

والآن، عندما يستنسخ أحدهم مستودعك أو يجذب منه، فسيحصل على جميع وسومك أيضاً.

يدفع أمر الدفع كلا النوعين من الوسوم



سيُدفع `--tags` «البعيد» `git push` الوسوم الخفيفة والوسوم المعنونة. لا يوجد حالياً خيار لدفع الوسوم الخفيفة فقط، لكن الأمر `git push --follow-tags` «البعيد» سيُدفع الوسوم المعنونة فقط إلى الخادوم البعيد.

٢، ٦، ٧، حذف الوسوم

لحذف وسم من مستودعك المحلي، نفذ «اسم-الوسم» `git tag -d`. مثلاً، يمكننا حذف الوسم الخفيف الذي أنشأناه سابقاً هكذا:

```
$ git tag -d v1.4-lw
Deleted tag 'v1.4-lw' (was e7d5add)
```

CONSOLE

لاحظ أن هذا لا يحذف الوسم من أي مستودع بعيد. توجد طريقتان شائعتان لحذف وسمٍ ما من مستودع بعيد:

الطريقة الأولى هي «اسم-الوسم»/refs/tags: «البعيد» `git push`:

```
$ git push origin :refs/tags/v1.4-lw
To /git@github.com:schacon/simplegit.git
- [deleted]          v1.4-lw
```

CONSOLE

لاستيعاب ما تفعله هذه الطريقة يمكن ترى أنها تدفع القيمة الفارغة التي قبل النقطتين الرأسيتين إلى اسم الوسم على المستودع البعيد، فعلياً تحذفه.

الطريقة الأخرى (والبدئية أكثر) لحذف وسم من مستودع بعيد، هي:

```
$ git push origin --delete «اسم-الوسم»
```

CONSOLE

٢، ٦، ٨، سحب الوسوم

إذا أردت رؤية نُسخ الملفات التي يشير إليها وسمٌ ما، يمكنك سحب هذا الوسم بأمر `git checkout`، مع أن هذا يضع مستودعك في حالة “detached HEAD”، التي لها بعض الآثار الجانبية السيئة:

```
$ git checkout v2.0.0
Note: switching to 'v2.0.0'.
```

CONSOLE

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by performing another checkout.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using `-c` with the switch command. Example:

```
git switch -c <new-branch-name>
```

Or undo this operation with:

```
git switch -
```

Turn off this advice by setting config variable `advice.detachedHead` to `false`

HEAD is now at 99ada87... Merge pull request #89 from schacon/appendix-final

```
$ git checkout v2.0-beta-0.1
```

Previous HEAD position was 99ada87... Merge pull request #89 from schacon/appendix-final

HEAD is now at df3f601... Add atlas.json and cover image

في حالة "detached HEAD"، إذا عدّلت شيئاً وأودعت، فإن الـ `HEAD` سيبقى كما هو، وإبداعك الجديد لن ينتمي إلى أي فرع ولن يمكن الوصول إليه أبداً، إلا ببصمته. لذا، فإن احتجت إجراء تعديلات — مثلاً لإصلاح علة في نسخة قديمة — فغالباً ستحتاج إلى إنشاء فرع:

```
$ git checkout -b version2 v2.0.0
Switched to a new branch 'version2'
```

CONSOLE

إذا فعلت هذا ثم أودعت، فإن فرع `version2` سيكون مختلفاً عن وسم `v2.0.0` لأنه سيكون متقدماً عنه بتعديلاتك، لذا كن حذراً.

٢، ٧، الكُنَيَات في جت

قبل أن نتقدم إلى الباب التالي، نود أن نعرّفك ميزة في جت ستجعل استعمالك أسهل وأريح وأكثر ألفة: الكُنَيَات. للوضوح، لن نستعملها في الكتاب إلا في هذا الفصل. لكن إذا نويت استعمال جت باستمرار، فيجب أن تعرفها.

لا يُخَمَّن جت الأمر الذي تريده إذا كتبت جزءاً منه. فإذا لم تشأ كتابة كل أمر بكامله، فيمكنك تعيين كُنْيَة لكل أمر تريده بسهولة بأمر التهيئة `git config`. هذه أمثلة ربما تحب إعدادها:

```
$ git config --global alias.co checkout
$ git config --global alias.br branch
$ git config --global alias.ci commit
$ git config --global alias.st status
```

CONSOLE

هذا يعني أن يمكنك كتابة `git ci` مثلاً بدلاً من أن تكتب `git commit`. وبالإستمرار مع جت، ستجد أوامر أخرى تستعملها كثيراً؛ لا تتردد في إنشاء كُنَيَات لها.

هذه الطريقة تصلح كذلك لإنشاء الأوامر التي تظن أنها يجب أن توجد. مثلاً، لتصحيح صعوبة الاستخدام التي واجهتها عند إلغاء تأهيل ملف، يمكنك إضافة كُنْيَة خاصة بك لإلغاء التأهيل `unstage` إلى جت:

```
$ git config --global alias.unstage 'reset HEAD --'
```

يجعل هذا الأمرين التاليين متكافئين:

```
$ git unstage fileA
$ git reset HEAD -- fileA
```

هذا أسهل وأوضح. وكذلك من الشائع إضافة أمر `last` «الأخير»، مثل هذا:

```
$ git config --global alias.last 'log -1 HEAD'
```

فيمكنك عندئذٍ رؤية إبداعك الأخير بسهولة:

```
$ git last
commit 66938dae3329c7aeb598c2246a8e6af90d04646
Author: Josh Goebel <dreamer3@example.com>
Date: Tue Aug 26 19:48:51 2008 +0800

    Test for current head

Signed-off-by: Scott Chacon <schacon@example.com>
```

وكما يمكنك أن تخمن، إنما يترجم جت الأمر الجديد إلى ما جعلته كُنْيَةً له. ولكنك أحياناً قد تريد تنفيذ أمر خارجي، بدلاً من أمر فرعي في جت. في هذه الحالة تبدأ الأمر بعلامة تعجب: `!`. يفيد هذا عندما تكتب أدواتك الخاصة التي تعمل مع مستودع جت. نوضح ذلك بعمل الكُنْيَةِ `git visual` لتشغيل `gitk`:

```
$ git config --global alias.visual '!gitk'
```

٢، ٨، الخلاصة

الآن تستطيع فعل جميع عمليات جت المحلية الأساسية: إنشاء مستودع أو استنساخه، وعمل تعديلات، وتأهيلها وإيداعها، وعرض تاريخ جميع التعديلات التي مر بها المستودع. التالي: سنشرح ميزة جت القاتلة للمنافسة: نموذج التفريع.

الباب الثالث:

التفرع في جت

معظم أنظمة إدارة النسخ بها نوع ما من دعم التفرع. التفرع يعني أنك تنشق عن مسار التطوير الرئيس، وتستمر بالعمل من غير أن تؤثر في ذلك المسار الرئيس. هذه عملية مكلفة نوعاً ما في أدوات إدارة نسخ كثيرة، وغالباً تحتاج منك إلى إنشاء نسخة جديدة من مجلد مشروعك، الذي قد يحتاج وقتاً طويلاً في المستودعات الكبيرة.

يسمى البعض نموذج التفرع في جت بأنه «ميزته القاتلة للمنافسة»، وهي بكل تأكيد تميزه في مجتمع إدارة النسخ. لماذا هي مميزة هكذا؟ لأن طريقة التفرع في جت خفيفة خفة مستحيلة، فتجعل إنشاء فرع جديد عملية شبه آنية، والانتقال بين الفروع ذهاباً وإياباً له تلك السرعة نفسها تقريباً. وخلافاً للكثير من الأنظمة الأخرى، يشجع جت على أساليب التطوير التي تعتمد على التفرع والدمج كثيراً، عدة مرات في اليوم حتى. وفهم هذه الميزة وإتقانها يعطيانك أداة قوية وفريدة، وقد يغيّران تماماً الطريقة التي تطوّر بها.

٣، ١، الفروع بإيجاز

لنفهم حقاً طريقة التفرع في جت، علينا أن نتراجع خطوة إلى الخلف ونتدبر طريقته في تخزين البيانات.

كما قد تتذكر من فصل [ما هو جت؟](#)، لا يخزن جت بياناته في صورة فروقات، بل في صورة لقطات.

وعندما تُودع، يخزن جت كائن إيداع فيه إشارة إلى لقطة المحتوى الذي أُلِّهته. وفيه كذلك اسم المؤلف وعنوان بريده ورسالة الإيداع والإشارات إلى الإيداعات السابقة له مباشرة (الإيداعات الآباء): لا أب لأول إيداع، وأب واحد للإيداعات العادية، وأبوين أو أكثر لإيداعات الدمج، وهي الإيداعات الناتجة من دمج فرعين أو أكثر.

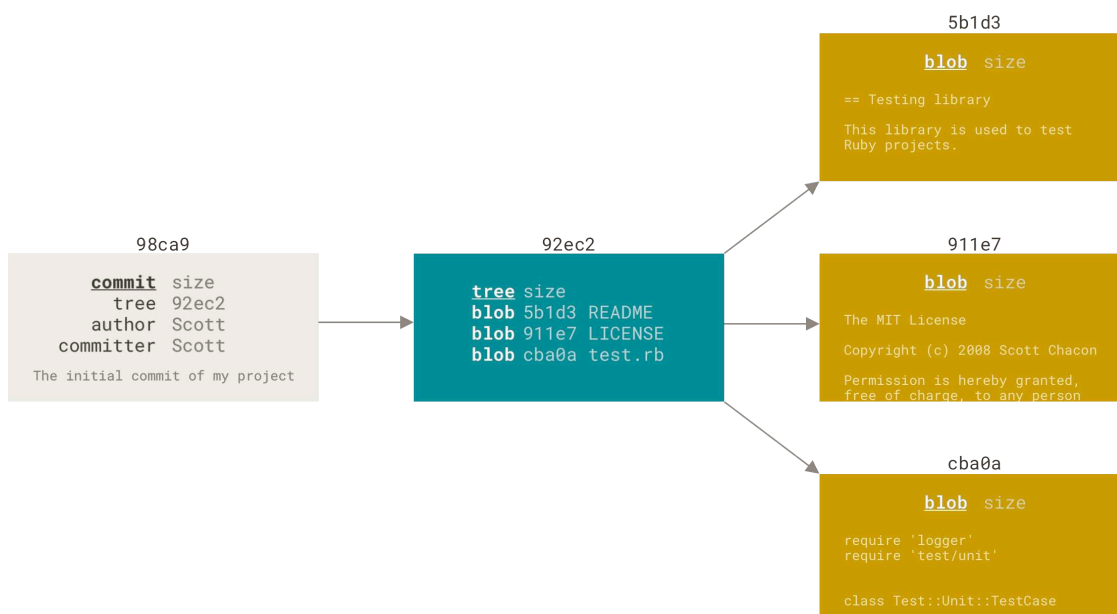
حتى نستطيع تصور هذا، لنفترض أن لديك مجلدًا به ثلاثة ملفات، وأنت أُلِّهتها جميعها ثم أودعتها. يحسب تأهيل الملفات بصمة كل ملف (بصمة SHA-1 التي ذكرناها في فصل [ما هو جت؟](#))، ويخزن نسخة الملف هذه في المستودع (وهي ما يسميها جت «كتلة رقمية» ("blob")، ونسميها «كتلة» اختصاراً)، ويضيف تلك البصمة إلى منطقة التأهيل:

```
$ git add README test.rb LICENSE
$ git commit -m 'Initial commit'
```

CONSOLE

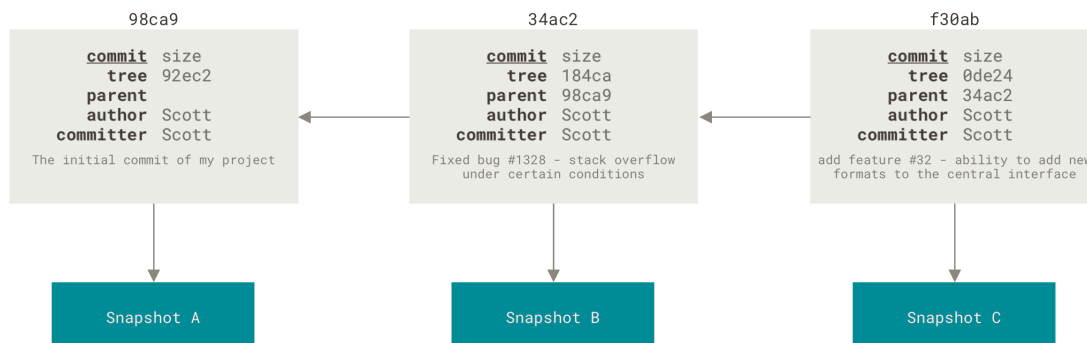
عندما تودع بأمر `git commit`، فإن جت يحسب أيضاً بصمات كل مجلد ومجلد فرعي (في هذه الحالة، مجلد جذر المشروع فقط)، ويخزنها في صورة كائنات أشجار في المستودع. ثم ينشئ جت كائن إيداع فيه بيانات وصفية وإشارة إلى شجرة جذر المشروع، حتى يستطيع إعادة إنشاء تلك اللقطة عند الحاجة.

صار في مستودعك الآن خمسة كائنات: ثلاث كتل (كلٌ منها يمثل محتويات ملف من الثلاثة)، ولشجرة واحدة (تسرد محتويات المجلد وما الكتل التي تشير إليها أسماء الملفات)، وإيداع واحد (فيه إشارة إلى شجرة الجذر تلك وكذلك البيانات الوصفية للإيداع).



شكل ٩. إيداع وشجرته

إذا أجريت تعديلات وأودعتها، فإن إيداعك التالي سيخزن إشارة إلى الإيداع السابق له مباشرةً.

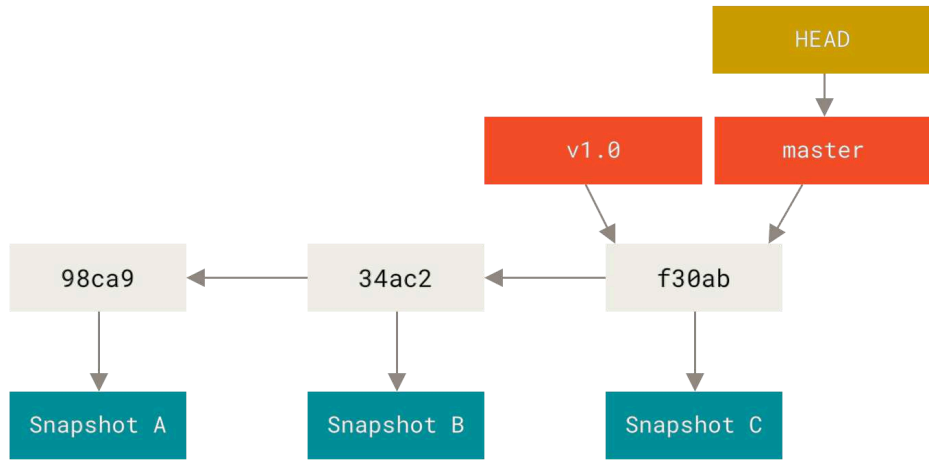


شكل ١٠. إيداعات وآبائها

فإنما الفرع في جت هو إشارة متحركة تشير إلى أحد هذه الإيداعات. والفرع المبدئي في جت يُسمى `master`. فعندما تنشر في صنع الإيداعات، فإن جت يعطيك فرعاً رئيساً يسمى `master` ويشير إلى آخر إيداع صنغته. ويتقدم فرع `master` مع كل إيداع تودعه.

فرع `master` في جت ليس مميزاً. فهو تماماً مثل أي فرع آخر. والسبب الوحيد لوجوده في أغلب المستودعات أن أمر `git init` ينشئه بهذا الاسم المبدئي وأكثر الناس لا يبالون بتغييره.





شكل ١١. فرع وتاريخ إيداعاته

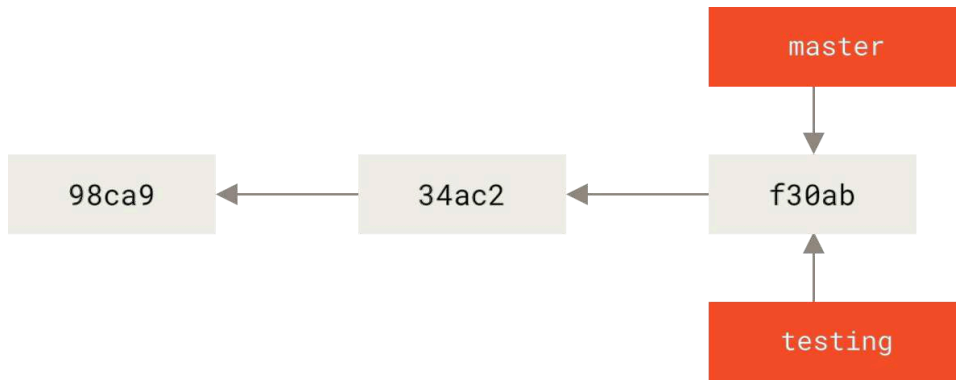
٣، ١، ١، إنشاء فرع جديد

ماذا يحدث عندما تنشئ فرعاً جديداً؟ الإجابة: ينشئ جت إشارة جديدة لك لتحركها كما تشاء. لنقل إنك أردت إنشاء فرع جديد اسمه `testing`. تفعل هذا بأمر الفروع `git branch`:

```
$ git branch testing
```

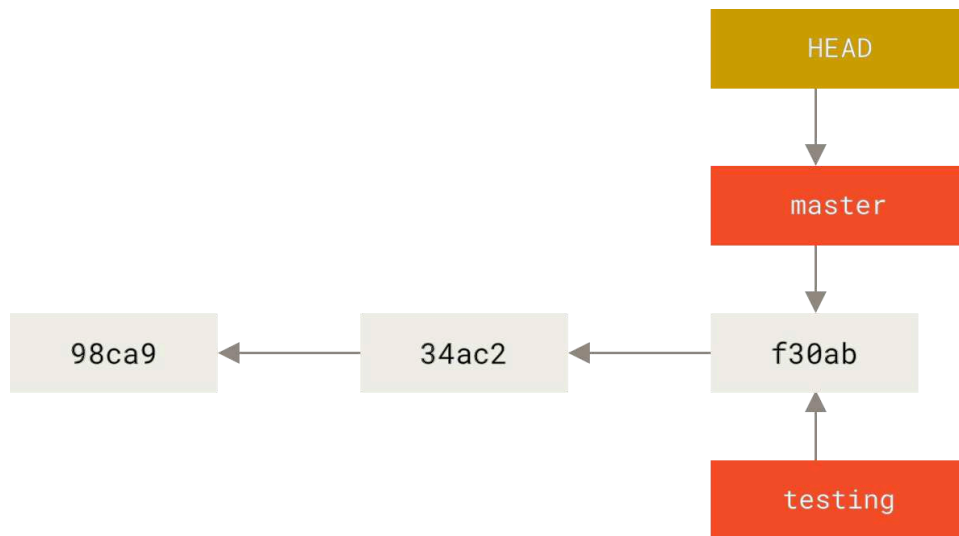
CONSOLE

ينشئ هذا إشارة إلى الإيداع الذي تقف عنده الآن.



شكل ١٢. فرعان يشيران إلى سلسلة الإيداعات نفسها

كيف يعرف جت في أي فرع أنت الآن؟ إنه يحتفظ بإشارة مخصوصة تسمى «إشارة الرأس» (HEAD). لاحظ أن هذه مختلفة كثيراً عن مفهوم HEAD في الأنظمة الأخرى مثل Subversion وCVS. في جت، هذه إشارة إلى الفرع المحلي الذي تقف فيه الآن. في حالتنا هذه، ما زلت واقفاً في فرع `master`. فما على أمر `git branch` إلا إنشاء فرع جديد؛ ليس عليه الانتقال إليه.



شكل ١٣. إشارة الرأس HEAD تشير إلى فرع

يمكنك رؤية هذا بسهولة بأمر السجل، الذي يُظهر لك ما تشير إليه إشارات الفروع، وذلك بالخيار `--decorate` («تسميات»، أي إظهار أسماء الإشارات؛ وهو مفترض مبدئياً منذ النسخة 2.13 من جت).

```

$ git log --oneline --decorate
f30ab (HEAD -> master, testing) Add feature #32 - ability to add new formats to the central interface
34ac2 Fix bug #1328 - stack overflow under certain conditions
98ca9 Initial commit
  
```

يمكنك رؤية فرعي `master` و `testing` عند إيداع `f30ab`.

٣، ٢، ١، الانتقال بين الفروع

لانتقال إلى فرع موجود، استخدم أمر السحب `git checkout`. هيا بنا نتنقل إلى فرعنا الجديد `testing`:

```

$ git checkout testing
  
```

يحرك هذا الأمر إشارة الرأس لتشير إلى فرع `testing`.

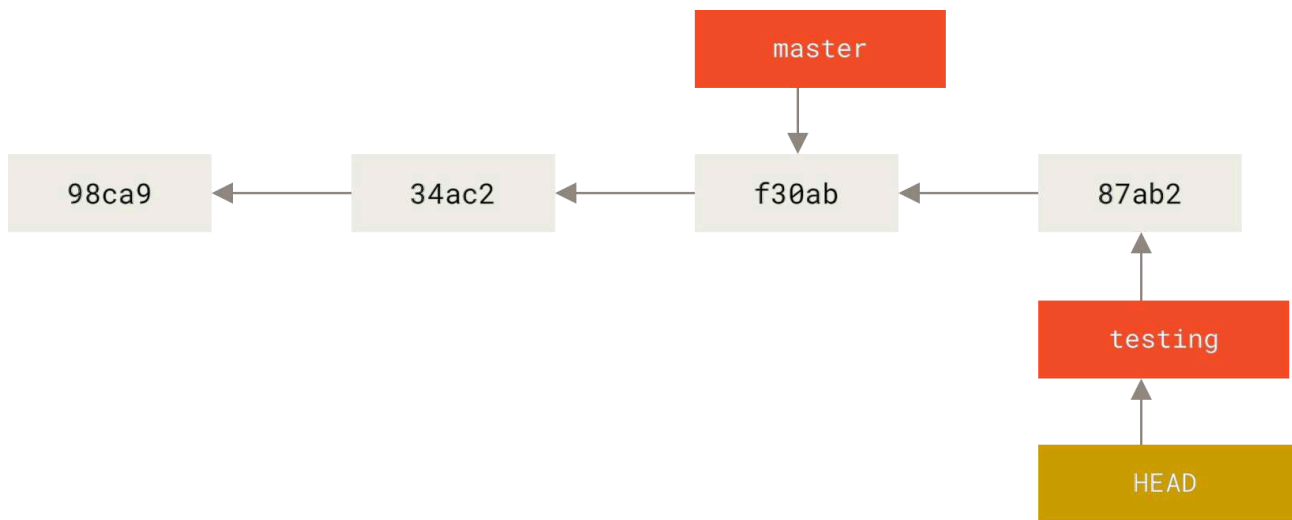


شكل ١٤. إشارة الرأس تشير إلى الفرع الحالي

ما دلالة هذا؟ لنصنع إيداعًا آخر إيداعًا.

```
$ vim test.rb  
$ git commit -a -m 'Make a change'
```

CONSOLE



شكل ١٥. فرع الرأس يتقدم عند صنع إيداع

هذا يدعو للتفكير، لأن الآن فرع testing قد تقدم، بينما قعد فرع master في مكانه مشيرًا إلى الإيداع القديم نفسه عندما انتقلنا إلى الفرع الجديد بأمر السحب. لنعد إلى فرع master :

```
$ git checkout master
```

CONSOLE

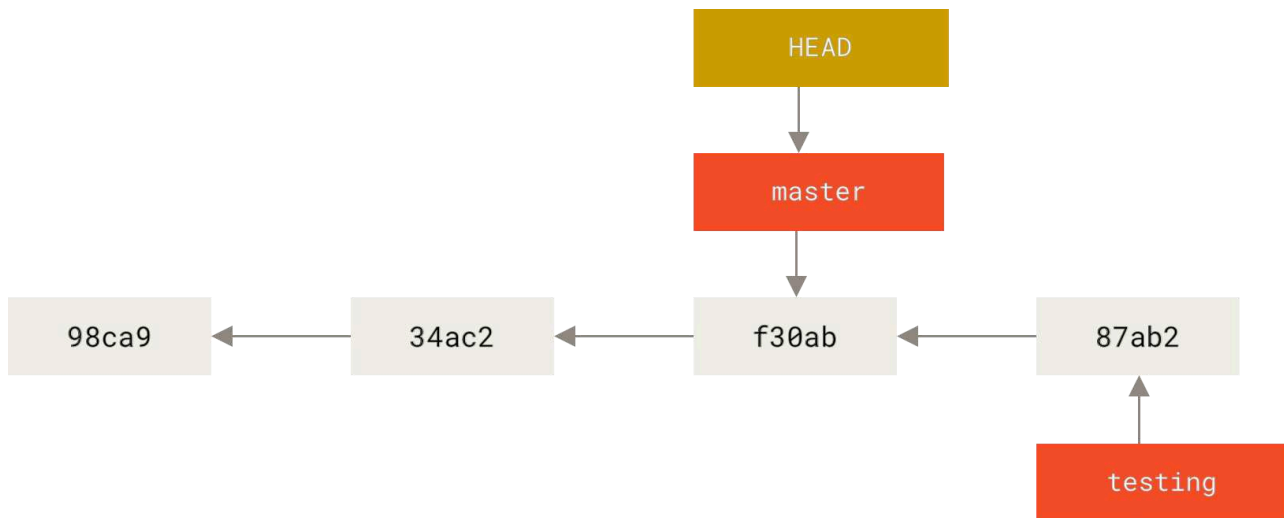
لا يُظهر أمر السجل جميع الفروع طوال الوقت

إذا نفذت `git log` الآن، فستسأل أين ذهب فرع testing الذي أنشأته، لأنه لن يظهر في ناتجه.

لم يتبخر الفرع، ولكن جت لا يعلم أنك مهتم به الآن، ولا يُظهر لك جت إلا ما يظن أنك مهتم به. بلفظ آخر، لا يُظهر لك أمر السجل بطبيعته إلا تاريخ الفرع الذي تقف فيه حاليًا.

لإظهار تاريخ فرع آخر، عليك طلب ذلك صراحةً، مثل `git log testing`. ولإظهار جميع الفروع، اطلب ذلك من `git log` بالخيار `--all`.





شكل ١٦. تتحرك إشارة الرأس عندما تنتقل إلى فرع آخر

فَعَلْ هذا الأمرَ فَعَلَيْن: أَعَادَ إشارةَ الرأسِ لتشيرَ إلى فرع `master`، وأَرْجَعَ الملفاتَ في مجلدِ العملِ إلى حالِها كما كانتَ في اللقطة التي يشيرُ إليها `master`. هذا يعني أيضًا أن التعديلات التي ستصنعها الآن ستُنْبَنَى على نسخة قديمة من المشروع. أي أنه عمليًا يتراجع عما فعلت في فرع `testing` لكي تستطيع السير في اتجاه آخر.

الانتقال بين الفروع يغيّر الملفات التي في مجلد عملك

مهمّ ملاحظة أنك عندما تنتقل إلى فرع آخر في جت، فإن الملفات التي في مجلد عملك ستتغير. فإذا انتقلت إلى فرع قديم، سيعود مجلد عملك إلى ما كان عليه عند آخر إيداع في هذا الفرع. وإن لم يستطع جت تغيير الملفات تغييرًا نظيفًا، فلن يسمح لك بالتبديل أصلاً.

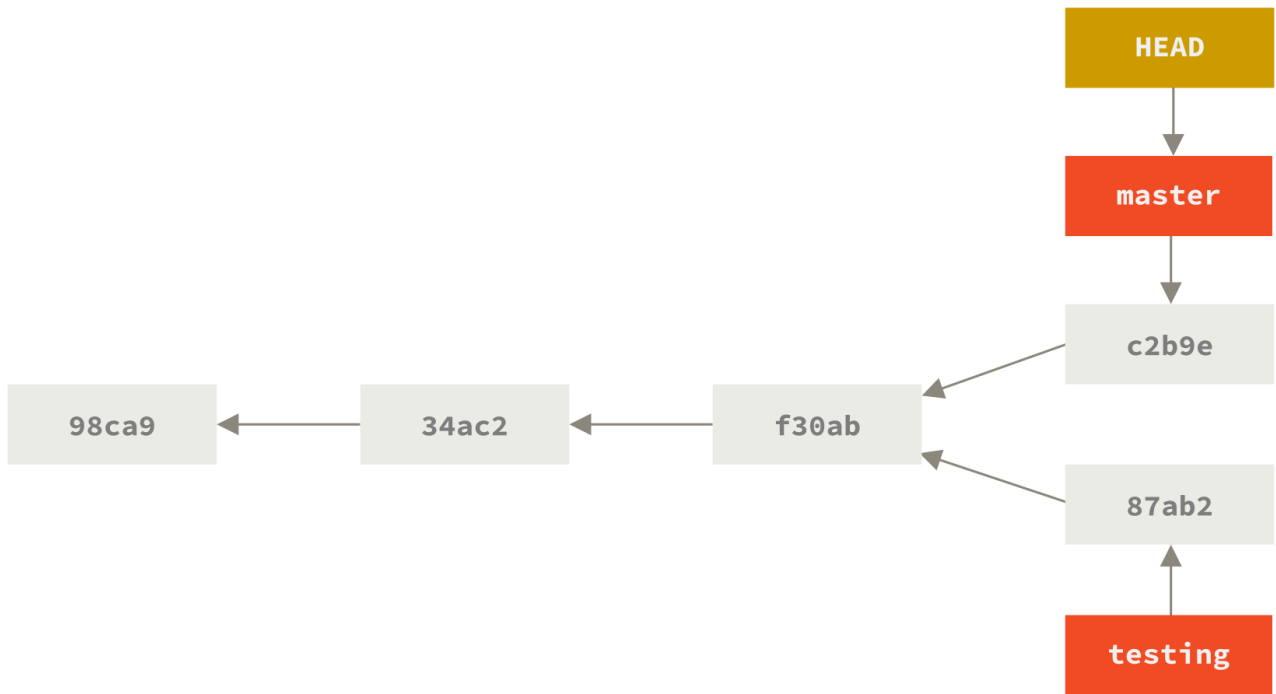


لنُجْري بعض التعديلات ونودع مجددًا:

```
$ vim test.rb
$ git commit -a -m 'Make other changes'
```

CONSOLE

الآن افترق تاريخ مشروعك (انظر شكل [تاريخ مفترق](#)). فلقد أنشأت فرعًا وانتقلت إليه وعملت فيه قليلًا، ثم عدت إلى الفرع الرئيس وعملت فيه عملاً آخر. كلا هذين التغيرين منعزلان في فرعين منفصلين: يمكنك التنقل بينهما كما تشاء، ودمجهما معًا عندما تكون جاهزًا. وكل هذا فعلته بسهولة بأوامر الفروع `branch` والسحب `checkout` والإيداع `commit`.



شكل ١٧. تاريخ مفترق

يمكنك أيضا رؤية هذا بسهولة بأمر السجل، فإذا نفذت `git log --oneline --decorate --graph --all` فسيظهر لك تاريخ إيداعاتك ومواضع إشارات فروعك وكيف افترق تاريخك.

```

$ git log --oneline --decorate --graph --all
* c2b9e (HEAD, master) Make other changes
| * 87ab2 (testing) Make a change
|/
* f30ab Add feature #32 - ability to add new formats to the central interface
* 34ac2 Fix bug #1328 - stack overflow under certain conditions
* 98ca9 Initial commit of my project
  
```

CONSOLE

ولأن الفرع في جيت ليس إلا ملفاً مجرداً فيه ٤٠ محرراً تمثل بصمة الإيداع الذي يشير إليه الفرع، فإن إنشاء الفروع وإزالتها عمليتان رخيصتان سريعتان. فعملية إنشاء فرع جديد تماثل في سرعتها عملية كتابة ٤١ بايتاً إلى ملف (٤٠ محرراً للبصمة ثم حرف نهاية السطر).

هذا اختلاف عظيم عن طريقة التفريع في معظم الأنظمة القديمة لإدارة النسخ، التي ننسخ فيها جميع ملفات المشروع إلى مجلد آخر. قد يحتاج هذا عدة ثوانٍ أو حتى دقائق، حسب حجم المشروع. ولكن تلك العملية في جيت دائماً عملية آنية. وأيضاً لأننا نسجل آباء الإيداعات عندما نودع، فإن إيجاد قاعدة مناسبة للدمج هي عملية يفعلها جيت من أجلنا آلياً، وهي سهلة جداً عموماً. تشجع هذه الميزات المطورين على إنشاء فروع واستعمالها بكثرة.

لنر لماذا عليك فعل هذا.

إنشاء فرع جديد والانتقال إليه في خطوة واحدة



من المعتاد أن ترغب في الانتقال إلى فرع جديد فور إنشائه؛ يمكنك إنشاء فرع والانتقال إليه بأمر واحد: «اسم-الفرع-الجديد» `git checkout -b`.

بدءًا من النسخة 2.23 من جت يمكنك استعمال أمر التبديل بدلًا من أمر السحب من أجل:

- الانتقال إلى فرع موجود: `git switch testing-branch`.
- إنشاء فرع جديد والانتقال إليه: `git switch -c new-branch`. الخيار `-c` للإنشاء، ويمكنك استخدام الخيار الكامل: `--create`.
- العودة إلى الفرع المسحوب سابقًا: `git switch -`.



٣، ٢، أسس التفرع والدمج

لننظر مثالًا سهلًا عن التفرع والدمج بأسلوب تطوير قد تستعمله في الحقيقة. لننقل إنك تفعل هذا:

١. تقوم ببعض الأعمال على موقع وب.

٢. تنشئ فرعًا لـ «قصة المستخدم» الجديدة التي تعمل عليها.

٣. تقوم ببعض الأعمال في هذا الفرع.

وعندئذٍ تأتيك مكالمة بأن علة أخرى حرجة وتحتاج منك إصلاحًا عاجلاً، فتفعل الآتي:

١. تنتقل إلى فرعك الإنتاجي.

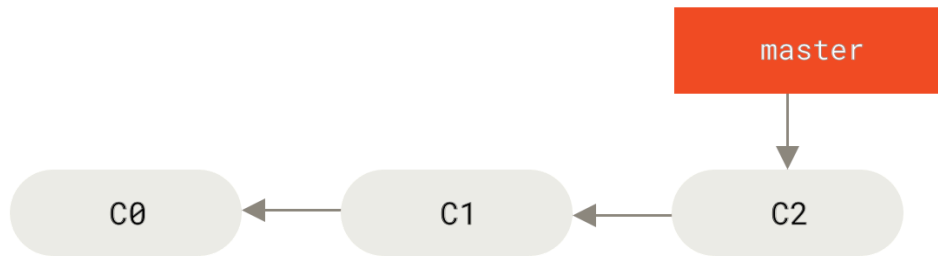
٢. تنشئ فرعًا لإضافة الإصلاح العاجل.

٣. وبعد اختباره، تدمج فرع الإصلاح العاجل، وتدفعه إلى فرع الإنتاج.

٤. تعود إلى قصة المستخدم الأصلية وتكمل عملك عليها.

٣، ٢، ١، أسس التفرع

أولاً، لننقل إنك تعمل على مشروعك، ولديك بضعة إبداعات بالفعل في فرع `master`.



شكل ١٨. تاريخ إيداعات بسيط

ثم رأيت أن تعمل على المسألة رقم ٥٣ في نظام متابعة المسائل الذي تستخدمه شركتك. فتنفذ أمر `git checkout` مع الخيار `-b` ، لإنشاء فرع جديد والانتقال إليه في الوقت نفسه:

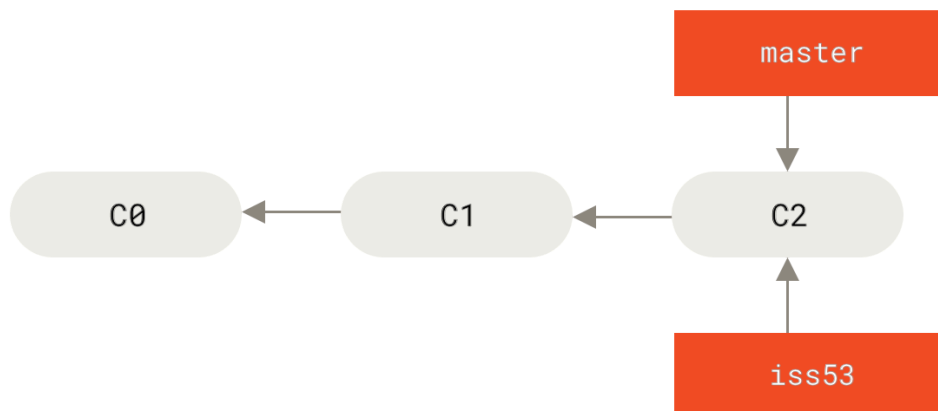
```
$ git checkout -b iss53
Switched to a new branch "iss53"
```

CONSOLE

وهذا اختصار للأمرين:

```
$ git branch iss53
$ git checkout iss53
```

CONSOLE

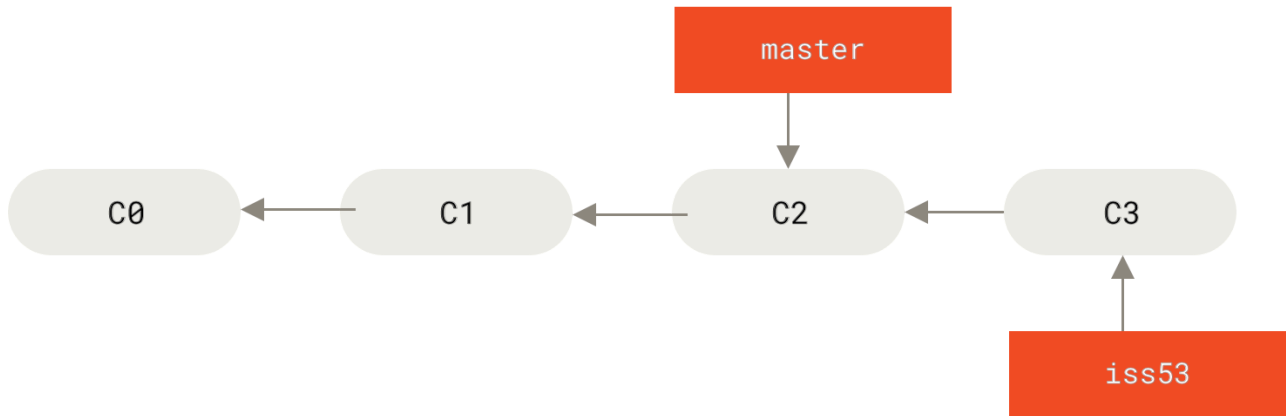


شكل ١٩. إنشاء إشارة إلى فرع جديد

يمكنك العمل على موقعك وصنع بعض الإيداعات. فعل هذا يحرك فرع `iss53` إلى الأمام، لأنه الفرع المسحوب (أي أنه الفرع الذي تشير إليه إشارة الرأس HEAD لديك):

```
$ vim index.html
$ git commit -a -m 'Create new footer [issue 53]'
```

CONSOLE



شكل ٢٠. تقدّم فرع iss53 بعملك عليه

ثم تأتيك مكالمة الآن بأن الموقع به علة، وعليك حلها فوراً. لا تحتاج مع جت أن تنشر إصلاحك لهذه العلة مع تعديلات iss53 التي صنعتها، ولا تحتاج أيضاً إلى بذل المجهود للتراجع عن هذه التعديلات حتى تستطيع العمل على إصلاح علة الموقع. ليس عليك إلا أن تنتقل عائداً إلى فرعك الرئيس master.

ولكن، قبل هذا، عليك ملاحظة أن إن كان مجلد عملك أو منطقة تأهيلك فيهما تعديلات غير مودعة وتختلف عما في الفرع الذي تريد الانتقال إليه، فلن يسمح لك جت بالانتقال. من الأفضل دوماً أن تجعل حالة مجلد العمل نظيفة قبل الانتقال بين الفروع. يمكن التحايل على هذا بطريقة أو بأخرى (تحديداً، التخبئة وتصحيح الإيداعات)؛ سنتطرق إلى ذلك فيما بعد في فصل التخبئة والتنظيف. ولكن لنفترض الآن أنك أودعت كل تعديلاتك، حتى يتسنى لك الانتقال عائداً إلى فرعك الرئيس:

```
$ git checkout master
Switched to branch 'master'
```

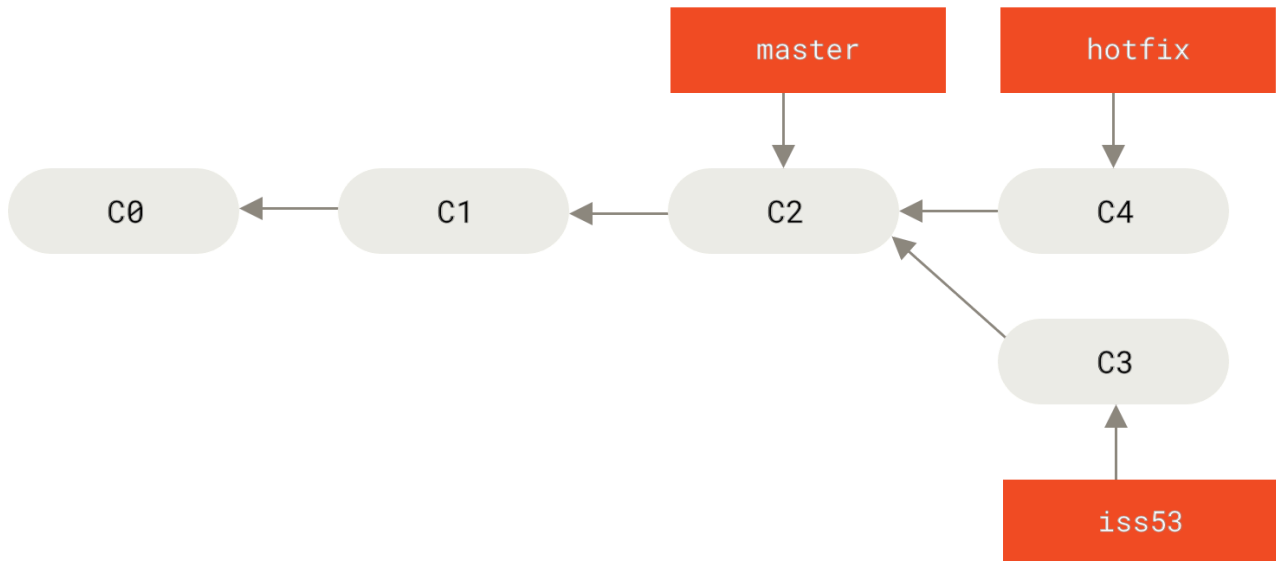
CONSOLE

ستجد الآن أن مجلد عملك مطابق تماماً لما كان عليه قبل أن تبدأ العمل على المسألة رقم ٥٣، ويمكنك الآن التركيز على إصلاح العلة الجديدة. هذه النقطة مهمة ويجب تذكرها: عندما تنتقل من فرع إلى آخر، يعيد جت مجلد عملك إلى ما كان عليه آخر مرة أودعت فيها في هذا الفرع، فيضيف ويحذف ويعدل الملفات آلياً، حتى يجعل نسخة العمل مشابهة تماماً لما كان عليه الفرع عند آخر إيداع تم فيه.

عليك الآن العمل على الإصلاح العاجل. لننشئ فرع hotfix لتعمل عليه حتى تنتهي:

```
$ git checkout -b hotfix
Switched to a new branch 'hotfix'
$ vim index.html
$ git commit -a -m 'Fix broken email address'
[hotfix 1fb7853] Fix broken email address
1 file changed, 2 insertions(+)
```

CONSOLE



شكل ٢١. فرع الإصلاح العاجل (hotfix) مبني على الفرع الرئيس (master)

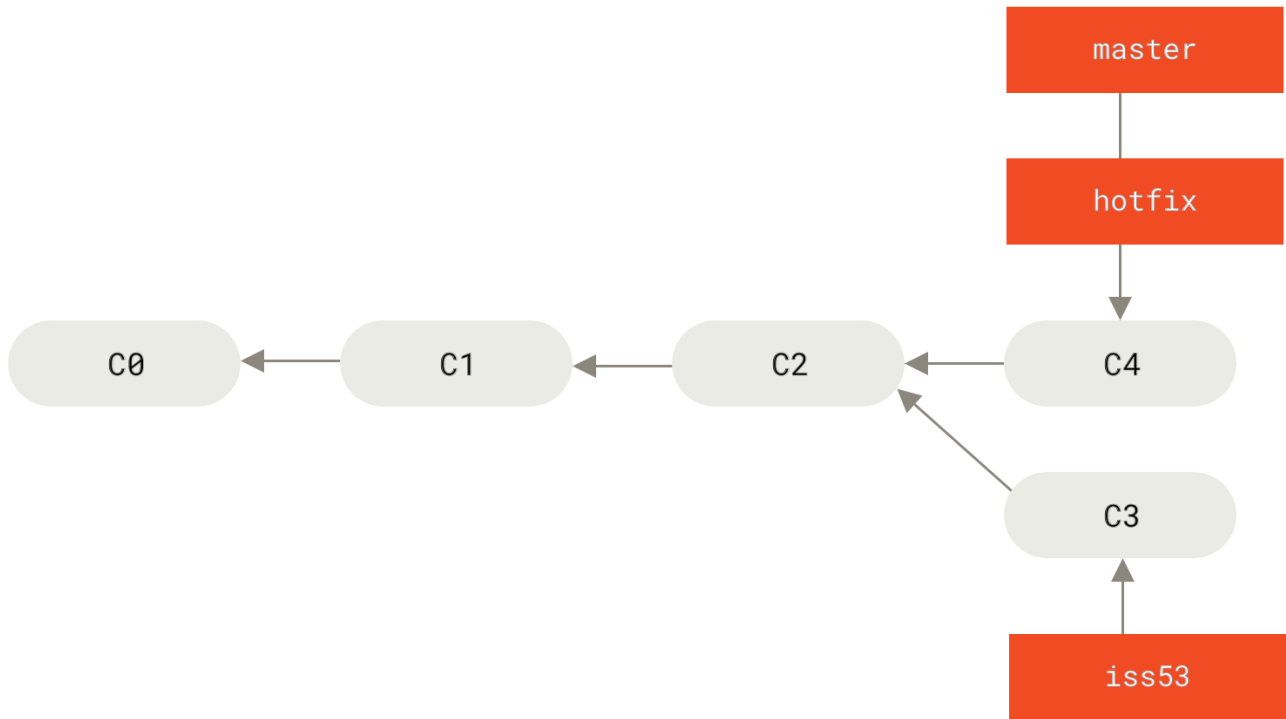
يمكنك الآن إجراء الاختبارات والتأكد من أن الإصلاح الذي صنعتَه هو المراد. ثم دمج فرع الإصلاح العاجل في الفرع الرئيس حتى تدفعه إلى الإنتاج. يمكنك فعل هذا بأمر الدمج `git merge`:

```
$ git checkout master
$ git merge hotfix
Updating f42c576..3a0874c
Fast-forward
 index.html | 2 ++
 1 file changed, 2 insertions(+)
```

CONSOLE

ستلاحظ عبارة “fast-forward” («تسريع») في ناتج الدمج. هذا لأن الإيداع C4 الذي يشير إليه فرع الإصلاح العاجل كان مباشرةً أمام الإيداع C2 الذي تقف فيه، فلم يفعل جت سوى تحريك الإشارة إلى الأمام. بلفظ آخر: عندما تريد دمج إيداع في إيداع آخر يمكن الوصول إليه بتتبع تاريخه، فإن جت لا يعقد الأمور بل يحرك الإشارة إلى الأمام، فلا أعمال مفترقة ليحاول دمجها — يسمى هذا «تسريعًا» (“fast-forward”).

تعديلاتك الآن موجودة في لقطة الإيداع التي يشير إليها الفرع الرئيس، فيمكنك الآن نشرها.



شكل ٢٢. تسريع master إلى hotfix

بعد نشر إصلاحك الشديد الأهمية، تكون جاهزاً للعودة إلى عملك الذي كنت فيه قبل هذه المقاطعة. ولكن عليك أولاً حذف فرع hotfix لأنك لم تعد تحتاج إليه؛ فالفرع الرئيس يشير إلى الشيء نفسه. يمكنك حذفه بالخيار `-d` مع أمر التفرع `git branch`:

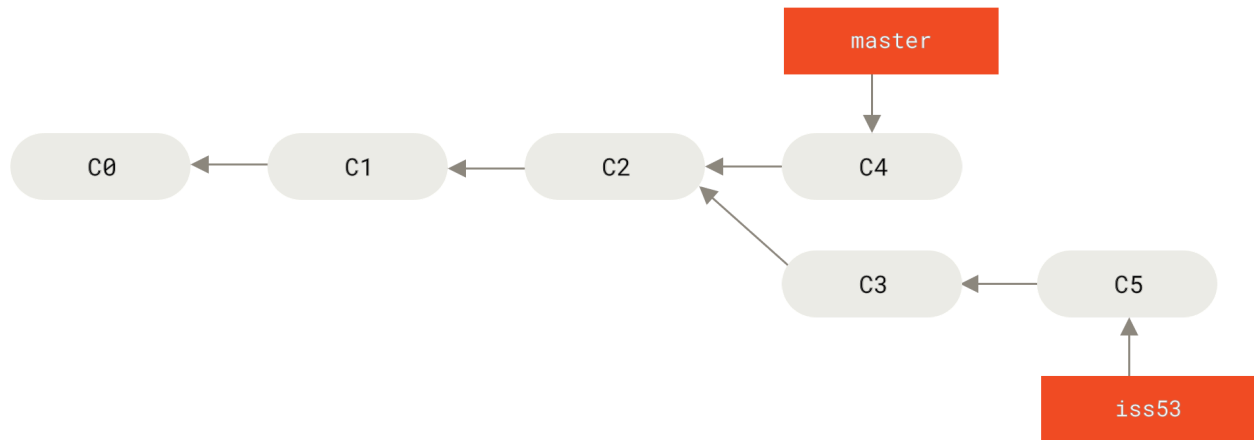
```
$ git branch -d hotfix
Deleted branch hotfix (3a0874c).
```

CONSOLE

يمكنك الآن العودة إلى فرع العمل الحالي الخاص بالمسألة رقم ٥٣، وإكمال العمل عليها.

```
$ git checkout iss53
Switched to branch "iss53"
$ vim index.html
$ git commit -a -m 'Finish the new footer [issue 53]'
[iss53 ad82d7a] Finish the new footer [issue 53]
1 file changed, 1 insertion(+)
```

CONSOLE



شكل ٢٣. استكمال العمل على iss53

من الواجب ملاحظة أن ملفات فرع iss53 لا تحتوى على عملك في فرع hotfix. فإذا احتجت إلى جذبه إليها، ادمج فرع master في فرع iss53 بالأمر `git merge master`، أو أجّله حتى تقرر جذب فرع iss53 إلى master فيما بعد.

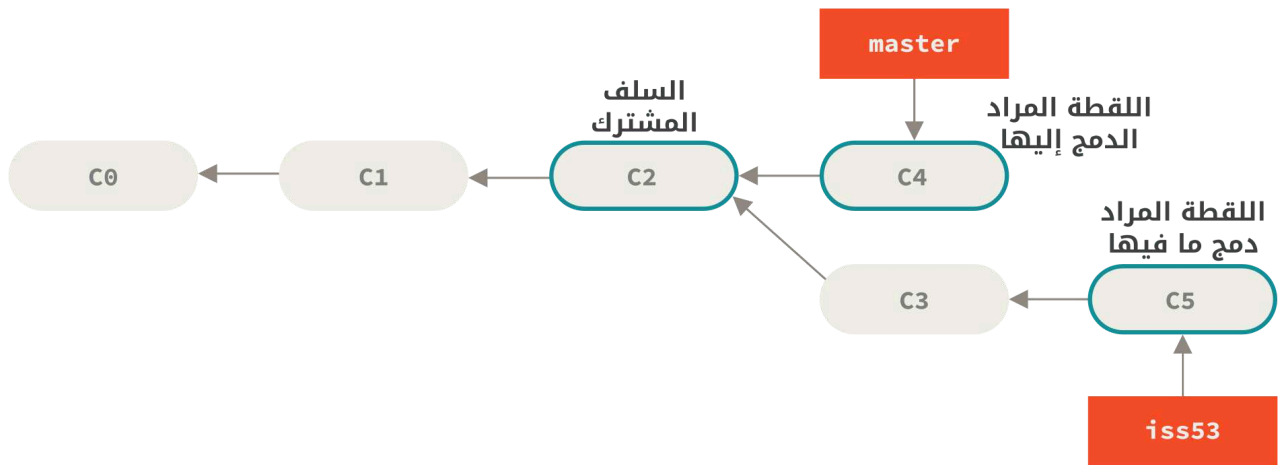
٣، ٢، ١، أسس الدمج

إذا رأيت أن عملك على المسألة رقم ٥٣ قد اكتمل وصار جاهزا لدمجه في الفرع الرئيس، فستدمج فرع iss53 في فرع master، تماما مثلما دمجت فرع hotfix سابقا: ليس عليك سوى سحب الفرع الذي تريد الدمج فيه ثم تنفيذ أمر الدمج:

```
$ git checkout master
Switched to branch 'master'
$ git merge iss53
Merge made by the 'recursive' strategy.
index.html | 1 +
1 file changed, 1 insertion(+)
```

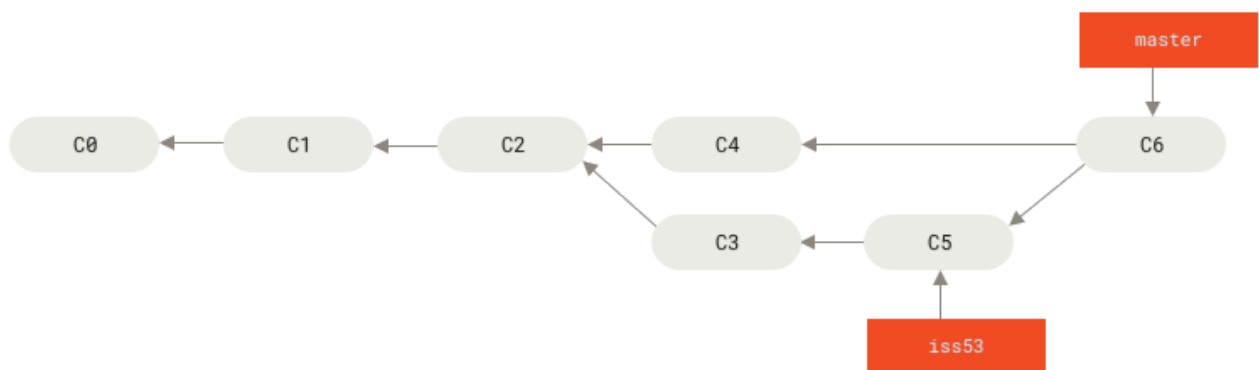
CONSOLE

يبدو هذا مختلفا قليلا عن دمج hotfix الذي أجرينّه سابقا. ففي حالتنا هذه قد افترق تاريخ التطوير منذ نقطة سابقة. ولأن الإيداع الأخير في الفرع الذي تقف فيه ليس سلفاً للفرع الذي تريد دمجه (أب أو جد أو جد أعلى)، فإن على جت القيام ببعض العمل. في هذه الحالة، يعمل جت دمجاً ثلاثياً نموذجياً، للقطتين اللتين يشيران إليهما رأسا الفرعين، مع السلف المشترك للاثنتين.



شكل ٢٤. اللقطات الثلاثة المستعملة في دمج نموذجي معتاد

فبدلاً من مجرد تحريك الإشارة إلى الأمام، ينشئ جت لقطة جديدة ناتجة عن هذا الدمج الثلاثي، وينشئ آلياً إيداعاً جديداً يشير إليها. يسمى هذا «إيداع دمج»، ويتميز بأن له أكثر من أب.



شكل ٢٥. إيداع دمج

الآن قد دُمج عملك، ولم تعد في حاجة إلى فرع `iss53`. فيمكنك غلق هذه المسألة في نظام متابعة المسائل، وحذف الفرع:

```
$ git branch -d iss53
```

CONSOLE

٣، ٢، ٣، أسس نزاعات الدمج

لا تسيّر هذه العملية بسلاسة في بعض الأحيان. فإذا عدّلت الجزء نفسه في الملف نفسه تعديلاً مختلفاً في الفرعين اللذين تنوي دمجهما، فلن يستطيع جت أن يدمجهما دمجا نظيفاً. فإن كان إصلاحك للمسألة رقم ٥٣ عدّل الجزء نفسه من الملف الذي عدّلته في فرع الإصلاح العاجل، فستواجه نزاع دمج يشبه هذا:

```
$ git merge iss53
Auto-merging index.html
```

CONSOLE

```
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

لم ينشئ جت آلياً إيداع دمجٍ جديداً، بل أوقف العملية حتى تحل النزاع. فإذا أردت رؤية الملفات غير المدموجة في أي وقت بعد نزاع الدمج، نفذ أمر الحالة `git status`:

```
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")

Unmerged paths:
  (use "git add <file>..." to mark resolution)

   both modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
```

ستظهر الملفات المتنازع عليها ولم تُدمج بعد أنها غير مدموجة "unmerged". ويضيف جت علامات معيارية لحل النزاعات إلى الملفات المتنازع عليها، حتى تتمكن من تحريرها يدوياً وحل تلك النزاعات. فستجد أن في ملفك جزءاً يشبه هذا:

```
<<<<<< HEAD:index.html
<div id="footer">contact : email.support@github.com</div>
=====
<div id="footer">
  please contact us at support@github.com
</div>
>>>>>> iss53:index.html
```

تجد نسخة HEAD (فرع master، لأنه الفرع الذي سحبته قبل أمر الدمج) في النصف الأعلى من هذه الكتلة (كل ما هو فوق سطر =====)، ونسخة iss53 في النصف الأسفل منها. وحتى تحل هذا النزاع، عليك اختيار أحد الجزأين أو دمج محتوَاهما بنفسك. فمثلاً قد تحله بتغيير الكتلة كلها إلى:

```
<div id="footer">
  please contact us at email.support@github.com
</div>
```

يحمل هذا الحل شيئاً من كلا الجزأين. أما الأسطر <<<<<< و ===== و >>>>>> فقد أزلناها بالكامل. وبعد حل كل نزاع مثل هذا في كل ملف متنازع عليه، نفذ أمر الإضافة `git add` على كل ملف لإعلام جت أنه قد حُل. فتأهيل الملف في جت يعلن نزاعه محلولاً.

وإذا أردت استعمال أداة رسومية لحل هذه المشاكل، فأمر أداة الدمج `git mergetool` يشغل أداة دمج رسومية مناسبة ويسير معك خلال النزاعات:

```
$ git mergetool
```

```
This message is displayed because 'merge.tool' is not configured.
See 'git mergetool --tool-help' or 'git help config' for more details.
'git mergetool' will now attempt to use one of the following tools:
opendiff kdiff3 tkdiff xxdiff meld tortoisemerge gvimdiff diffuze diffmerge ecmerge p4merge araxis
bc3 codecompare vimdiff emerge
Merging:
index.html

Normal merge conflict for 'index.html':
  {local}: modified file
  {remote}: modified file
Hit return to start merge resolution tool (opendiff):
```

إذا أردت استعمال أداة دمج أخرى غير الأداة المبدئية (اختار جت في هذه الحالة أداة `opendiff` لأننا نفذناه على نظام ماك)، فيمكنك رؤية قائمة بجميع الأدوات المدعومة في الأعلى بعد جملة "one of the following tools". ليس عليك سوى كتابة اسم الأداة التي تريدها.

إذا احتجت أدوات متقدمة أكثر لحل النزاعات العويصة، فستحدث أكثر عن الدمج في فصل [تصايف متقدمة في الدمج](#).



بعد إغلاق أداة الدمج، فسيُسألك جت عما إذا كان الدمج ناجحًا. إذا أجبت بنعم، فسيؤهل الملف لك لإعلان أنه قد حُل. ويمكنك عندئذٍ استعراض الحالة مجددًا للتحقق أن جميع النزاعات قد حُلّت:

```
$ git status
On branch master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:

  modified:   index.html
```

فإذا كنت راضيا عن هذا، وتأكدت من أن كل شيء كان عليه نزاع قد أُهِّل، نفذ `git commit` لاختتام إيداع الدمج. ورسالة الإيداع المبدئية تشبه هذا:

```
Merge branch 'iss53'

Conflicts:
  index.html
#
# It looks like you may be committing a merge.
# If this is not correct, please remove the file
#   .git/MERGE_HEAD
# and try again.
```

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
# On branch master
# All conflicts fixed but you are still merging.
#
# Changes to be committed:
#   modified:   index.html
#
```

فيها يمكنك شرح حلك للنزاع وتعليل تعديلاتك إن لم تكن واضحة، إذا ظننت أن هذا يفيد من يرى هذا الإيداع فيما بعد.

٣، ٣، إدارة الفروع

الآن وقد أنشأت فروعاً ودمجتها وحذفتها، لنر أدوات إدارة فروع ستيفيك عندما تشرع في استعمال الفروع طوال الوقت.

ليس أمر الفروع `git branch` لإنشاء فروع وحذفها فحسب. فإذا نفذته بلا معاملات، سيسرد لك فروعك الحالية:

```
$ git branch
  iss53
* master
  testing
```

CONSOLE

لاحظ حرف النجمة * أمام فرع `master`؛ إنه يعني أن هذا الفرع هو الفرع المسحوب حالياً (أي أنه الفرع الذي تشير إليه إشارة الرأس HEAD). يعني هذا أنك إذا أودعت الآن، فإن فرع `master` سيتقدم إلى الأمام بعملك الجديد. لرؤية آخر إيداع في كل فرع، نفذ `git branch -v`:

```
$ git branch -v
  iss53    93b412c Fix javascript issue
* master   7a98805 Merge branch 'iss53'
  testing  782fd34 Add scott to the author list in the readme
```

CONSOLE

والخياران المفيدان `--merged` («مدموج») و `--no-merged` («غير مدموج») يصفيان هذه القائمة فلا ترى إلا الفروع التي دمجتها أو التي لم تدمجها في الفرع الذي تقف فيه. فللفروع المدموجة في الفرع الحالي، نفذ `git branch --merged`:

```
$ git branch --merged
  iss53
* master
```

CONSOLE

ترى `iss53` في القائمة لأنك دمجته سابقاً. والفروع التي في هذه القائمة وليس أمامها نجمة (*)، يمكنك في العموم حذفها بأمان بالأمر `git branch -d`؛ لن تفقد شيئاً بحذفها لأنك بالفعل ضمنت ما فيها من عمل إلى فرع آخر.

لرؤية جميع الفروع التي بها عمل غير مدموج بعد، نفذ `git branch --no-merged`:

```
$ git branch --no-merged
testing
```

يُظهر لك هذا الأمر فرعك الآخر. ستفشل محاولة حذفه بالأمر `git branch -d` لأن به عملاً غير مدمج بعد:

```
$ git branch -d testing
error: The branch 'testing' is not fully merged.
If you are sure you want to delete it, run 'git branch -D testing'.
```

إن رغبت حقاً وبقيناً في حذف الفرع وفقد ما فيه من عمل، فأجبر جت على حذفه بالخيار `-D`، كما تخبرك الرسالة.

إذا لم تعطِ إيداعاً أو اسمَ فرعٍ إلى الخيارين `--merged` و `--no-merged`، فإنهما، على الترتيب، سيسردان ما الذي دُمج أو لم يُدمج في الفرع الحالي.

يمكنك دائماً إعطاؤهما اسم فرع للسؤال عن حالة دمجه من غير أن تحتاج إلى الانتقال أولاً إلى هذا الفرع بأمر السحب، مثلاً: ما الذي لم يُدمج في فرع `master`؟



```
$ git checkout testing
$ git branch --no-merged master
topicA
featureB
```

٣، ٣، ١، تغيير اسم فرع

لا تغيّر اسم فرع ما زال الآخرون يستعملونه. ولا تغيّر اسم فرع مثل `master` أو `main` أو `mainline` قبل أن تقرأ فصل [تغيير اسم الفرع الرئيس](#).



هَبْ فرعاً لديك اسمه `bad-branch-name` وتريد جعله `corrected-branch-name` مع الإبقاء على تاريخه بالكامل. وتريد أيضاً تغيير اسمه على الخادوم البعيد (جتهب GitHub أو جت لاب GitLab أو غيرهما). كيف تفعل هذا؟

غيّر اسم الفرع على جهازك بالأمر `git branch --move`:

```
$ git branch --move bad-branch-name corrected-branch-name
```

هذا يغيّر `bad-branch-name` إلى `corrected-branch-name`، ولكن هذا التغيير محلي فقط حتى الآن. فلجعل الآخرين يرون الفرع الصحيح في المستودع البعيد، عليك دفعه:

```
$ git push --set-upstream origin corrected-branch-name
```

لنلق نظرةً على حالنا الآن:

```
$ git branch --all
* corrected-branch-name
  main
  remotes/origin/bad-branch-name
  remotes/origin/corrected-branch-name
  remotes/origin/main
```

CONSOLE

لاحظ أنك في فرع `corrected-branch-name` وأنه متاح في المستودع البعيد. ولكنّ الفرع ذا الاسم الخطأ متاح كذلك هناك، لكن يمكنك حذفه بالأمر التالي:

```
$ git push origin --delete bad-branch-name
```

CONSOLE

الآن قد حلّ اسم الفرع الصحيح محل اسم الفرع الخطأ في كل مكان.

تغيير اسم الفرع الرئيس

تغيير اسم فرع مثل `master` أو `main` أو `mainline` أو `default` سيُعطل التكاملات والخدمات والأدوات المساعدة وبرمجيات البناء والإصدار التي يستخدمها مستودعك. لذا عليك التشاور مع زملائك في المشروع قبل الإقدام على هذا الأمر. وعليك كذلك أن تبحث بحثاً وافياً في مستودعك وتحديث أيّ إشارة إلى الاسم القديم للفرع في المصدر البرمجي للمشروع وفي البرمجيات.



غير اسم فرع `master` المحلي إلى `main` بالأمر:

```
$ git branch --move master main
```

CONSOLE

لم يعد لدينا أي فرع محلي `master`، لأننا غيرنا اسمه إلى `main`.

ولجعل الآخرين يرون فرع `main` الجديد، عليك دفعه إلى المستودع البعيد. هذا يجعل الفرع الجديد متاحاً هناك:

```
$ git push --set-upstream origin main
```

CONSOLE

نجد الآن أنفسنا في الحالة التالية:

```
$ git branch --all
* main
  remotes/origin/HEAD -> origin/master
  remotes/origin/main
  remotes/origin/master
```

CONSOLE

اختلف فرعي المحلي master، وحلّ محله الفرع main. وصار main في المستودع البعيد. ولكن فرع master القديم بقي موجوداً في المستودع البعيد. فسيظل المشاركون الآخرون يتخذون فرع master أساساً لأعمالهم، حتى تتخذ إجراء آخر.

بين يديك الآن عددٌ من المهام لاجتياز تلك المرحلة الانتقالية:

- على جميع المشروعات المعتمدة على هذا المشروع تحديث مصدرها البرمجي و/أو إعداداتها.
- عليك تحديث أي ملفات إعدادات خاصة بالاختبارات.
- عليك مواءمة بُرُمِجات البناء والإصدار.
- عليك مواءمة إعدادات خادم مستودعك، مثل الفرع المبدئي وقواعد الدمج والأمر الأخرى التي تعتمد على أسماء الفروع.
- عليك تحديث الإشارات إلى الفرع القديم في التوثيق.
- عليك إغلاق أو دمج كل طلبات الجذب الموجهة إلى الفرع القديم.

بعد فعل جميع هذه المهام، والتيقن أن فرع main يقوم بعمله تماماً مثل فرع master، يمكنك حذف فرع master:

```
$ git push origin --delete master
```

CONSOLE

٣، ٤، أساليب التطوير التفرعية

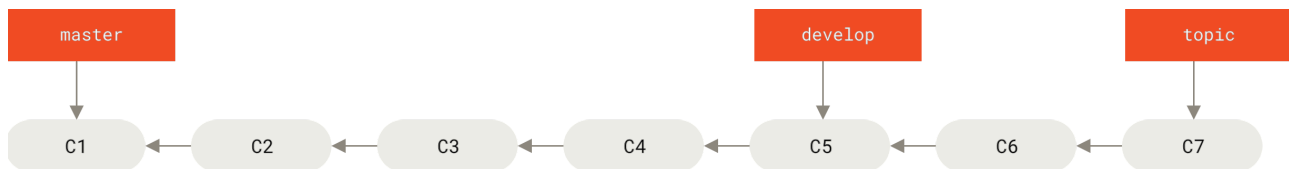
بما أنك الآن تعلم أسس التفرع والدمج، ماذا يمكنك أو يجدر بك فعله بهما؟ سنتناول في هذا الفصل بعض أشهر أساليب التطوير التي يجعلها هذا التفرع الخفيف ممكنة، لكي تقرر إذا ما كنت تود أن تجعلها جزءاً من دورة التطوير التي تتبعها.

٣، ٤، ١، الفروع طويلة العمر

يستعمل جت دمجاً ثلاثياً غير معقد، فيسهّل الدمج بين فرعين مرات عديدة عبر مدة زمنية طويلة. يتيح لك هذا وجود عدد من الفروع المفتوحة دائماً لتستعملها لمراحل مختلفة من دورة التطوير، لأنك تستطيع أن تدمج باستمرار فيما بينها.

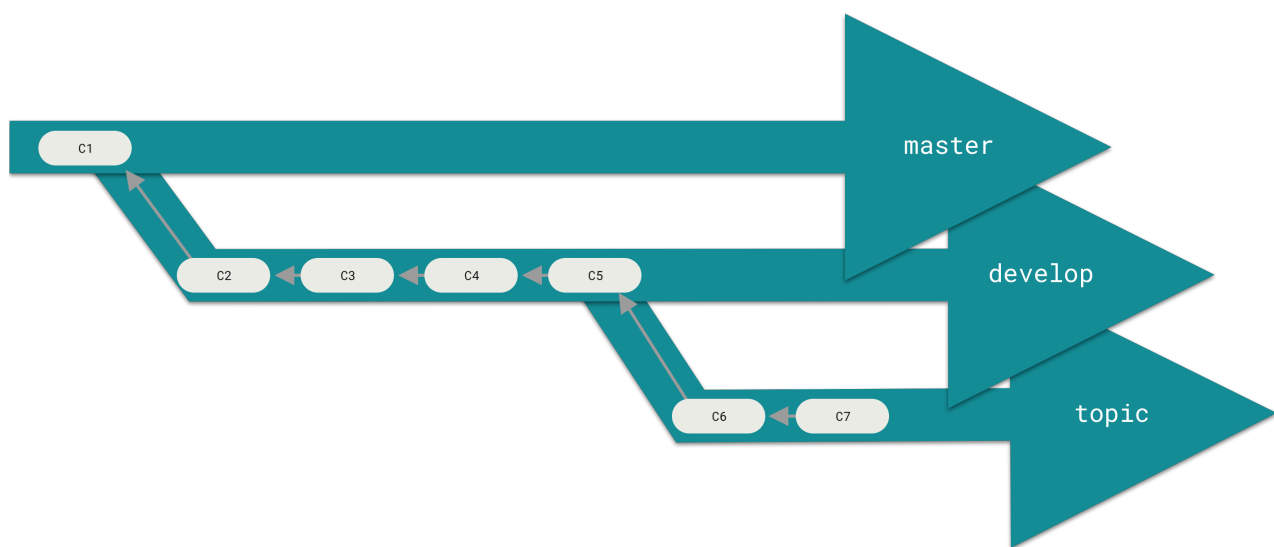
الكثير من المطورين مستخدمي جت يعتمدون هذا النهج في أسلوبهم في التطوير، فيخصصون مثلاً الفرع الرئيس للمصدر المستقر تماماً وحسب، أو للذي أُصدر فعلاً، أو للذي سيصدر. ويكون لديهم فرعاً موازياً اسمه develop أو next مثلاً، ليعملوا منه أو ليستعملوه لاختبار الاستقرار، فليس بالضرورة أن يكون مستقراً دوماً، ولكن عند استقراره، يمكن دمجه في الفرع الرئيس. ويستعملون هذا الفرع ليجذبوا فيه فروع الموضوعات (الفروع قصيرة العمر، مثل فرع iss53 المذكور سابقاً) عندما تكون جاهزة، لضمان اجتيازها جميع الاختبارات وأنها لا تُحدث عللاً.

نحن فعلياً نتحدث عن إشارات ترتقي في سلم ايداعاتك. فالفروع المستقرة في أسفله، أما طليعة التطوير ففي أعلاه.



شكل ٢٦. منظور خطي لتفريع الاستقرار المتزايد

لعل الأسهل تصور أنها صومعات عمل منعزلة، فتتخرج دفعة من الإيداعات إلى صومعة أخرى أكثر استقرارا عندما تجتاز جميع الاختبارات.



شكل ٢٧. منظور «صومعي» لتفريع الاستقرار المتزايد

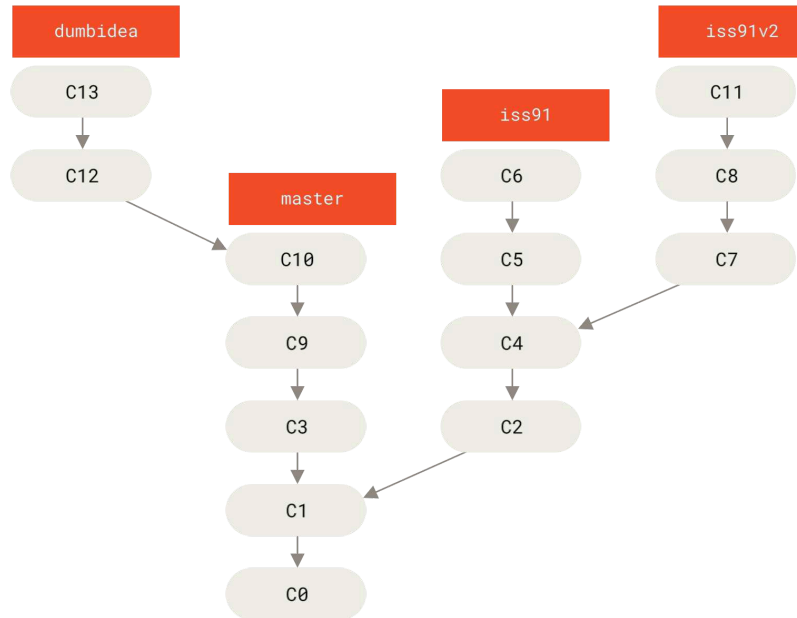
يمكنك فعل هذا بعدة مستويات من الاستقرار. فلدى بعض المشروعات الكبيرة فرع `proposed` أو `pu` («تحديثات مقترحة») ويدمجوا فيه فروعاً قد لا تكون جاهزة لأن تكون في فرع `next` أو `master`. فالأمر أن فروعك في مستويات مختلفة من الاستقرار، فعندما يصل أحدها إلى مستوى استقرار أعلى، فإنه يُدمج في الفرع الأعلى. ونكرر: ليس ضروريا استعمال عدد من الفروع طويلة العمر، ولكنه كثيرا ما يفيد، خصوصا عندما تتعامل مع مشروعات معقدة أو كبيرة جدا.

٣، ٤، ٢، فروع الموضوعات

لكن فروع الموضوعات تفيد جميع المشروعات بغض النظر عن حجمها. فرع الموضوع هو فرع قصير العمر تنشئه وتستعمله لميزة واحدة أو ما يخصها من عمل. لعلك لم تفعل هذا قط مع نظام إدارة نسخ آخر، لأن التفريع والدمج غالبا ما يكونا بطيئين جدا في الأنظمة الأخرى. ولكن الشائع مع جت هو إنشاء فروع والعمل عليها ودمجها وحذفها عدة مرات في اليوم الواحد.

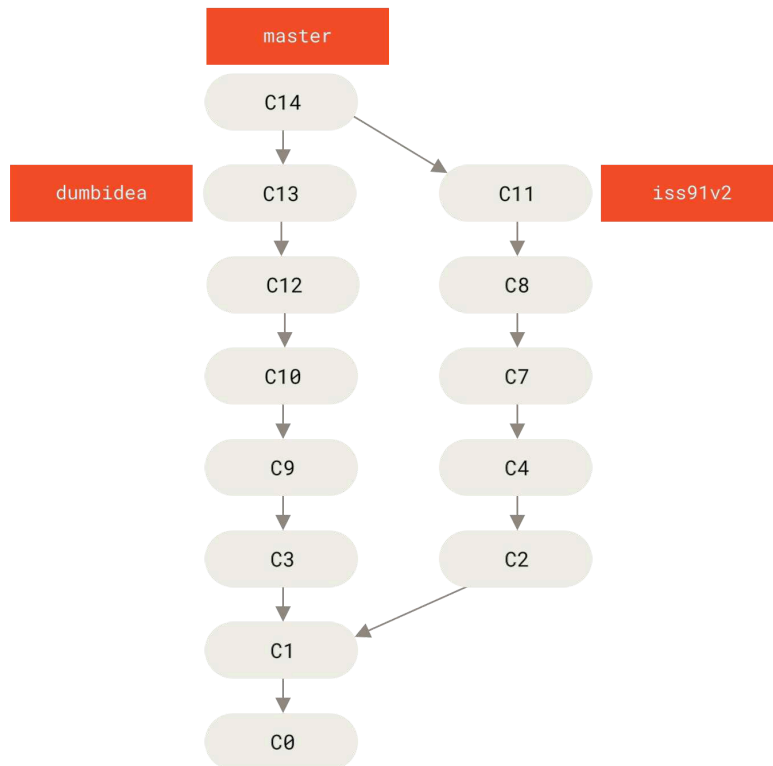
وقد رأيت هذا في الفصل السابق في فرعي `iss53` و `hotfix` اللذين أنشأتهم، فقد صنعت بضعة إيداعات فيهما ثم حذفتهما فور دمجهما في فرعك الرئيس. يسمح لك هذا الأسلوب بـ«تبديل السياق» سريعا وبالكامل، لأنك قسّمت عملك إلى صومعات، وكل صومعة (فرع) ليس فيها إلا التعديلات التي تخص موضوعاً واحداً، فيسهّل ذلك رؤيتها عند المراجعة (code review) وغير ذلك. ويمكنك إبقاء التعديلات هناك دقائق أو أياماً أو شهوراً، ثم دمجها عندما تكون جاهزة، بغض النظر عن ترتيب إنشائها أو العمل عليها.

لننقل مثلاً إنك عملت (في master)، ثم تفرّعت لإصلاح علة (iss91)، وعملت عليها قليلاً، ثم تفرّعت مجدداً (من الفرع الثاني) لتجرب طريقة أخرى لإصلاح العلة نفسها (iss91v2)، ثم عدت إلى فرعك الرئيس (master) وعملت فيه قليلاً، ثم تفرّعت منه لتجربة شيء لست واثقاً أنه جيد (فرع dumbidea). سيبدو تاريخ إيداعك الآن مثل هذا:



شكل ٢٨. فروع موضوعات متعددة

لننقل إنك الآن وجدت إصلاحك الثاني للعلة (iss91v2) أفضل، وأنتك أريت زملاءك فرع dumbidea فأخبروك أنه عبثي. فيمكنك إنذاراً إلقاء فرع iss91 الأول (وفق الإيداعين C5 و C6)، ودمج الفرعين الآخرين في الفرع الرئيس. سيبدو تاريخك الآن كهذا:



شكل ٢٩. التاريخ بعد دمج dumbidea و iss91v2

سنتحدث بتفصيل أكبر عن مختلف أساليب التطوير الممكنة في مشروعات جت في باب [جت المتوزع](#)، فعليك قراءة هذا الفصل قبل أن تقرر أي أسلوب تفرّيع سيتبعه مشروعك التالي.

من المهم تذكر أنك عندما تفعل أيًا من هذا فإن هذه الفروع تبقى محلية بالكامل. فعندما تتفرّع وتدمج، يحدث كل شيء داخل مستودع جت الخاص بك وحسب؛ لا يحدث أي تواصل مع الخادوم.

٣، ٥، الفروع البعيدة

الإشارات البعيدة هي تلك الإشارات الموجودة في مستودعاتك البعيدة، كالفروع والوسوم. يمكنك سرد جميع الإشارات البعيدة بالأمر «البعيد» `git ls-remote`، أو سرد الفروع البعيدة ومعلوماتها بالأمر `git remote show` «البعيد» (حيث «البعيد» هو الاسم المختصر للمستودع البعيد). ولكن الشائع هو الانتفاع بـ«الفروع المتعقّبة للبعيد».

الفرع المتعقّب لبعيد هو إشارة إلى حالة فرع بعيد. أي أنه إشارة محلية (أي في المستودع الذي على حاسوبك) لكن لا يمكنك تحريكها؛ إن جت يحركها لك عند الاتصال مع الخادوم، حتى يضمن أنها دائماً تمثل حالة المستودع البعيد. اعتبرها إشارات مرجعية مثل علامات المتصفح، لتذكرك أين كانت فروع مستودعك البعيد عندما تواصلت معه آخر مرة.

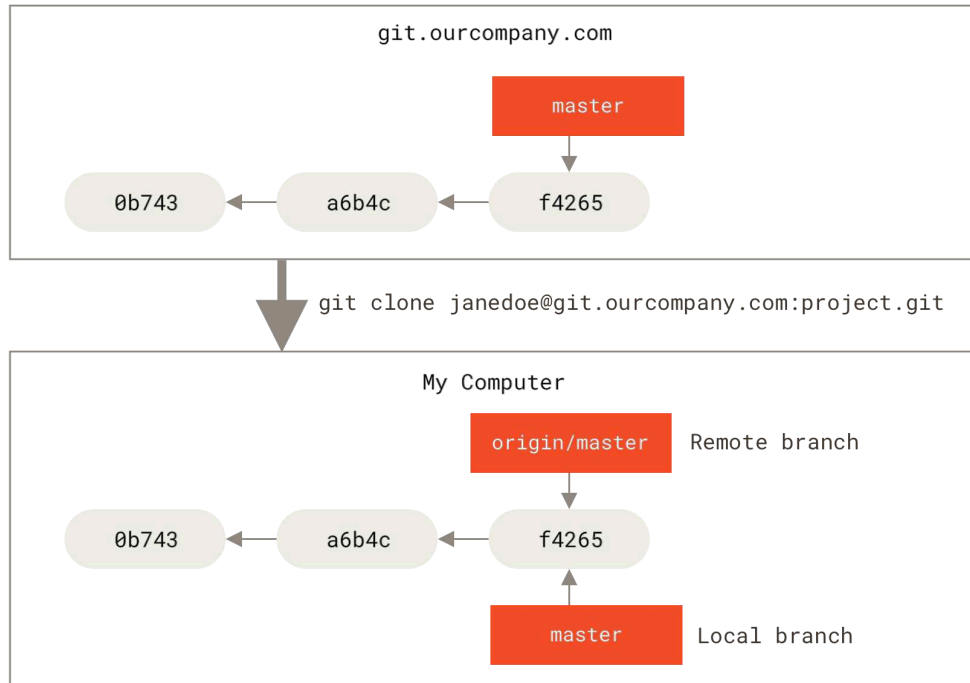
يكون شكل أسماء الفروع المتعقّبة للبعيد `<remote>/<branch>` (أي اسم المستودع البعيد ثم شرطة مائلة ثم اسم الفرع). فمثلاً إذا أردت رؤية كيف بدا فرع `master` في مستودعك البعيد `origin` عندما اتصلت به آخر مرة، فانتقل إلى فرع `origin/master`. وإذا كنت تعمل مع زميل على مسألةٍ ودفع فرع `iss53` إلى المستودع البعيد، فقد يكون لديك فرع محلي بالاسم نفسه، ولكن الفرع الذي على الخادوم سيمثله عندك الفرع المتعقّب للبعيد الذي اسمه `.origin/iss53`.

لعل الكلام غامض، فدعنا ننظر إلى مثال. لنقل إن لديك خادوم جت على شبكتك عنوانه `git.ourcompany.com`. إذا استنسخته، فإن أمر الاستنساخ سيسميه `origin` لك، ويجذب كل ما فيه من بيانات، وينشئ إشارة إلى ما يشير إليه فرع `master` عليه ويسميه `origin/master` محلياً. وسيعطيك جت أيضاً فرع `master` محلي خاص بك بادئاً من المكان نفسه الذي فيه فرع `master` الخاص بالأصل، حتى يتسنى لك البدء بالعمل.

الاسم "origin" ليس مميزاً

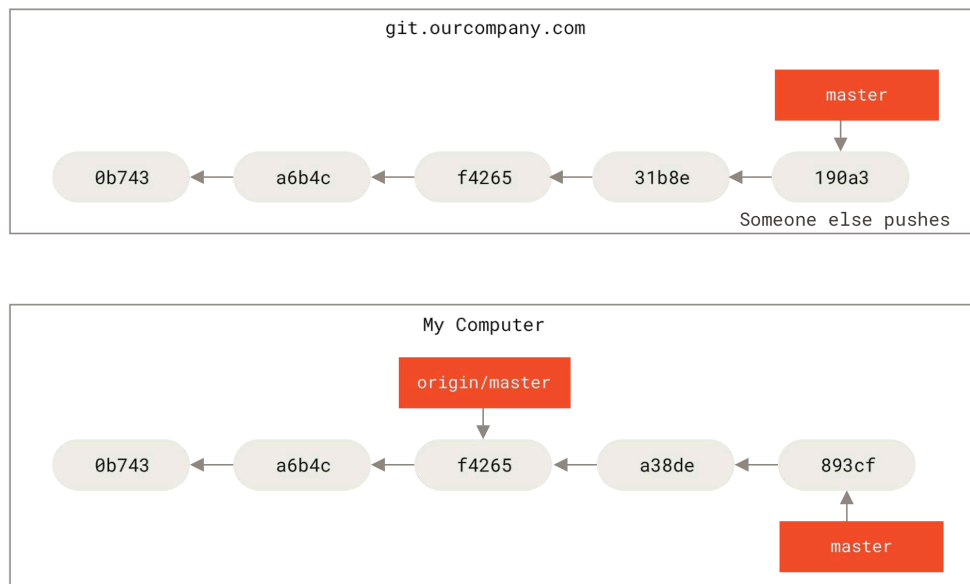
تماماً مثلما أن اسم الفرع الرئيس "master" لا يحمل أي معنى خاص في جت، ف كذلك اسم المستودع البعيد الأصل "origin". فإن "master" هو الاسم المبدئي لأول فرع ينشئه جت عندما تستخدم `git init` (وهو السبب الوحيد لشيوعه)، وكذلك "origin" هو الاسم المبدئي للمستودع البعيد عندما تستخدم `git clone`. فإذا استخدمت `git clone -o yalla` مثلاً، فإنك ستجد أن `yalla/master` هو اسم الفرع البعيد المبدئي.





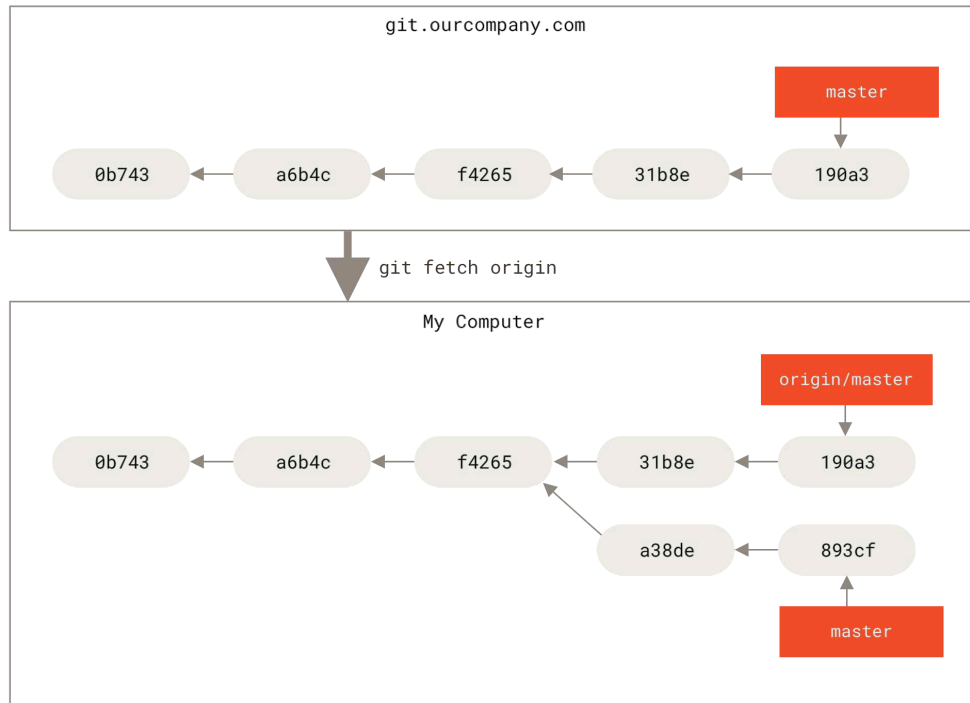
شكل ٣٠. المستودعان البعيد والمحلي بعد الاستنساخ

إذا عملت في فرعك الرئيس المحلي، ودفع أحد إلى الفرع الرئيس في المستودع البعيد، فإن تاريخي الفرعين سيتقدمان مفترقين. وإن تجنبنا الاتصال مع مستودعك البعيد على الخادوم الأصل، فلن تتحرك إشارة `origin/master` التي لديك.



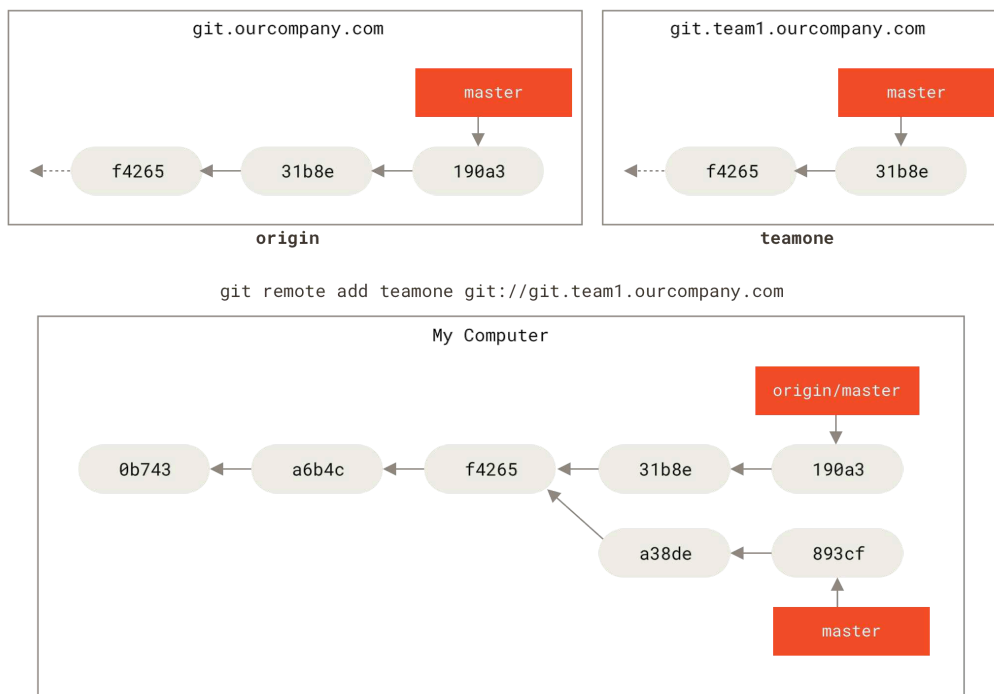
شكل ٣١. قد يفترق العمل المحلي والبعيد

لمزامنة عملك مع مستودع بعيد، نَفِّذ الأمر «البعيد» `git fetch` (في حالتنا `git fetch origin`). فهذا الأمر يبحث عن المستودع المسمى «origin» (في حالتنا `git.ourcompany.com`)، ويستحضر البيانات التي عليه وليست عندك بعد، ويحدِّث قاعدة بياناتك المحلية، ويحرك إشارة `origin/master` الخاصة بك لتشير إلى موقعها الجديد المحدث.



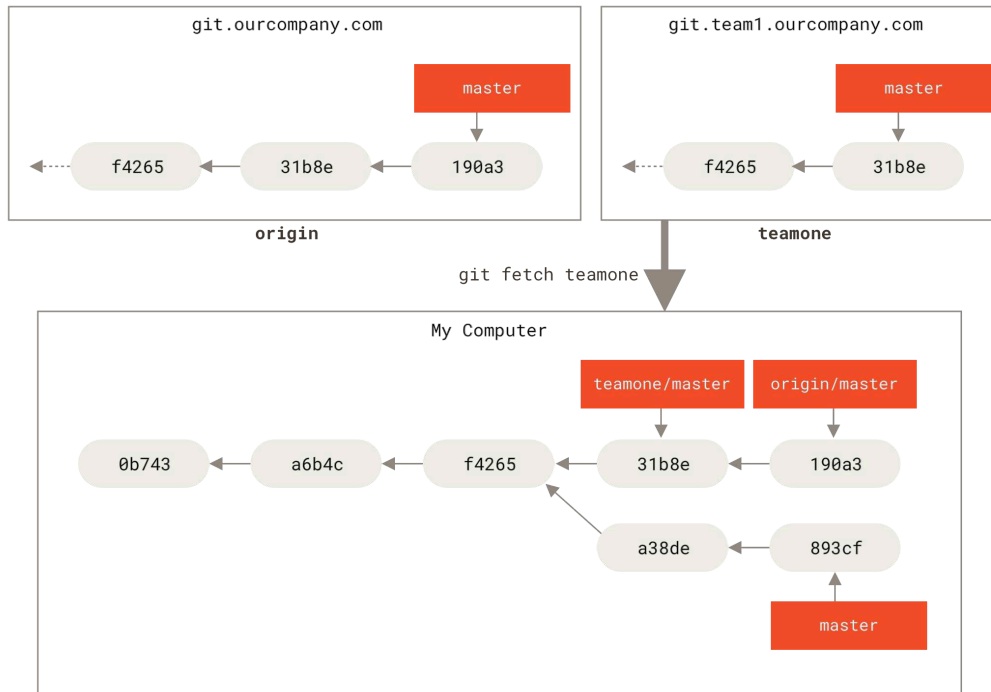
شكل ٣٢. يحدّث أمر الاستحضار `git fetch` فروعك المتعقّبة للبعيد

لتمثيل وجود خواديم بعيدة عديدة ولإيضاح منظر الفروع المتعقّبة لهذه المستودعات البعيدة، لنقل إن لديك خادم جت داخلي آخر، ولا يستخدمه إلا فريق واحد من أجل التطوير، وإن عنوانه هو `git.team1.ourcompany.com`. يمكنك إضافته إشارةً بعيدة جديدة في مشروعك، بأمر `git remote add` كما رأينا في باب [أسس جت](#)، وتسميته `teamone`، الذي يُعتبر اسماً مختصراً لرابطة الكامل.



شكل ٣٣. إضافة إشارة إلى خادم بعيد آخر

والآن، نَفِّذْ `git fetch teamone` لاستحضار كل ما لدى خادم `teamone` البعيد وليس لديك بعد. ولأنه الآن ليس لديه من البيانات إلا جزءًا مما لدى الخادوم الأصل (`origin`)، فلن يستحضر جت شيئًا، لكنه سيعدّ فرعًا متعقبًا للبعيد يسمى `teamone/master` ليشير إلى الإيداع الذي يشير إليه فرع `master` في مستودع `teamone`.



شكل ٣٤. فرع متعقب للبعيد للفرع `teamone/master`

٣، ٥، ١، الدفع

عندما تريد أن تشارك فرعًا مع العالم، فعليك دفعه إلى مستودع بعيد لديك إذن تحريره. ففروعك المحلية لا تُزَامَنُ آليًا إلى المستودعات البعيدة، حتى التي دفعت إليها؛ عليك دفع الفروع التي تريد مشاركتها بأمر صريح. يسمح لك هذا أن تستعمل فروعًا خصوصية للأعمال التي لا تريد مشاركتها، وألا تدفع إلا فروع الموضوعات التي تريد التعاون عليها.

مثلا إذا كان لديك فرعًا اسمه `serverfix` وتريد العمل عليه مع الآخرين، يمكنك دفعه بالطريقة نفسها التي دفعت بها فرعك الأول؛ نَفِّذْ `git push <remote> <branch>` (أي اسم المستودع البعيد ثم الفرع):

```
$ git push origin serverfix
Counting objects: 24, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (15/15), done.
Writing objects: 100% (24/24), 1.91 KiB | 0 bytes/s, done.
Total 24 (delta 2), reused 0 (delta 0)
To https://github.com/schacon/simplegit
 * [new branch]      serverfix -> serverfix
```

CONSOLE

هذا اختصار، لأن جت يفك اسم الفرع `serverfix` إلى `refs/heads/serverfix:refs/heads/serverfix` الذي يعني «ادفع فرعي المحلي `serverfix` إلى المستودع البعيد `origin` لتحديث فرع `serverfix` عليه». سنفصّل شرح جزء `refs/heads/` في باب موازل جت، ولكن عامةً يمكنك تركه. كذلك يمكنك تنفيذ `git push origin`

serverfix:serverfix الذي يفعل الشيء نفسه؛ إنه يقول: «خذ فرعي المسمى serverfix واجعله فرع serverfix في المستودع البعيد». هذه الصياغة مفيدة لدفع فرع محلي إلى فرع بعيد باسم مختلف. فمثلاً إن لم تُردّه أن يسمى serverfix في المستودع البعيد، فنقدّ git push origin serverfix:awesomebranch، فهذا يدفع فرعك المحلي serverfix إلى فرع awesomebranch في المستودع البعيد.

لا تكتب كلمة مرورك كل مرة

إذا كنت تستعمل رابط HTTPS للدفع، فإن خادم جت سيسألك عن اسم مستخدمك وكلمة مرورك للاستيثاق. المعناد أن عميل جت سيسألك عن هذا في الطرفية حتى يعرف الخادوم إذا ما كان مسموحاً لك بالدفع.



إذا لم تنشأ أن تكتب كلمة مرورك في كل مرة تدفع فيها، فعليك إعداد «تذكّر مؤقت للاستيثاق» ("credential cache"). أسهل خيار هو جعله في ذاكرة الحاسوب لعدة دقائق، الذي يمكنك إعداده بالأمر `git config --global credential.helper cache`.

لمعلومات أكثر عن خيارات تذكّر الاستيثاق المتاحة، انظر فصل [تخزين الاسم وكلمة المرور](#).

وفي المرة التالية التي يستحضر أحد زملائك من الخادوم، سيحصل على إشارة إلى حيث يشير فرع serverfix على الخادوم؛ سيجدها عنده في الفرع البعيد origin/serverfix:

```
$ git fetch origin
remote: Counting objects: 7, done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0)
Unpacking objects: 100% (3/3), done.
From https://github.com/schacon/simplegit
* [new branch]      serverfix -> origin/serverfix
```

CONSOLE

من المهم ملاحظة أنك عندما تستحضر، يجلب لك هذا فروعاً جديدة متعقبة للبعيد، أي أنك لا تحصل تلقائياً على نسخ محلية منها يمكنك تعديلها. بلفظ آخر، لا تحصل آلياً على فرع serverfix جديد في هذه الحالة: لم تُعطَ إلا إشارة origin/serverfix التي لا يمكنك التعديل فيها.

لدمج هذا العمل في فرعك الحالي، يمكنك تنفيذ `git merge origin/serverfix`. وإذا أردت فرع serverfix خاصاً بك تستطيع العمل فيه، يمكنك تفريعه من الفرع المتعقب للبعيد:

```
$ git checkout -b serverfix origin/serverfix
Branch serverfix set up to track remote branch serverfix from origin.
Switched to a new branch 'serverfix'
```

CONSOLE

يعطيك هذا فرعاً محلياً يمكنك العمل فيه، ويبدأ من حيث يقف origin/serverfix.

٣، ٥، ٢، تعقب الفروع

إن سحب فرع محلي من فرع متعقب للبعيد ينشئ آلياً ما يسمى «فرع متعقب» (والفرع الذي يتعقبه يسمى «الفرع المنبع»). الفروع المتعقبة هي فروع محلية ذات علاقة مباشرة بفرع بعيد. فإذا كنت في فرع متعقب وكتبت `git pull`، فسيعرف جت تلقائياً أي مستودع بعيد يستحضر منه وأي فرع يدمج فيه.

عندما تستنسخ مستودعاً، ينشئ جت فرعاً باسم الفرع المبدئي فيه (مثل `master`) ويجعله يتعقب الفرع المبدئي في المستودع الأصل (`origin/master`). ولكن يمكنك إعداد فروع متعقبة أخرى إذا أردت، لتعقب مستودعات بعيدة أخرى، أو لتعقب فرع غير الرئيس. أيسر حالة مثلما رأيت آنفاً، عند اتحاد اسم الفرع المحلي والبعيد، أي `git checkout -b <branch> <remote>/<branch>`. وهذه العملية شائعة بما يكفي أن جت يتيح اختصارها بالخيار `--track`:

```
$ git checkout --track origin/serverfix
Branch serverfix set up to track remote branch serverfix from origin.
Switched to a new branch 'serverfix'
```

CONSOLE

وفي الحقيقة أن هذا شائع جداً حتى إن جت يتيح اختصاراً لهذا الاختصار. فإذا كان اسم الفرع الذي تريد سحبه، أولاً غير موجود محلياً بالفعل، وثانياً يطابق تماماً اسماً في مستودع بعيد واحد، فسينشئ لك جت فرعاً متعقباً:

```
$ git checkout serverfix
Branch serverfix set up to track remote branch serverfix from origin.
Switched to a new branch 'serverfix'
```

CONSOLE

ولإعداد فرع محلي باسم مختلف عن الفرع البعيد، فسهل استعمال الصيغة الأولى مع اسم فرع محلي مختلف:

```
$ git checkout -b sf origin/serverfix
Branch sf set up to track remote branch serverfix from origin.
Switched to a new branch 'sf'
```

CONSOLE

الآن، فرعك المحلي `sf` سيجذب من `origin/serverfix` تلقائياً.

إن كان لديك بالفعل فرعاً محلياً وتریده أن يجذب من فرع بعيد جذبته للتو، أو تريد تغيير الفرع المنبع الذي تتعقبه، عليك بخيار «تعيين المنبع» `--set-upstream-to` أو اختصاره `-u`، مع أمر الفروع `git branch` لتغييره بأمر صريح وقتما شئت.

```
$ git branch -u origin/serverfix
Branch serverfix set up to track remote branch serverfix from origin.
```

CONSOLE



عندما يكون لديك فرع متعقب مُعدّ، يمكنك الإشارة إلى فرعه المنبع بالاختصار `@{upstream}` أو `@{u}`. فإذا كنت في `master` وكان يتعقب `origin/master`، يمكنك تنفيذ أمر مثل `git merge` `@{u}` بدلاً من `git merge origin/master` إن أردت.

لرؤية الفروع المتعقّبة التي أعدتها، يقبل أمر الفروع خيار الإطناب مرتين `-vv` ليسرد لك فروعك المحلية مع معلومات مزيدة فيها ما يتعقبه كل فرع وإذا كان فرعك متقدماً عنه (`ahead`) أو متأخراً (`behind`) أو كليهما.

CONSOLE

```
$ git branch -vv
iss53      7e424c3 [origin/iss53: ahead 2] Add forgotten brackets
master     1ae2a45 [origin/master] Deploy index fix
* serverfix f8674d9 [teamone/server-fix-good: ahead 3, behind 1] This should do it
testing    5ea463a Try something new
```

فنرى هنا أن فرع `iss53` يتعقب `origin/iss53` وأنه «متقدم» (`ahead`) باثنين، أي أن لدينا إيداعين محليين ولم ندفعهما إلى الخادوم بعد. ونرى كذلك أن فرع `master` يتعقب `origin/master` وأنه متزامن معه. ثم نرى بعدهما أن فرع `serverfix` يتعقب فرع `server-fix-good` على خادوم `teamone` وأنه متقدم عنه بثلاثة ومتأخر بواحد، أي أن لدى الخادوم إيداعاً لم ندمجه في فرعنا بعد، وأن لدينا ثلاثة إيداعات محلية لم ندفعها إليه. ثم نرى في النهاية أن فرع `testing` لا يتعقب أي فرع بعيد.

مهم ملاحظة أن هذه الأعداد ليست إلا منذ آخر استحضار (`fetch`) من كل خادوم نتعقبه. فلا يحاول هذا الأمر الاتصال بالخواديم؛ إنما يخبرك بما يحفظه على حاسوبك عما فيها. فإذا أردت من أعداد التقدم والتأخر أن تكون أحدث ما يكون، فعليك الاستحضار من جميع خواديمك البعيدة قبل تنفيذ هذا الأمر مباشرةً. ويمكنك فعل ذلك هكذا:

CONSOLE

```
$ git fetch --all; git branch -vv
```

٣، ٥، ٣، الجذب

نعلم أن أمر الاستحضار `git fetch` يستحضر الإيداعات التي في المستودع البعيد وليست لديك، لكنه لا يغيّر شيئاً في مجلد عملك قَطُّ؛ إنما يجلب البيانات لك ويتركك تدمجها بنفسك. لكن في جت أمر يسمى أمر الجذب `git pull`، وهذا الأمر عملياً يكافئ استحضاراً `git fetch` متبوعاً مباشرةً بدمج `git merge`، في معظم الحالات. فإذا كان لديك فرع متعقب مُعدّ كما في الفصل السابق، إما بإعداده صراحةً وإما بأن يعدّه لك أمر الاستنساخ `git clone` أو أمر السحب `git checkout`، فإن أمر الجذب `git pull` سينظر أيّ مستودع وأيّ فرع يتعقبهما فرعك الحالي، ويستحضر ما في المستودع البعيد ويحاول دمجهم في فرعك.

٣، ٥، ٤، حذف فروع بعيدة

لنقل إنك قضيت ما تريد من فرع بعيد، مثلاً انتهيت أنت وزملاؤك من إضافة ميزة جديدة ودمجتموها في الفرع الرئيس. يمكنك حذف فرع بعيد بالخيار `--delete` مع أمر الدفع `git push`. فإذا أردت حذف فرع `serverfix` من الخادوم، يمكنك تنفيذ الأمر التالي:

```
$ git push origin --delete serverfix
To https://github.com/schacon/simplegit
- [deleted]          serverfix
```

CONSOLE

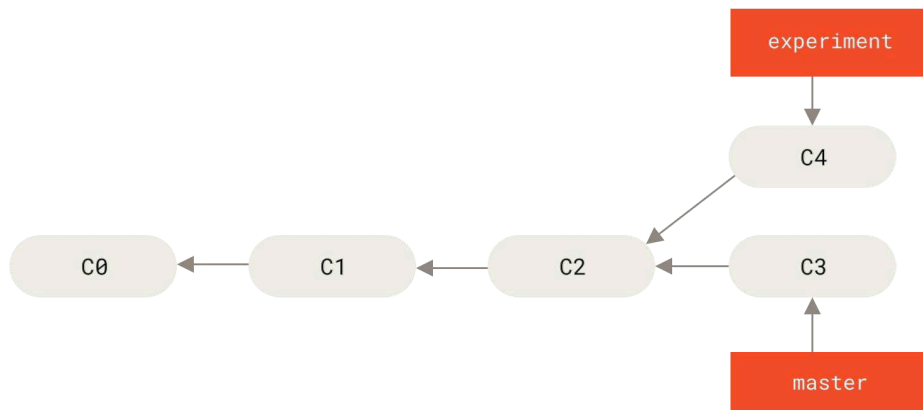
لا يفعل هذا الأمر سوى أنه يحذف الإشارة من على الخادوم. ولكن خواديم جت عموماً تبقى البيانات موجودة وقتاً، إلى أن يعمل جامع المهمات، فغالباً ستستطيع استعادته بسهولة إن حذفته بالخطأ.

٣، ٦، إعادة التأسيس

في جت طريقتان لضم التعديلات من فرع إلى آخر: الدمج `merge` وإعادة التأسيس `rebase`. سنتعلم في هذا الفصل ما هي إعادة التأسيس، وكيف نفعلها، ولماذا هي أداة مذهلة فعلاً، ومتى لن نود استخدامها.

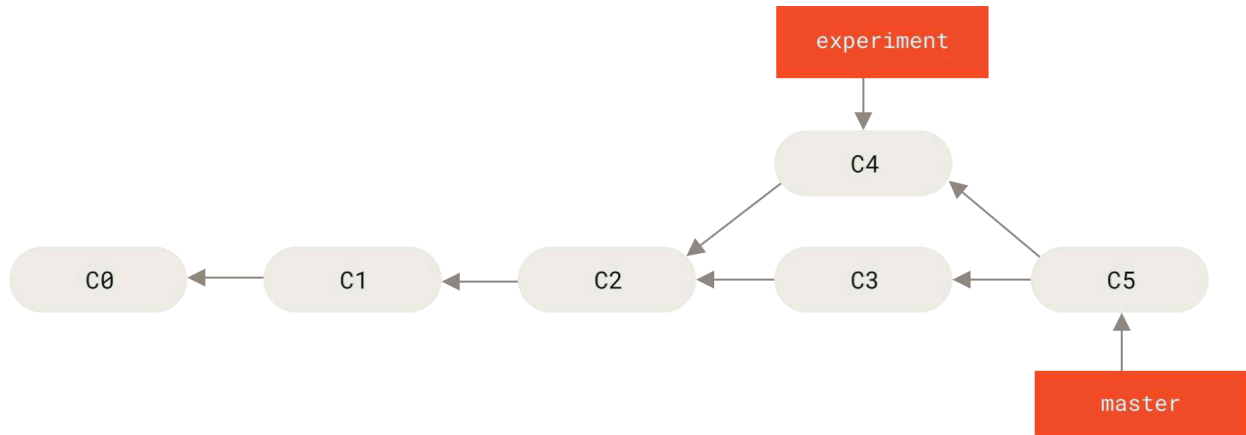
٣، ٦، ١، أسس إعادة التأسيس

إذا عدت إلى مثال من الأمثلة السابقة في فصل [أسس الدمج](#)، ستجد أن عملك افترق إلى إبداعات في فرعين مختلفين.



شكل ٣٥. تاريخ بسيط مفترق

أسهل طريقة لضم الفرعين، كما ناقشنا بالفعل، هي أمر الدمج `merge`، الذي يقوم بدمج ثلاثي بين آخر لقطتين في الفرعين (C3 و C4) وآخر سلف مشترك لهما (C2)، وينشئ لقطة جديدة (وإبداعاً).



شكل ٣٦. الدمج لضم تاريخ العمل المفترق

لكن توجد طريقة أخرى: يمكنك أخذ رُقعة التعديلات ("patch") التي صنعتها في هذا الإيداع (C4) وإعادة تطبيقها على الإيداع الآخر (C3). هذه ما نسميها «إعادة التأسيس» ("rebasing") في جت. فبأمر إعادة التأسيس `rebase` يمكنك أخذ جميع التعديلات التي أودعتها في فرع ما، وإعادة صنعها في فرع آخر.

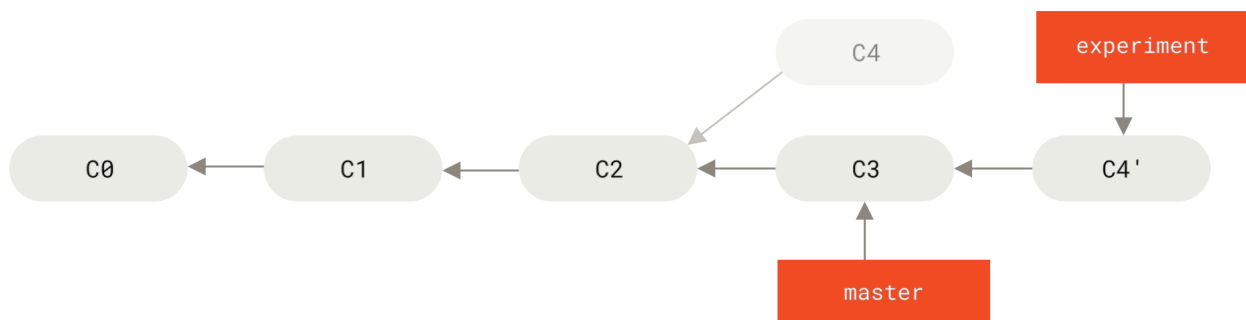
في هذا المثال سنسحب فرع `experiment`، ثم نعيد تأسيسه على الفرع الرئيس `master`.

```

$ git checkout experiment
$ git rebase master
First, rewinding head to replay your work on top of it...
Applying: added staged command
  
```

CONSOLE

تتم هذه العملية بالذهاب إلى السلف المشترك للفرعين (الفرع الحالي الذي تقف فيه وتريد إعادة تأسيسه، والفرع الذي تريد إعادة التأسيس عليه)، وحساب التعديلات التي تمت في كل إيداع في الفرع الحالي وحفظها في ملفات مؤقتة، ثم إرجاع الفرع الحالي إلى الإيداع الذي عنده الفرع الذي تريد إعادة التأسيس عليه، وأخيراً تطبيق كل تعديل عليه بالترتيب.



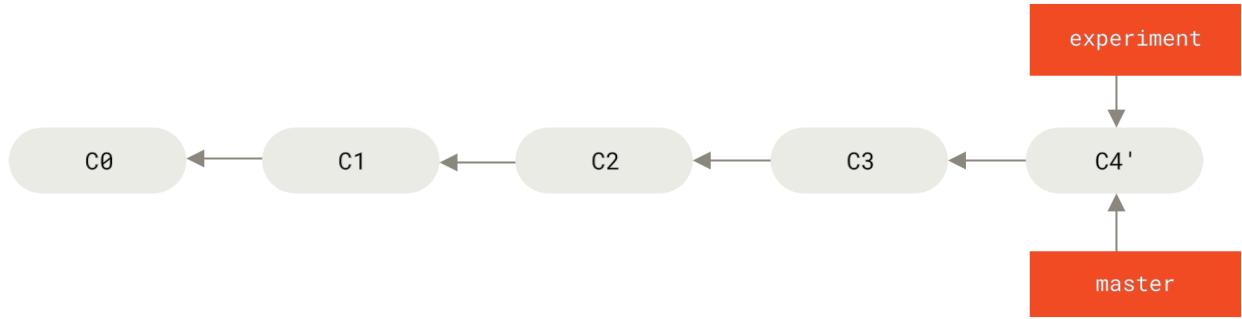
شكل ٣٧. إعادة تأسيس تعديلات C4 على C3

يمكنك الآن العودة إلى الفرع الرئيس وعمل دمج تسريع ("fast-forward").

```

$ git checkout master
$ git merge experiment
  
```

CONSOLE



شكل ٣٨. تسريع فرع master

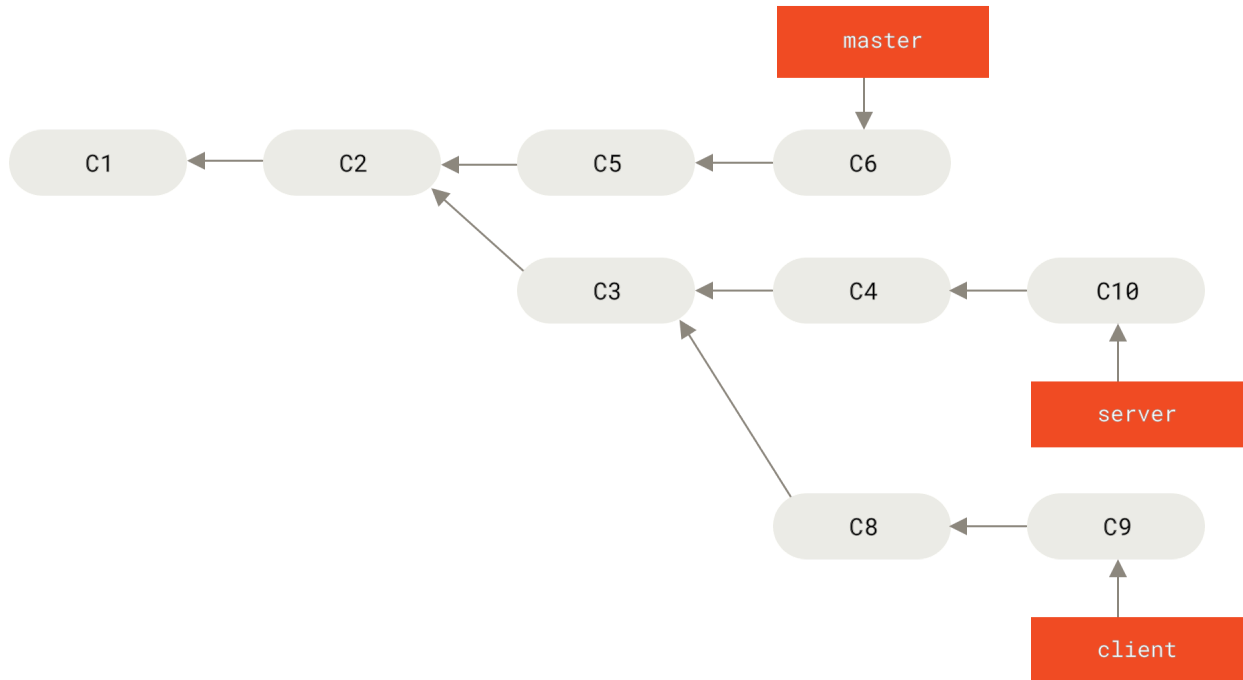
اللقطه التي يشير إليها إيداع C4' الآن مطابقة تماماً لتلك التي كان يشير إليها C5 في [مثال الدمج](#). لا فرق في الناتج النهائي، ولكن تعطينا إعادة التأسيس تاريخاً أنظف. فإذا نظرت إلى سجل فرع مُعاد تأسيسه، ستجده تاريخاً خطئاً: يبدو أن كل العمل تم على التوالي، ولو أنه في الأصل قد تم على التوازي.

غالباً ستفعل هذا لضمان أن إيداعاتك ستُطبّق بنظافة على فرع بعيد؛ مثلاً في مشروع تريد المساهمة فيه لكنك لست من المطورين القائمين عليه. فستعمل عندئذٍ في فرع، ثم تعيد تأسيسه على origin/master عندما تكون جاهزاً لتسليم رُقتك إليهم. فبهذا لن يُجهد المطورين ضمّ عملك، فما الأمر إلا تسريعاً، أو تطبيقاً نظيفاً للرقعة.

لاحظ أن اللقطه التي يشير إليها الإيداع النهائي، سواءً كانت بعد إعادة تأسيس أم بعد دمج، هي اللقطه نفسها؛ لا فرق إلا في شكل التاريخ. فإعادة التأسيس تعيد صناعة إيداعات فرع في فرع آخر بترتيبها نفسه، لكن الدمج يدمج آخر نقطتين معاً.

٣، ٢، ١، إعادة تأسيس شيقه أكثر

يمكنك كذلك جعل إعادة التأسيس تطبّق التعديلات على فرع غير «الفرع المستهدف». لنرَ مثلاً تاريخاً مثل شكل [تاريخ فيه فرع موضوع متفرع من فرع موضوع آخر](#). لقد أنشأت فرع موضوع (server) لإضافة ميزات في جزء الخادوم في مشروعك، وصنعت إيداعاً. بعدئذٍ أنشأت فرعاً من هذا الفرع للعمل في جزء العميل (client) وصنعت بضعة إيداعات. ثم في آخر الأمر عدت إلى فرع server وصنعت بضعة إيداعات أخرى.



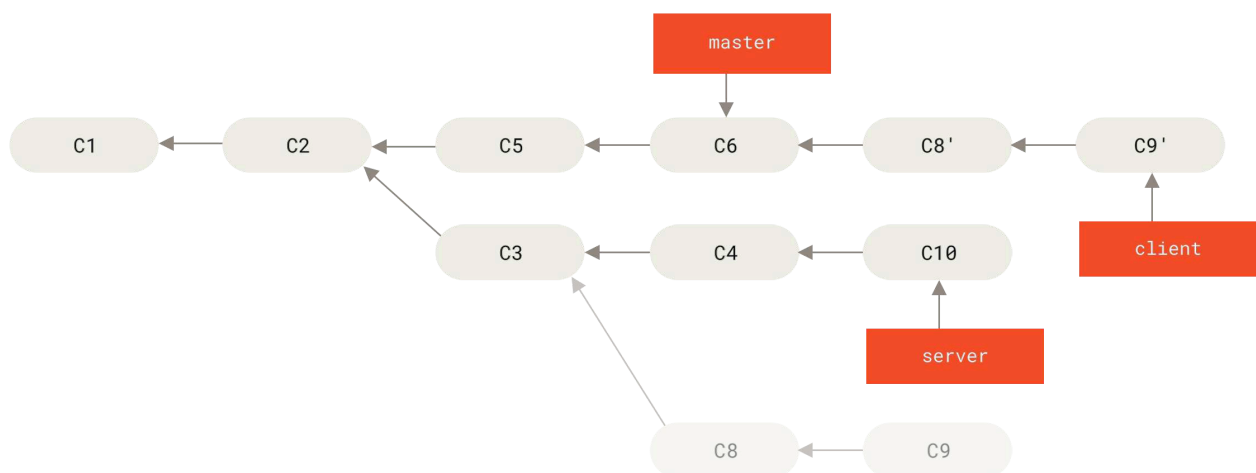
شكل ٣٩. تاريخ فيه فرع موضوع متفرع من فرع موضوع آخر

لنقل إن إيداعاتك التي في جزء العميل، رأيت أن تدمجها في الفرع الرئيس لكي تصدرها، مع الإبقاء على إيداعات جزء الخادوم حتى تختبرها أكثر. إن الإيداعات التي في client وليست في server (وهي C8 و C9) تستطيع إعادة تطبيقها على فرع master بالخيار --onto مع أمر git rebase :

```
$ git rebase --onto master server client
```

CONSOLE

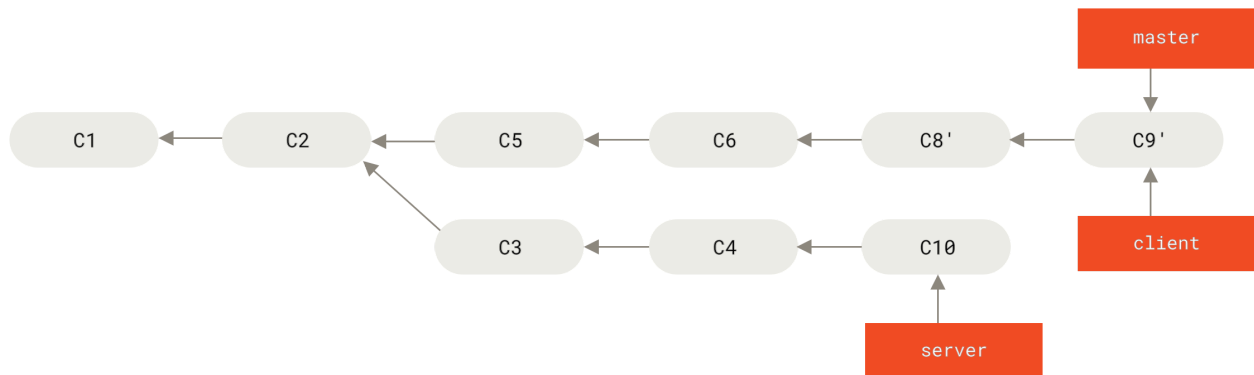
إنما يقول هذا: «احسب فروقات فرع client (التعديلات التي سُجّلت فيه) منذ أن افترق عن فرع server، ثم أعد تطبيقها في فرع client كأنه قد تفرّع من فرع master وليس من server. «صعبة قليلا، لكن النتيجة عظيمة.



شكل ٤٠. إعادة تأسيس فرع موضوع على فرع موضوع آخر

عندئذٍ يمكنك تسريع الفرع الرئيس master (انظر شكل [تسريع الفرع الرئيس لضم تعديلات فرع client](#)):

```
$ git checkout master
$ git merge client
```

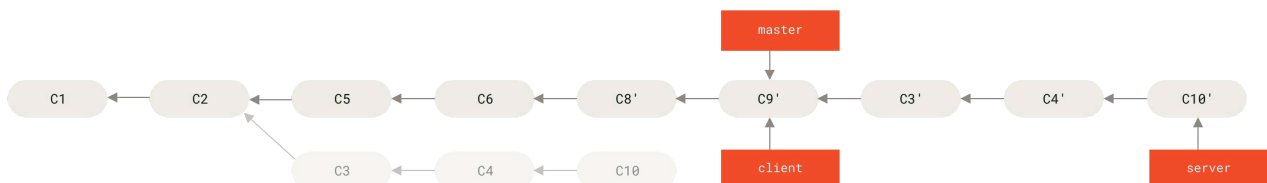


شكل ٤١. تسريع الفرع الرئيس لضم تعديلات فرع client

لنقل إنك قررت جذب فرع server كذلك. يمكنك إعادة تأسيس فرع server على الفرع الرئيس بلا حاجة إلى سحبه أولاً، بالأمر `git rebase <basebranch> <topicbranch>` (أي الفرع الأساس ثم فرع الموضوع)، وهذا يسحب لك فرع الموضوع (server في حالتنا) ويعيد تطبيق إبداعاته على الفرع الأساس (master):

```
$ git rebase master server
```

هذا يُعيد تطبيق إبداعات فرع الخادوم server على الفرع الرئيس master، كما يظهر في شكل [إعادة تأسيس فرع server على الفرع الرئيس](#).



شكل ٤٢. إعادة تأسيس فرع server على الفرع الرئيس

عندئذٍ يمكنك تسريع الفرع الأساس (master):

```
$ git checkout master
$ git merge server
```

ثم تستطيع حذف الفرعين client و server لأن كل ما فيهما قد ضُم بالفعل ولم تعد بحاجة إليهما، فيصير تاريخك في نهاية هذا كما في شكل [تاريخ الإبداعات في النهاية](#):

```
$ git branch -d client
$ git branch -d server
```



شكل ٤٣. تاريخ الإيداعات في النهاية

٣، ٦، ٣، محذورات إعادة التأسيس

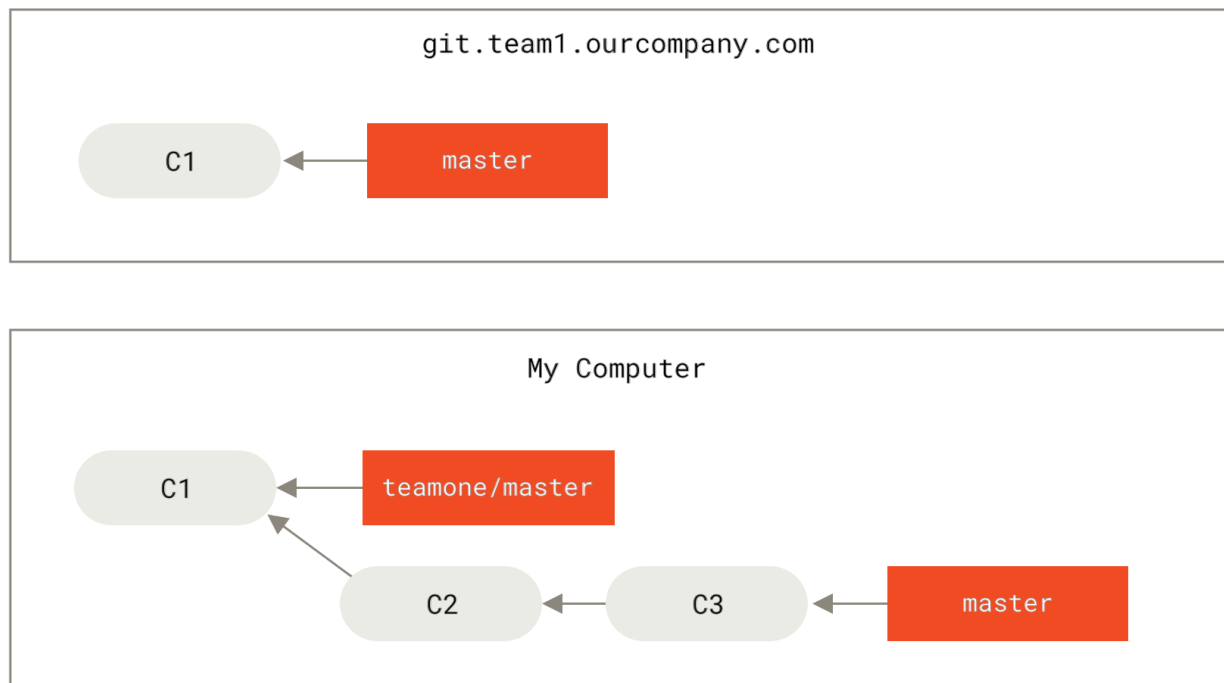
ولكن... نعيم إعادة التأسيس ليس بغير عيوب، التي يمكن اختصارها في سطر واحد:

لا تعد تأسيس إيداعات لها وجود خارج مستودعك، فربما قد بنى الناس عليها عملاً.

إذا اتبعت هذه النصيحة الإرشادية، فستكون بخير. وإن لم تفعل، فسيكرهك الناس ويحتقرك الأهل والأصحاب.

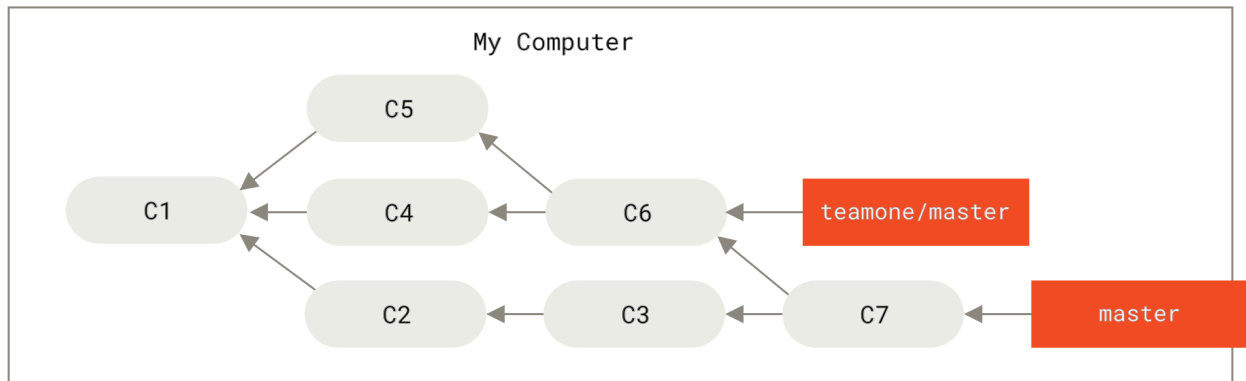
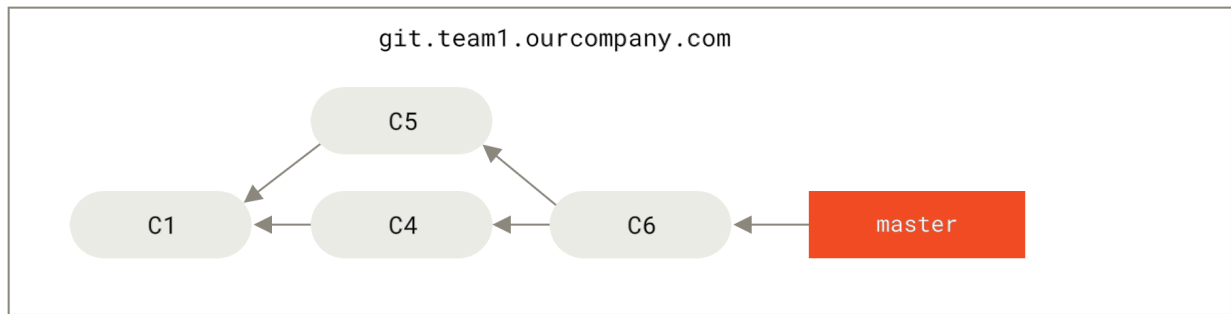
فعندما تعيد التأسيس، فإنك تهجر الإيداعات الموجودة وتصنع إيداعات جديدة شبيهة بالقديمة لكن مختلفة عنها. وإذا دفعت هذه الإيداعات إلى مستودع ما وجذبها الآخرون وبنوا عليها أعمالاً، ثم جئت فغيّرت هذه الإيداعات بأمر `git rebase` ثم دفعتها من جديد، فسيضطر زملاؤك إلى إعادة دمج أعمالهم، وستؤول الأمور إلى فوضى عندما تحاول جذب أعمالهم إلى عملك.

لنركز كيف يمكن لإعادة تأسيس عمل منشور أن تسبب مشاكل. لنقل إنك استنسخت من الخادوم المركز، ثم بنيت عليه عملاً. سيبدو تاريخ إيداعك مثل هذا:



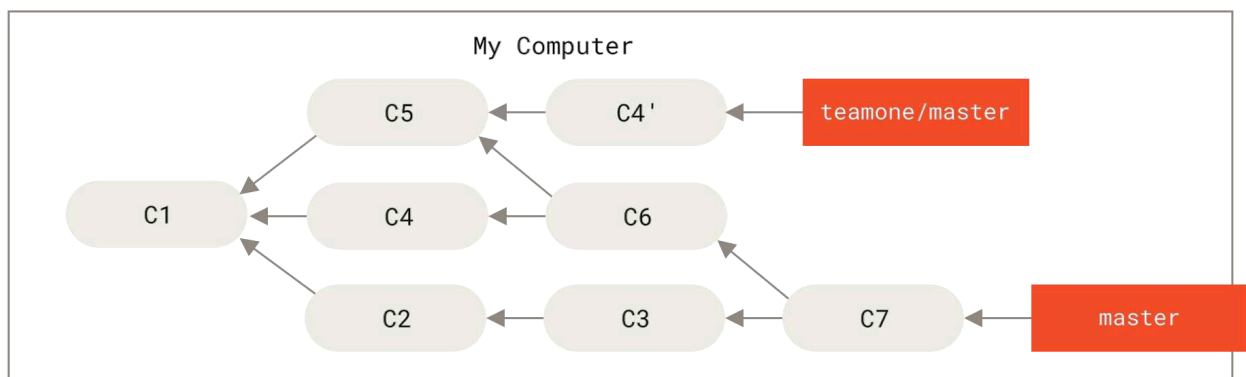
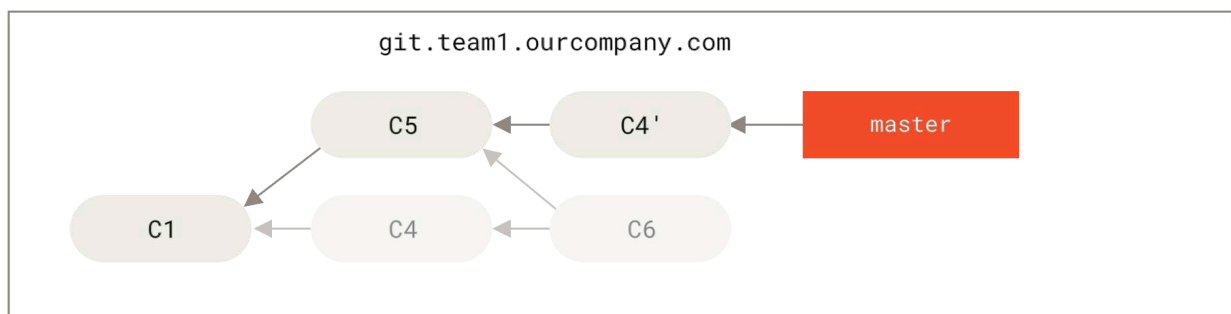
شكل ٤٤. استنسخ مستودعاً، وابن عملاً عليه

ثم جاء شخص آخر وصنع المزيد من الإيداعات، التي شملت دمجاً، ثم دفعها إلى الخادوم المركز. فاستحضرت (`fetch`) الفرع البعيد الجديد ودمجته في عملك، فصار تاريخك كالاتي:



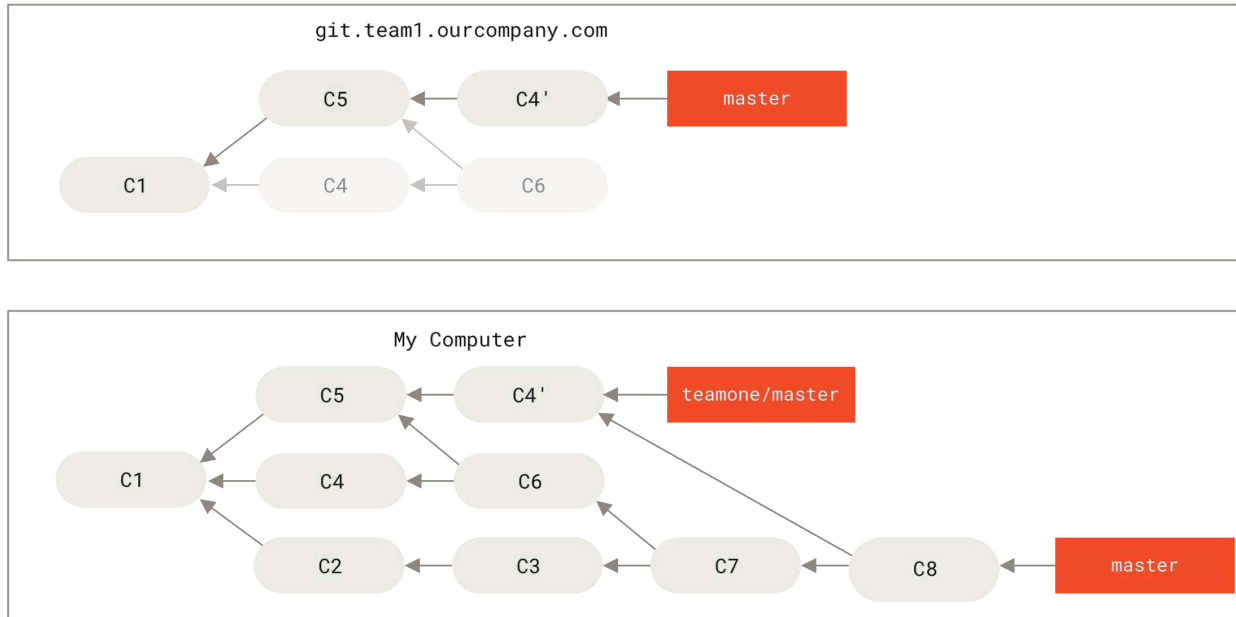
شكل ٤٥. استحضار المزيد من الإيداعات وادمجها في مستودعك

بعدئذ، قرر الذي دفع العمل المدموج أن يتراجع ويعيد تأسيس عمله بدل دمج، فدفع عَنوةً (`git push --force`) لتحرير التاريخ على الخادوم. ثم استحضرت (`fetch`) من هذا الخادوم، جالباً الإيداعات الجديدة.



شكل ٤٦. شخص يدفع إيداعات معاد تأسيسها، هاجراً بذلك الإيداعات التي بنيت عليها عملك

كلاكما الآن في مأزق. فإذا جذبت، ستصنع إيداع دمج يضم كلا التاريخين، وسيبدو مستودعك مثل هذا:



شكل ٤٧. عندما تدمج العمل نفسه مجددا في إيداع دمج جديد

فإذا نظرت في السجل `git log` عندما يصير تاريخك كهذا، فسترى إيداعين متطابقين في اسم المؤلف وتاريخ الإيداع ورسالته، فيسبب اللبس. وأضف إلى ذلك أنك إذا دفعت هذا التاريخ إلى الخادوم، فستعيد تقديم كل هذه الإيداعات المعاد تأسيسها إلى الخادوم المركز من جديد، فتسبب المزيد من اللبس لأكثر للناس. يمكننا افتراض أن المطور الآخر لا يريد الإيداعين C4 و C6 في التاريخ، ولذا أعاد تأسيسهما.

٣، ٤، ٦، أعد التأسيس عندما تعيد التأسيس

إذا وجدت نفسك في مثل هذا الموقف، فلدى جت وسائل سحرية أخرى قد تساعدك. فلو أن زميلك دفع عنوةً تعديلات تغير إيداعاتٍ بنيت عليها، فالتحدي هو أن تعرف ما هي تعديلاتك وما هي تعديلات ذاك الشخص.

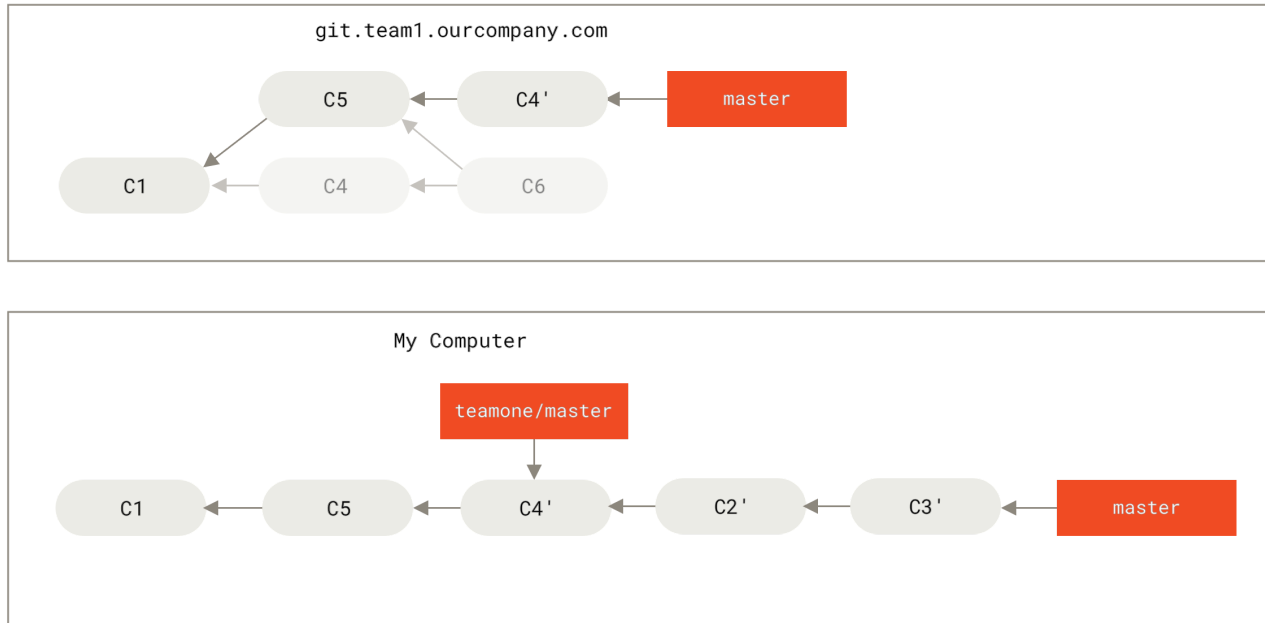
الخبر الجميل أن جت لا يحسب للإيداع وحده بصمة SHA-1، ولكن يحسبها أيضا للرقعة (الفروقات) التي صنعها ذلك الإيداع. وهذه البصمة تسمى «معرف الرقعة» («patch-id»).

فإذا جذبت إيداعات قد غُيّرت وأعدت تأسيسها على الإيداعات الجديدة من زميلك، فغالبا سينجح جت في تمييز ما هي تعديلاتك الفريدة ويطبقها على الفرع الجديد.

فمثلا في الموقف الافتراضي السابق، لو أننا أعدنا التأسيس (بالأمر `git rebase teamone/master`)، بدل الدمج عندما كنا في خطوة «شخص يدفع إيداعات معاد تأسيسها، هاجراً بذلك الإيداعات التي بنيت عليها عملك»، فإن جت سوف:

- يحدد الإيداعات التي تفرّد بها فرعنا (C7 ، C6 ، C4 ، C3 ، C2)
- يحدد ما الذي ليس بإيداعات دمج (C4 ، C3 ، C2)
- يحدد الإيداعات التي لم تتغير في الفرع المستهدف (فقط C2 و C3، لأن C4 له رقعة C4' نفسها)
- يطبق هذه الإيداعات على فرع `teamone/master`

فبدلاً مما رأينا في «عندما تدمج العمل نفسه مجدداً في إيداع دمج جديد»، سنحصل على نتيجة مثل «أعد التأسيس على عمل معاد تأسيسه ومدفوع عنوة».



شكل ٤٨. أعد التأسيس على عمل معاد تأسيسه ومدفوع عنوة

لن ينجح هذا إلا إذا كان الإيداعان C4 و C4' اللذان صنعتهما زميلك لهما الرقعة نفسها تقريباً، وإلا فلن يعرف جت عندما يعيد التأسيس أنهما متطابقين وسيضيف رقعة أخرى شبيهة بالإيداع C4 (التي غالباً ستفشل في أن تُطبَّق بنظافة، لأن تعديلاتها ستكون مطابقة بالفعل ولو جزئياً).

ويمكنك أن تختصر هذا باستعمال خيار إعادة التأسيس `--rebase` مع أمر الجذب، أي تنفيذ `git pull --rebase` بدلاً من `git pull` المجرد. أو يمكنك فعل ذلك يدوياً بالاستحضار `git fetch` ثم إعادة التأسيس، أي تنفيذ `git rebase teamone/master` في هذه الحالة.

وإذا كنت تستخدم `git pull` وتريد جعل خيار `--rebase` خياراً مفترضاً دوماً، يمكنك تفعيل قيمة التهيئة `pull.rebase`، مثلاً بالأمر `git config --global pull.rebase true`.

إذا كنت أبداً لا تعيد تأسيس إلا الإيداعات التي لم تغادر حاسوبك، فستكون بخير. وإن كنت تعيد تأسيس إيداعات قد دفعتها، ولكن لم يبن عليها أحد آخر إيداعاتٍ، فستكون بخير كذلك. أما إن كنت تعيد تأسيس إيداعات قد دفعتها إلى العالم وربما بنى عليها الناس أعمالاً، فقد تجد نفسك في ورطة مُغيظة منهكة، ثم ازدراء زملائك لك.

إن وجدت أنت أو زميلٌ لك أن ذلك ضروري يوماً ما، تأكد أن الجميع يعرفون استخدام `git pull --rebase`، لكي تحاول جعل معاناة ما بعد الحادثة أقل سوءاً ولو قليلاً.

٣، ٦، ٥، بين إعادة التأسيس والدمج

الآن وقد رأيت إعادة التأسيس والدمج عملياً، قد تتساءل أيهما أفضل. قبل أن نستطيع الإجابة عن هذا السؤال، لنرجع إلى الوراء قليلاً ونتحدث عن معنى التاريخ.

إحدى وجهتي النظر أن تاريخ إيداعات مستودعك هي **سجل لما حدث فعلاً**. أي أنها وثيقة تاريخية، ولها قيمة في ذاتها، ويجب ألا يُعبث بها. فمن هذا المنظور، يكاد يُعدّ تغيير التاريخ استهزاءً ومسبةً، لأنك تكذب بشأن ما حدث فعلاً. فمانا إنذا لو كانت لدينا فوضى من إيداعات الدمج؟ هكذا جرت الأمور، ويجب أن يحتفظ بها المستودع من أجل الأجيال القادمة.

وجهة النظر المقابلة هي أن تاريخ الإيداعات هو **قصة صناعة مشروعك**. ولأنك لا تنشر المسودة الأولى من كتاب، فلم إنذا تنشر عملاً أشعث أغبر؟ ففي أثناء عملك على مشروع، قد تحتاج سجلاً لجميع عثرائك وطرقك المسدودة. ولكن عندما يحين وقت إظهار عملك إلى العالم، فقد تود أن تحكي قصةً متماسكة عن كيفية الوصول من «أ» إلى «ب». ولذلك يستخدم أصحاب هذا المذهب أدوات مثل إعادة التأسيس ومعالجة الفروع لتحرير إيداعاتهم قبل دمجها في الفرع الرئيس، يستخدمون rebase و filter-branch ليحكون القصة بالطريقة الأقرب للقراء في المستقبل.

لنعد الآن إلى السؤال عن التفضيل بين الدمج وإعادة التأسيس: لعلك وجدت أنه أعقد من أن تكون له إجابة يسيرة. فإن جت أداة قوية، ويتيح لك فعل الكثير بتاريخ مستودعك، ولكن كل فريق وكل مشروع هو حالة خاصة. الآن وقد علمت الأسلوبين وطريقة عملهما، عليك أن تقرر بنفسك ما أفضلهما لحالتك الخاصة.

ويمكنك الجمع بين ميزات كليهما: أعد تأسيس التعديلات المحلية قبل دفعها حتى تنظف عملك، ولكن لا تعد أبداً تأسيس أي شيء دفعته إلى مستودع ما.

٣، ٧، الخلاصة

تناولنا أسس التفريع والدمج في جت. ينبغي أن يسهل عليك الآن إنشاء فروع جديدة والانتقال إليها والتنقل بين الفروع ودمج فروع محلية معاً. ستقدر أيضاً على مشاركة فروعك بدفعها إلى مستودع مشترك، وعلى العمل مع آخرين على فروع مشتركة، وعلى إعادة تأسيس فروعك قبل مشاركتها. التالي: سنتحدث عما تحتاج لتشغيل خادمك الخاص بك لاستضافة مستودعات جت.

الباب الرابع: جت على الخادوم

تستطيع الآن فعل معظم مهامك اليومية التي تستخدم فيها جت. ولكن عليك استعمال مستودع بعيد لأي عمل تعاوني به. ومع أنك نظريا تستطيع دفع التعديلات إلى مستودعات الآخرين الشخصية المحلية والجذب منها، إلا أن فعل هذا متجنب لأن من السهل التسبب في خلط الأمور ومن يعمل على ماذا، إن لم تكن حذرا. وكذلك إذا أردت أن يصل زملاؤك إلى مستودعك حتى عندما يكون حاسوبك مغلقاً أو غير متصل بالشبكة؛ فكثيرا ما يفيد وجود مستودع مشترك مضمون. لذا فالمستحب للتعاون مع غيرك أن تعدّ مستودعاً وسيطاً يستطيع كلاكما الوصول إليه، وتجذبا منه وتدفعا إليه.

تشغيل خادوم جت عملية يسيرة. أولاً تحدد الموافيق (البروتوكولات) التي تريد منه دعمها. وسيتناول الفصل الأول من هذا الباب الموافيق المتاحة ومزايا وعيوب كل منها. ثم تشرح الفصول التالية بعض الترتيبات المعتادة العاملة بهذه الموافيق وكيف تشغل خادومك بها. وأخيرا سنرى بعض الخيارات المستضافة، إذا لم تمنع استضافة مشروعاتك البرمجية على خواديم الآخرين وكنت لا تريد المعاناة بإعداد خادومك الخاص ورعايته.

إن لم تكن مهتماً بتشغيل خادومك الخاص، فيمكنك تخطي هذه الفصول والانتقال إلى الفصل الأخير من هذا الباب، الذي يريك بعض الخيارات لإعداد حساب مستضاف، ثم انتقل إلى الباب التالي، الذي فيه التفاصيل المختلفة للعمل في بيئة إدارة متوزعة للنسخ.

المستودع البعيد هو عموما مستودع مجلد، أي مستودع جت ليس له مجلد عمل. ولأن المستودع لا يُستعمل إلا ملتقى للتعاون، فلا داعي إلى سحب لقطة منه على الحاسوب؛ إنما هو لبيانات جت وحدها. أي بأيسر الكلمات: المستودع المجرد هو محتويات مجلد `git`. الخاص بمشروعك، ولا شيء غير ذلك.

٤، ا، الموافيق (البروتوكولات)

يتيح جت أربعة موافيق مختلفة لنقل البيانات: المحلي، و HTTP، و SSH (أي Secure Shell)، و Git. سنناقش ما هم وما الظروف التي فيها ستود (أو لا تود) استعمالهم.

٤، ا، ا، الميفاق المحلي

الأكثر بدائية هو الميفاق المحلي ("Local protocol")، الذي يكون المستودع البعيد فيه هو مجلد آخر على الحاسوب نفسه. ويستخدم غالبا إذا كان كل من في فريقك لديه وصول إلى نظام ملفات مشترك مثل NFS [الرابط: https://en.wikipedia.org/wiki/Network_File_System](https://en.wikipedia.org/wiki/Network_File_System) مضموم (mounted)، أو في الحالة الأندر أن يكون كل شخص مستخدما لحاسوب واحد. ولكن تلك الأخيرة ليست حالة مثالية لأن وقتئذ ستبيت كل مشروعاتك البرمجية على جهاز واحد، وهذا يزيد احتمال فقد البيانات فقدا كارثيا.

إذا كان لديك نظام ملفات مشترك مضموم، فيمكنك إنَّ استنساخ مستودع محلي مكوَّن من ملفات عادية، والدفع إليه، والجذب منه. فلاستنساخ مستودع مثل هذا، أو لإضافته مستودعاً بعيداً في مشروع موجود بالفعل، فاستعمل مسار المستودع كأنه رابط URL. مثلاً، لاستنساخ مستودع محلي، يمكنك فعل شيء مثل هذا:

```
$ git clone /srv/git/project.git
```

CONSOLE

أو مثل هذا:

```
$ git clone file:///srv/git/project.git
```

CONSOLE

ولكن تصرف جت يختلف قليلاً إذا بدأت العنوان بالميفاق `file://` صريحاً. فإذا كتبت المسار فقط، فإن جت يحاول صنع روابط صلبة (hardlinks) أو نسخ الملفات التي يحتاجها مباشرةً. أما إذا كتبت `file://`، فإن جت يتفَّذ العمليات التي يستعملها في المعتاد لنقل الملفات عبر الشبكة، ويكون هذا في الغالب أقل كتيماً في الكفاءة. السبب الأساسي لكتابة البادئة `file://` هي إذا كنت تريد نسخة نظيفة من المستودع بغير الإشارات أو الكائنات الإضافية؛ غالباً ما يكون ذلك بعد الاستيراد من نظام إدارة نسخ آخر أو أمر شبيهه (انظر باب [تداخل جت لعمليات الصيانة](#)). سنستعمل المسار العادي هنا لأنه أسرع في أغلب الأحيان.

لإضافة مستودع محلي إلى مشروع جت موجود، نفِّذ أمرًا مثل هذا:

```
$ git remote add local_proj /srv/git/project.git
```

CONSOLE

عندئذٍ يمكنك الدفع إلى ذلك البعيد والجذب منه بالاسم المختصر الجديد `local_proj` كما كنت تفعل تماماً عبر الشبكة.

المزايا

مزايا المستودعات «الملفاتية» أنها سهلة وتستعمل تصاريح الملفات واتصال الشبكة الموجودين فعلاً. وإذا كان لديك نظام ملفات مشترك يصل إليه جميع فريقك، فإعداد مستودع عملية سهلة جداً: تضع نسخة المستودع المجرد في مكانٍ يستطيع الجميع الوصول إليه، وتنضبط أنونات القراءة والتحرير كما تفعل لأي مجلد مشترك آخر. سنناقش كيفية تصدير نسخة مستودع مجردة لهذا الهدف في فصل [تنشيط جت على خادم](#).

هذا أيضاً خيار ظريف وسريع لجلب عمل شخص آخر من مستودعه. فإذا كنت وزميلك تعملان على مشروع واحد ويريدك أن تسحب شيئاً ما، فتنفذ أمر مثل `git pull /home/badr/project` أسهل كثيراً في الغالب من أن يدفع عمله إلى خادم بعيد ثم تستحضره أنت بعد ذلك.

العيوب

عيوب هذه الطريقة هي أن الوصول المشترك غالباً ما يكون أصعب كثيراً من الوصول الشبكي العادي، في إعداداته وفي الوصول إليه من أماكن مختلفة. فإذا أردت أن تدفع من حاسوبك المحمول عندما تكون في المنزل، فستحتاج إلى ضم القرص البعيد، وهذا قد يكون صعباً وبطيئاً مقارنةً بوصول عبر الشبكة.

من المهم ذكر أن هذا ليس بالضرورة الخيار الأسرع إذا كنت تستعمل نوعاً من الضم (mount) المشترك. فالمستودع المحلي لا يكون سريعاً إلا إذا كان لديك وصول سريع إلى البيانات. فإن مستودع على NFS يكون في الغالب أبطأ من مستودع عبر SSH على الخادوم نفسه، بفرض أن جت يعمل من الأقراص المحلية في كل نظام.

وأخيراً، لا يحمي هذا الميثاق المستودع من الإتلاف غير المقصود. فكل مستخدم لديه وصول صَدفي كامل للمجلد «البعيد»، فلا شيء يمنعه من تعديل ملفات جت الداخلية أو إلالتها وتخريب المستودع.

٤، ١، ٢، ميثاق HTTP

يستطيع جت التواصل عبر HTTP بطريقتين مختلفتين. لم يعرف جت قبل النسخة 1.6.6 منه إلا واحدة منهما، وكانت ساذجة جداً وعموماً للقراءة فقط. ولكن في نسخة 1.6.6، جاء ميثاق جديد جعل جت يتفاوض بذكاء في شأن نقل البيانات، بطريقة تشبه ما كان يفعله عبر SSH. وصار ميثاق HTTP الجديد هذا في الأعوام الأخيرة أشهر كثيراً لأنه أيسر للمستخدم وأذكى في تواصله. فانتشرت تسمية النسخة الجديدة «ميثاق HTTP الذكي» (Smart HTTP)، والقيمة «ميثاق HTTP البليد» (Dumb HTTP). وستتناول الميثاق الذكي أولاً.

ميثاق HTTP الذكي

يعمل الميثاق الذكي بطريقة كبيرة الشبه بميثاقَي SSH و Git، لكنه يعمل عبر منافذ HTTPS (الآمن) المعيارية ويستعمل آليات استيثاق HTTP المتنوعة، ما يعني أنه غالباً أسهل للمستخدم من شيء مثل SSH، لأنك مثلاً تستطيع استعمال اسم المستخدم وكلمة المرور بدلاً من الاضطرار إلى إعداد مفاتيح SSH.

لعله أشهر طريقة لاستعمال جت اليوم، لأن من الممكن إعدادهِ لينتج المستودع بغير هُوية مثل ميثاق Git، وكذلك لينتج الدفع إليه بهُوية وتعمية مثل ميثاق SSH. فبدلاً من الاضطرار إلى إعداد رابطين (URL) مختلفين للعمليات، يمكنك الآن استعمال رابط واحد لكليهما. وإذا حاولت الدفع فطلبَ المستودعُ الاستيثاقَ منك (وهو ما يجب أن يحدث في المعتاد)، فسيسألك الخادوم عن اسم المستخدم وكلمة المرور. والأمر نفسه للقراءة وحدها.

وفي الواقع، في خدمات مثل جت هب، الرابط الذي تستعمله لتصفح المستودع عبر المتصفح (مثلاً <https://github.com> [/schacon/simblegit/](https://schacon/simblegit/)) يمكنك استعماله هو نفسه للاستنساخ، وكذلك للدفع إليه إذا كان لديك الإذن.

ميثاق HTTP البليد

إن لم يستجب الخادوم لطلب ميثاق HTTP الذكي من جت، فسيحاول عميل جت استعمال ميثاق HTTP البليد الأيسر. يتوقع الميثاق البليد أن يُقدّم له مستودع جت المجرد كما تُقدّم الملفات العادية من خادم الويب. فجمال الميثاق البليد هو سهولة إعدادهِ. فليس عليك إلا وضع مستودع جت مجرد في مكانٍ ما في جذر مستندات HTTP، وإعداد خطاف post-update، وتكون قد أتممت المهمة (انظر فصل [خطاطيف جت](#)). عندئذٍ يستطيع استنساخ المستودع كل من يستطيع الوصول إلى خادم الويب الذي وضعته عليه. فللسماح بقراءة مستودعك عبر HTTP، افعل شيئاً مثل هذا:

```
$ cd /var/www/htdocs/
$ git clone --bare /path/to/git_project gitproject.git
$ cd gitproject.git
$ mv hooks/post-update.sample hooks/post-update
$ chmod a+x hooks/post-update
```

CONSOLE

هذا كل ما في الأمر. وخطاف post-update الذي يأتي مبدئياً مع جت ينفذ الأمر المناسب (git update-server-info) لجعل الاستحضار والاستنساخ عبر HTTP يعمل بطريقة صحيحة. ويعمل هذا الأمر عندما تدفع إلى المستودع (عبر SSH مثلاً). عندئذٍ يستطيع الآخرون الاستنساخ بمثل هذا الأمر:

```
$ git clone https://example.com/gitproject.git
```

CONSOLE

نستعمل في حالتنا هذه مسار /var/www/htdocs وهو الشائع مع خواديم Apache. لكن يمكنك استعمال أي خادم وب سكوبي (استاتيكي)؛ ليس عليك سوى وضع المستودع المجرد في مساره، فبيانات جت تُقدّم لطالبيها مثل أي ملفات ساكنة عادية (انظر باب [دواخل جت](#) للتفاصيل عن كيف تُقدّم بالتحديد).

وفي العموم ستختار إما تشغيل خادم HTTP ذكي للقراءة والتحرير، وإما جعل الملفات متاحة للقراءة فقط بالطريقة البليدة. فمن النادر تشغيل نوعي الخدمتين معاً.

المزايا

سنركز على مزايا النسخة الذكية من ميفاق HTTP.

بساطة الرابط الواحد للوصول بنوعيه تسهّل الأمور كثيراً على المستخدم النهائي، وكذلك عدم الاستيثاق إلا عند الحاجة. والاستيثاق باسم مستخدم وكلمة مرور لهو أيضاً مزية كبيرة فيه على SSH، فلا يحتاج المستخدمون أن يولدوا مفاتيح SSH محلياً ثم يرفعوا مفاتيحهم العمومية إلى الخادوم ليسمح لهم بالتواصل. وإن هذه لمزية عظيمة في قابلية الاستخدام، للمستخدمين الأقل حنكة، وللمستخدمين على أنظمة عليها SSH غير موجود أو غير شائع. وهو أيضاً كفؤ وسريع جداً، مثل SSH.

ويمكنك كذلك إتاحة مستودعاتك للقراءة فقط عبر HTTPS الآمن، ما يعني أن بإمكانك تعمية المحتوى خلال نقله، أو حتى جعل العملاء يستعملون شهادات SSL موقّعة مخصصة.

شيء جميل آخر هو أن HTTP و HTTPS مستعملان بكثرة حتى إن جدران الحماية (firewalls) الخاصة بالمؤسسات تُضبط في الغالب لتتيح مرورهما عبر منافذها.

العيوب

قد يكون إعداد جت عبر HTTPS أصعب قليلاً من SSH على بعض الخواديم. ولكن غير ذلك، فلا تكاد توجد مزية لأحد الموافيق الأخرى على HTTP الذكي في تقديم محتوى جت.

فإذا كنت تستعمل HTTP للدفع المستوثق، فإعطاء اسم المستخدم وكلمة المرور قد يكون في بعض الأحيان أعقد قليلاً من استعمال المفاتيح عبر SSH. ولكن توجد عدة أدوات للاحتفاظ بهما يمكنك استعمالها لجعل العملية أخف كثيراً، مثل Keychain Access على نظام ماك أو إس و Credential Manager على نظام ويندوز. اقرأ فصل [تخزين الاسم وكلمة المرور](#) لترى كيف تضبط نظامك للاحتفاظ بكلمة مرور HTTP آمن على نظامك.

٤، ١، ٣، ميفاق SSH

النقل عبر SSH شائع عند استضافة جت ذاتيا. هذا لأن معظم الخواديم مُعدّة فعلا لَقَبول الوصول عبره، وإن لم تكن معدة لإعدادها سهل. أيضا SSH هو ميفاق شبكي استيثاق، ويوجد في كل مكان، وسهل عموما إعدادة واستعماله.

لاستنساخ مستودع جت عبر SSH، استعمل رابط `ssh://` مثل:

```
$ git clone ssh://[user@]server/project.git
```

CONSOLE

أو استعمل الصيغة المختصرة شبيهة `scp` لميفاق SSH:

```
$ git clone [user@]server:project.git
```

CONSOLE

وفي كلتا الحالتين، إن لم تحدد اسم المستخدم الاختياري، فسيعدّ جت مطابقاً لاسم مستخدم النظام الحالي على حاسوبك.

المزايا

مزايا SSH عديدة. أولاً، سهل الإعداد نسبياً؛ فعفاريته (daemons) منتشرة، ولأكثر مديري الشبكات خبرة فيها، وأغلب أنظمة التشغيل تأتي بها معدّة أو بأدوات لإدراجها. ثانياً، التواصل عبره آمن؛ فكل البيانات تُنقل بعد التعمية والاستيثاق. وأخيراً، إنه كفؤ، فيجعل البيانات ذات أقل حجم ممكن قبل نقلها، مثل موافيق HTTPS و Git والمحلي.

العيوب

نقيصة SSH أنه لا يدعم وصول المجهولين إلى مستودعك. فإذا كنت تستعمل SSH، فليجب على الناس الحصول على وصول SSH لجهازك، حتى لرؤيتها فحسب، وهذا لا يجعل SSH مستحسنًا للمشروعات المفتوحة، التي يود الناس أن يستنسخواها للنظر فيها فقط. أما إذا كنت لا تستعمله إلا داخل شبكة مؤسستك، فقد يكون SSH هو الميفاق الوحيد الذي نحتاج إلى التعامل معه. وإذا وددت أن تأذن للمجهولين بالاطلاع فقط على مشروعاتك ولكن تريد SSH أيضاً، فعليك إعداد SSH للدفع ثم إعداد ميفاق آخر للآخرين حتى يستحضروا منه.

٤، ١، ٤، ميفاق Git

أخيراً، لدينا ميفاق Git. هذا عفريت (daemon) مخصص مرفق مع جت، ويستمع على منفذ مخصص (9418) ليتيح خدمة شبيهة بميفاق SSH، لكن بغير استيثاق أو تعمية. وعليك إنشاء ملف اسمه `git-daemon-export-ok` في مستودعك لتجعل جت يقدّمه عبر ميفاق Git؛ فالعفريت لن يقدّم مستودعاً ليس فيه هذا الملف، ولكن غير ذلك لا يوجد أمان. إما أن يكون المستودع متاحاً للجميع لاستنساخه، وإما لا يكون. هذا يعني أنه عموماً لا يمكن الدفع عبر هذا الميفاق. يمكنك تفعيل إذن الدفع، ولكن لانعدام الاستيثاق، فكل من يجد رابط مشروعك على الشبكة (الإنترنت)، يستطيع الدفع إليه. يكفي أن نقول أن هذا نادر.

المزايا

ميفاق Git غالباً ما يكون أسرع ميفاق نقل شبكي متاح. فإن كنت تخدم كمًّا كبيراً من النقل الشبكي لمشروع عمومي، أو تقدّم مشروعاً كبيراً لا يحتاج استيثاق المستخدمين للاطلاع عليه، فغالبا ستودّ إعداد عفريت جت ليقدمه. فهو يستعمل الآلية نفسها التي يستعملها SSH لنقل البيانات، لكن بغير عبء التعمية والاستيثاق.

العيوب

بسبب عدم وجود TLS أو أي نوع من التعمية، فإن الاستنساخ عبر `git://` قد يسبب ثغرة تنفيذ تعليمات برمجية عشوائية (arbitrary code execution)، لذا ينبغي تجنبه إلا إن كنت تعي جيداً ماذا تفعل.

- عندما تنفذ `git clone git://example.com/project.git`، فإن مخترقا يتحكم في جهاز التوجيه (router) الخاص بك سيستطيع تعديل المستودع الذي استنسخته للتو، مثلاً يضيف تعليمات برمجية خبيثة فيه. وعندما تصرّف (compile) أو تشغل ما في هذا المستودع الذي استنسخته للتو، فستنفذ تلك التعليمات البرمجية الخبيثة. ابتعد عن تنفيذ `git clone http://example.com/project.git` للسبب نفسه (أي ميفاق HTTP غير الآمن).

- تنفيذ `git clone https://example.com/project.git` (الآمن) لا يعاني من تلك العلة (إلا إن استطاع المخترق أن يحضر شهادة TLS للنطاق الذي تتصل به، `example.com` في هذا المثال). تنفيذ `git clone git@example.com:project.git` (بميفاق SSH) لا يعاني من تلك العلة أيضاً، إلا إن كنت قبلت بصمة مفتاح SSH خاطئة.

وكذلك لا يدعم الاستيثاق، فأى أحد سيستطيع استنساخ المستودع (ولكن غالباً هذا ما تريده بالتحديد). ولعله أيضاً من أصعب الموافيق في الإعداد. فيجب أن يشغل عفريته الخاص، الذي يحتاج تهيئة `xinetd` أو `systemd` أو ما يشبههما. وليس هذا يسيراً دائماً. ويحتاج أيضاً وصولاً عبر جدار الحماية (firewall) إلى منفذ 9418، وهذا ليس منفذاً معتاداً تسمح به دائماً جدران الحماية الخاصة بالمؤسسات؛ فخلف تلك الجدران الكبيرة، هذا المنفذ الغامض غالباً ما يحظر.

٤، ٢، تثبيت جت على خادمك

سنتناول الآن إعداد خدمة جت لتشغيل هذه الموافيق على خادمك.

سنعرض هنا الأوامر والخطوات اللازمة لتثبيت أساسي مبسط على خادم لينكس، لكن ممكن أيضاً تشغيل هذه الخدمات على ماك أو إس أو ويندوز. وفي الحقيقة إن إعداد خادم إنتاجي ضمن بنية تحتية، يقيناً سيشمل اختلافات في تدابير الأمان أو أدوات نظام التشغيل، ولكننا نرجو أن يعطيك شرحنا رؤية عامة لما ينطوي عليه الأمر.



حتى تعد أي خادم جت، عليك أولاً تصدير مستودع موجود إلى مستودع جديد مجرد. والمستودع المجرد هو مستودع ليس فيه مجلد عمل. هذا في المعتاد عملية سهلة. فاستنساخ مستودع لإنشاء مستودع جديد مجرد، نفذ أمر الاستنساخ بالخيار `--bare` («مجرد»): والعرف أن أسماء مجلدات المستودعات المجردة تنتهي باللاحقة `.git`، مثل هذا:

```
$ git clone --bare my_project my_project.git
Cloning into bare repository 'my_project.git'...
done.
```

سيكون لديك الآن نسخة من بيانات مجلد جت في مجلد `my_project.git`.

هذا يكافئ تقريباً شيئاً مثل هذا:

```
$ cp -Rf my_project/.git my_project.git
```

توجد بعض الاختلافات الطفيفة في ملف التهيئة ("config")، ولكن لغرضنا اليوم فيكاداً يتطابقان. فهذا الأخير يأخذ مستودع جت وحده بغير مجلد عمله وينشئ مجلدًا مخصصًا له وحده.

٤، ٢، ١، وضع المستودع المجرد على خادم

الآن وقد صار لديك نسخة مجردة من مستودعك، فلا تحتاج سوى وضعه على الخادوم وضبط الموافيق (البروتوكولات). لنقل إنك أعددت خادمًا يسمى `git.example.com`، ولديك وصول SSH له، وتريد تخزين كل مستودعات جت الخاصة بك في مجلد `/srv/git` فيه. إذا كان مجلد `/srv/git` موجودًا على هذا الخادوم، فيمكنك إعداد مستودعك الجديد بنسخ مستودعك المجرد إليه:

```
$ scp -r my_project.git user@git.example.com:/srv/git
```

يستطيع الآن المستخدمون الذين لديهم إذن قراءة مجلد `/srv/git` عبر SSH أن يستنسخوا مستودعك بالأمر:

```
$ git clone user@git.example.com:/srv/git/my_project.git
```

وإذا كان لمستخدم إذن تحرير هذا المجلد، ودخل عبر SSH إلى الخادوم، فسيستطيع الدفع إليه متى شاء.

وسيضيف جت إذن التحرير للمجموعة إلى المستودع بطريقة صحيحة إذا استخدمت خيار `--shared` («مشارك») مع أمر الابتداء `git init`. لاحظ أن هذا لن يُتلف أي إبداعات أو إشارات أو أي شيء آخر.

```
$ ssh user@git.example.com
$ cd /srv/git/my_project.git
$ git init --bare --shared
```

قد رأيت كم هو سهل إنشاء نسخة مجردة من مستودع جت ووضعها على خادم عندكم وصول SSH إليه. فيمكنكم الآن التعاون في مشروع واحد.

مهمٌ ملاحظة أنك حقا لا تحتاج غير هذا لتشغيل خادم جت نافع يصل إليه العديون: تنشئ حسابات SSH، وتضع مستودعًا مجردًا في مكانٍ للجميع إذن قراءته وتحريره. وتمت العملية بنجاح؛ لا شيء آخر مطلوب.

سنرى في الفصول القليلة التالية كيف يمكنك التوسع لترتيبات أكثر تعقيداً. وسيشمل هذا عدم الاضطرار إلى إنشاء حساب مستخدم لكل مستخدم، وإضافة إذن القراءة للجميع إلى المستودعات، وإعداد واجهات الوب، والمزيد. لكن تذكر أن للتعاون مع بعض الناس على مشروع خصوصي، كل ما تحتاجه هو خادم SSH ومستودع مجرد.

٤، ٢، ٢، الترتيبات الصغيرة

إذا كانت شركتك صغيرة أو كنت تجرب جت في مؤسسة ولستم إلا عدداً قليلاً من المطورين، فقد تكون الأمور يسيرة عليك. فأحد أعقد مناحي إعداد خادم جت هو إدارة المستخدمين. فإن أردت لبعض المستودعات ألا يقرأها إلا مستخدمون معينون وألا يحررها إلا جماعة أخرى، فقد تجد ترتيب الأنونات والوصول أصعب.

الوصول عبر SSH

إذا كان لديك خادمٌ يستطيع جميع المطورين الوصول إليه عبر SSH، فمن الأسهل عموماً إعداد مستودعك الأول عليه، لأنك بهذا لن تحتاج إلى عمل شيء تقريباً (كما شرحنا في الفصل السابق). وإذا احتجت إلى أنونات وصول أعقد لمستودعاتك، فيمكنك تحقيقها بأنونات نظام الملفات العادية الخاص بنظام تشغيل خادمك.

وإذا أردت وضع مستودعاتك على خادم ليس فيه حساب لكل مطور يحتاج إذن التحرير في فريقك، فعليك إعداد وصول SSH لكلٍ منهم. إذا كنت تريد فعل هذا على خادم لديك، فسنفترض أن لديك بالفعل خادم SSH مثبت عليه، وأنه وسيلتك للتواصل مع الخادم الجهاز.

توجد أكثر من طريقة لإعطاء إذن الوصول لكل واحد في فريقك. الأولى هي إعداد حساب لكل واحد، وهي عملية سهلة لكن قد تكون مرهقة. فربما لا تريد تنفيذ `adduser` (أو بديله المحتمل `useradd`) والاضطرار إلى ضبط كلمة مرور مؤقتة لكل مستخدم جديد.

الطريقة الثانية هي إنشاء حساب مستخدم `git` واحد على الجهاز، وطلب مفتاح SSH العمومي من كل مستخدم تريد إعطاءه إذن التحرير، ثم إضافة هذه المفاتيح إلى ملف `~/.ssh/authorized_keys` في حساب `git` الجديد هذا. وعندئذٍ سيستطيع كل شخص الوصول إلى ذلك الجهاز عبر حساب `git`. وهذا لا يؤثر على بيانات الإيداعات بأي شكل؛ فحساب مستخدم SSH الذي تتصل عبره لا يؤثر على الإيداعات التي تسجلها.

طريقة أخرى هي أن يستوثق خادم SSH الخاص بك من خادم LDAP أو مصدر استيثاق مركزي آخر أعدته. وما دام لكل مستخدم وصول صدقي إلى الجهاز، فأى آلية استيثاق SSH تخطر على بالك ستعمل.

٤، ٣، توليد مفتاح SSH عمومي لك

العديد من خواديم جت تستوثق المستخدمين بمفاتيح SSH العمومية. فعلى كل مستخدم توليد مفتاح إن لم يكن لديه مفتاح فعلاً. تتشابه هذه العملية عبر أنظمة التشغيل المختلفة. أولاً عليك التحقق أنك لا تملك مفتاحاً فعلاً. المكان المبدئي لتخزين مفاتيح SSH الخاصة بالمستخدم هو مجلد `~/.ssh`. فانظر فيه لتعرف إذا كان لديك مفتاحٌ فعلاً:

```
$ cd ~/.ssh
$ ls
```

CONSOLE

```
authorized_keys2 id_dsa known_hosts
config id_dsa.pub
```

عليك البحث عن ملفين اسم أحدهما يشبه `id_dsa` أو `id_rsa`، والآخر هو الاسم نفسه لكن بالامتداد `.pub`. فملف `.pub` هو مفتاحك العمومي، والملف الآخر هو نظيره الخاص. وإذا لم يكن لديك هذين الملفين (أو لم يكن لديك مجلد `.ssh` من الأساس)، فيمكنك إنشاءهما بتشغيل برنامج يسمى `ssh-keygen` («مولّد مفاتيح SSH»)، الذي يأتي مع حزمة SSH على أنظمة لينكس وماك أو إس ومع Git for Windows على ويندوز:

```
$ ssh-keygen -o
Generating public/private rsa key pair.
Enter file in which to save the key (/home/schacon/.ssh/id_rsa):
Created directory '/home/schacon/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/schacon/.ssh/id_rsa.
Your public key has been saved in /home/schacon/.ssh/id_rsa.pub.
The key fingerprint is:
d0:82:24:8e:d7:f1:bb:9b:33:53:96:93:49:da:9b:e3 schacon@mylaptop.local
```

هو أولاً يتحقق المكان الذي تريد حفظ المفتاح فيه (`.ssh/id_rsa`)، ثم يطلب مرتين إدخال عبارة المرور (`passphrase`)، ويمكنك تركها فارغة إذا لم تنشأ إدخال عبارة مرور عند استعمال المفتاح. أما إذا أردتها، فعليك إضافة الخيار `-o`؛ فهو يحفظ المفتاح الخاص بصيغة أقوى من الصيغة المبدئية في مقاومة كسر عبارات المرور بالقوة العمياء. ويمكنك أيضاً استخدام أداة `ssh-agent` («عميل SSH») لمنع الحاجة إلى إدخال عبارة المرور في كل مرة.

الآن، على كل مستخدم فعل هذا أن يرسل مفتاحه العمومي إليك أو إلى من يدير خادم جت (بفرض أنك أعدت خادم SSH ليستعمل المفاتيح العمومية). وليس عليهم سوى نسخ محتويات ملف `.pub` وإرسالها إليك بالبريد الإلكتروني. تبدو المفاتيح العمومية مثل هذا:

```
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAKl0UpkDHrfHY17SbrmTIpNLTGK9Tjom/BWDSU
GPLnafzLHDTYW7hdI4yZ5ew18JH4JW9jbhUFRviQzM7xLELEVf4h9LFX5QVkbPppSwg0cda3
Pbv7k0dJ/MTyBtWFCR+HAo3FXRitBqxiX1nKhXpHAZsMciLq8V6RjsNAQwdsdMFvSLVK/7XA
t3FaoJoAsncM1Q9x5+3V0Ww68/eIFmb1zuUFljQJKprX88XypNDvjYNby6vw/Pb0rwert/En
mZ+AW40ZPnTPI89ZPmVMLuayrD2cE86Z/il8b+gw3r3+1nKatmIkjn2so1d01QraTLMqVSsbx
NrRFi9wrf+M7Q== schacon@mylaptop.local
```

لشرح أعمق لإنشاء مفتاح SSH على أنظمة التشغيل المختلفة، انظر دليل جت هب لمفاتيح SSH (بالإنجليزية):

<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>

٤، ٤، إعداد الخادوم

لنعدّ معا وصول SSH على الخادوم. سنستعمل في هذا المثال طريقة `authorized_keys` («المفاتيح المستوثقة») لاستيثاق مستخدميك. سنفترض أيضاً أنك تستعمل توزيع لينكس معتادة مثل أوبنتو.

معظم المشروع هنا يمكن عمله آلياً بأمر `ssh-copy-id` ، بدلا من نسخ المفاتيح العمومية وتثبيتها يدويا.



أولا، أنشئ حساب مستخدم باسم `git` وأنشئ مجلد `ssh` له.

```
$ sudo adduser git
$ su git
$ cd
$ mkdir .ssh && chmod 700 .ssh
$ touch .ssh/authorized_keys && chmod 600 .ssh/authorized_keys
```

CONSOLE

سنحتاج الآن إلى إضافة بعض مفاتيح SSH العمومية للمطورين إلى ملف المفاتيح المستوثقة الخاص بالمستخدم `git`. لنفرض أن لديك بعض المفاتيح الموثوقة وأنتك حفظتها في ملفات مؤقتة. للتذكير، تبدو المفاتيح العمومية هكذا:

```
$ cat /tmp/id_rsa.badr.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCB007n/ww+ouN4gSLKssMxXnB0vf9LGt4L
ojG6rs6hPB09j9R/T17/x4lhJA0F3FR1rP6kYBRswj2aThGw6HXLm9/5zytK6Ztg3RPPK+4k
Yjh6541NYsnEZAuZx0jTtYAUfrtU3Z5E003C4ox0j6H0rfIF1kKI9MAQLMdpGW1GYEIgS9Ez
Sdfd8AcCIcTDWbqlAcU4UpkaX8KyGLLwsNuuGztobF8m72ALC/nLF6JLtPofwFB\gc+myiv
07TCUSBdLQlgMV0Fq1I2uPWQ0kOWQAHE0mfjy2jctxSDBQ220ymjaNsHT4kgZg2AYYgPq
dAv8JggJICUvax2T9va5 gsg-keypair
```

CONSOLE

ليس عليك إلا إضافتها إلى ملف `authorized_keys` الخاص بالمستخدم `git` الموجود في مجلد `ssh` الخاص به:

```
$ cat /tmp/id_rsa.badr.pub >> ~/.ssh/authorized_keys
$ cat /tmp/id_rsa.shams.pub >> ~/.ssh/authorized_keys
$ cat /tmp/id_rsa.wafaa.pub >> ~/.ssh/authorized_keys
```

CONSOLE

أعد الآن مستودع مجرد لهم بأمر `git init` مع الخيار `--bare` ، لتنشئ المستودع بلا مجلد عمل:

```
$ cd /srv/git
$ mkdir project.git
$ cd project.git
$ git init --bare
Initialized empty Git repository in /srv/git/project.git/
```

CONSOLE

عندئذٍ يستطيع بدر أو شمس أو وفاء دفع النسخة الأولى من مشروعهم إلى المستودع بإضافته مستودعاً بعيداً في نسختهم المحلية، ودفع الفرع الذي لديهم إليه. لاحظ أن في كل مرة تريد فيها إضافة مشروع، على شخص ما الوصول إلى الجهاز عبر الصدفية وإنشاء مستودع مجرد. ليكن `gitserver` اسم المضيف ("hostname") للخادوم الذي أعدت عليه المستودع ومستخدم `git`. إذا كنت تشغل الخادوم داخليا وأعدت DNS ليشير الاسم `gitserver` إلى هذا الخادوم، فيمكنك استعمال الأوامر كما هي تقريبا (بفرض أن `myproject` هو مشروع موجود وفيه ملفات):

```
# على حاسوب بدر
$ cd myproject
$ git init
$ git add .
$ git commit -m 'Initial commit'
$ git remote add origin git@gitserver:/srv/git/project.git
$ git push origin master
```

الآن سيستطيع الآخرون استنساخه إلى أجهزتهم ودفع تعديلاتهم إليه بالسهولة نفسها:

```
$ git clone git@gitserver:/srv/git/project.git
$ cd project
$ vim README
$ git commit -am 'Fix for README file'
$ git push origin master
```

بهذه الطريقة ستحصل سريعاً على خادم جت يتيح إندي القراءة والتحرير لبضعة مطورين.

عليك أيضاً ملاحظة أن حتى الآن، أولئك المستخدمين جميعهم يمكنهم أيضاً الولوج إلى الخادوم والحصول على صدفـة المستخدم git. إذا أردت تقييد هذا، فعليك تغيير الصدفـة إلى شيء آخر في ملف `/etc/passwd`.

يمكنك بسهولة تقييد حساب المستخدم git إلى الأنشطة المرتبطة بجـت باستعمال أداة صدفـة مقيدة اسمها git-shell («صدفـة جت») وهي مرفقة مع جـت. فإذا ضبطتها لتكون صدفـة ولوج لحساب المستخدم git فإن هذا الحساب لن يكون له وصول صدفـة طبيعي إلى خادمك. لاستعمالها، حدد git-shell لتكون صدفـة ولوج لهذا الحساب بدلاً من bash أو csh. ولفعل هذا، عليك أولاً إضافة المسار الكامل لأمر git-shell إلى ملف `/etc/shells` إذا لم يكن موجوداً فيه بالفعل:

```
$ cat /etc/shells      # انظر إن كانت صدفـة جت هنا، وإلا...
$ which git-shell      # فتتحقق أن صدفـة جت مثبتة على نظامك
$ sudo -e /etc/shells  # ثم أضف مسارها من الأمر السابق إلى ملف الصدفـات
```

يمكنك الآن تغيير صدفـة المستخدم بالأمر `chsh <username> -s <shell>`:

```
$ sudo chsh git -s $(which git-shell)
```

عندئذٍ يستطيع المستخدم git استعمال SSH للدفع والجذب من مستودعات جت، بغير أن يكون له وصولاً صدفياً إلى خادمك. وإن حاول، فسيرى رسالة رفض ولوج كهذه:

```
$ ssh git@gitserver
fatal: Interactive git shell is not enabled.
hint: ~/git-shell-commands should exist and have read and execute access.
Connection to gitserver closed.
```

لكن ما زال يمكن للمستخدمين توجيه منفذ SSH (أي "port forwarding") للوصول إلى أي خادم آخر يقدر خادم جت أن يتصل به. فإذا أردت منع هذا، فأضف في ملف المفاتيح المستوتقة (authorized_keys) هذه الخيارات في أول كل مفتاح تريد تقييده:

```
no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty
```

CONSOLE

ستبدو نتيجة التعديل مثل هذا:

```
$ cat ~/.ssh/authorized_keys
no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQCB007n/ww+ouN4gSLKssMxXnB0vf9LGt4LojG6rs6h
PB09j9R/T17/x4lhJA0F3FR1rP6kYBRsWj2aThGw6HXLm9/5zytK6Ztg3RPPK+4kYjh6541N
YsnEAZuXz0jTTyAUfrtU3Z5E003C4ox0j6H0rFI1kKI9MAQLMdpGW1GYEIGS9EzSdfd8AcC
IicTDWbqLAcU4UpkaX8KyGLLwsNuuGztobF8m72ALC/nLF6JLtPofwFBlgc+myiv07TCUSbd
LQlgMV0Fq1I2uPWQOk0WQAHukE0mfjy2jctxSDBQ220ymjaNsHT4kgTzG2AYYgPqdAv8JggJ
ICUvax2T9va5 gsg-keypair
```

CONSOLE

```
no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQDEwENNmomTboYI+LJieaAY16qiXiH3wuvENhBG...
```

عندئذٍ ستظل تعمل أوامر جت الشبكية، ولكن المستخدمين لن يعودوا قادرين على الوصول إلى صدفه. وكما ترى في رسالة الرفض، يمكنك أيضاً إعداد مجلد في مجلد منزل المستخدم git لتخصيص أمر git-shell قليلاً. فيمكنك مثلاً تقييد أوامر جت التي يقبلها الخادم، أو تخصيص الرسالة التي يراها المستخدمين عندما يحاولون الوصول عبر SSH. نفذ الأمر git help shell للحصول على معلومات مزيدة عن تخصيص الصدفه.

٤، ٥، عفريت جت

(من المترجم) كلمة "daemon" ظهرت في MIT اسماً للعمليات الخدمية التي تعمل في خلفية نظام التشغيل ولا يلاحظها المستخدم. جاءت التسمية نسبة إلى [عفريت ماكسويل](https://ar.wikipedia.org/wiki/عفريت_ماكسويل) (الرابط: https://ar.wikipedia.org/wiki/عفريت_ماكسويل) في الفيزياء، الذي يستعمل المعنى القديم للكلمة (في اليونانية والعربية)، وهو الكائن الغيبي الذي يعمل في الخفاء، وليس بالضرورة شيطانياً. فترجمها «عفريت»، وأترجم فعل الصيرورة منها ("daemonize") إلى «عفرتة». الاسم المناظر على ويندوز هو «خدمة» ("service")، وقد بدأ استخدامه حديثاً في لينكس كذلك.



سنعدّ الآن عفريئاً ليقدم المستودعات بميفاق "Git". هذا هو الخيار الشائع لإتاحة وصول سريع بغير استيثاق لبيانات جت. تذكر أنه غير مستوثق، فأى شيء تتيحه عبر هذا الميفاق سيكون عمومياً للجميع داخل الشبكة.

فإذا كنت تستخدمه على خادم خارج جدار حمايتك (firewall)، فعليك ألا تستخدمه إلا للمشروعات التي يمكن أن يراها العالم. وإذا كان خادمك داخل جدار حمايتك، فيمكنك استخدامه للمشروعات التي يحتاج الكثير من الناس أو الأجهزة الوصول إليه وصول قراءة فقط (مثل خواديم البناء أو الدمج المستمر (CI))، إن لم تكن تريد إضافة مفتاح SSH لكلٍ منهم.

وفي جميع الأحوال، إن ميفاق Git سهل الإعداد نسبياً. فلستَ تحتاج إلا إلى عفرة هذا الأمر:

```
$ git daemon --reuseaddr --base-path=/srv/git/ /srv/git/
```

CONSOLE

خيار `--reuseaddr` («أعد استخدام العنوان») يسمح بإعادة تشغيل الخادوم بغير انتظار انقضاء مهلة الاتصالات القديمة (timeout). وخيار `--base-path` («أساس المسار») يتيح للناس استنساخ المشروعات بغير تحديد المسار بكامله. أما المسار الذي في آخر الأمر يخبر عفريت جت مكان المستودعات التي سيُصدّرُها. وإذا كنت تستخدم جدار حماية (firewall)، فستحتاج أيضاً إلى فتح منفذ 9418 فيه على الجهاز الذي تعدّه عليه.

طريقة عفرة هذه العملية تختلف حسب نظام تشغيلك.

لأن `systemd` هو نظام الابتدء (init) الأشهر على توزيعات لينكس الحديثة، فيمكنك استخدامه لهذا الغرض. ليس عليك سوى إنشاء الملف `/etc/systemd/system/git-daemon.service` فيه هذا:

```
[Unit]
Description=Start Git Daemon

[Service]
ExecStart=/usr/bin/git daemon --reuseaddr --base-path=/srv/git/ /srv/git/

Restart=always
RestartSec=500ms

StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=git-daemon

User=git
Group=git

[Install]
WantedBy=multi-user.target
```

CONSOLE

ربما لاحظت أن اسم المستخدم واسم المجموعة اللذين يُنفَّذ بهما عفريت جت هما `git`. يمكنك تغييرهما إلى ما يناسبك، ولكن تأكد من وجود اسم المستخدم المختار على نظامك. وتأكد أيضاً من أن الملف التنفيذي لبرنامج جت موجود فعلاً في المسار `/usr/bin/git` وإلا فغيّره إلى ما يناسب.

وأخيراً، نفَّذ `systemctl enable git-daemon` لبدء تشغيل الخدمة (العفريت) آلياً مع بدء تشغيل النظام، ويمكنك تشغيل الخدمة بالأمر `systemctl start git-daemon` وإيقافها بالأمر `systemctl stop git-daemon`.

على الأنظمة الأخرى، قد يناسبك `xinetd` أو بُريمج في نظام `sysvinit` أو شيئاً آخر — طالما أنك جعلت هذا الأمر مُعفَرت ومُراقَب بطريقةٍ ما.

ثم نحتاج إلى إخبار جت بالمستودعات التي يسمح بالوصول إليها بغير استيثاق عبر خادم جت. يمكنك فعل هذا بإنشاء ملف اسمه `git-daemon-export-ok` في كل مستودع.

```
$ cd /path/to/project.git
$ touch git-daemon-export-ok
```

CONSOLE

وجود هذا الملف يخبر جت بقبول إتاحة هذا المشروع بغير استيثاق.

٤، ٦، ميفاق HTTP الذكي

لدينا الآن وصولاً مستوثقاً عبر SSH ووصولاً غير مستوثق عبر `git://`، ولكن يوجد أيضاً ميفاق يمكنه عمل كلا الأمرين في وقت واحد. ليس إعداد HTTP الذكي إلا أن تفعّل على الخادم بريمج CGI المرفق مع جت المسمى `git-http-backend`. يقرأ هذا البريمج المسار والترويسات (headers) التي يرسلها أمر الاستحضار `git fetch` أو الدفع `git push` إلى رابط HTTP ويحدد إذا كان العميل يستطيع التواصل عبر HTTP (وهذا صحيح لأي عميل جت منذ الإصدار 1.6.6). وإذا رأي البريمج أن العميل ذكي، فسيتواصل معه بذكاء؛ وإلا فسيتواصل معه بالميفاق البليد (ولذا فهو متوافق مع الإصدارات القديمة التي تريد القراءة فحسب).

لنرّ إعداداً يسيراً جداً. سنعدّ فيه أبانتشي (Apache) ليكون خادم CGI. إن لم يكن لديك أبانتشي مُعدّ، فيمكنك إعداده على حاسوب لينكسي بفعل شيء كهذا:

```
$ sudo apt-get install apache2 apache2-utils
$ a2enmod cgi alias env
```

CONSOLE

هذا أيضاً يفعّل وحدات `mod_cgi` و `mod_alias` و `mod_env`، التي تحتاجها جميعها حتى يعمل هذا الإعداد بشكل صحيح.

سنحتاج أيضاً إلى ضبط مجموعة المستخدمين اليونكسية الخاصة بمجلدات `/srv/git` إلى `www-data`، حتى يتسنى لخادم الويب قراءة المستودعات وتحريرها، لأن عملية أبانتشي التي تشغل بريمج CGI الخاص بنا تعمل (مبدئياً) تحت هذا المستخدم:

```
$ chgrp -R www-data /srv/git
```

CONSOLE

وكذلك سنحتاج إلى إضافة بعض الأشياء إلى تهيئة أبانتشي لتشغيل `git-http-backend` معالجاً ("handler") لأي شيء يأتي من المسار `/git` على خادمك.

```
SetEnv GIT_PROJECT_ROOT /srv/git
SetEnv GIT_HTTP_EXPORT_ALL
ScriptAlias /git/ /usr/lib/git-core/git-http-backend/
```

CONSOLE

إن أهملت متغير البيئة `GIT_HTTP_EXPORT_ALL`، فلن يتيح جت للعملاء غير المستوثقين إلا تلك المستودعات التي فيها الملف `git-daemon-export-ok`، تماماً مثلما فعل عفريت جت.

وأخيراً سنحتاج إلى إخبار أباتشي أن يسمح بالطلبات إلى `git-http-backend` وأن يستوثق عمليات التحرير بطريقة ما، مثلاً بكتلة `Auth` كهذه:

```
<Files "git-http-backend">
  AuthType Basic
  AuthName "Git Access"
  AuthUserFile /srv/git/.htpasswd
  Require expr !(%{QUERY_STRING} -strmatch '*service=git-receive-pack*' || %{REQUEST_URI} =~ m#/git-receive-pack$#)
  Require valid-user
</Files>
```

يتطلب هذا إنشاء ملف `htpasswd`. (يبدأ اسمه بنقطه) فيه كلمات المرور لجميع المستخدمين المقبولين. هذا مثال على إضافة المستخدم "schacon" إليه:

```
$ htpasswd -c /srv/git/.htpasswd schacon
```

لدى أباتشي طرائق عديدة لاستيثاق المستخدمين؛ عليك اختيار إحداها وتطبيقها. ليس هذا إلا أيسر مثال استطعنا الإتيان به. ومن شبه المؤكد أنك أيضاً ستحتاج إلى إعداد هذا عبر SSH حتى تكون هذه البيانات كلها معمة.

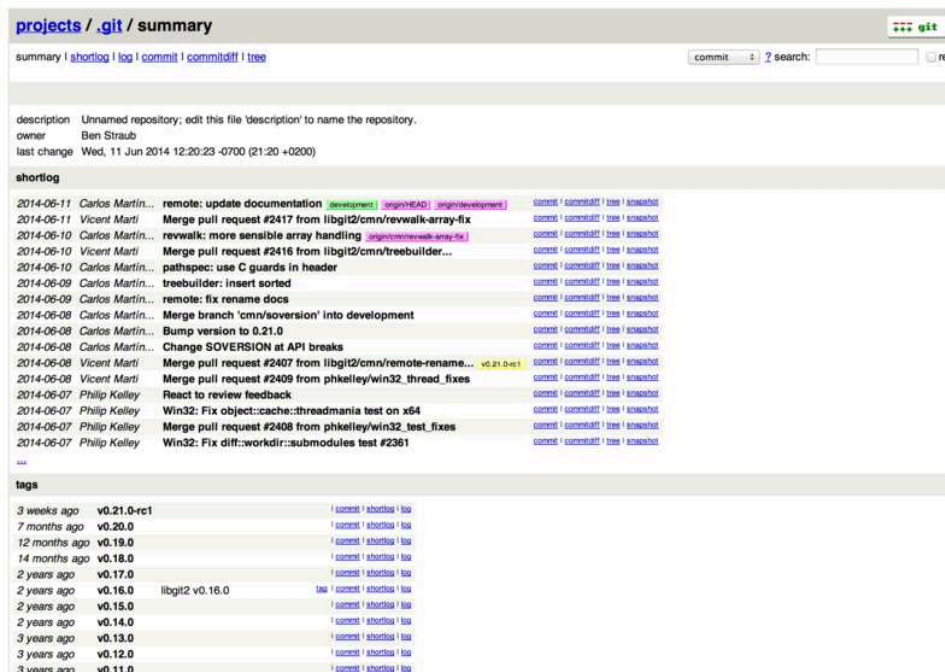
لا نود الخوض عميقاً في دوامة دقائق تهيئة أباتشي، لأنك قد تستخدم خادمًا آخر أو أن لديك احتياجات استيثاق مختلفة. وإنما الأمر أن مع جت بریمج CGI اسمه `git-http-backend`، الذي عند ندائه يفعل كل المفاوضات لإرسال واستقبال البيانات عبر HTTP. ولكنه لا ينفذ أي استيثاق بنفسه. لكن هذا سهل التحكم فيه في خادم الويب الذي يناديه. يمكنك فعل هذا مع ربما أي خادم وب يدعم CGI، لذا فانطلق مع الخادم الذي تعرفه حق المعرفة.

لمزيد من المعلومات عن تهيئة الاستيثاق في أباتشي، انظر وثائق أباتشي (بالإنجليزية) هنا: <https://httpd.apache.org/docs/current/howto/auth.html>



٤، ٧، GitWeb

لأنك الآن تستطيع الوصول إلى مشروعك مع إذن التحرير أو مع إذن القراءة فقط. فقد تود إعداد واجهة وب رسومية صغيرة له. إن مع جت بریمج CGI يسمى جتوب، ويستخدم أحياناً لهذا الغرض.



شكل ٤٩. واجهة وب جتوب

فإذا أردت رؤية كيف يبدو جتوب لمشروعك، فمع جت أمر يشغل نسخة مؤقتة منه إذا كان لديك خادم وب خفيف على نظامك مثل `lighttpd` أو `webrick`. على الأنظمة الليكسية غالباً يكون `lighttpd` مثبّتاً، فقد تستطيع تشغيله بتنفيذ `git instaweb` في مجلد مشروعك. وإن كنت على ماك، فإن نسخة `Leopard` تأتي بلغة `Ruby` مثبته مبدئياً، فيكون `webrick` هو الظن الأقرب. لبدء `instaweb` بشيء غير `lighttpd` عليك تشغيله بخيار `--httpd`.

```
$ git instaweb --httpd=webrick
[2009-02-21 10:02:21] INFO  WEBrick 1.3.1
[2009-02-21 10:02:21] INFO  ruby 1.8.6 (2008-03-03) [universal-darwin9.0]
```

هذا يشغل خادم `HTTPD` على منفذ `1234` ويفتح متصفح الويب على تلك الصفحة آلياً. فهو يسهّل عليك. وعندما تقضي ما تريد وتود إيقافه، نَفِّذ الأمر نفسه لكن بالخيار `--stop`:

```
$ git instaweb --httpd=webrick --stop
```

وإذا كنت تود تشغيل واجهة الويب على خادم طوال الوقت لفريقك أو لمشروع مفتوح تستضيفه، فستحتاج إلى إعداد برمج `CGI` ليقدمه خادم الويب العادي الذي تستخدمه. بعض توزيعات لينكس تأتي بحزمة `gitweb` التي قد تستطيع تثبيتها بأمر مثل `apt` أو `dnf`، لذا فقد تود تجربة هذا أولاً. سنتناول بإيجاز كبير تثبيت جتوب يدوياً. عليك أولاً الحصول على مصدر جت، الذي فيه جتوب، ثم توليد برمج `CGI` المخصص:

```
$ git clone https://git.kernel.org/pub/scm/git/git.git
$ cd git/
$ make GITWEB_PROJECTROOT="/srv/git" prefix=/usr gitweb
  SUBDIR gitweb
  SUBDIR ../
make[2]: `GIT-VERSION-FILE' is up to date.
  GEN gitweb.cgi
```

```
GEN static/gitweb.js
$ sudo cp -Rf gitweb /var/www/
```

لاحظ أنك تحتاج إلى إخبار هذا الأمر بمكان مستودعاتك في المتغير `GITWEB_PROJECTROOT`. والآن، تحتاج إلى جعل أباتشي يستخدم CGI لهذا البريمج، ويمكنك فعل ذلك بإضافة مستضيف وهمي له:

```
<VirtualHost *:80>
  ServerName gitserver
  DocumentRoot /var/www/gitweb
  <Directory /var/www/gitweb>
    Options +ExecCGI +FollowSymLinks +SymLinksIfOwnerMatch
    AllowOverride All
    order allow,deny
    Allow from all
    AddHandler cgi-script cgi
    DirectoryIndex gitweb.cgi
  </Directory>
</VirtualHost>
```

CONSOLE

نكرر: يمكن تقديم جتوب عبر أي خادم وب يتيح CGI أو Perl. فإذا أردت شيئاً آخر فليس إعداداه بالصعب. يمكنك الآن زيارة `http://gitserver/` لرؤية مستودعاتك على الشبكة.

٤، ٨، GitLab

لعلك وجدت أن جتوب GitWeb قليل الإمكانات بعض الشيء. فإذا كنت تبحث عن خادم جت حديث ومكتمل الخصائص، فيوجد عدد من الحلول المفتوحة المصدر يمكنك تثبيتها بدلا منه. ولأن جتلاب من أشهرها، فإننا سنتناول تثبيته واستخدامه مثالا. هذا الخيار أصعب من جتوب وسيحتاج منك رعاية أكثر، لكنه مكتمل الخصائص.

٤، ٨، ا، التثبيت

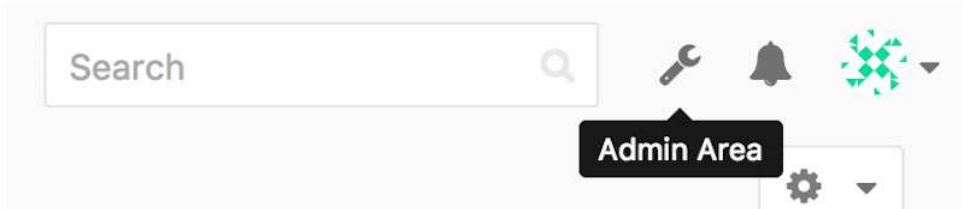
جتلاب هو تطبيق وب بقاعدة بيانات، لذا فتثبيته أعقد من بعض خواديم جت الأخرى. لكن لحسن الحظ هذه العملية موثقة بالكامل ومدعومة جيدا. جتلاب يشدد التوصية بتثبيته على خادمك عبر الحزمة الرسمية الحافلة، المسماة حزمة “Omnibus GitLab”.

خيارات التثبيت الأخرى هي:

- `GitLab Helm chart`، لاستخدامه مع `Kubernetes`.
- حزم `Docker` لـ `GitLab`، لاستخدامها مع `Docker`.
- من ملفات المصدر.
- مقدّمو الخدمات السحابية، مثل `AWS` و `Google Cloud Platform` و `Azure` و `OpenShift` و `Digital Ocean`.

للمزيد من المعلومات (بالإنجليزية) انظر [GitLab Community Edition \(CE\) readme](https://gitlab.com/gitlab-org/gitlab-foss/-/blob/master/README.md) (الرابط: <https://gitlab.com/gitlab-org/gitlab-foss/-/blob/master/README.md>).

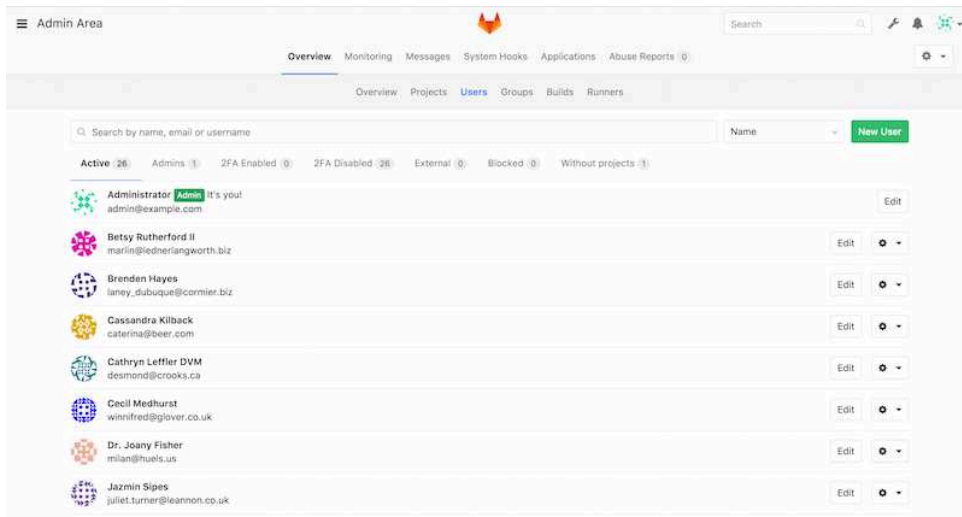
واجهة إدارة جتلاب هي واجهة وب. ليس عليك إلا توجيه متصفحك إلى اسم المضيف ("hostname") أو عنوان IP الذي ثبت عليه جتلاب، ثم لُج بحساب المدير root. كلمة المرور المبدئية تختلف حسب خيار التثبيت، لكن الحزمة الرسمية الحافلة، بطبيعتها تولّد لك كلمة مرور وتخزنها في ملف `/etc/gitlab/initial_root_password` حتى أربع وعشرين ساعة على الأقل. انظر التوثيق لتفاصيل أزيد. بعد الولوج، اضغط على رمز "Admin area" (منطقة الإدارة) في القائمة التي على اليمين بالأعلى.



شكل 0.5. زر "Admin area" (منطقة الإدارة) في قائمة جتلاب

المستخدمون

على كل من يريد استخدام خادم جتلاب الخاص بك الحصول على حساب مستخدم. حسابات المستخدمين أمر يسير؛ هي في الأساس معلومات شخصية مرتبطة ببيانات الولوج. كل حساب مستخدم لديه **مساحة أسماء** ("namespace")، وهي تجميع منطقي للمشروعات الخاصة به. فمثلاً إذا كان لدى المستخدم shams مشروع اسمه project، فإن رابط ذلك المشروع سيكون `http://server/shams/project`.



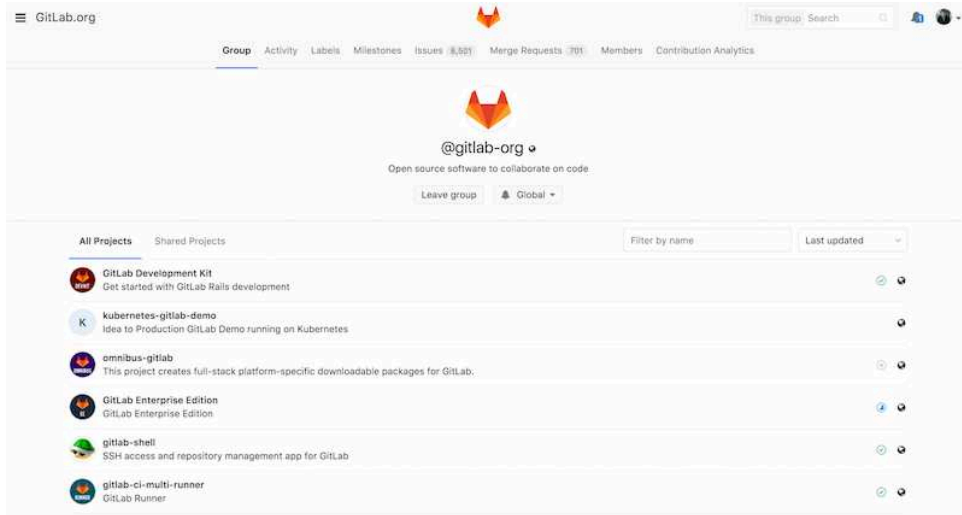
شكل 0.5. شاشة إدارة المستخدمين على جتلاب

يمكنك إزالة حساب مستخدم بطريقتين: «حظر» ("block") حساب مستخدم يمنعه من الولوج إلى خادمك، ولكن كل بياناته التي في مساحة أسمائه ستبقى، والإيداعات الموقّعة ببيده ستظل تشير إلى صفحته الشخصية على خادمك.

أما «محو» ("destroy") مستخدم فيزيله تماماً من قاعدة البيانات ومن نظام الملفات؛ ستُزال كل المشروعات والبيانات التي في مساحة أسمائه، وكذلك كل المجموعات التي يملكها. طبعاً هذا الفعل مستديم الأثر وأشدّ إتلافاً، ونادراً ما ستحتاجه.

المجموعات

المجموعة في جتلاب هي تجميعية من المشروعات، مع معلومات عن كيفية وصول المستخدمين إلى هذه المشروعات. كل مجموعة لديها «مساحة أسماء مشروعات»، تماماً مثلما للمستخدمين، لذا فإن كان في المجموعة training مشروع اسمه materials، فسيكون رابطته `http://server/training/materials`.



شكل ٥٢. شاشة إدارة المجموعات على جتلاب

ترتبط كل مجموعة بعدد من المستخدمين، ولكل منهم مستوى من الصلاحيات لمشروعات المجموعة وكذلك المجموعة نفسها. وتتراوح هذه المستويات من "Guest" («زائر»: للمسائل "issues" والمحادثات "chat" فقط)، إلى "Owner" («مالك»: للتحكم الكامل في المجموعة وأعضائها ومشروعاتها). وهذه المستويات كثيرة يصعب سردها هنا، لكنّ شاشة الإدارة في جتلاب فيها رابطاً مفيداً.

المشروعات

المشروع في جتلاب هو تقريباً مستودع جت واحد. ينتمي كل مشروع إلى مساحة أسماء واحدة، إما لمستخدم وإما لمجموعة. وإذا كان المشروع ينتمي إلى مستخدم، فله مالك المشروع تحكم مباشر في من لديه حق الوصول إلى المشروع. وإذا كان ينتمي إلى مجموعة، فصلاحيات أعضاء المجموعة هي التي تحدد.

كل مشروع له مستوى ظهور، ليحدد من يستطيع رؤية صفحات هذا المشروع ومستودعه. فإذا كان المشروع «خصوصياً» ("Private")، فلا يظهر إلا لمن يحددهم مالك المشروع بالاسم. وإذا كان «داخلياً» ("Internal")، فإنه لا يظهر إلا للمستخدمين الوالجين إلى الخادوم. وإذا كان «عمومياً» ("Public") فإنه يظهر للجميع. لاحظ أن هذا يتحكم في الوصول عبر `git fetch` وكذلك عند الوصول عبر واجهة الوب لهذا المشروع.

الخطاطيف

يدعم جتلاب الخطاطيف، على مستوى المشروع وعلى مستوى النظام كله. سيرسل خادوم جتلاب طلب HTTP بطريقة POST مع وصف بصيغة JSON، كلما حدث حدثٌ مناسب. وهذه طريقة عظيمة لربط مستودعات جت وخادوم جتلاب بالأدوات الآلية الأخرى التي تستخدمها في التطوير، مثل خواديم الدمج المستمر (CI)، وغرف المحادثة، وأدوات النشر (deployment).

٤، ٨، ٣، الاستخدام الأساسي

أول ما قد تود عمله على جت لآب هو إنشاء مشروع، وذلك بالضغط على رمز "+" على شريط الأدوات. ستُسأل عن اسم المشروع، ومساحة الأسماء التي ينتمي إليها، ومستوى ظهوره. معظم ما تحدده هنا ليس مستديماً، فتستطيع تغييره فيما بعد من واجهة الإعدادات. اضغط على "Create Project" («إنشاء المشروع») لإتمام العملية.

وما إن يكون المشروع حياً، قد تود ربطه بمستودع محلي. يمكن التواصل مع أي مشروع عبر HTTPS أو SSH. ويمكنك استعمال أي منهما لإضافة مستودع جت بعيد في مستودعك المحلي. ستجد الروابط في أعلى الصفحة الرئيسية للمشروع. فمثلاً تنفيذ هذا الأمر في مستودع محلي سيضيف إليه مستودعاً بعيداً اسمه gitlab ويشير إلى المستودع المستضاف:

```
$ git remote add gitlab https://server/namespace/project.git
```

CONSOLE

أما إذا لم يكن لديك نسخة محلية من المستودع، فيمكنك استنساخه بسهولة هكذا:

```
$ git clone https://server/namespace/project.git
```

CONSOLE

وتتيح واجهة الوب طرائق مختلفة مفيدة للنظر إلى المستودع نفسه. فتُظهر الصفحة الرئيسية لكل مشروع آخر الأنشطة، والروابط التي في أعلى الصفحة تُريك ملفات المشروع وتاريخ إيداعاته.

٤، ٨، ٤، التعاون

أسهل طريقة للتعاون على مشروع على جت لآب هي إعطاء كل مستخدم إذن الدفع مباشرةً إلى المستودع. يمكنك ضم المستخدمين إلى المشروع بالذهاب إلى قسم الأعضاء ("Members") في إعدادات هذا المشروع، وربط المستخدمين الجدد بمستويات الوصول المناسبة. (مررنا على مستويات الوصول في فصل [المجموعات](#)). إذا كان للمستخدم مستوى الوصول «مطور» ("Developer") فيستطيع دفع الإيداعات والفروع مباشرةً إلى المستودع.

لكن طريقة أخرى للتعاون بغير ضم المطورين إلى المستودع هي طلبات الدمج ("merge request")، التي تتيح المساهمة بطريقة محكمة لكل مستخدم يستطيع رؤية المشروع. فالمستخدمون ذوو الوصول المباشر يمكنهم إنشاء فرع ودفع إيداعاتهم إليه ثم إنشاء طلب لدمج فرعهم في الفرع الرئيس أو أي فرع آخر. أما المستخدمون الذين ليس لديهم إذن الدفع للمستودع، فيمكنهم «اشتقاق» ("fork") المستودع لإنشاء نسختهم الخاصة منه، ودفع إيداعاتهم إلى نسختهم الخاصة بهم، ثم إنشاء طلب دمج من اشتقاقهم إلى المشروع الأصل. يسمح هذا الأسلوب للمالك بالتحكم الكامل في كل ما يدخل المستودع ومتى يدخل، وفي الوقت نفسه يسمح بمساهمات المستخدمين غير الموثوق فيهم.

طلبات الدمج، والمسائل ("issues")، هما أهم أدوات النقاشات الطويلة في جت لآب. فكل طلب دمج يسمح بنقاش على كل سطر من سطور التعديل المقترح (ويدعم نوعاً خفيفاً من مراجعة التعديلات (code review))، إضافةً إلى نقاش عام. يمكن تكليف مستخدم بإتمام طلب دمج أو مسألة، أو جعل ذلك ضمن هدف من الأهداف ("milestone").

ركّز هذا الفصل في معظمه على خصائص جت لآب المرتبطة بجت. ولكنه مشروع ناضج ومكتمل الخصائص، وفيه الكثير غير ذلك ليساعد فريقك في التعاون، مثل موسوعات المشروعات وأدوات رعاية الأنظمة. من محاسن جت لآب أنك ما إن تُتم إعداد الخادوم وتشغيله، فيندر أن تحتاج إلى تعديل ملف تهيئة أو الوصول إلى الخادوم عبر SSH؛ فواجهة الوب تتيح معظم أفعال الإدارة والاستخدام العام.

٤، ٩، خيارات الاستضافة الخارجية

إن لم تشأ خوض غمار العمل المطلوب لإعداد خادوم جت الخاص بك، فلديك عدة خيارات لاستضافة مشروعاتك على مواقع استضافة خارجية متخصصة. لهذا منافع عديدة: أن مواقع الاستضافة عموماً أسرع في الإعداد وأسهل في بدء المشروعات عليها، وليس عليك رعاية الخادوم ولا صيانتته ولا مراقبته. حتى إن أعددت وشغلت خادومك الخاص داخليا، فقد تود استخدام موقع استضافة عمومي لمشروعاتك البرمجية المفتوحة؛ فهذا يسهل على المجتمع أن يجدها ويساعدك فيها.

لدينا اليوم عدد مهول من الاستضافات، كلٌ له مزايا وعيوب مختلفة. تجد قائمة محدّثة في صفحة الاستضافات في موسوعة جت الرسمية: <https://archive.kernel.org/oldwiki/git.wiki.kernel.org/index.php/GitHosting.html>.

سنتناول جت هب بالتفصيل في باب [GitHub](#)، لأنه أكبر استضافة جت إطلاقاً، وقد تحتاج إلى التعامل مع مشروعات مستضافة عليه على أي حال، ولكن توجد عشرات الخيارات الأخرى إذا لم تشأ إعداد خادوم جت الخاص بك.

٤، ١٠، الخلاصة

لديك عدة خيارات لإعداد وتشغيل مستودع جت بعيد حتى يمكنك التعاون من الآخرين أو مشاركة عملك.

يتيح لك تشغيل خادومك الخاص تحكماً كبيراً ويسمح لك بتشغيله خلف جدار الحماية الخاص بك (firewall)، لكن مثل هذا الخادوم يحتاج قدرًا لا بأس به من الوقت لإعداده ورعايته. أما إذا وضعت مشروعاتك البرمجية على خادوم مستضاف، فستجده أسهل إعداداً ورعايةً. لكن يجب أن يكون مسموحًا لك بذلك، فإن بعض المؤسسات لا تسمح.

نتوقع أنه صار من السهل نسبياً تحديد الحل أو توليفة الحلول المناسبة لك ولمؤسستك.

الباب الخامس:

جت المتوزع

الآن وقد أعددت مستودع جت بعيداً ليكون مركزاً للمطورين ليشاركوا فيه مصادرههم البرمجية، وقد صرت عالماً بأوامر جت الأساسية في العمل المحلي، سنرى الآن كيف يمكنك استعمال أساليب التطوير المتوزع في جت.

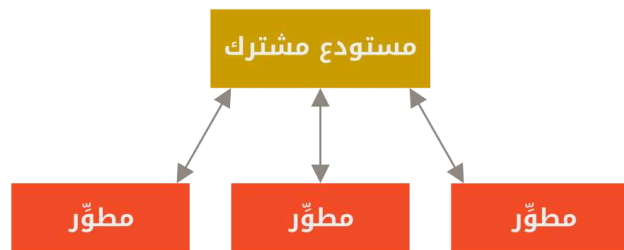
سنرى في هذا الباب كيف تستعمل جت في بيئة متوزعة، مساهماً أو دامجاً. فستتعلم كيف تساهم بمصدر برمجي أو بتغييرات برمجية في مشروع، وكيف تجعل ذلك أيسر ما يمكن، عليك وعلى مدير المشروع، وكذلك كيف تدير بنجاح مشروعاً يساهم فيه عددٌ من المطورين.

٥، ١، أساليب التطوير المتوزع

خلافًا للأنظمة المركزية، طبيعة جت المتوزعة تسمح بمرونة كبيرة في التعاون على تطوير المشروعات. ففي الأنظمة المركزية، الكل متساوون في أنهم نقاط فرعية تعمل مع مركز التقاء. أما في جت، فكل مطور يمكن أن يكون نقطة أو مركزاً؛ كل مطور قد يساهم في مستودعات الآخرين وقد يرعى مستودعاً عمومياً يبني الآخرون عليه ويساهمون فيه. يتيح هذا ضروباً كثيرة من أساليب التطوير لمشروعاتك ومشروعات فريقك. لذا سنتناول بعضاً من أشهر الأساليب التي تعتمد على هذه المرونة. وسنرى مزايا وعيوب كلٍ منها. يمكنك اختيار أسلوب واحد، أو مزج خصائص بضعة أساليب كيفما يناسبك.

٥، ١، ١، الأسلوب المركزي

في الأنظمة المركزية، يكاد ألا يوجد إلا أسلوب تعاون واحد: الأسلوب المركزي. مركز التقاء واحد، أي مستودع، هو الذي يقبل التعديلات، ويزامن كل المساهمين مستودعاتهم معه. المطورون هم نقاط فرعية («عملاء» لنقطة الالتقاء المركزية)، ويتزامنون مع المركز.



شكل ٥٣. أسلوب التطوير المركزي

فإذا استنسخ المستودع مبرمجان، وكلاهما عدل فيه، فإن المبرمج الذي يدفع تعديلاته إلى المستودع أولاً لن يجد صعوبة، أما المبرمج الآخر فعليه أولاً دمج عمل المبرمج الأول قبل دفع تعديلاته إلى المستودع، حتى لا يغير عمل الأول. هذا الأمر حقيقة في جت كما في Subversion (مثل جميع أنظمة إدارة النسخ المركزية)، وهذا الأسلوب يعمل جيداً في جت.

فإذا كنت مرتاحاً مع أسلوب تطوير مركزي معين في شركتك أو فريقك، فيمكنك الاستمرار في اتباعه مع جت. ليس عليك إلا إعداد مستودع واحد، وإعطاء كل واحد في فريقك إذن الدفع؛ ولن يسمح جت لأحد بتغيير عمل الآخرين.

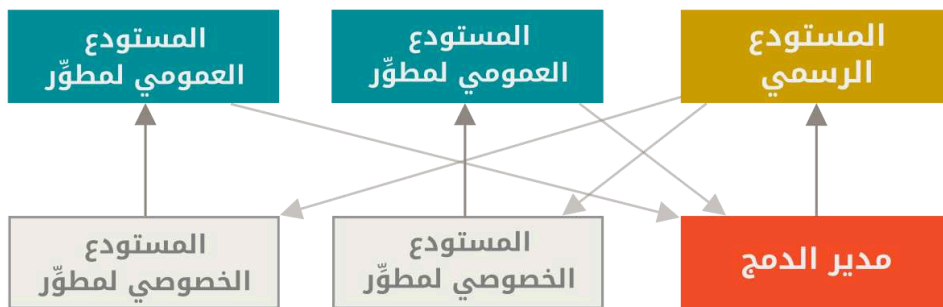
تخيل أن سميير وسميرة بدءا العمل في وقت واحد. وأتم سميير تعديلاته ودفعها إلى الخادوم. ثم أرادت سميرة دفع تعديلاتها، فسيفض الخادوم، ويخبرها أنها تريد دفع إيداعات ليست إيداعات تسريع، ولن يسمح لها إلا بعد أن تستحضر (fetch) الجديد وتدمجه (merge). أسلوب التطوير هذا جذاب للكثيرين لأنه مألوف ومريح للكثيرين.

وليس هذا الأسلوب مقصوداً على جماعات التطوير الصغيرة؛ فبنموذج التفرع في جت يسمح لمئات المطورين بالعمل بيسر معاً على مشروع واحد، باستعمال عشرات الفروع في وقت واحد.

٥، ١، ٢، أسلوب مدير الدمج

يمكّنك جت من التعامل مع العديد من المستودعات البعيدة. فيسمح هذا بأسلوب تطوير فيه كل مطور له إذن الدفع إلى مستودعه العمومي، وإذن القراءة لمستودعات الآخرين كلهم. هذا غالباً يشمل وجود مستودع رئيس يُعدّ المستودع «الرسمي». فللمساهمة في هذا المشروع، عليك استنساخه ودفع تعديلاتك إلى نسختك. عندئذ ترسل إلى القائم على المشروع طلباً بجذب تعديلاتك. فيضيف مستودعك إلى المستودعات البعيدة في المشروع، ويختبر تعديلاتك على جهازه، ثم يدمجها ويدفعها إلى المستودع الرئيس. تجري هذه العملية هكذا (انظر شكل [أسلوب مدير الدمج](#)):

١. يدفع القائم على المشروع إلى مستودعه العمومي.
٢. يستنسخ مبرمج ما ذلك المستودع ويعدّل فيه.
٣. يدفع هذا المبرمج إلى نسخته العمومية الخاصة به.
٤. يرسل المبرمج (في رسالة بريد شابكي مثلاً) إلى القائم على المشروع طلباً بجذب تعديلاته.
٥. يضيف القائم على المشروع مستودع المساهم إلى المشروع ويدمج التعديلات على جهازه.
٦. يدفع القائم على المشروع هذه التعديلات إلى المستودع الرئيس.



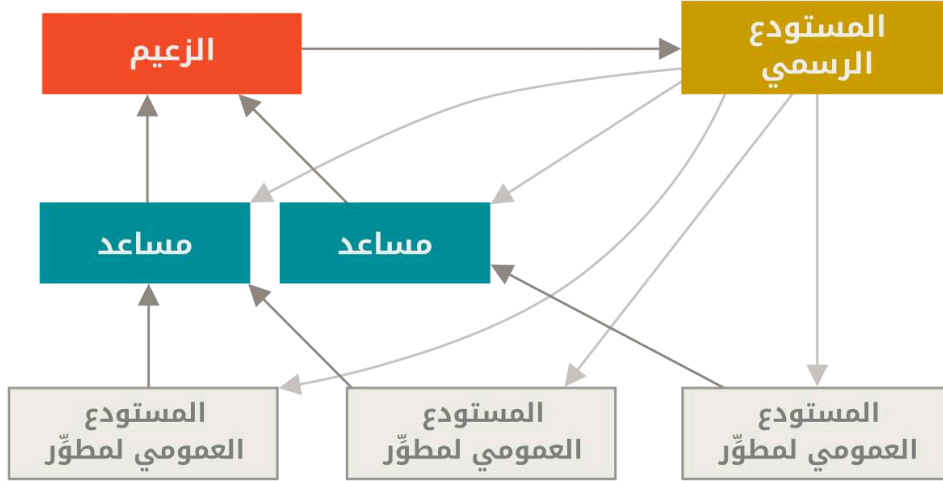
شكل ٥٤. أسلوب مدير الدمج

هذا الأسلوب هو الأشهر في أدوات التطوير الملتقي مثل جت هب وجت لاب. فسهل فيها اشتقاق مشروع ودفع تعديلاتك إلى نسختك المشتقة لبراها الجميع. من مزايا هذا الأسلوب إمكانك مواصلة العمل، وإمكان القائم على المستودع الرئيس أن يجذب تعديلاتك في أي وقت. فلا يحتاج المساهمون إلى انتظار المشروع أن يدمج تعديلاتهم، فكلّ يعمل على راحته.

٥، ١، ٣، أسلوب الزعيم والمساعدين

هذا شكل من أشكال أسلوب «المستودعات العديدة». تتبَّعه غالباً المشروعات العملاقة ذات المئات من المساهمين. من أشهرها نواة لينكس. وفيه نجد عدداً من مديري الدمج، كلٌّ منهم مسؤول عن جزء معين من المستودع، ويسمون المساعدِين (lieutenants). ولجميع المساعدين مدير دمج واحد يسمى الزعيم أو الدكتاتور الخيّر (benevolent dictator). ويدفع الزعيم إلى المستودع المرجع، الذي يجذب منه جميع المساهمين. تجري هذه العملية هكذا (انظر شكل [أسلوب الزعيم والمساعدين](#)):

١. يعمل المطورون العاديون في فروع موضوعات، ويعيدون تأسيس عملهم على الفرع الرئيس للمستودع المرجع الذي يدفع الزعيم إليه.
٢. يدمج المساعدون فروع موضوعات المطورين في فروعهم الرئيسة.
٣. يدمج الزعيم فروع المساعدين الرئيسة في فرعه الرئيس.
٤. وأخيراً، يدفع الزعيم فرعه الرئيس إلى المستودع المرجع ليتسنى للمطورين إعادة التأسيس عليه.



شكل ٥٥. أسلوب الزعيم والمساعدين

ليس هذا الأسلوب شائعاً، لكنه مفيد في المشروعات الضخمة وفي البيئات ذات المراتب العديدة من المطورين. فإنه يتيح لقائد المشروع (الزعيم) تفويض معظم العمل إلى الآخرين، وجمع أجزاء كبيرة من التعديلات في نقاط متعددة قبل دمجها معاً.

٥، ١، ٤، أنماط إدارة فروع المصدر البرمجي

صنّف Martin Fowler دليلاً باسم "Patterns for Managing Source Code Branches" («أنماط إدارة فروع المصدر البرمجي»). يتناول هذا الدليل أساليب التطوير الشائعة في جت، ويشرح كيف تستعملها ومتى. وفيه كذلك قسم يقارن بين الدمج المتواتر والدمج المتباعد.



الدليل بالإنجليزية: <https://martinfowler.com/articles/branching-patterns.html>

٥، ١، ٥، خلاصة أساليب التطوير

هذه من أشهر أساليب التطوير المستعملة، الممكنة مع نظام إدارة نسخ متوزع مثل جت. لكن الكثير من التنويعات ممكنة حتى تلائم أسلوبك في الواقع. لأنك الآن قد تستطيع تحديد شكل الأسلوب الذي قد يلائمك، فسنتناول أمثلة محددة للقيام بالوظائف الأساسية التي تتكون منها الأساليب المختلفة. سنتعلم في الفصل التالي بعض أشهر أنماط المساهمة في مشروع.

٥، ٢، المساهمة في مشروع — مقدمة

أكبر مشكلة في شرح كيف تساهم في مشروع هي تعدد أساليب المساهمة. فإن الناس يتعاونون بطرائق شتى بسبب مرونة جت الكبيرة. فيصعب شرح كيف عليك أن تساهم؛ فكل مشروع مختلف. من العوامل المؤثرة: عدد المساهمين النشطين، وأسلوب التطوير المتبع، وهل لديك إذن بالإيداع في المشروع أم لا، وربما أسلوب المساهمة من خارج الفريق الأساسي.

العامل الأول هو عدد المساهمين النشطين: كم مبرمجًا يساهم بهمة ونشاط في المشروع، وكل متى؟ كثيرًا ما ستجد للمشروع مطوريين أو ثلاثة، وبضعة إيداعات في اليوم، أو ربما أقل للمشروعات الخاملة نسبيًا. أما الشركات والمشروعات الكبيرة، فقد تجد لها آلاف المطورين، ومئات أو آلاف من الإيداعات كل يوم. هذا مهم، لأن مع زيادة المطورين، ستواجه مشاكل في التحقق أن تعديلاتك تُدمج بسهولة وبنظافة. فتعديلاتك قد تصير باطلة أو شديدة التلف بسبب دمج تعديلات أخرى خلال عملك أو خلال انتظارك قبول عملك أو دمج. فكيف إذاً يمكنك إبقاء مصدرك البرمجي محدثًا وإيداعاتك صالحة؟

العامل التالي هو أسلوب التطوير المتبع. هل هو مركزي وكل المطورين متساوون في أن لديهم إذن الدفع إلى الفرع الرئيس؟ هل للمشروع مشرف أو مدير دمج ينظر في كل الرقع؟ هل يجب أن يراجع كل الرقع المساهمون الآخرون ويقبلوها (peer-review)؟ هل أنت جزء من هذه العملية؟ هل تتبعون نظام الزعيم والمساعدين وعليك تقديم عملك إلى المساعد أولاً؟

العامل التالي هو الإذن لك بالإيداع، فالأسلوب المطلوب للمساهمة في مشروع سيختلف كثيرًا إذا كنت تستطيع الدفع إلى المستودع مباشرة أم لا. فإذا لم تكن، فكيف يجب المشروع قبول الأعمال المساهم بها؟ هل لديهم سياسة معينة؟ ما قدر العمل الذي تساهم به في كل مرة؟ كم مرة تساهم؟

كل هذه الأسئلة لها أثر في تحديد كيفية مساهمتك الفعالة في مشروع وفي تحديد الأساليب المناسبة أو المتاحة لك. سنتناول جوانب من كل هؤلاء في سلسلة من المواقف المحتملة، منتقلين من الأيسر إلى الأبعد؛ سنتستطيع من هذه الأمثلة بناء الأساليب المحددة التي تحتاجها.

٥، ٣، المساهمة في مشروع — إرشادات الإيداع

قبل أن نشرع في تناول حالات معينة، هذه ملاحظة قصيرة عن رسائل الإيداعات: وجود إرشادات جيدة لصناعة الإيداعات والالتزام بها يسهل كثيرا العمل مع جت والتعاون مع الآخرين. يتيح مشروع جت مستندًا يسرد نصائح حسنة لصناعة الإيداعات وإرسال الرقع؛ يمكنك قراءته في مستودعه في ملف Documentation/SubsubmittingPatches (الرابط:)

characters or so. In some contexts, the first line is treated as the subject of an email and the rest of the text as the body. The blank line separating the summary from the body is critical (unless you omit the body entirely); tools like rebase will confuse you if you run the two together.

Write your commit message in the imperative: "Fix bug" and not "Fixed bug" or "Fixes bug." This convention matches up with commit messages generated by commands like `git merge` and `git revert`.

Further paragraphs come after blank lines.

- Bullet points are okay, too
- Typically a hyphen or asterisk is used for the bullet, followed by a single space, with blank lines in between, but conventions vary here
- Use a hanging indent

إذا اتبعت جميع رسائل إيداعاتك هذا النموذج، فستكون الأمور أيسر كثيرًا عليك وعلى المطورين الذين يتعاونون معك. ورسائل إيداع مشروع جت نفسه حسنة الصياغة؛ اسردها بالأمر `git log --no-merges` لترى كيف يبدو تاريخ إيداعات مشروع مصاغ صياغة حسنة.

افعل كما نقول، لا كما نفعل

في هذا الكتاب أمثلة كثيرة ليست ذات رسائل إيداع حسنة الصياغة كهذه. إنما نستعمل خيار الرسالة - `m` مع أمر الإيداع `git commit` للاختصار.



بإيجاز: افعل كما نقول، لا كما نفعل.

٥، ٤، المساهمة في مشروع — فريق خصوصي صغير

أصغر شكل مشروع قد نقابله هو مشروع خصوصي له مطور واحد آخر أو مطوران. «خصوصي» في هذا السياق تعني مغلق المصدر، أي ليس متاحًا لعموم الناس. لديك أنت والمطورين الآخرين إذن الدفع إلى المستودع.

يمكنك في هذا اتباع أسلوب يشبه ما قد تتبعه مع Subversion أو نظام مركزي آخر. وستتفجع بمزايا مثل الإيداع بغير اتصال بالشبكة، والتفريع والدمج الأسهلان كثيرًا. لكن أسلوب التطوير متطابق تقريبًا؛ ليس أكبر اختلاف إلا أن الدمج يحدث في المستودعات المحلية وليس في الخادوم عند الإيداع. لنرَ كيف يبدو عندما يعمل مطوران معًا في مستودع مشترك. سمير، المطور الأول، استنسخ المستودع وعدّل فيه وأودع محليًا. اختصرنا الرسميات التي في الرسائل إلى ...

```
# حاسوب سمير
$ git clone samir@github:simplegit.git
Cloning into 'simplegit'...
...
$ cd simplegit/
```

CONSOLE

```
$ vim lib/simplegit.rb
$ git commit -am 'Remove invalid default value'
[master 738ee87] Remove invalid default value
1 files changed, 1 insertions(+), 1 deletions(-)
```

وسميرة، المطورة الأخرى في المشروع، فعلت الشيء نفسه: استنسخت المستودع وعدّلت فيه وأودعت محلياً:

```
# حاسوب سميرة
$ git clone sameera@github:simplegit.git
Cloning into 'simplegit'...
...
$ cd simplegit/
$ vim TODO
$ git commit -am 'Add reset task'
[master fbff5bc] Add reset task
1 files changed, 1 insertions(+), 0 deletions(-)
```

ثم دفعت سميرة بعملها إلى الخادوم، فقبله على الرحب والسعة:

```
# حاسوب سميرة
$ git push origin master
...
To sameera@github:simplegit.git
1edee6b..fbff5bc master -> master
```

يُظهر السطر الأخير رسالة مفيدة من عملية الدفع. وصيغتها «شِر-قديم»..«شِر-جديد» «شِر-أصل» <- «شِر-هدف»، حيث «شِر-قديم» هي الإشارة (“reference”) القديمة، و«شِر-جديد» هي الإشارة الجديدة، و«شِر-أصل» هي اسم الإشارة المحلية المدفوعة إلى الخادوم، و«شِر-هدف» هي اسم الإشارة التي على الخادوم ويُراد تحديثها. ستري مثل هذا الناتج في الشرح التالي، فالمعرفة اليسيرة بمعناه ستفيدك في فهم الحالات المختلفة للمستودعات. وللاستزادة، انظر توثيق أمر الدفع [git-push](https://git-scm.com/docs/git-push) (الرابط: <https://git-scm.com/docs/git-push>).

لنكمل مثالنا. بعد ما حدث بقليل، عدّ سمير في المستودع وأودع تعديلاته في مستودعه المحلي، وحاول دفعها إلى الخادوم نفسه:

```
# حاسوب سمير
$ git push origin master
To samir@github:simplegit.git
! [rejected] master -> master (non-fast forward)
error: failed to push some refs to 'samir@github:simplegit.git'
```

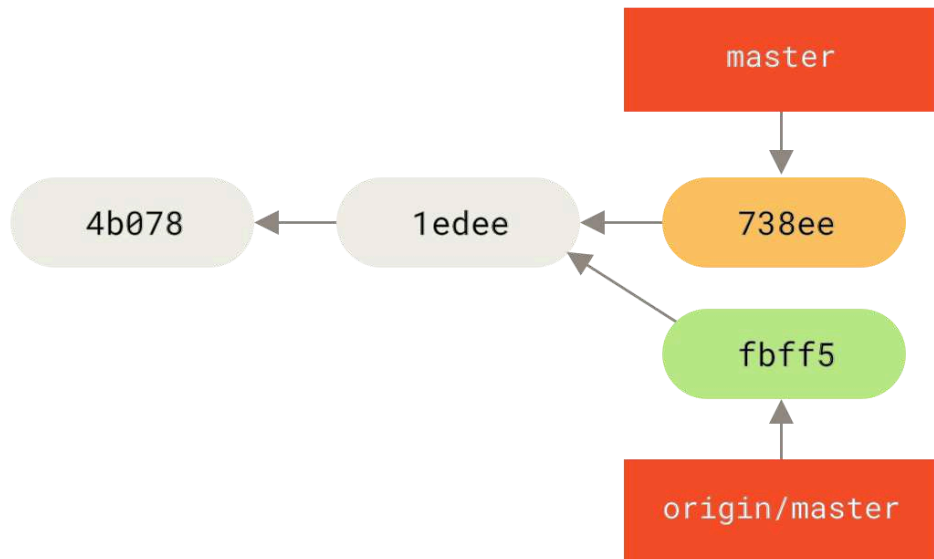
في هذه الحالة، فشل دفع سمير بسبب دفع سميرة السابق لتعديلاتها. هذا مهم جداً فهمه خصوصاً إذا كنت معتاداً على Subversion، لأنك تلاحظ أنهما لم يعدّلا في الملف نفسه. فإنّ عدّلت ملفات مختلفة فإنّ Subversion سيدمجها لك آلياً على الخادوم، لكن مع جت يجب عليك أولاً دمج الإيداعات محلياً. بلفظ آخر: يجب على سمير أن يستحضر (fetch) تعديلات سميرة من المستودع المنبع ويدمجها في مستودعه المحلي قبل أن يُسمح له بالدفع.

فخطوة سمير الأولى هي استحضار عمل سميرة (وهذا فقط يستحضر عملها من المستودع المنبع، لكن لا يدمجه في عمله):

```
$ git fetch origin
...
From samir@github:simplegit
+ 049d078...fbff5bc master    -> origin/master
```

CONSOLE

عندئذٍ يبدو مستودع سمير المحلي هكذا:



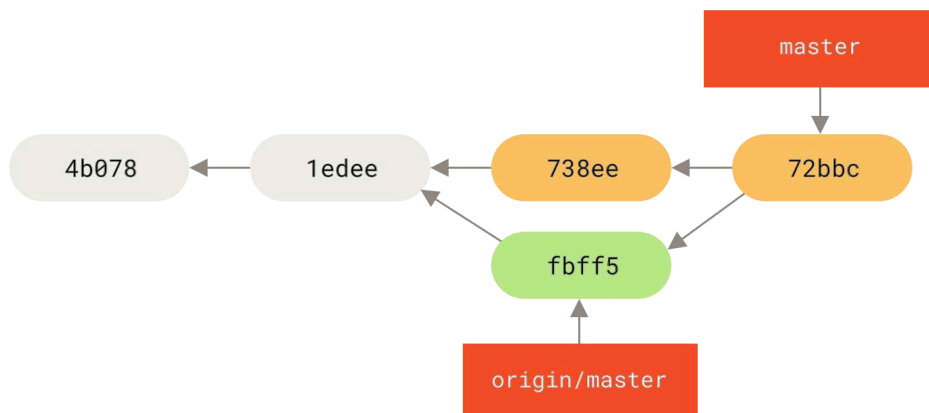
شكل ٥٧. التاريخ المفترق عند سمير

يستطيع سمير الآن أن يدمج في مستودعه المحلي عمل سميرة الذي استحضره.

```
$ git merge origin/master
Merge made by the 'recursive' strategy.
TODO | 1 +
1 files changed, 1 insertions(+), 0 deletions(-)
```

CONSOLE

ما دام هذا الدمج المحلي قد تم بسلام، فسيبدو التاريخ الجديد عند سمير هكذا:



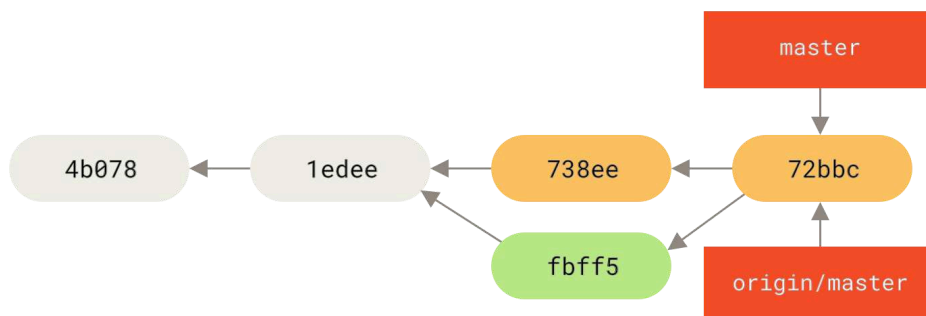
شكل ٥٨. مستودع سمير بعد دمج origin/master

عندئذٍ قد يودّ سمير أن يجرب هذا المصدر البرمجي الجديد ليطمئن أن لا شيء من عمل سميرة قد أثر على عمله. فإذا بدا كل شيء بخير، فيمكنه أخيراً أن يدفع هذا العمل الدموج إلى الخادوم:

```
$ git push origin master
...
To samir@github:simplegit.git
fbff5bc..72bbc59 master -> master
```

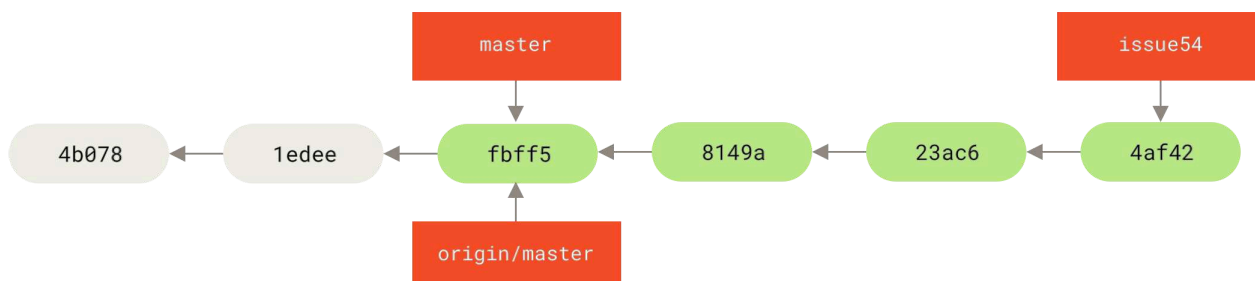
CONSOLE

وفي النهاية، سيبدو تاريخ إيداعات سمير هكذا:



شكل ٥٩. التاريخ عند سمير بعد الدفع إلى الخادوم origin

وفي هذه الأثناء أنشأت سميرة فرع موضوع جديد وسمّته issue54 وأودعت فيه ثلاثة إيداعات. ولم تستحضر تعديلات سمير بعد. فيبدو تاريخ إيداعاتها هكذا:



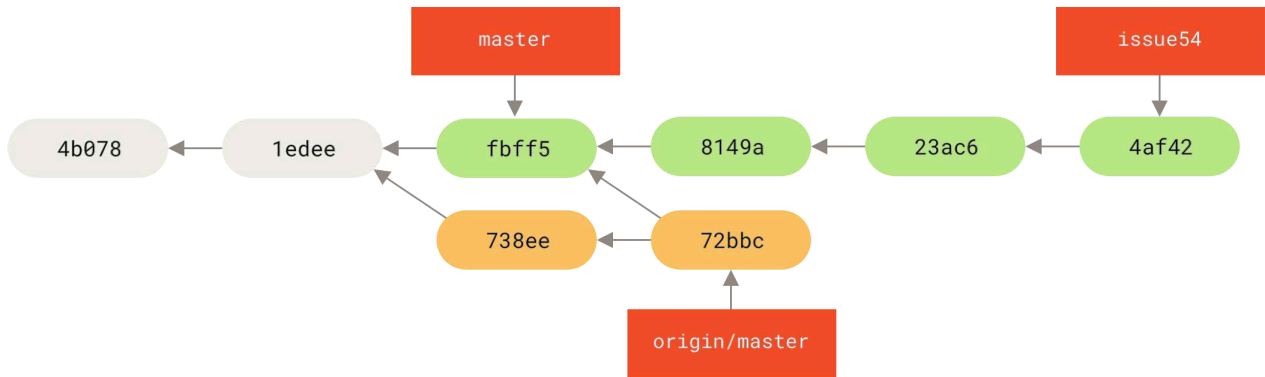
شكل ٦٠. فرع الموضوع عند سميرة

وفجأةً علمت سميرة بخبر دفع سمير إيداعاتٍ إلى الخادوم، وتريد أن تنظر فيها، فاستحضرت ما لدى الخادوم من جديد:

```
# حاسوب سميرة
$ git fetch origin
...
From sameera@github:simplegit
fbff5bc..72bbc59 master -> origin/master
```

CONSOLE

هذا يجذب العمل الذي دفعه سمير في هذه الأثناء، فصار التاريخ عند سميرة هكذا:



شكل 7. التاريخ عند سميرة بعد استحضار تعديلات سمير

ثم رأت سميرة أن فرع موضوعها قد صار جاهزاً، لكنها أرادت أن ترى أيّ جزء من عمل سمير الذي استحضرت، يجب عليها دمجها في عملها حتى تستطيع الدفع. فاستعملت أمر السجل:

```

$ git log --no-merges issue54..origin/master
commit 738ee872852dfaa9d6634e0dea7a324040193016
Author: Samir Samara <samsam@example.com>
Date:   Fri May 29 16:01:27 2009 -0700

```

Remove invalid default value

CONSOLE

الصيغة `issue54..origin/master` هي «مصفاة سجل» تسأل جت أن يعرض فقط الإيداعات التي في الفرع الآخر (`origin/master` هنا) وليست في الفرع الأول (`issue54` هنا). سنفضّل هذه الصيغة في فصل نطاقات الإيداعات.

من هذا الناتج، نرى أن إيداعاً واحداً فقط صنعه سمير ولم يُدمج بعد في المستودع المحلي عند سميرة. فعندما تدمج سميرة `origin/master` فإن هذا الإيداع وحده الذي سيُغيّر مستودعها المحلي.

تستطيع سميرة الآن أن تدمج فرع الموضوع في فرعها الرئيس ثم تدمج عمل سمير (`origin/master`) في فرعها الرئيس، ثم تدفع هذا العمل المدموج إلى الخادوم.

فأولاً، بعد أن أودعت سميرة كل عملها في فرع الموضوع `issue54`، انتقلت إلى فرعها الرئيس استعداداً لدمج كل هذا:

```

$ git checkout master
Switched to branch 'master'
Your branch is behind 'origin/master' by 2 commits, and can be fast-forwarded.

```

CONSOLE

تستطيع سميرة الآن أن تدمج أيّ الفرعين أولاً (`origin/master` أو `issue54`)؛ كلاهما «منبع»، فلا يهم الترتيب. ولقطة المشروع في آخر المطاف ستتطابق في الحالين؛ لن يختلف سوى شكل التاريخ. فاختارت دمج فرع `issue54` أولاً:

```

$ git merge issue54
Updating fbff5bc..4af4298
Fast forward

```

CONSOLE

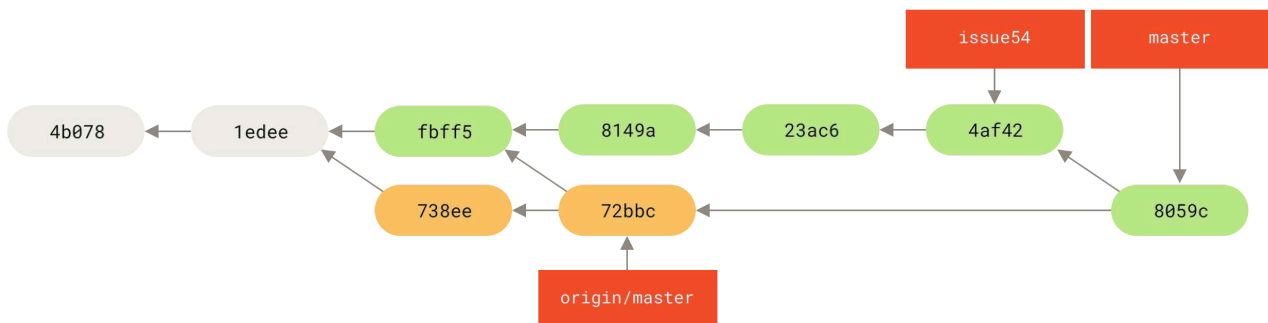
```
README | 1 +
lib/simplegit.rb | 6 +++++-
2 files changed, 6 insertions(+), 1 deletions(-)
```

لم تظهر مشكلة؛ إنما كان دمج تسريع كما رأيت. فأكملت سميرة الدمج المحلي بدمج عمل سمير التي استحضرت من قبل وبقى في فرع `origin/master`:

```
$ git merge origin/master
Auto-merging lib/simplegit.rb
Merge made by the 'recursive' strategy.
lib/simplegit.rb | 2 +-
1 files changed, 1 insertions(+), 1 deletions(-)
```

CONSOLE

دُمج كل شيء بنظافة، وصار التاريخ عند سميرة هكذا:



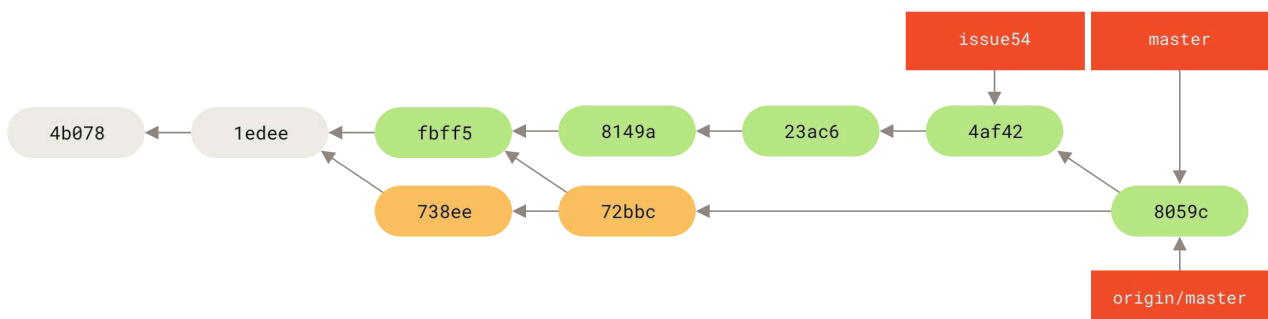
شكل ٦٢. التاريخ عند سميرة بعد دمج تعديلات سمير

صار الآن `origin/master` موصولاً بفرع `master` الخاص بسميرة، فنتسطيع الآن دفعه بسلام (بفرض أن سمير لم يدفع إبداعات أخرى جديدة في هذه الأثناء):

```
$ git push origin master
...
To sameera@github:simplegit.git
72bbc59..8059c15 master -> master
```

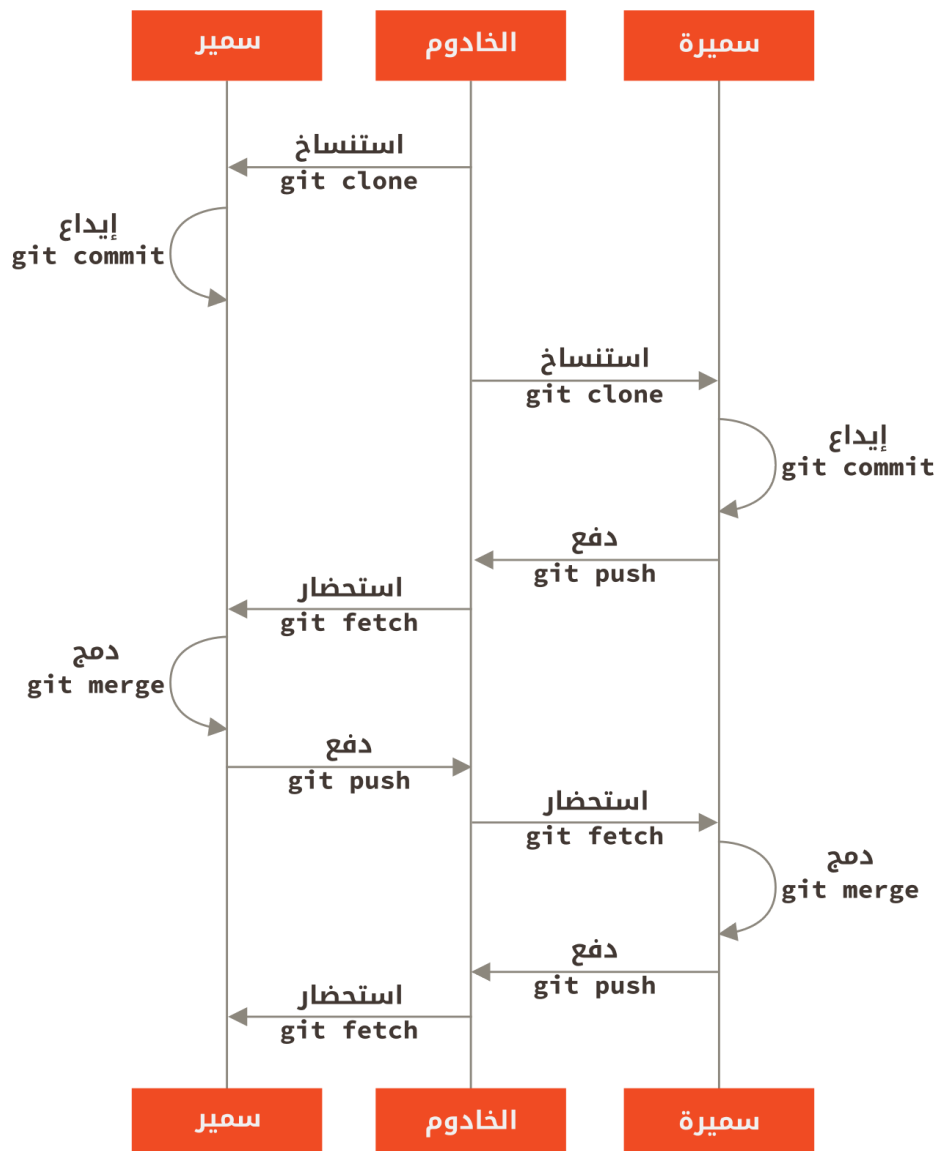
CONSOLE

كلُّ منهما أودع عدة مرات ودمج عمل الآخر بسلام.



شكل ٦٣. التاريخ عند سميرة بعد دفع كل التعديلات إلى الخادوم

هذا من أسهل أساليب التطوير: فإنك تعمل قليلاً (غالباً في فرع موضوع)، ثم تدمج عملك في فرعك الرئيس عندما يكون جاهزاً. وعندما تريد مشاركته فإنك تستحضر الفرع الرئيس المنبع وتدمج ما فيه (إذا تغيّر) في فرعك الرئيس المحلي، وأخيراً تدفع هذا إلى الفرع الرئيس المنبع. ويكون الشكل العام لتسلسل الأحداث هكذا:



شكل ٦٤. شكل عام لتسلسل الأحداث في أسلوب تطوير بسيط لعدة مطورين

٥، ٥، المساهمة في مشروع

— فريق خصوصي بمدير دمج

سنتناول الآن أدوار المساهمين في مجموعة خصوصية أكبر. ستتعلم كيف تعمل في بيئة فيها مجموعات صغيرة تتعاون في أمور مختلفة ثم يدمج عملهم أحد آخر.

لنقل إن سمير وسميرة يعملان معاً في ميزة واحدة (ولتكن "featureA")، بينما تعمل سميرة مع سمر، وهي مطوّرة ثالثة في الفريق، تعملان في ميزة أخرى (ولتكن "featureB"). في هذه الحالة تتبع الشركة نوعاً من أسلوب مدير الدمج، حيث لا يدمج أعمال مجموعات المبرمجين إلا مهندسون محدّدون، والفرع الرئيس للمستودع الرئيس لا يدفع إليه إلا هؤلاء المهندسون. في هذا الموقف، كل العمل يحدث في فروع مجموعات المبرمجين، ثم يجذبه الدامجون فيما بعد.

لنتابع سميرة وهي تعمل على الميزتين وتتعاون مع مبرمجين مختلفين في آن واحد في هذه البيئة. استنسخت سميرة مستودعها، ثم رأت أن تعمل على featureA أولاً. فأنشأت فرعاً جديداً لهذه الميزة وبدأت بالعمل فيه:

```
# حاسوب سميرة
$ git checkout -b featureA
Switched to a new branch 'featureA'
$ vim lib/simplegit.rb
$ git commit -am 'Add limit to log function'
[featureA 3300904] Add limit to log function
1 files changed, 1 insertions(+), 1 deletions(-)
```

عندئذٍ احتاجت أن تشارك عملها مع سمير، فدفعت إيداعات فرعها featureA إلى الخادوم. ولكن ليس لسميرة إذن الدفع إلى الفرع الرئيس (الدامجون فقط لديهم هذا الإذن)، فاحتاجت إلى الدفع إلى فرع آخر حتى تتعاون مع سمير:

```
$ git push -u origin featureA
...
To sameera@github:simplegit.git
* [new branch]      featureA -> featureA
```

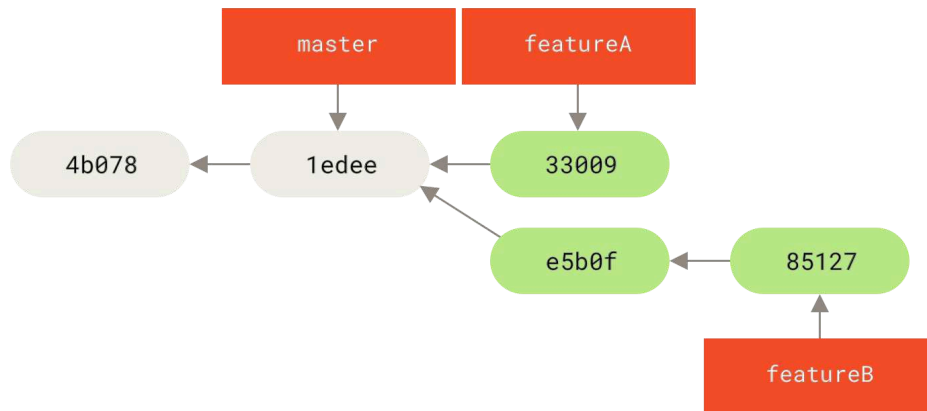
ثم أرسلت سميرة إلى سمير بريداً تخبره بدفعها شيئاً إلى فرع اسمه featureA حتى ينظر فيه. وحتى يرد سمير، رأت سميرة أن تشرع في العمل على featureB مع سمير. ولكي تبدأ، أنشأت فرع موضوع جديد مبني على الفرع الرئيس للخادوم:

```
# حاسوب سميرة
$ git fetch origin
$ git checkout -b featureB origin/master
Switched to a new branch 'featureB'
```

ثم صنعت سميرة إيداعين في فرع featureB الذي أنشأته:

```
$ vim lib/simplegit.rb
$ git commit -am 'Make ls-tree function recursive'
[featureB e5b0fdc] Make ls-tree function recursive
1 files changed, 1 insertions(+), 1 deletions(-)
$ vim lib/simplegit.rb
$ git commit -am 'Add ls-files'
[featureB 8512791] Add ls-files
1 files changed, 5 insertions(+), 0 deletions(-)
```

يبدو الآن مستودع سميرة هكذا:



شكل 70. تاريخ إيداعات سميرة أول الأمر

ثم وجدت أن عملها جاهزٌ للدفع، لكن وصلها يريد من سمر أنها قد أنشأت فرعاً فيه بعض العمل على الميزة الثانية (“featureB”) ودفعته إلى الخادوم في الفرع featureBee. ففتحنا سميرة إلى دمج تلك التعديلات في عملها قبل دفعه إلى الخادوم. فأولاً استحضرت سميرة تعديلات سمر بأمر الاستحضار:

```

$ git fetch origin
...
From sameera@github:simplegit
* [new branch]      featureBee -> origin/featureBee
  
```

CONSOLE

وسميرة ما زالت في فرع featureB الخاص بها، فتستطيع الآن دمج عمل سمر في فرعها بأمر الدمج:

```

$ git merge origin/featureBee
Auto-merging lib/simplegit.rb
Merge made by the 'recursive' strategy.
 lib/simplegit.rb | 4 ++++
 1 files changed, 4 insertions(+), 0 deletions(-)
  
```

CONSOLE

عندئذٍ أرادت سميرة دفع عمل “featureB” المدموج إلى الخادوم، لكنها لم تنشأ دفع فرعها featureB نفسه؛ فلأن سمر قد أنشأت فرع featureBee على الخادوم، فأرادت سميرة أن تدفع إلى ذلك الفرع نفسه، وذلك بالأمر:

```

$ git push -u origin featureB:featureBee
...
To sameera@github:simplegit.git
 fba9af8..cd685d1 featureB -> featureBee
  
```

CONSOLE

يسمى هذا «مُعَرِّف إشارة» (“refspec”). انظر فصل ~~سُرف الإشارة~~ (refspec) لنقاش مفصّل عن معرّفات الإشارة في جت وعن أمور أخرى يمكن أن تفعلها بها. لاحظ كذلك الخيار -u ؛ هذا اختصار --set-upstream («تعيين المنبع»)، الذي يهيئ الفروع للدفع والجذب بسهولة أكبر فيما بعد.

فجأةً أتى سميرة بريدٌ من سمر يخبرها أنه دفع إيداعاتٍ إلى فرع featureA الذي يتعاونان فيه، ويسألها أن تنظر فيها. فكررت سميرة أمر الاستحضار git fetch لاستحضار كل جديد لدى الخادوم، شاملاً بالطبع عمل سمر الأخير:

```
$ git fetch origin
...
From sameera@github:simplegit
 3300904..aad881d  featureA  -> origin/featureA
```

تستطيع سميرة الآن أن تعرض سجل أعمال سمير الجديدة بمقارنة محتوى الفرع المستحضر حديثاً featureA مع نسختها المحلية من الفرع نفسه:

```
$ git log featureA..origin/featureA
commit aad881d154acdaeb2b6b18ea0e827ed8a6d671e6
Author: Samir Samara <samsam@example.com>
Date: Fri May 29 19:57:33 2009 -0700
```

Increase log output to 30 from 25

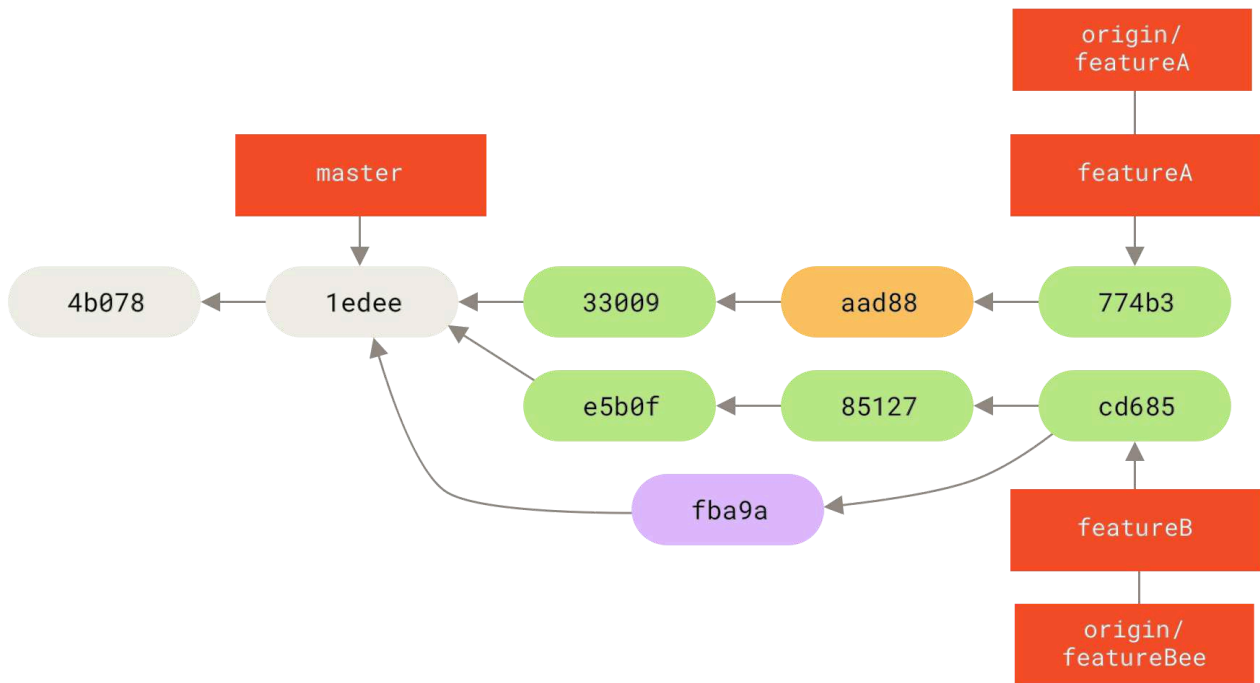
فإذا راق سميرة ما رأت، فتستطيع دمج عمل سمير في فرع featureA الخاص بها بالأمر:

```
$ git checkout featureA
Switched to branch 'featureA'
$ git merge origin/featureA
Updating 3300904..aad881d
Fast forward
 lib/simplegit.rb | 10 ++++++++-
1 files changed, 9 insertions(+), 1 deletions(-)
```

وأخيراً قد تود سميرة أن تغير أشياءً يسيرة فيه بعد الدمج، فيمكنها عمل هذه التعديلات، وإيداعها في فرع featureA المحلي الخاص بها، ثم دفع الناتج النهائي إلى الخادوم:

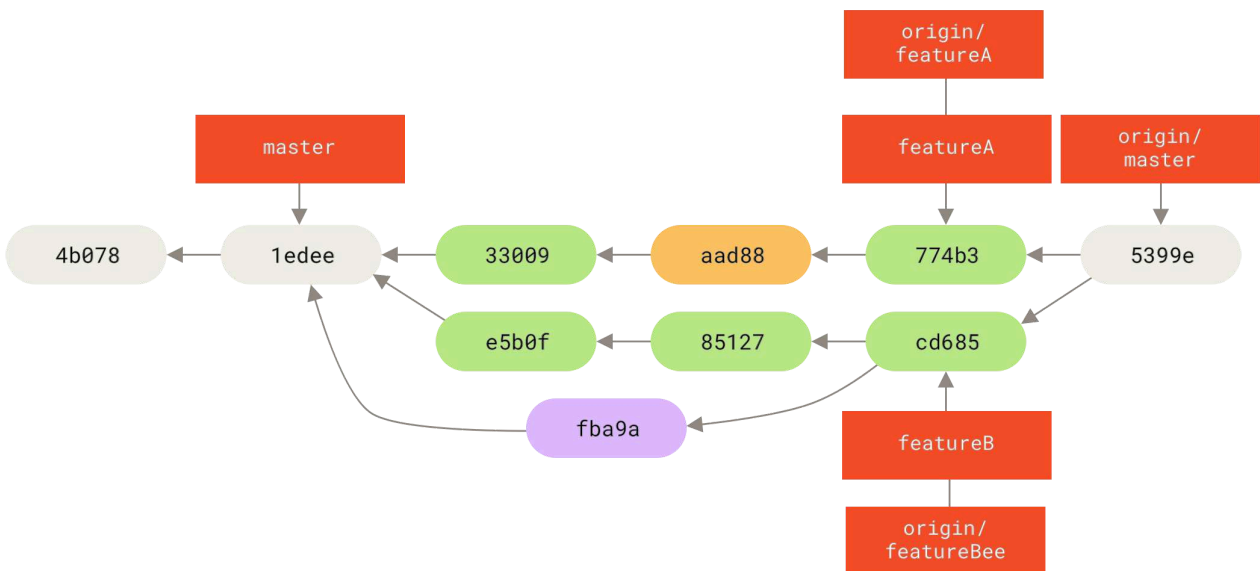
```
$ git commit -am 'Add small tweak to merged content'
[featureA 774b3ed] Add small tweak to merged content
 1 files changed, 1 insertions(+), 1 deletions(-)
$ git push
...
To sameera@github:simplegit.git
 3300904..774b3ed  featureA -> featureA
```

والآن يبدو تاريخ الإيداعات عند سميرة هكذا:



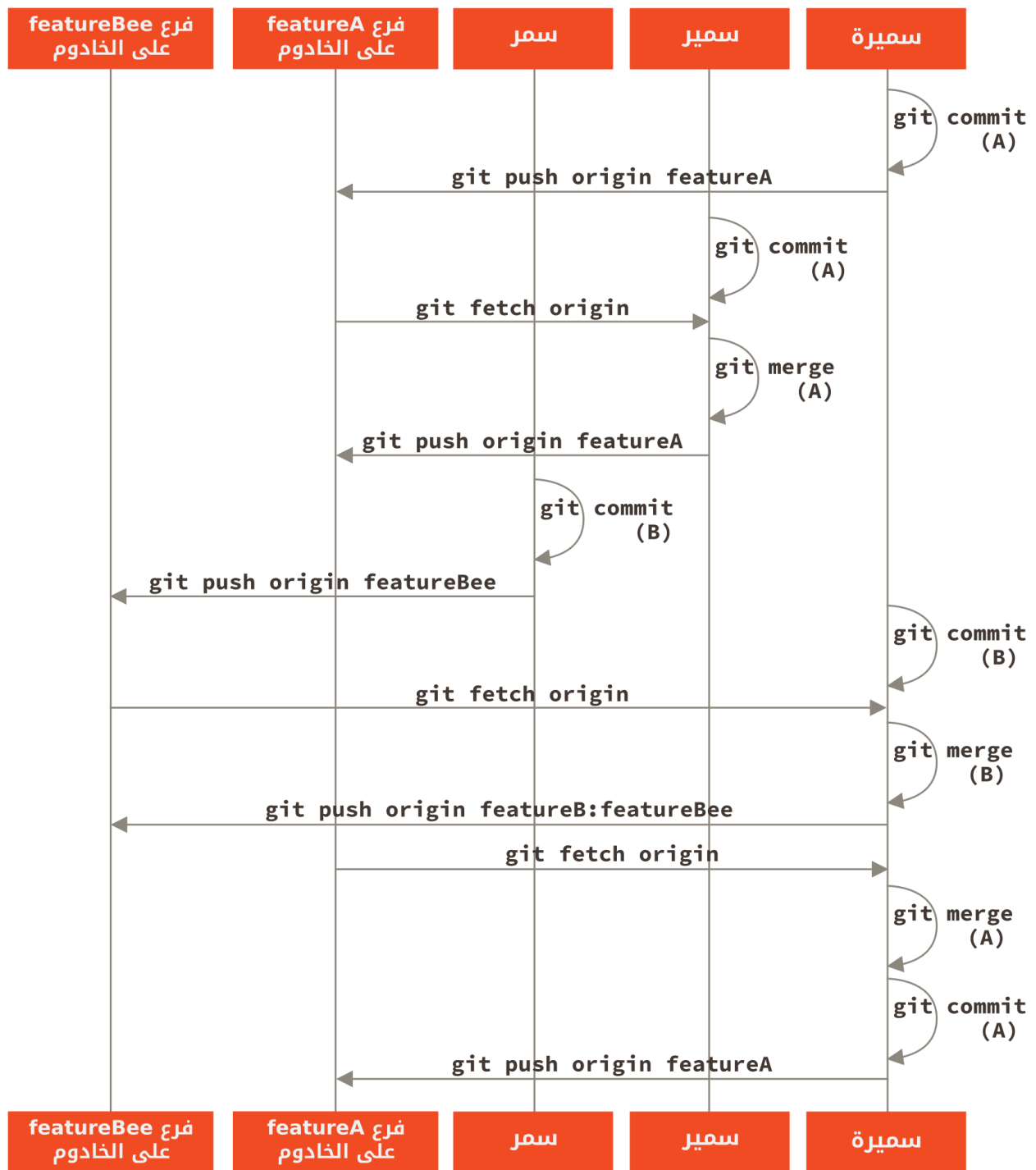
شكل ٦٦. التاريخ عند سميعة بعد الإيداع في فرع موضوع

وفيما بعد، سميعة وسمير وسمر يخبرون الدامجين أن الفرعين featureA و featureBee على الخادوم جاهزان للدمج في الفرع الرئيس. وبعد أن يدمجها الدامجون في الفرع الرئيس، فإن الاستحضار منه سيجلب إيداع الدمج، فيصير التاريخ هكذا:



شكل ٦٧. التاريخ عند سميعة بعد دمج فرعَي الموضوعين

ينتقل الكثيرون إلى جت لقدرته هذه أن يقبل عمل عدة مجموعات على التوازي، ثم دمج مسارات عمل مختلفة بعد هذا. إنها لمزية عظيمة في جت أن يسمح لمجموعات فرعية صغيرة من فريق واحد أن تتعاون عبر الفروع البعيدة بغير الاضطرار إلى إشراك الفريق كله أو تعطيله. الشكل العام لتسلسل الأحداث الذي رأيته للتو هكذا:



شكل ٦٨. شكل عام لتسلسل الأحداث في أسلوب تطوير فريق بمدير دمج

٥، ٦، المساهمة في مشروع — مشروع عمومي باشتاقات

المساهمة في مشروع عمومي أمرها مختلف. فليس لديك إذن الدفع فيه لتستطيع تحديث فروعه مباشرةً، فعليك إرسال عملك إلى القائمين عليه بطريقة أخرى. يشرح المثال الأول هذا المساهمة بالاشتقاق في مواقع استضافة جت التي تسهّل الاشتقاق. مواقع كثيرة تدعمه (منها جت هب GitHub، و BitBucket، و repo.or.cz، وآخرون)، والكثير من مديري المشروعات يتوقعون أسلوب المساهمة هذا. أما الفصل التالي فسيتناول المشروعات التي تحب قبول الرقع المساهم بها عبر البريد الشابيكي.

أولاً عليك غالباً استنساخ المستودع الرئيس، وإنشاء فرع موضوع للرقعة أو سلسلة الرقع التي تريد المساهمة بها، ثم العمل فيه. فسلسلة الأفعال تبدو هكذا:

```
$ git clone «رابط-المستودع»
$ cd project
$ git checkout -b featureA
... عمل ...
$ git commit
... عمل ...
$ git commit
```

CONSOLE

قد تود استعمال إعادة التأسيس التفاعلية `git rebase -i` لهرس عملك ("squash") إلى إيداع واحد، أو لإعادة ترتيب الإيداعات، حتى تسهّل مراجعة الرقعة على المشرف؛ انظر فصل [تحرير التاريخ](#) لمعلومات أزيد عن إعادة التأسيس التفاعلية.



عندما تفرغ من عملك في الفرع وتكون مستعداً لمشاركته مع القائمين على المشروع، اذهب إلى صفحة المشروع الأصل واضغط زر "Fork" («اشتقاق»)، لتُنشئ نسختك الخاصة بك من المشروع التي تستطيع الدفع إليها. عندئذٍ تحتاج إضافة رابط هذا المستودع ليكون بعيداً جديداً في مستودعك المحلي؛ لنسمّيه فيه هذا المثال `myfork`:

```
$ git remote add myfork «الرابط»
```

CONSOLE

عندئذٍ تدفع عملك إلى هذا المستودع. من الأسهل أن تدفع (إلى اشتقاقك) فرع الموضوع الذي عملت فيه، بدلاً من دمج عملك في الفرع الرئيس المحلي ودفع هذا الفرع. وذلك حتى لا تضطر إلى إرجاع فرعك الرئيس إلى الخلف، إن لم يقبلوا عملك أو أنهم اصطفوه (نتناول عملية الاصطفاء `cherry-pick` في جت بمزيد من التفصيل في فصل [أساليب الاصطفاء وإعادة التأسيس](#)). فإذا القائمون على المشروع دمجوا عملك (`merge`)، أو أعادوا تأسيسه (`rebase`)، أو اصطفوه (`cherry-pick`)، فستحتاج إلى إحضاره من جديد بال جذب من مستودعهم بطريقة ما.

وعموماً، يمكنك دفع عملك بالأمر:

```
$ git push -u myfork featureA
```

CONSOLE

ما إن يتم دفع عملك إلى اشتقاقك من المستودع، عليك إعلام القائمين على المشروع الأصل أن لديك شيئاً تود منهم أن يدمجوه. في الغالب يسمى هذا «طلب جذب» (`pull request`)، وفي المعتاد تُنشئ هذا الطلب إما عبر الموقع (قلدي جت هب آلية طلبات الجذب الخاصة به التي سنتناولها في باب [GitHub](#)) وإما بالأمر `git request-pull` وإرسال ناتجه بالبريد بنفسك إلى القائمين على المشروع.

(من المترجم) يسمى هذا «طلب جذب» في جت نفسه وفي عدة مواقع استضافة جت، ويسمى «طلب دمج» في مواقع جت أخرى. واختلف الناس في علة الاختلاف بين الاسمين؛ انظر الإجابات على هذا السؤال: [Pull request vs Merge request](https://stackoverflow.com/questions/22199432/pull-request-vs-merge-request) {الرابط: <https://stackoverflow.com/questions/22199432/pull-request-vs-merge-request> }.



و«الجذب» أقرب لعمل جت، فإنك تطلب من القائمين على المشروع الأصل أن يجذبوا (pull) عملك، والجذب يشمل الدمج. لكن «الدمج» أوضح لمن لم يضطلع بجت. فمن اختار «طلب جذب» اتبع جت نفسه، ومن اختار «طلب دمج» اختار الأوضح للمستخدمين.

ومما يؤيد «الجذب» أن الجذب هو استحضار fetch متبوعاً إما بدمج merge وإما بإعادة التأسيس rebase. لكن في العربية «الدمج» يعني «merge» ويعني «integrate» في سياق إدارة المراجعات.

أمر `git request-pull` تعطيه: (١) الفرع الأساس الذي تريدهم أن يجذبوا فرعك إليه، و (٢) رابط مستوع جت الذي تريدهم أن يجذبوا منه؛ ويُنتج لك خلاصة التعديلات التي تسألهم أن يجذبوها. فمثلاً، إذا أردت سميرة أن ترسل إلى سمير طلب جذب، وقد صنعت إيداعين في فرع الموضوع الذي دفعت إليه للتو، يمكنها تنفيذ هذا الأمر:

CONSOLE

```
$ git request-pull origin/master myfork
The following changes since commit 1edee6b1d61823a2de3b09c160d7080b8d1b3a40:
Sameera Samara (1):
    Create new function

are available in the git repository at:

https://github.com/simlegit.git featureA

Sameera Samara (2):
    Add limit to log function
    Increase log output to 30 from 25

lib/simlegit.rb | 10 ++++++--
1 files changed, 9 insertions(+), 1 deletions(-)
```

يمكن إرسال هذا الناتج إلى القائم على المشروع، فإنه يخبره من أين تفرّع العمل، ويوجز له الإيداعات، ويحدد له من أين يستطيع جذب العمل الجديد.

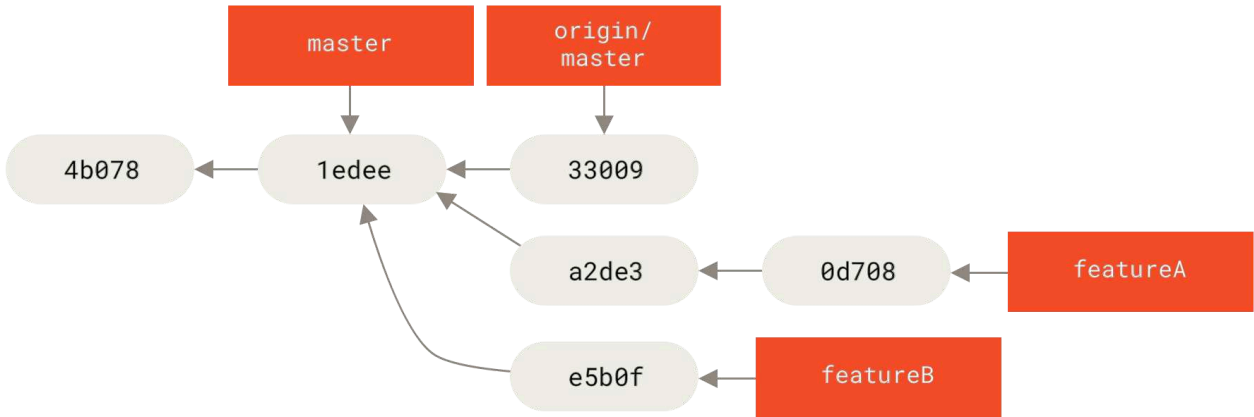
وإن الأسهل غالباً في مشروع لست القائم عليه أن تُبقي دوماً فرعك الرئيس المحلي يتبع الفرع البعيد المناظر له (مثلاً `master` الخاص بك يتبع `origin/master`)، وأن تفعل كل شيء في فروع موضوعات، حتى تستطيع أن تلغيها بسهولة إن رُفضت. وكذلك فصل مسارات العمل في فروع موضوعات يسهّل عليك إعادة تأسيسها إذا تحرك رأس المستودع الرئيس خلال عملك ولم تبقى إيداعاتك قابلة للتطبيق بنظافة. فمثلاً إذا أردت أن ترسل إلى المشروع تعديلات في موضوع آخر، فلا تستمر بالعمل في فرع الموضوع السابق الذي دفعته؛ بل ابدأ من جديد باشتقاق الفرع الرئيس للمستودع الأصل:

CONSOLE

```
$ git checkout -b featureB origin/master
... عمل ...
```

```
$ git commit
$ git push myfork featureB
$ git request-pull origin/master myfork
... إرسال طلب الجذب الناتج بالبريد الشبكي إلى القائم على المشروع ...
$ git fetch origin
```

فالآن كل موضوع منهما محتوًى في صومعة (مثل طابور الرقع) فيمكن تحرير أحدهما أو تعديله أو إعادة تأسيسه بغير تشابك الموضوعين أو اعتماد أحدهما على الآخر، مثل هذا:



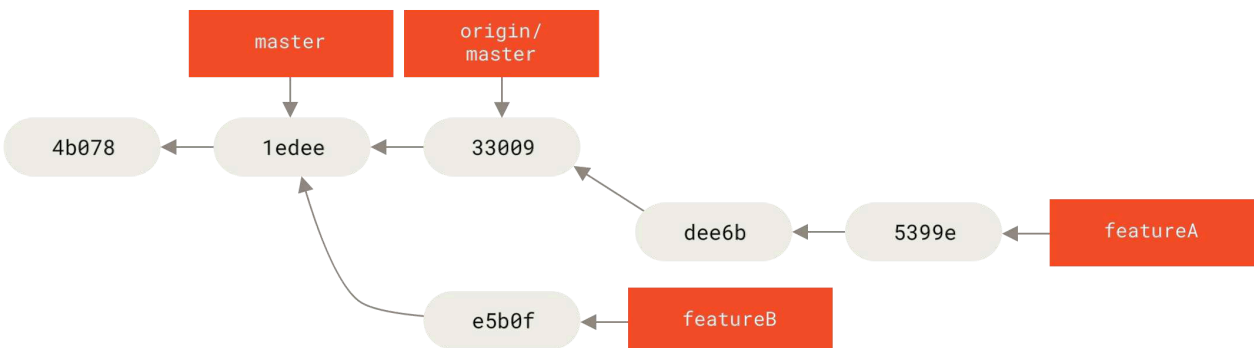
شكل ٦٩. تاريخ الإيداعات عند بدء العمل في featureB

لنقل إن القائم على المشروع قد جذب عددًا من الرقع الأخرى ثم حاول جذب فرعك الأول فلم يجده يقبل الدمج بنظافة. يمكنك عندئذٍ أن تعيد تأسيس هذا الفرع على `origin/master` وتحلّ النزاعات نيابةً عن القائم على المشروع، ثم تُعيد إرسال تعديلاتك:

```
$ git checkout featureA
$ git rebase origin/master
$ git push -f myfork featureA
```

CONSOLE

هذا يحرّر التاريخ ليبدو هكذا مثل شكل [تاريخ الإيداعات بعد العمل في featureA](#).



شكل ٧٠. تاريخ الإيداعات بعد العمل في featureA

ولأنك أعدت تأسيس الفرع، فعليك الدفع عَنوةً (خيار -f مع أمر push)، حتى تبدل فرع featureA على الخادوم بإيداع ليس مبنياً عليه. ويمكنك بدلاً من ذلك أن تدفع العمل الجديد إلى فرع آخر على الخادوم (مثلاً featureAv2).

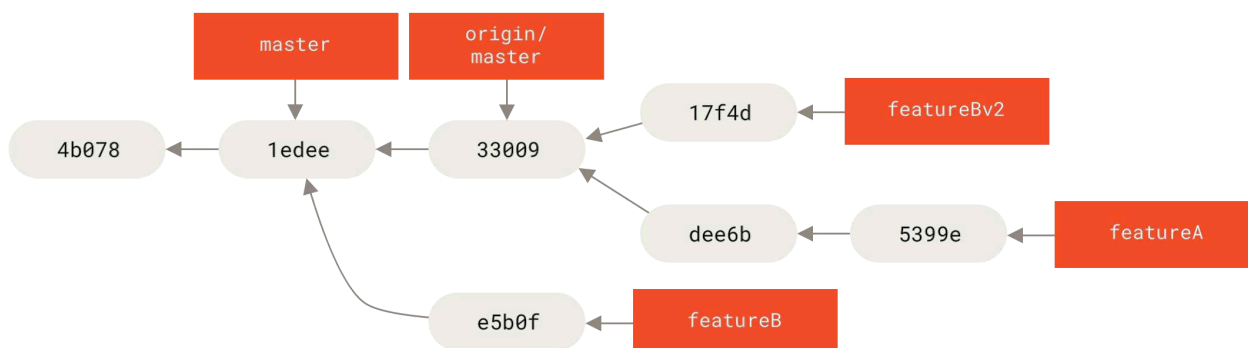
لنرَ موقفاً محتملاً آخر: نظر القائم على المشروع إلى عملك في فرعك الثاني، وراقه المبدأ، لكنه ودَّ أن تغيّر بعض التفاصيل. ورأيت أن تنتهز الفرصة لتعيد تأسيسه على الفرع الرئيس الحالي للمشروع. فبدأت فرعاً جديداً من فرع origin/master الحالي، وهرست إيداعات featureB إليه ("squash")، وحللت أي نزاعات ظهرت، وغيّرت له ما أراد، ثم دفعت هذا إلى فرع جديد على الخادوم:

```
$ git checkout -b featureBv2 origin/master
$ git merge --squash featureB
... change implementation ...
$ git commit
$ git push myfork featureBv2
```

CONSOLE

خيار الهرس --squash يتناول كل العمل الذي في الفرع المدموج، ويهرسه (يجعله إيداعاً واحداً) فيصير حال المستودع كأن ما حدث هو دمج حقيقي، بغير أن يصنع إيداع دمج. يعني هذا أن إيداعك الجديد سيكون له إيداع أب واحد، ويسمح لك باشتغال كل التعديلات التي في فرع آخر، ثم زيادة تعديلات أخرى عليها قبل صناعة الإيداع الجديد. وفي حالة الدمج العادي (وليس الهرس)، قد يفيدك أيضاً الخيار --no-commit لتأجيل إيداع الدمج.

عندئذٍ يمكنك إعلام القائم على المشروع أنك قد أتممت التعديلات المطلوبة، وأنه سيجدها في فرع featureBv2.



شكل ٧. تاريخ الإيداعات بعد العمل في featureBv2

٥، ٧، المساهمة في مشروع — مشروع عمومي عبر البريد الشبكي

للكثير من المشروعات أسلوب متفق عليه لقبول الرُقْع؛ عليك النظر في قواعد كل مشروع، لأن كل مشروع مختلف. ولأن العديد من المشروعات الكبيرة، ذات التاريخ، تقبل الرقع عبر قائمة المطوّرين البريدية، فسرى مثلاً على ذلك الآن.

هذا الأسلوب شبيهه بالسابق: تُنشئ فرع موضوع لكل سلسلة من الرقع تعمل عليها؛ إنما الفرق هو كيف ترسلها إلى المشروع: لا تشتق المستودع وتدفع إلى اشتقاقك، لكن تحوّل كل إيداع إلى رسالة بريدية وترسلها إلى قائمة المطورين البريدية:

```
$ git checkout -b topicA
... عمل ...
$ git commit
... عمل ...
$ git commit
```

لديك الآن إيداعان تود إرسالهما إلى القائمة البريدية، فتستعمل أمر تنسيق الرقع `git format-patch` لإنشاء ملفات بصيغة mbox حتى ترسلها بالبريد الشابيكي إلى القائمة. يحوّل هذا الأمر كل إيداع إلى رسالة بريد عنوانها السطر الأول من رسالة الإيداع، وباقي رسالة الإيداع في متن الرسالة مع رقعة الإيداع (التعديلات التي قدّمها هذا الإيداع، بصيغة مخصصة). الجميل في هذا أن تطبيق رقعة من رسالة أنشأها أمر تنسيق الرقع، يحافظ على معلومات الإيداع.

```
$ git format-patch -M origin/master
0001-add-limit-to-log-function.patch
0002-increase-log-output-to-30-from-25.patch
```

أمر تنسيق الرقع يطبع أسماء الملفات التي أنشأها، والخيار `-M` (اختصار `--find-renames`) يأمر جت أن يكتشف الملفات التي تغيّرت أسماؤها أو مساراتها. وتبدو الملفات في حالتنا هكذا:

```
$ cat 0001-add-limit-to-log-function.patch
From 330090432754092d704da8e76ca5c05c198e71a8 Mon Sep 17 00:00:00 2001
From: Sameera Samara <sameera@example.com>
Date: Sun, 6 Apr 2008 10:17:23 -0700
Subject: [PATCH 1/2] Add limit to log function

Limit log functionality to the first 20

---
lib/simplegit.rb | 2 +-
1 files changed, 1 insertions(+), 1 deletions(-)

diff --git a/lib/simplegit.rb b/lib/simplegit.rb
index 76f47bc..f9815f1 100644
--- a/lib/simplegit.rb
+++ b/lib/simplegit.rb
@@ -14,7 +14,7 @@ class SimpleGit
   end

   def log(treeish = 'master')
-    command("git log #{treeish}")
+    command("git log -n 20 #{treeish}")
   end

   def ls_tree(treeish = 'master')
--
2.1.0
```

ويمكن في ملفات الرقع هذه زيادة معلومات تريد أن تخبر بها القائمة البريدية ولا تظهر في رسالة الإيداع؛ فإذا أضفت نصًا بين السطر `---` وأول الرقعة (السطر `diff --git`)، فسيستطيع المطورون قراءته، لكن برنامج الترقيع سيتجاهله.

لإرسال هذا إلى القائمة البريدية، يمكنك أن تلصق محتوى الملف في برنامج بريدك الشابكي، أو أن ترسله عبر برنامج سطر الأوامر. لكن لصقه قد يضيع شيئاً من تنسيقته، خصوصاً مع برامج عملاء البريد «الذكية» التي لا تحافظ على فواصل السطور والمسافات البيضاء الأخرى كما ينبغي. لكننا نحمد الله أن جت يتيح لنا أداة تساعدنا في إرسال رقع منسقة كما ينبغي عبر ميفاق IMAP، الذي قد يكون أسهل لك. فسنشرح الآن كيف نرسل رقعة عبر Gmail، فهو وكيل البريد الشابكي الذي نعرف التعامل معه حق المعرفة؛ يمكنك الاطلاع على شرح مفصل لعدد من برامج البريد في آخر الملف السابق ذكره (<https://github.com/git/git/blob/master/Documentation/SubmittingPatches>) (الرابط: <https://github.com/git/git/blob/master/Documentation/SubmittingPatches>) في مستودع مصدر جت.

عليك أولاً إعداد قسم imap في ملف التهيئة الخاص بك ~/.gitconfig. يمكنك تنفيذ سلسلة من أوامر التهيئة git config لتعيين كل قيمة على حدة، أو إضافتهن جميعاً يدوياً. في النهاية سيكون ملف التهيئة مثل هذا:

```
[imap]
  folder = "[Gmail]/Drafts"
  host = imaps://imap.gmail.com
  user = user@gmail.com
  pass = YX]8g76G_2^sFbd
  port = 993
  sslverify = false
```

INI

إن لم يكن خادم IMAP يستعمل SSL، فغالباً لن تحتاج السطرين الآخرين، وستبدأ خانة host بـ imap:// وليس imaps://. بعد إعداد هذا، يمكنك تنفيذ أمر رفع المسودة git imap-send حتى تضع سلسلة الرقع في مجلد المسودات (Drafts) في خادم IMAP الذي حددته:

```
$ cat *.patch | git imap-send
Resolving imap.gmail.com... ok
Connecting to [74.125.142.109]:993... ok
Logging in...
sending 2 messages
100% (2/2) done
```

CONSOLE

عندئذٍ تجدها في مجلد المسودات؛ عدّل خانة «إلى» (To) لتجعل فيها عنوان القائمة البريدية التي تريد إرسال الرقعة إليها، وربما تضيف في خانة «نسخة للعلم» (CC) عنوان القائم على المشروع أو المسؤول عن هذا الجزء، ثم ترسلها.

ويمكنك كذلك إرسال هذه الرقع عبر خادم SMTP. يمكنك مثلما سبق تنفيذ سلسلة من أوامر التهيئة git config لتعيين كل قيمة على حدة، أو إضافتهن جميعاً يدوياً في قسم sendemail في ملف تهيئتك ~/.gitconfig:

```
[sendemail]
  smtpencryption = tls
  smtpserver = smtp.gmail.com
  smtpuser = user@gmail.com
  smtpserverport = 587
```

INI

بعدئذٍ ننفذ أمر الإرسال git send-email لإرسال رُقعتك:

```
$ git send-email *.patch
0001-add-limit-to-log-function.patch
0002-increase-log-output-to-30-from-25.patch
Who should the emails appear to be from? [Sameera Samara <sameera@example.com>]
Emails will be sent from: Sameera Samara <sameera@example.com>
Who should the emails be sent to? sameera@example.com
Message-ID to be used as In-Reply-To for the first email? y
```

عندئذٍ يطبع لك جت كتلة معلومات تقريرية لكل رقعة ترسها، مثل هذه:

```
(mbox) Adding cc: Sameera Samara <sameera@example.com> from
      \line 'From: Sameera Samara <sameera@example.com>'
OK. Log says:
Sendmail: /usr/sbin/sendmail -i sameera@example.com
From: Sameera Samara <sameera@example.com>
To: sameera@example.com
Subject: [PATCH 1/2] Add limit to log function
Date: Sat, 30 May 2009 13:29:15 -0700
Message-Id: <1243715356-61726-1-git-send-email-sameera@example.com>
X-Mailer: git-send-email 1.6.2.rc1.20.g8c5b.dirty
In-Reply-To: <y>
References: <y>

Result: OK
```

انظر موقع git-send-email.io [الرابط: https://git-send-email.io](https://git-send-email.io) للمساعدة في تهيئة نظامك وبريدك، ولنصائح وحيل أخرى، ولييئة تجارب معزولة لتجربة إرسال رقعة بالبريد الشبكي.



٥، ٨، المساهمة في مشروع — الخلاصة

شرحنا في هذا الفصل عدة أساليب تطوير، وتكلمنا عن الفروق بين العمل في فريق صغير على مشروع مغلق المصدر وبين المشاركة في مشروع حر كبير. لقد عرفت كيف تكشف أخطاء المسافات قبل الإيداع، وكيف تكتب رسائل إيداع حسنة. وتعلمت كيف تصيغ الرقع وترسلها إلى قوائم المطورين البريدية. تناولنا كذلك التعامل مع عمليات الدمج في سياق أساليب التطوير المختلفة. أنت الآن جاهز للمساهمة في أي مشروع.

التالي: سنرى كيف تعمل على الجهة الأخرى: رعاية مشروع جت. سنتعلم كيف تكون زعيماً أو مدير دمج.

٥، ٩، رعاية مشروع

— مقدمة

مع معرفة كيف تساهم بفعالية في مشروع، غالباً ستحتاج معرفة كيف ترعى مشروعاً. قد يكون هذا قبول رقع أرسلت إليك بالبريد الشابكي مصاغة بأمر تنسيق الرقع `format-patch`. وقد يكون دمج تعديلات من فروع مستودعات بعيدة مضافة في مشروعك. عليك أن تعلم كيف تقبل المساهمات بأوضح طريقة للمساهمين وأدومها لك، سواءً أكنّت مشرفاً على المستودع المرجع لمشروع أم كنت تساعد في تحقق الرقع أو قبولها.

٥، ٩، العمل في فرع موضوع

عندما تنوي دمج عمل جديد، من الحسن أن تجربته أولاً في فرع موضوع: فرع مؤقت تُنشئه خصيصاً لتجربة هذا العمل الجديد. فإنك هكذا تستطيع بسهولة تعديل كل رقعة على حدة، أو تركها إن لم تكن صالحة، حتى تعود إليها فيما بعد. إذا كنت تسمي فروعك بأسماء سهلة وذات صلة بالعمل التي تنويه فيها، مثل `ruby_client` أو شيء واضح مثله، فإنك تتذكره بسهولة عندما تعود إليه إن اضطررت لتركه وقتاً. بل إن القائمين على مشروع جت يميلون إلى استعمال «مساحات الأسماء» في أسماء الفروع، مثل `sc/ruby_client`، حيث `sc` اختصار اسم المساهم. كما تتذكر، تستطيع إنشاء فرعاً من فرعك الرئيس هكذا:

```
$ git branch sc/ruby_client master
```

CONSOLE

وإذا أردت الانتقال إليه بمجرد إنشائه، فنفذ أمر السحب مع خيار الفرع `checkout -b`:

```
$ git checkout -b sc/ruby_client master
```

CONSOLE

أنت الآن جاهز لدمج المساهمات في فرع الموضوع هذا، ثم تحديد ما إذا كنت تريد دمجها في أحد الفروع الطويلة العمر في مشروعك.

٥، ١٠، رعاية مشروع

— تطبيق الرقع من البريد

إذا وصلت رقعة من البريد الشابكي وتريد دمجها في مشروعك، فعليك تطبيقها في فرع موضوع حتى تنتظر فيها. في جت أمران لتطبيق الرقع الآتية من البريد: `git apply` للرقع المجردة، و `git am` للرقع بالصيغة البريدية.

٥، ١٠، ١، أمر تطبيق الرقع المجردة `apply`

إذا وصلت رقعة ناتجة من برنامج `git diff` أو إحدى نسخ برنامج `diff` اليونكسي (وهذا ليس مستحباً؛ انظر الفصل التالي)، فيمكنك تطبيقها بأمر `git apply`. بفرض أنك حفظها في ملف `/tmp/patch-ruby-client.patch`، يمكنك تطبيقها هكذا:

```
$ git apply /tmp/patch-ruby-client.patch
```

هذا يعدّل ملفات مجلدك الحالي. يكاد هذا يطابق تنفيذ أمر `patch -p1` لتطبيقها، لكن أمر جت شكاك وصارم فيقبل تطابقات جزئية أقل مما يقبل أمر `patch`. وإنه يتعامل مع إضافة الملفات وحذفها ونقلها، إذا كانت موصوفة بصيغة فروقات جت (`git diff`). وهذا لا يفعله `patch`. وآخر فرق هو أن أمر جت `git apply` إما يطبق الرقعة كلها وإما يتراجع عنها كلها. لكن `patch` يستطيع تطبيق الرقع جزئياً، فيترك ملفاتك في حالة غريبة. فعموماً أمر جت حريص ومحافظ أكثر من أمر `patch`.

ولن يصنع لك إيداعاً؛ بعد تنفيذ أمر تطبيق الرقع المجردة من جت، عليك تأهيل التغييرات بنفسك وإيداعها.

يمكنك كذلك أن تستعمله للتحقق أن الرقعة ستطبق بنظافة قبل تطبيقها فعلاً؛ نفذ `git apply --check` على ملف الرقعة:

```
$ git apply --check 0001-see-if-this-helps-the-gem.patch
error: patch failed: ticgit.gemspec:1
error: ticgit.gemspec: patch does not apply
```

إذا لم يعطك ناتجاً، فيمكن تطبيق الرقعة بنظافة. وإن فشل التحقق، فهذا الأمر كذلك ينهي تنفيذه بقيمة خروج غير الصفر، حتى تستعمله في البرمجيات إن أردت.

٥، ١٠، ٢، أمر تطبيق الرقع البريدية am

إذا كان المساهم يستخدم جت وكان لطيفاً واستعمل أمر تنسيق الرقع `git format-patch` لإنشاء رقعته، فإن مهمتك أيسر لأن الرقعة تحوي لك معلومات المؤلف ورسالة الإيداع. فإذا استطعت أن تحثّ مساهميك على استعمال أمر تنسيق الرقع من جت بدلاً من أمر الفروق (`git diff`، أو `diff` اليونكسي)، فافعل. فلا تستعمل أمر تطبيق الرقع المجردة `git apply` إلا للرقع الأثرية وأشباهاها.

لتطبيق رقعة أنشأها أمر تنسيق الرقع `format-patch`، استعمل أمر تطبيق الرقع البريدية `git am` (اسمه `am` لأنه لتطبيق ("apply") سلسلة من الرقع من صندوق بريد شابكي ("mailbox"). ونظرياً، `git am` صُمم ليقرأ ملف `mbox`، وهو صيغة نصية ميسورة لتسجيل رسالة أو عدة رسائل بريدية في ملف نصي واحد. وتشبه هذا:

```
From 330090432754092d704da8e76ca5c05c198e71a8 Mon Sep 17 00:00:00 2001
From: Sameera Samara <sameera@example.com>
Date: Sun, 6 Apr 2008 10:17:23 -0700
Subject: [PATCH 1/2] Add limit to log function

Limit log functionality to the first 20
```

هذا أول ناتج أمر تنسيق الرقع الذي رأيته في فصل المساهمة السابق. وهو أيضاً موافق لصيغة `mbox`. فإذا أرسل إليك أحد رقعاً كما ينبغي: بأمر إرسال البريد `git send-email`، ونزلت هذه الرسالة بصيغة `mbox`، فيمكنك إذاً أن تنفذ `git am` على هذا الملف، وسيبدأ في تطبيق جميع الرقع التي يجدها فيه. وإذا كان تطبيق عميل بريدك يستطيع حفظ

عدة رسائل بصيغة mbox ، فتستطيع حفظ سلسلة كاملة من الرقع في ملف واحد ثم تنفيذ أمر تطبيق الرقع البريدية git am عليه لتطبيقها واحدةً واحدةً.

وإذا رفع أحدُ ملف الرقعة المصاغ بأمر تنسيق الرقع، إلى نظام متابع علل أو نحوهِ، فتستطيع حفظ الملف إلى جهازك وتنفيذ أمر تطبيق الرقع البريدية عليه:

```
$ git am 0001-limit-log-function.patch
Applying: Add limit to log function
```

CONSOLE

نرى أن الرقعة طبّقت بنظافة وأنشئ لك إيداعاً جديداً. أخذت معلومات المؤلف من خانة المرسل (From:) وخانة التاريخ (Date:) اللذين في ترويسة البريد، وأنشأ رسالة الإيداع من عنوان البريد (Subject:) ومن الجزء السابق للرقعة في متن البريد ("body"). مثلاً إذا كانت هذه الرقعة هي المثال الذي ذكرناه عن صيغة mbox قبل قليل، فإن إيداعها سيكون مثل هذا:

```
$ git log --pretty=fuller -1
commit 6c5e70b984a60b3cecd395edd5b48a7575bf58e0
Author: Sameera Samara <sameera@example.com>
AuthorDate: Sun Apr 6 10:17:23 2008 -0700
Commit: Scott Chacon <schacon@gmail.com>
CommitDate: Thu Apr 9 09:19:06 2009 -0700
```

CONSOLE

```
    Add limit to log function
```

```
    Limit log functionality to the first 20
```

معلومات المودع (Commit) تبيّن الذي طبّق الرقعة ومتى طبّقها. ومعلومات المؤلف (Author) تبيّن الذي صنع التعديلات نفسها وأنشأ الرقعة، ومتى أنشأها.

لكن من الوارد ألا تطبق الرقعة بنظافة. ربما تفرّق فرعك الرئيس وشطّ بعيداً عن الفرع الذي بُنيت الرقعة عليه. وربما تعتمد هذه الرقعة على رقعةٍ أخرى لم تطبّقها بعد. فعندئذٍ سيفشل أمر التطبيق وسيسألك ماذا تريد أن تفعل:

```
$ git am 0001-see-if-this-helps-the-gem.patch
Applying: See if this helps the gem
error: patch failed: ticgit.gemspec:1
error: ticgit.gemspec: patch does not apply
Patch failed at 0001.
When you have resolved this problem run "git am --resolved".
If you would prefer to skip this patch, instead run "git am --skip".
To restore the original branch and stop patching run "git am --abort".
```

CONSOLE

وهو يضع لك علامات التنازع في أي ملف فيه مشكلة، تماماً مثلما الحال عند النزاع أثناء الدمج أو إعادة التأسيس. وتحله بالطريقة نفسها: تحرر الملفات لحل النزاع، وتأهّلها، ثم تنفذ git am --resolved للانتقال إلى الرقعة التالية.

```
$ (أصلح الملف)
$ git add ticgit.gemspec
$ git am --resolved
Applying: See if this helps the gem
```

وإذا أردت أن يحاول جت حل النزاع ببعض الذكاء، فأعطه الخيار -3 ، الذي يجعله يحاول إجراء دمجٍ ثلاثي. لا يفترض جت هذا الخيار لأنه لا يعمل إن لم يكن في مستودعك الإيداع الذي تقول الرقعة أنها مبنية عليه. إذا كان لديك ذلك الإيداع (أي أن الرقعة مبنية على إيداع عمومي) فإن الخيار -3 عموماً يجعل جت أذكى كثيراً عند تطبيق رقعة فيها نزاع:

```
$ git am -3 0001-see-if-this-helps-the-gem.patch
Applying: See if this helps the gem
error: patch failed: ticgit.gemspec:1
error: ticgit.gemspec: patch does not apply
Using index info to reconstruct a base tree...
Falling back to patching base and 3-way merge...
No changes -- Patch already applied.
```

في حالتنا هذه، بغير الخيار -3 فإن الرقعة كانت ستكون «متنازعة». لكن بالخيار -3 استطاع جت تطبيقها بنظافة.

إذا كنت تطبق عدداً من الرقع من ملف mbox ، فيمكنك تطبيقها تفاعلياً، ليقف الأمر قبل كل رقعة ليسألك إذا ما كنت تريد تطبيقها؛ هذا بالخيار --interactive أو اختصاره -i :

```
$ git am -3 -i mbox
Commit Body is:
-----
See if this helps the gem
-----
Apply? [y]es/[n]o/[e]dit/[v]iew patch/[a]ccept all
```

هذا جميل إذا كان لديك عددٌ من الرقع المحفوظة، لأنك عندئذٍ تستطيع رؤية الرقعة أولاً إن لم تنتدكرها، وكذلك ألا تكرر تطبيقها إذا كنت قد فعلت سابقاً.

عندما تفرغ من تطبيق كل الرقع وتودعها في فرع الموضوع، ابدأ في النظر فيما إذا كنت تريد دمجها في فرع طويل العمر، وكيف.

٥، ١١، رعاية مشروع — سحب فروع بعيدة

إذا أنتك المساهمات من مستخدم جت قد أنشأ له مستودعاً، ودفع إيداعات إليه، وأرسل إليك رابط مستودعه واسم الفرع البعيد الذي فيه التعديلات، فيمكنك إضافته مستودعاً بعيداً في مستودعك لتدمج تعديلاته محلياً.

مثلاً إذا أرسلت إليك سميرة بريداً تخبرك أن لديها ميزة جديدة حسنة في فرع ruby-client في مستودعها، يمكنك اختبار هذه الميزة بإضافة مستودعها البعيد وسحب هذا الفرع محلياً:

```
$ git remote add sameera https://github.com/SameeraSamara/myproject.git
$ git fetch sameera
$ git checkout -b rubyclient sameera/ruby-client
```

وإذا راسلتك من جديد تخبرك بفرع آخر فيه ميزة أخرى حسنة، فيمكنك أن تستحضر (fetch) وتسحب (checkout) مباشرةً لأن المستودع البعيد موجود في مستودعك بالفعل.

هذا ينفع أحسن النفع إذا كنت تعمل مع مساهمٍ باستمرار. أما إذا كان المساهم يرسل رقعة واحدة كل زمن طويل، فقبولها عبر البريد قد يختصر الوقت مقارنةً بإلزام الجميع باستعمال خواديم ثم إضافة المستودعات البعيدة وإزالتها طوال الوقت من أجل بضع رقع. ثم لن يصير لديك مئات من المستودعات البعيدة، كل واحد منها لمساهم برقعة واحدة أو رقتين. وعموماً، فإن البرمجيات (scripts) والخدمات المستضافة تسهّل هذا؛ إنما يعتمد الأمر على أسلوبك وأسلوب المساهمين في التطوير.

والمزية الأخرى لهذا الأسلوب هو حصولك على تاريخ الإيداعات. ربما تواجه نزاعات دمج حقيقية، لكنك هكذا ستعرف أيّ إيداعاتك التي بُنيت المساهمات عليها، فيكون الدمج الثلاثي الحقيقي هو الخيار التلقائي، بدلاً من استعمال خيار 3- والدعاء بأن تكون الرقعة مبنية على إيداعٍ عمومي موجود في مستودعك.

إن كنت لا تعمل مع مساهمٍ ما إلا نادراً، لكنك مع ذلك تود الجذب منه بهذا الأسلوب، فأعطِ أمر الدفع رابط مستودعه البعيد؛ هذا يجذب منه مرة واحدة ولا يحفظ الرابط في الإشارات البعيدة:

```
$ git pull https://github.com/onetimeguy/project
From https://github.com/onetimeguy/project
* branch          HEAD      -> FETCH_HEAD
Merge made by the 'recursive' strategy.
```

٥، ١٢، رعاية مشروع — تحديد التغييرات الجديدة

لديك الآن فرع موضوع فيه مساهمات. يمكنك الآن أن تحدد ماذا ترى أن تفعل به. يراجع هذا الفصل أمرين حتى تعرف كيف ترى بهما ماذا سيتغيّر بالضبط عندما تدمجه في فرعك الرئيس.

كثيراً ما يفيد أن ترى كل إيداعات هذا الفرع التي ليست في فرعك الرئيس. يمكن استبعاد إيداعات master بخيار --not قبل اسم الفرع. هذا يكافئ تماماً صيغة master..contrib التي استعملناها سابقاً. مثلاً إذا أرسل لك مساهم رقتين وأنشأت فرعاً أسميته contrib وطبقت هاتين الرقتين فيه، فيمكنك أن تنفذ هذا:

```
$ git log contrib --not master
commit 5b6235bd297351589efc4d73316f0a68d484f118
Author: Scott Chacon <schacon@gmail.com>
Date:   Fri Oct 24 09:53:59 2008 -0700
```

See if this helps the gem

```
commit 7482e0d16d04bea79d0dba8988cc78df655f16a0
```

```
Author: Scott Chacon <schacon@gmail.com>
```

```
Date: Mon Oct 22 19:38:36 2008 -0700
```

```
Update gemspec to hopefully work better
```

لرؤية تعديلات كل إيداع، لعلك تتذكر أن خيار الرقعة -p مع أمر السجل git log يُظهرها لك.

لرؤية الفرق الكامل لما سيحدث فعلاً عند دمج فرع الموضوع هذا مع فرع آخر، قد تضطر إلى حيلة غريبة لرؤية الفرق الصحيح. قد تفكر بتنفيذ هذا:

```
$ git diff master
```

CONSOLE

هذا الأمر يعطيك فروقاً، لكنه قد يخدعك. فإذا تحرك فرعك الرئيس بعد إنشاء فرع الموضوع منه، فترى ما يبدو عجباً. هذا لأن جت يقارن لقطه آخر إيداع في الفرع الحالي مع لقطه آخر إيداع في الفرع الآخر. فمثلاً إذا أضفت سطرًا في ملف في الفرع الرئيس، فالمقارنة المباشرة بين اللقطتين ستوحي أن فرع الموضوع سيزيله.

إذا كان master سلفاً مباشراً لفرع موضوعك (أب أو جد أو جد أعلى)، فلا بأس في هذا. أما إذا افترق تاريخا الفرعين، فناتج أمر الفرق بينهما سيوحي بإضافة كل شيء جديد في فرع الموضوع وإزالة كل شيء جديد تفرّد به الفرع الرئيس.

ما تريد رؤيته فعلاً هو تعديلات فرع الموضوع: التعديلات التي ستقدّمها إلى الفرع الرئيس إذا دمجت فرع الموضوع فيه. ترى هذا بجعل جت يقارن آخر إيداع في فرع الموضوع مع آخر سلف مشترك مع الفرع الرئيس.

نظرياً يمكن فعل هذا بحساب السلف المشترك ثم طلب الفرق معه:

```
$ git merge-base contrib master
36c7dba2c95e6bbb78dfa822519ecfec6e1ca649
$ git diff 36c7db
```

CONSOLE

أو بشيء من الإيجاز:

```
$ git diff $(git merge-base contrib master)
```

CONSOLE

لكن لا هذا ولا ذاك استعمله مريح، فيختصر لنا جت ذلك بـ«صيغة النقاط الثلاثة». فإذا أردت رؤية فروق الفرع الحالي منذ آخر إيداع مشترك مع فرع آخر، أعط أمر الفرق: الفرع الآخر وثلاث نقاط والفرع الحالي، بغير مسافة:

```
$ git diff master...contrib
```

CONSOLE

لا يظهر لك هذا الأمر إلا تعديلات فرع contrib بعد آخر سلف مشترك مع فرع master. هذه الصيغة عظيمة النفع وتستحق التذكر.

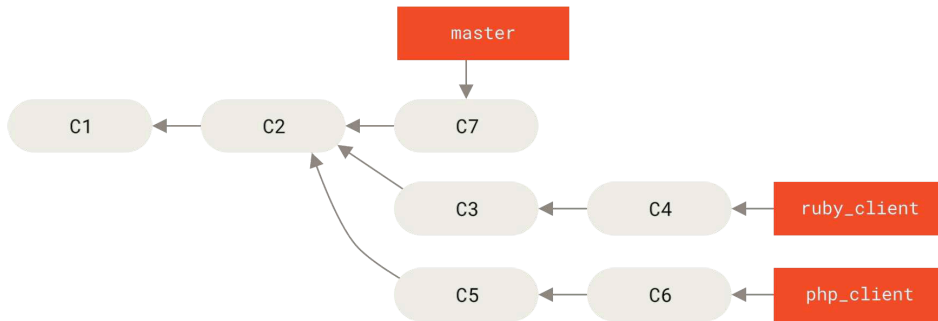
٥، ١٣، رعاية مشروع — دمج المساهمات

عندما يكون كل ما في فرع موضوعك جاهزاً لدمجه في فرع من الفروع العامة، يكون السؤال: كيف؟ ثم ما الأسلوب العام الذي تريده لرعاية مشروعك؟ أمامك عدة خيارات، وسنتناول بعضاً منهم.

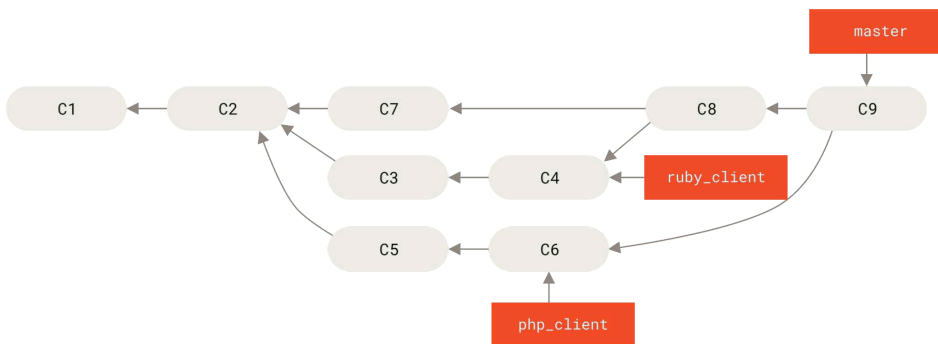
٥، ١٣، ١، أساليب الدمج

من أيسر الأساليب أن تدمج كل المساهمات إلى فرعك الرئيس مباشرةً. فقط. في هذا الأسلوب يكون فرعك الرئيس فيه المصدر المستقر عموماً. وعندما يكون لديك فرع موضوع فيه عمل تراه مكتملاً، أو مساهمة قد تحققت صحتها، فادمج ذلك في الفرع الرئيس، واحذف فرع الموضوع الذي دمجته للتو، ثم أعد الكرة.

مثلاً لدينا مستودعُ فرع الرئيس `master` وفيه أعمال في فرعين اسمهما `ruby_client` و `php_client` مثل شكل [تاريخ فيه فرعاً موضوعين](#)، فإذا دمجتنا `ruby_client` ثم `php_client`، سيبدو التاريخ مثل شكل [بعد دمج فرعٍ الموضوعين](#).



شكل ٧٢. تاريخ فيه فرعاً موضوعين

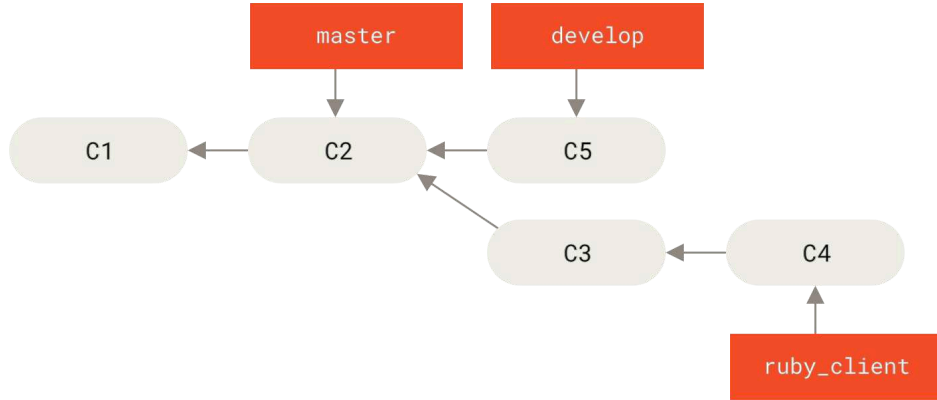


شكل ٧٣. بعد دمج فرعٍ الموضوعين

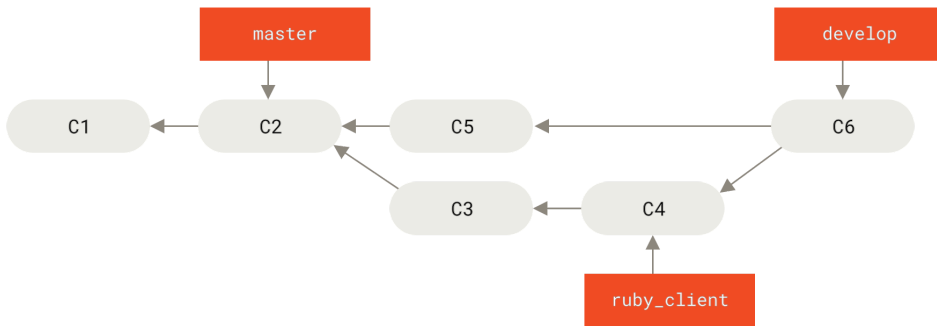
ربما هذا أيسر أسلوب تطوير. لكنه قد يصعب الأمور في المشروعات الكبيرة أو شديدة الاستقرار التي تحتاج أن تحرص أشد الحرص قبل أن تدمج شيئاً فيها.

فإذا كان مشروعك كبير أو استقراره مهم، فقد تجعل أسلوبك الدمج بمرحلتين. وهو أن يكون لك فرعان طويلاً الأمد: الفرع الرئيس (master مثلاً)، وفرع التطوير (develop مثلاً)؛ وتلتزم في هذا الأسلوب ألا يتغير الفرع الرئيس إلا عند ضمان استقرار ما فرع التطوير استقراراً يقينياً وأن فيه كل التعديلات المرادة وأنتك جاهز لإصداره إلى المستخدمين. وكلا الفرعين

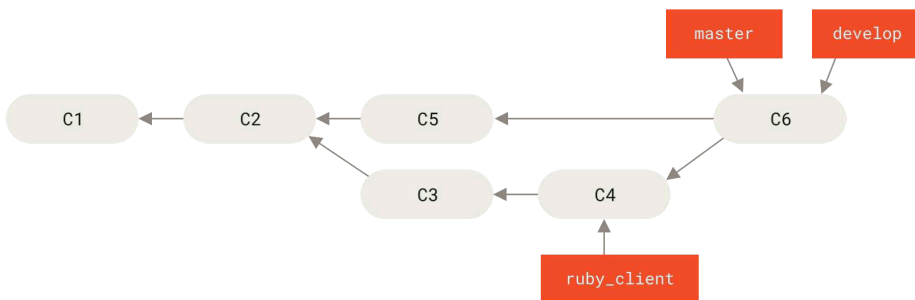
تدفعهما إلى المستودع العمومي. كل فرع موضوع تريد دمجه (شكل [قبل دمج فرع موضوع](#))، تدمجه أولاً في فرع التطوير develop (شكل [بعد دمج فرع موضوع](#)). ثم عندما تجهز إصداراً موسومة، تسرّع الفرع الرئيس إلى الإيداع الأخير في فرع التطوير الذي صار مستقرًا (شكل [بعد إصدار المشروع](#)).



شكل ٧٤. قبل دمج فرع موضوع



شكل ٧٥. بعد دمج فرع موضوع

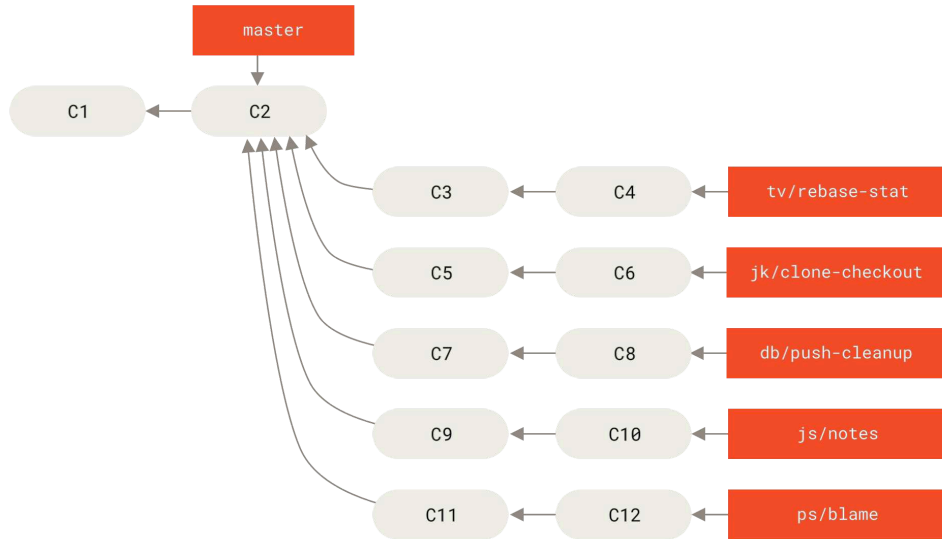


شكل ٧٦. بعد إصدار المشروع

هكذا، عندما يستنسخ الناس مستودع مشروعك، لهم أن يسحبوا الفرع الرئيس master ليعبثوا آخر نسخة مستقرة ويطبقوها محدثة من هذا الفرع بسهولة، ولهم أن يسحبوا فرع التطوير develop الذي فيه المصدر الأحدث. وكذلك يمكنك الإمعان في هذا الأسلوب، بإضافة فرع الدمج integrate الذي تدمج فيه التعديلات كلها، ثم عندما ترى استقراراً ما في هذا الفرع واجتياز الاختبارات فإنك تدمجه في فرع التطوير develop. وعندما تتحقق استقرار فرع التطوير زمنياً فإنك تسرّع الفرع الرئيس ليوافقه.

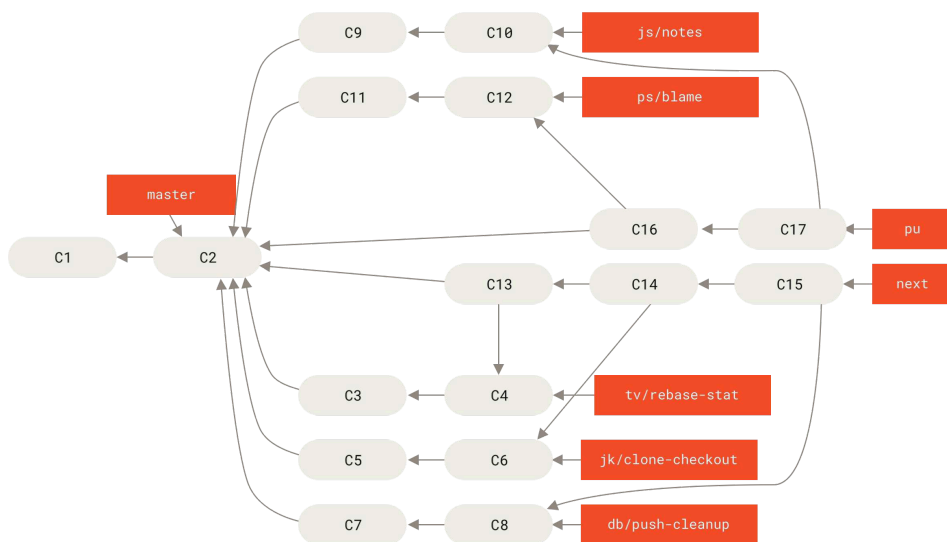
٥، ١٣، ٢، أساليب الدمج الكثير

في مشروع مصدر جت أربعة فروع طويلة العمر: master و next و seen و سابقاً pu = «تحديثات مقترحة»، و maint لحمل الإصلاحات إلى الإصدارات القديمة (backports). عند المساهمة، فإن المساهمات تُجمع في فروع موضوعات في مستودع المشرف بطريقة تشبه ما شرحنا سابقاً (انظر [إدارة مجموعة كبيرة من المساهمات المتوازية في فروع موضوعات](#)). عندئذٍ تُقِيم موضوعات المساهمات لتحديد صلاحيتها وصحتها، أم أنها تحتاج عملاً. فإذا كانت صالحة فإنها تدمج في next ويُدفع هذا الفرع إلى المستودع العمومي لي تجرب الجميع الموضوعات المدموجة كلها معاً.



شكل ٧٧. إدارة مجموعة كبيرة من المساهمات المتوازية في فروع موضوعات

وإذا كانت الموضوعات تحتاج عملاً فإنها تدمج في فرع seen. وعندما يتقرر أنها مستقرة تماماً، فإنها تدمج في master. ثم يعاد تأسيس الفرعين next و seen على master الجديد. هذا يعني أن master يتحرك إلى الأمام دائماً أو شبه دائماً، وأن next يعاد تأسيسه أحياناً، وأن seen يعاد تأسيسه أكثر من ذلك.



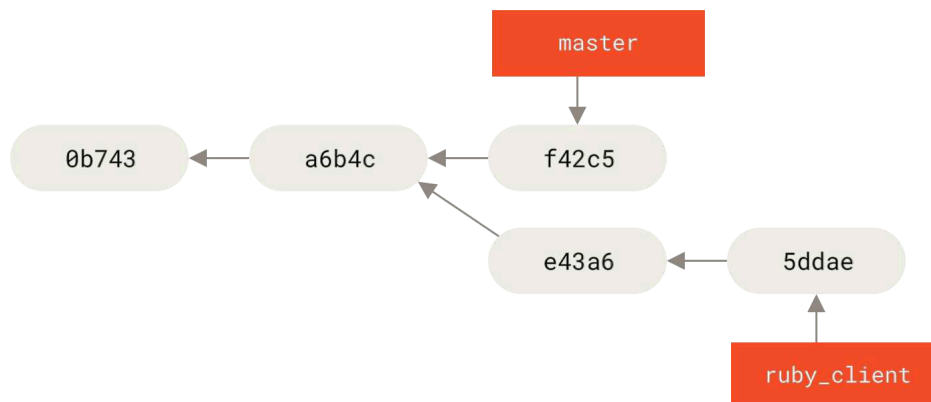
شكل ٧٨. دمج مساهمات فروع الموضوعات في فروع دمج طويلة الأمد

عندما يُدمج فرع موضوع في master أخيراً، فإنه يحذف من المستودع. وكذلك في مشروع مصدر جت فرع maint، الذي يُشتق من آخر إصدارة لـيُتيح رُقَع للإصدارات القديمة إن دعت الحاجة إلى إصدارة صيانة. لذا فإنك عندما تستنسخ مستودع جت فإنك تجد أربعة فروع يمكنك سحبها لفحص المشروع في حالاته المختلفة من التطوير، حسب حماسك للأحدث أو حسب رغبتك في المساهمة. وللمشرف أسلوب منظم يساعده في تدقيق المساهمات الجديدة. لمشروع جت أسلوب تطوير مخصص. لتفهمه بوضوح أشد، انظر دليل مشرف جت (بالإنجليزية) <https://github.com/git/git/blob/master/Documentation/howto/maintain-git.adoc>.

٥، ١٣، ٣، أساليب الاصطفاء وإعادة التأسيس

يجب مشرفون آخرون إعادة تأسيس المساهمات (rebase) على فرعهم الرئيس أو اصطفائها (cherry-pick)، بدلاً من دمجها فيه، ليبقوا تاريخ الإيداعات خطياً قدر الإمكان. فإذا عملت في فرع موضوع ورأيت أن تدمجه، فانتقل إلى هذا الفرع ونفذ أمر إعادة التأسيس فيه لإعادة صناعة إيداعاته على الفرع الرئيس. فإذا تم هذا بنجاح، فيمكنك تسريع فرعك الرئيس، وسيكون تاريخ مشروعك خطياً.

والطريقة الأخرى لنقل تغييرات من فرع إلى آخر هي باصطفائه (cherry-pick). فالاصطفاء في جت هو مثل إعادة التأسيس لكن لإيداع وحيد. فإنه يتناول رقعة إيداع سابق ويحاول تطبيقها على الفرع الحالي. يفيد هذا إذا كان لديك عدد من الإيداعات في فرع موضوع وتريد دمج واحد فقط منها، أو عندك إيداع واحد فقط في فرع موضوع وترى أن اصطفاءه خير من إعادة تأسيسه. للمثال، قل إن لديك مشروعاً مثل هذا:



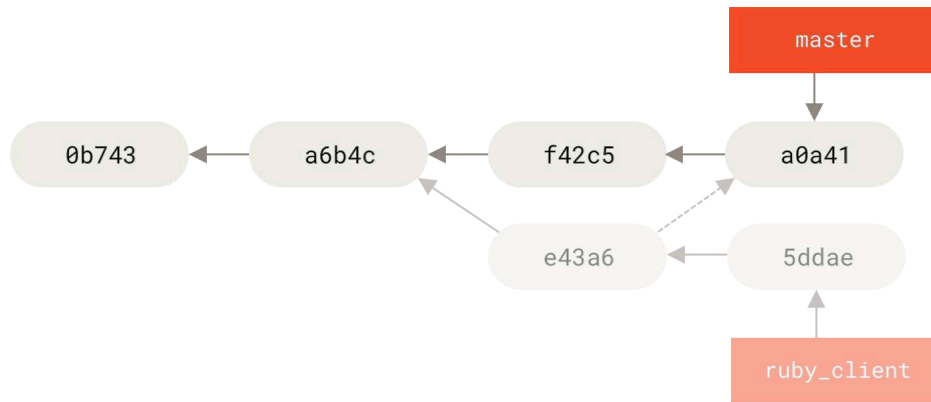
شكل ٧٩. مثال تاريخ قبل الاصطفاء (cherry-pick)

فإذا أردت جذب الإيداع e43a6 إلى فرع master، فيمكنك تنفيذ هذا وأنت في الفرع الهدف master:

```
$ git cherry-pick e43a6
Finished one cherry-pick.
[master]: created a0a41a9: "More friendly message when locking the index fails."
3 files changed, 17 insertions(+), 3 deletions(-)
```

CONSOLE

هذا يجذب تغييرات الإيداع e43a6 نفسها، لكن بإيداع جديد وبصمة جديدة لأن تاريخ تطبيقها مختلف. عندئذ سيبدو تاريخك هكذا:



شكل ٨٠. التاريخ بعد اصطفاء إيداع من فرع موضوع

يمكنك عندئذٍ إزالة فرع الموضوع وهجر إيداعاته التي لم تردّها.

٥، ١٣، ٤، معاودة مصالحة مسجلة (rerere)

إذا كنت كثير الدمج وإعادة التأسيس، أو فروع موضوعاتك طويلة العمر، ففي جت وسيلة نافعة اسمها “rerere”.

إن “rerere” اختصار “reuse recorded resolution” أي «إعادة تطبيق الحل المسجل للنزاع». وهي لتقليل الجهد البدوي في حل النزاعات. فعند تمكينها، سيلتقط جت صورة من المشروع عند حدوث نزاع وصورة بعد حله، وإن لمح جت هذا النزاع مرة أخرى فسيصلحه لك من نفسه كما أصلحته أنت أول مرة تماماً.

هذه الميزة جزءان: إعداد تهيئة، وأمر. فأما الإعداد فهو `rerere.enabled`، وهو صغير ويسهل إضافته إلى تهيئة جت العامة في نظامك:

```
$ git config --global rerere.enabled true
```

CONSOLE

فمن الآن، وقتما واجهت نزاع دمج وحلته وأودعت، فسيُسجل جت الحل إن احتجته في المستقبل.

وإن احتجت التعامل مع ذاكرة «معاودة المصالحات المسجلة»، فاستعمل الأمر `git rerere`. فعند تنفيذه منفرداً (بغير معاملات)، فإن جت ينتظر في ذاكرة حلول النزاعات ويبحث عن ما يطابق النزاع الحالي ويحلّه (ولكن هذا يحدث تلقائياً إذا كان `rerere.enabled` بـ `true`). ولهذا الأمر أوامر فرعية لرؤية ما سيسجل، ولحذف حل نزاع معين، ولمسح ذاكرة الحلول كلها. سنفضّل الحديث عن هذا كله في فصل ~~معاودة مصالحة مسجلة (rerere)~~ في الباب السابع.

٥، ١٤، رعاية مشروع

— إصدار مشروعك

٥، ١٤، ١، وسم إصداراتك

عندما تنوي إطلاق إصدار، من المستحب أن تعيّن لها وسمًا، حتى تُنشئها مجدداً متى شئت فيما بعد. تستطيع إنشاء وسمٍ كما شرحنا في باب [أسس جت](#). وإذا كنت مشرف المشروع ووقّعت الوسم، فسيبدو مثل هذا:

```
$ git tag -s v1.5 -m 'my signed 1.5 tag'
You need a passphrase to unlock the secret key for
user: "Scott Chacon <schacon@gmail.com>"
1024-bit DSA key, ID F721C45A, created 2009-02-09
```

إذا كنت فعلاً توفّع وسومك، فقد يتعبك توزيع المفتاح العمومي الذي توفّع به. مشرف مشروع جت حل هذا بجعل مفتاحه العام كتلة رقمية في المستودع ("blob")، ووسمها بوسم يشير إليها مباشرةً. إذا أردت فعل هذا، فعليك أولاً تحديد المفتاح الذي تريد التوقيع به، فاسرد مفاتيحك بأمر `gpg --list-keys`:

```
$ gpg --list-keys
/Users/schacon/.gnupg/pubring.gpg
-----
pub 1024D/F721C45A 2009-02-09 [expires: 2010-02-09]
uid Scott Chacon <schacon@gmail.com>
sub 2048g/45D02282 2009-02-09 [expires: 2010-02-09]
```

عندئذٍ تستطيع استيراده مباشرةً إلى قاعدة بيانات جت بتصديره ونقله في أنبوب يونكسي إلى أمر `git hash-object`، الذي يكتب هذا المحتوى في كتلة رقمية جديدة في جت ويخبرك ببصمة الكتلة:

```
$ gpg -a --export F721C45A | git hash-object -w --stdin
659ef797d181633c87ec71ac3f9ba29fe5775b92
```

الآن لديك محتوى مفاتيحك في جت، فيمكنك إنشاء وسم يشير إليه مباشرةً بتعيين البصمة التي أخبرك بها أمر `hash-object`:

```
$ git tag -a maintainer-gpg-pub 659ef797d181633c87ec71ac3f9ba29fe5775b92
```

فإذا دفعت الوسوم بالأمر `git push --tags`، فإنك ستشارك الوسوم `maintainer-gpg-pub` مع الجميع. وإذا أراد أحد أن يتحقق وسمًا، فإنه يجذب من بيانات جت الكتلة الرقمية المشار إليها بالوسوم ويستوردها إلى GPG:

```
$ git show maintainer-gpg-pub | gpg --import
```

فيمكنه بهذا المفتاح أن يتحقق كل وسومك الموقعة. وحتى إنك تستطيع أن تشرح خطوات التحقق بدقة في رسالة الوسوم نفسه، ليراها من يُظهره بالأمر «وسم-المفتاح» `git show`.

٥، ١٤، ٢، توليد رقم بناء

لا يرافق إيداعات جت أرقامٌ متزايدة بإطراد مثل v123 أو نحوها. فإذا أردت لإيداع اسمًا يلائم البشر، فيمكنك تنفيذ أمر `git describe` على هذا الإيداع. فيصنع لك جت اسمًا يتكون من اسم آخر وسم قبل هذا الإيداع، ثم عدد الإيداعات بينهما، ثم يختتمه بمختصر بصمة الإيداع مسبقة بحرف "g" (اختصار Git):

```
$ git describe master
v1.6.2-rc1-20-g8c5b85c
```

هكذا تستطيع تصدير لقطة أو بناء من مشروعك وتسميه اسمًا يفهمه الناس. بل إنك إذا بنيت جت من مصدره البرمجي المستنسخ من مستودعه الرسمي، فإن `git --version` سيجيب بشيء مثل هذا. وإذا طلبت وصف إيداع قد وسمته هو نفسه، فإن `git describe` يعطيك اسم الوسم فقط.

أمر الوصف بطبيعته لا يعتمد إلا على الوسوم المعنونة (وهي الوسوم المنشأة بخيارات أشهرها `-a` و `-s`). أما إذا أردته أن ينظر إلى الوسوم الخفيفة كذلك، فأضف خيار `--tags` إليه. ويمكنك كذلك استعمال هذا الاسم الوصفي مع أمر السحب `git checkout` وأمر الإظهار `git show`، على أن ذلك يعتمد على مختصر البصمة الذي في آخره فقد لا يدوم صلاحه. فمثلا انتقل مشروع نواة لينكس من ثماني محارف إلى عشرة محارف لضمان تفرّد بصمات الكائنات، فأبطلت الأسماء القديمة الناتجة من `git describe`.

٥، ١٤، ٣، تجهيز إصدار

الآن أنت تريد إصدار بناء من مشروعك. من المستحب أن تحزم آخر لقطة من مشروعك لأولئك المساكين المحرومين من استخدام جت. افعل هذا بأمر `git archive`، مثلا:

```
$ git archive master --prefix='project/' | gzip > `git describe master`.tar.gz
$ ls *.tar.gz
v1.6.2-rc1-20-g8c5b85c.tar.gz
```

فإذا فتح أحد هذه الحزمة، سيجد آخر نسخة من مشروعك الذي في مجلد `project`. وتستطيع كذلك إنشاء حزمة بصيغة `zip`، بأمر كبير الشبه بالسابق، لكن بالخيار `--format=zip` لأمر الحزم:

```
$ git archive master --prefix='project/' --format=zip > `git describe master`.zip
```

لديك الآن حزمة `tar` مضغوطة وحزمة `zip` من إصدار مشروعك، فيمكنك رفعهما إلى موقعك أو إرسالهما بالبريد الشبكي إلى الناس.

٥، ١٤، ٤، السجل الموجز

حان الآن موعد إخبار قائمتك البريدية بما حدث في مشروعك. أمر السجل الموجز `git shortlog` هو طريقة سريعة لصياغة «سجل تغييرات» مشروعك منذ آخر إصدار أو رسالة بريدية. فإنه يوجز كل الإيداعات التي في النطاق الذي تحدده. فمثلا الأمر التالي يخبرك بموجز الإيداعات جميعًا منذ آخر إصدار، إذا كانت آخر إصدار من هذا المشروع اسمها `v1.0.1`:

```
$ git shortlog --no-merges master --not v1.0.1
Chris Wanstrath (6):
    Add support for annotated tags to Grit::Tag
    Add packed-refs annotated tag support.
```

```
Add Grit::Commit#to_patch
Update version and History.txt
Remove stray `puts`
Make ls_tree ignore nils
```

Tom Preston-Werner (4):

```
fix dates in history
dynamic version method
Version bump to 1.0.2
Regenerated gemspec for version 1.0.2
```

موجز واضح لكل الإيداعات منذ v1.0.1 ، ومجموعة بمؤلف الإيداع، حتى ترسلها إلى قائمتك البريدية.

o، o، الخلاصة

تعرف الآن كيف تساهم في مشروع يستعمل جت، وكذلك كيف ترعى مشروعك الخاص أو تدمج مساهمات الآخرين. أنت الآن مطّور جت فعّال؛ مبارك! ستعرف في الباب التالي كيف تستخدم جت هب، أكبر وأشهر خدمة استضافة جت.

الباب السادس:

GitHub

جِتهَب هو أكبر مستضيف لمستودعات جت، وهو نقطة التعاون المركزية لملايين المطورين والمشاريع. نسبة كبيرة من جميع مستودعات جت مستضافة على جتهَب، والعديد من المشاريع المفتوحة المصدر تستخدمه لاستضافة جت وتتبع العلل ومراجعة الرُّقْع وأُمُور أخرى. لذا فبالرغم من أنه ليس جزءًا صريحًا من مشروع جت الحر والمفتوح المصدر، إلا أنك غالبًا سوف تود أو تحتاج إلى التعامل معه في وقتٍ ما أثناء استخدامك جت مهنيًا.

هذا الباب عن استخدام جتهَب بفعالية. نتناول إنشاء حساب وإدارته، وإنشاء مستودعات جت واستخدامها، وأساليب العمل الشائعة للمساهمة في مشاريع ولقبول المساهمات في مشاريعك، وواجهة جتهَب البرمجية، والكثير من النصائح الصغيرة لجعل حياتك أسهل عموماً.

إن لم تكن مهتمًا باستخدام جتهَب لاستضافة مشاريعك أو للمساهمة في مشاريع الآخرين المستضافة عليه، فيمكنك الانتقال إلى الباب التالي: **أسوات جت**.

تغير الواجهات

من المهم ملاحظة أن مكوّنات الواجهات في لقطات الشاشة عرضة للتغير مع الوقت، مثل أي موقع نشيط. ولكننا نرجو أن يبقى فيها المفهوم العام لما نريد أن نحققه. ولكن إذا أردت نُسخًا أحدث من هذه الشاشات، فقد تجدها في نسخة الكتاب على الشابكة.



١، ١، إعداد الحساب وتهيئته

أول شيء تحتاجه هو إعداد حساب مستخدم مجاني. ليس عليك سوى زيارة <https://github.com> واختيار اسم مستخدم متاح وإدخال عنوان بريد شباككي وكلمة مرور، ثم الضغط على الزر الأخضر الكبير "Sign up for GitHub".

شكل ٨١. استمارة التسجيل في جتهَب

ثاني شيء ستراه هو صفحة أسعار الاشتراكات غير المجانية، ولكن يمكنك تجاهلها الآن. سيرسل لك جت هب رسالة لتتحقق عنوان البريد الذي أدخلته. اذهب إلى بريدك وأكدّه؛ هذا مهم (كما سنرى فيما بعد).

يتيح جت هب أكثر ميزاتاته للحسابات المجانية، باستثناء بعض الخصائص المتقدمة.

فتشمل اشتراكاته المدفوعة أدوات وميزات متقدمة إضافةً إلى تقليل قيود الخدمات المجانية، ولكن لن نتناول تلك الأمور في هذا الكتاب. للمزيد من المعلومات عن الاشتراكات المتاحة والمقارنة بينها، رُز

<https://github.com/pricing>

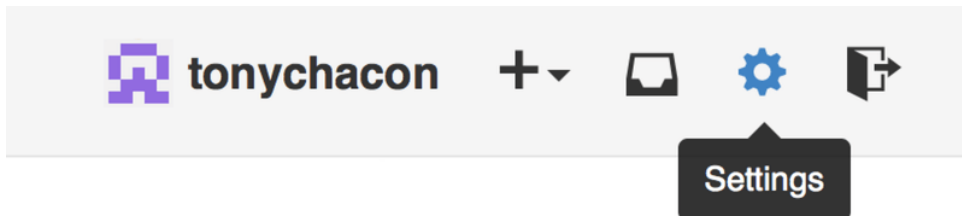


ستجد في أعلى يسار الشاشة شارة القط ذي الأرجل الثمانية (والسمى “Octocat” — «أخطوقط»)، اضغط عليه لتنتقل إلى صفحتك الرئيسية. مبارك عليك! أنت الآن مستعد لاستخدام جت هب.

١، ١، ١، وصول SSH

تستطيع الآن التواصل مع مستودعات جت بميفاق <https://> والاستيثاق باسم المستخدم وكلمة المرور اللذين أعدتهما للتو. لكن لمجرد استنساخ المستودعات العمومية، لا تحتاج حتى للتسجيل في الموقع. إنما سيقفنا الحساب الذي قد أنشأناه فيما بعد عندما نشق مستودعاتٍ وندفع إلى اشتقاقاتها.

إذا احتجت استخدام خواديم بعيدة عبر SSH، فإنك تحتاج إعداد مفتاح عمومي. إذا لم يكن لديك واحداً بالفعل، فانظر فصل [توليد مفتاح SSH عمومي لك](#). افتح إعدادات حسابك بالضغط على رابط الترس الذي في أعلى يمين الشاشة.



شكل ٨٢. رابط إعدادات الحساب

ثم اختر تبويب “SSH keys” من القائمة اليسرى.

شكل ٨٣. إعدادات مفاتيح SSH

ثم اضغط زر “Add an SSH key” لإضافة مفاتيحك، وأعطه اسمًا، وانسخ في الحقل النصي الكبير محتوى ملف مفاتيحك العمومي (`~/.ssh/id_rsa.pub` أو كما سمّيته)، ثم اضغط “Add key”.

اختر لمفاتيحك اسمًا تتذكره. مثلاً «حاسوبي المحمول» أو «حساب العمل». اختر لكل مفتاح اسمًا واضحًا حتى إذا أردت إبطال أحدها تعرف أيهم تريد.

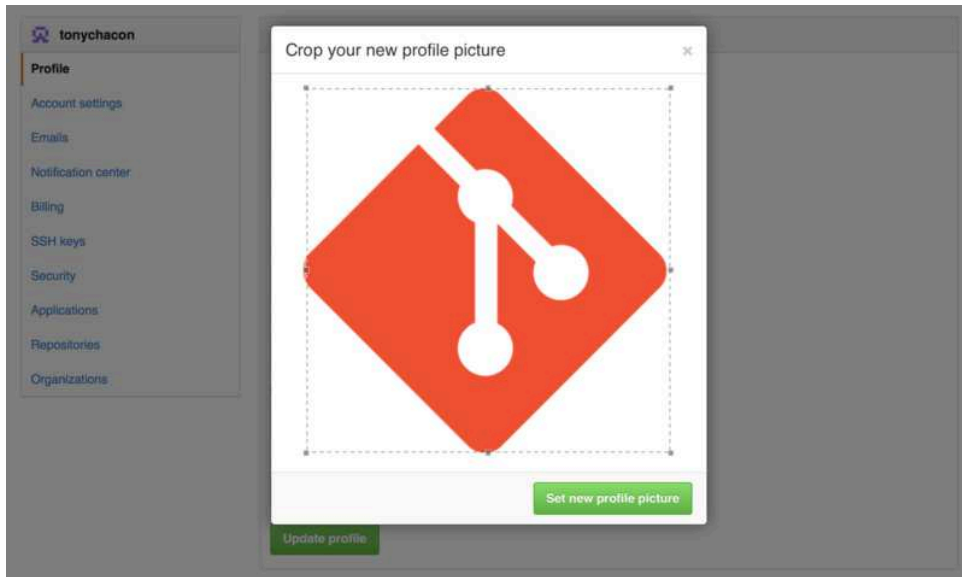


٦، ٢، صورتك الشخصية

الخطوة التالية، إن أردت، هي تغيير الصورة المولدة آلياً إلى صورة من اختيارك. اذهب إلى تبويب “Profile” (فوق تبويب “SSH Keys”) واضغط على “Upload new picture”.

شكل ٨٤. إعدادات معلومات الحساب

سنختار صورة شارة جت موجودة لدينا على القرص الصلب، وسيتيح الموقع لنا الفرصة لاقتصاصها.



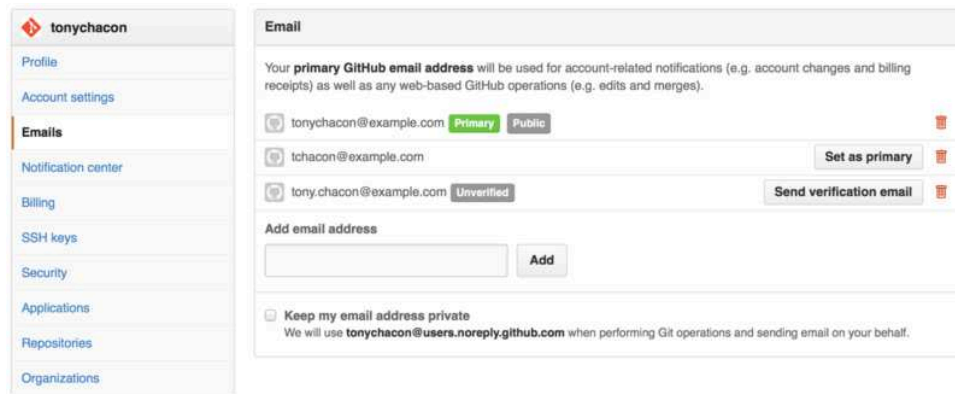
شكل ٨٥. اقتصاص صورتك الشخصية التي رفعتها

من الآن سيرى الناس صورتك بجوار اسمك عندما تتفاعل على الموقع.

وإذا كنت قد رفعت صورة لك على خدمة Gravatar الشهيرة (والشائعة مع حسابات WordPress)، فستكون هي الصورة المبدئية ولن تحتاج إلى هذه الخطوة من الأساس.

٦، ٣، ١، عناوين بريدك الشابكي

يربط جت هب إيداعاتك في جت بحسابك في جت هب عبر عنوان بريدك الشابكي. فإذا كنت تستعمل عدة عناوين بريد في إيداعاتك، وتريد من جت هب أن يربطها جميعًا بحسابك، فعليك إضافتها كلها في قسم البريد في تبويب الإدارة.



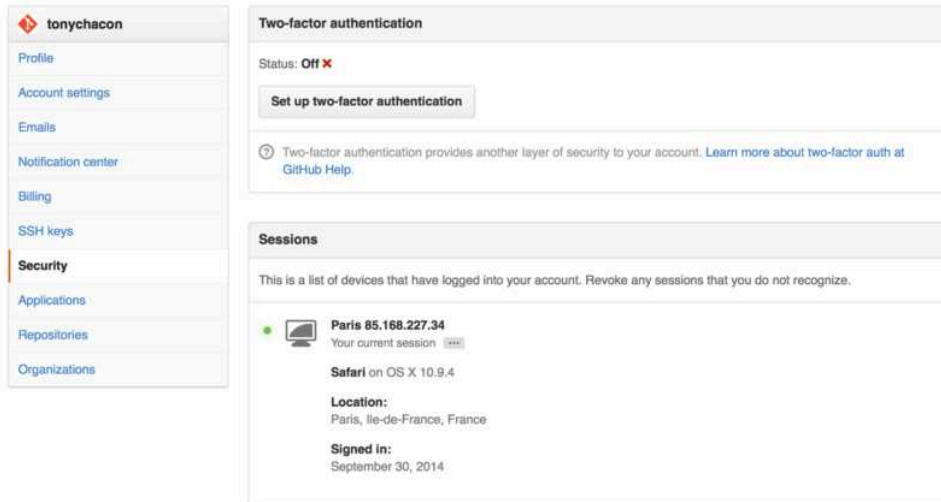
شكل ٨٦. إضافة جميع عناوين البريد الشابكي

في شكل [إضافة جميع عناوين البريد الشابكي](#) نرى بعض الحالات الممكنة للعناوين. العنوان الأول متحقق ومضبوط ليكون العنوان المبدئي، أي أنه العنوان الذي تُرسل إليه الإشعارات والتقارير. والعنوان الثاني متحقق، فيمكن جعله العنوان المبدئي إذا أردت التبديل. والعنوان الأخير غير متحقق، فلا يمكنك جعله عنوانك المبدئي. فإذا رأى جت هب أي عنوان من هؤلاء في إيداع في أي مستودع على الموقع، فسيربطه بحسابك.

٦، ٤، الاستيثاق الثنائي

وأخيراً، للمزيد من الأمان، يقيناً عليك إعداد «الاستيثاق الثنائي» (وتسمى كذلك «المصادقة الثنائية» أو «التحقق بخطوتين») (2FA)، اختصار Two-factor Authentication). وهي طريقة استيثاق بدأت بالانتشار حديثاً لتقليل احتمال اختراق حسابك إن سُرقَت كلمة مرورك بطريقةٍ ما. وتفعيلها يجعل جتَهب يسألك عن طريقتين مختلفتين للاستيثاق، لكيلا يتعرض حسابك للاختراق إن استولى أحد على إحدهما.

يمكنك إعداد الاستيثاق الثنائي من تبويب "Security" في صفحة إعدادات حسابك.



شكل ٨٧. الاستيثاق الثنائي في تبويب الأمان

إذا ضغطت زر "Set up two-factor authentication" ستنتقل إلى صفحة إعدادات يمكنك فيها اختيار تطبيق هاتف محمول لتوليد الرمز الثانوي («كلمة مرور استخدام واحد مؤقتة» "time based one-time password")، أو أن تجعل جتَهب يرسل إليك الرمز عبر رسالة محمول قصيرة (SMS) في كل مرة تحتاج إلى الدخول إلى الموقع.

بعد اختيار طريقتك المفضلة واتباع التعليمات لإعداد الاستيثاق الثنائي، سيصير حسابك أكثر أماناً نسبياً، وستحتاج إلى إدخال رمز مع كلمة مرورك في كل مرة تدخل إلى حسابك على جتَهب.

٦، ٢، المساهمة في مشروع

٦، ٣، رعاية مشروع

٦، ٤، إدارة منظمة

٦، ٥، برمجة جتَهب

١،١، الخلاصة

أنت الآن مستخدم جت هب؛ تعرف كيف تنشئ حساباً وتدير منظمة وتنشئ مستودعات وتدفع إليها وتساهم في مشروعات الآخرين وتقبل مساهماتهم. في الباب التالي سنتعلم أدوات أقوى ونصائح أعمق للتعامل في المواقف المعقدة، لتكون أستاذاً بحق في جت.

الباب الثامن:

تخصيص جت

تناولنا حتى الآن أسس عمل جت وكيف نستعمله، ثم شرحنا عدداً من الأدوات التي يتيحها جت لمساعدتك في استعماله بسهولة وفعالية. أما في هذا الباب فنرى كيف نخصّص أفعال جت، بالتعرّف على إعدادات تهيئة مهمة وعلى نظام الخطاطيف. فهذه الأدوات يسهل عليك جعل جت يتصرف بالضبط كما تحتاج أنت أو شركتك أو فريقك.

٨، ١، تهيئة جت

٨، ٢، سمات الملفات في جت

٨، ٣، خطاطيف جت

لدى جت، مثل أنظمة إدارة نسخ كثيرة أخرى، طريقة لتشغيل برمجيات (scripts) مخصصة عند أحداث معينة مهمة. وتسمى الخطاطيف، وتقسم نوعين: خطاطيف عند العمل، وخطاطيف على الخادوم. تستثير خطاطيف العمل عمليات مثل الإيداع والدمج، أما خطاطيف الخادوم فالعمليات الشبكية مثل استقبال الإيداعات المدفوعة. ويمكنك استخدام الخطاطيف لأسباب عديدة مختلفة.

٨، ٣، ١، تركيب خطاف

تحفظ جميع الخطاطيف في مجلد hooks داخل مجلد جت. أي في `git/hooks`. في معظم المستودعات. وعندما تبتدئ مستودعاً جديداً بأمر `git init`، فإن جت يملأ مجلد الخطاطيف بأمثلة برمجيات (التي قد يفيد بعضها بلا تغيير)، وفيها شرح مدخلات كل برمج. أكثر الأمثلة مكتوبة بلغة الصدفة (shell)، والبعض بلغة Perl. ولكن أي ملف تنفيذي بالاسم المناسب سيعمل؛ اكتبه بـ Python أو Ruby أو بما تشاء. وإذا أردت استعمال برمجيات الخطاطيف المرفقة مع جت، عليك تغيير أسمائها، فأسمائها جميعاً تنتهي بالامتداد `..sample`.

لتفعيل برمج خطاف، ضع ملفاً تنفيذياً بالاسم المناسب (بغير امتداد) في مجلد `hooks` في مجلد `git`. في مستودعك، ومن وقتئذٍ فصاعداً سيُنَفَّذ. نمر الآن على أهم الخطاطيف ونسرد أسمائها.

٨، ٣، ٢، خطاطيف العمل

خطاطيف كثيرة تُنفَّذ عند العمل. يقسمهم هذا الفصل إلى أقسام ثلاثة لتيسير الشرح: خطاطيف الإيداع، وخطاطيف البريد، والخطاطيف الأخرى.



مهم جداً أن تعلم أن خطاطيف العميل ليست تُنسخ عند استنساخ المستودع. فإذا كان مرادك من هذه البرمجيات أن تفرض سياسات معينة، فربما تفضل فعل هذا على الخادوم؛ انظر المثال في فصل [مستودع](#) لفرض السياسات في جت.

خطاطيف الإيداع

الخطاطيف الأربعة الأولى تخص عملية الإيداع.

يعمل خطاف ما قبل الإيداع (pre-commit) أولاً، حتى قبل أن تكتب رسالة الإيداع. ويُستعمل لفحص اللقطة التي ستودعها للتأكد من أنك لم تنس شيئاً، أو للتأكد من أنك نفذت الاختبارات، أو للتأكد مما تريد التأكد منه عموماً. وإنهاء تنفيذ هذا البريمج بقيمة خروج غير الصفر يوقف الإيداع، على أنك تستطيع تخطيه بالأمر `git commit --no-verify`. يمكنك هذا من تنسيق المصدر البرمجي (مثل تنفيذ lint أو ما يكافئه)، أو التأكد من عدم انتهاء السطور بمسافات زائدة (وهذا ما يفعله المثال الآتي مع جت)، أو فحص وجود التوثيق المناسب للدوال الجديدة.

أما خطاف تجهيز رسالة الإيداع (prepare-commit-msg) فيعمل قبل فتح محرر رسالة الإيداع ولكن بعد إنشاء الرسالة المبدئية، حتى يتيح لك تعديل الرسالة المبدئية قبل أن يراها صانع الإيداع. ويُرسَل إلى هذا الخطاف ثلاثة معاملات: مسار الملف المؤقت الذي فيه رسالة الإيداع، ونوع الإيداع، وبصمة SHA-1 للإيداع إذا كان إيداعاً مصححاً (أي مودّع بالخيار --amend). وعموماً ليس هذا الخطاف كبير النفع للإيداعات العادية، لكنه يفيد الإيداعات التي رسالتها المبدئية مولدة آلياً، مثل قوالب رسائل الإيداع، وإيداعات الدمج، والإيداعات المهروسة ("squashed")، والإيداعات المصححة ("amended"). وقد تستخدمه مع قالب رسالة إيداع لإدراج معلومات فيها برمجياً.

وأما خطاف رسالة الإيداع (commit-msg) فيقبل معاملاً واحداً، وهو كذلك مسار الملف المؤقت الذي فيه رسالة الإيداع التي كتبها المطور. وإن أنهى تنفيذه بقيمة خروج غير الصفر فإن جت يوقف الإيداع. فيمكنك إذاً استخدامه لتحقيق حالة مشروعك أو رسالة الإيداع قبل السماح للإيداع بأن يتم. وسنرى في الفصل الأخير من هذا الباب شرح مثال لاستعمال هذا الخطاف لتحقيق موافقة رسائل الإيداع لنمط محدد.

أما خطاف ما بعد الإيداع (post-commit) فيعمل بعد تمام عملية الإيداع بكاملها. ولا يقبل أي معاملات. لكن يمكن جلب الإيداع الأخير بسهولة بالأمر `git log -1 HEAD`. وعموماً يُستعمل هذا البريمج للإعلامات أو ما يشابهها.

خطاطيف البريد

يمكنك إعداد خطاطيف ثلاثة لأساليب التطوير المعتمدة على البريد. جميعها يناديها أمر تطبيق الرقع البريدية `git am`، فإذا لم يكن هذا الأمر في أسلوب التطوير الذي تتبعه فانتقل إلى الفصل التالي. أما إذا كنت تقبل الرقع المُعدّة بالأمر `git format-patch` عبر البريد الشابكي، فقد يفيدك بعضها.

أول خطاف ينفذ هو خطاف رسالة الرقعة (applypatch-msg). ويقبل معاملاً واحداً هو مسار الملف المؤقت الذي فيه رسالة الإيداع المقترحة. وإنهاء تنفيذه بقيمة خروج غير الصفر يوقف الرقعة. فيمكنك استعماله للتأكد من أن رسالة الإيداع صحيحة التنسيق، أو لاستنظام (normalize) الرسالة بتعديلها في مكانها.

والتالي هو خطاف ما قبل إيداع الرقعة (pre-applypatch). وقد يسبب اللبس أنه، خلافاً لاسمه الأصلي بالإنجليزية، يعمل بعد تطبيق الرقعة ولكن قبل صنع الإيداع، فيمكنك استعماله لفحص اللقطة قبل إيداعها. فيمكنك فيه تنفيذ الاختبارات أو فحص شجرة العمل أو غير ذلك. وإن افترقت شيئاً أو فشل اختبار، فإن إنهاء تنفيذ هذا البريمج بقيمة خروج غير الصفر يوقف أمر git am بغير إيداع الرقعة.

أما خطاف ما بعد إيداع الرقعة (post-applypatch) فهو الخطاف الأخير الذي ينفذ عند تطبيق رقعة بريدية بالأمر git am، وينفذ بعد صنع الإيداع. فيمكنك استعماله لإعلام صاحب الرقعة أو غيره بإتمام إيداعها. ولا يمكنك إيقاف الرقعة بهذا البريمج.

خطاطيف العميل الأخرى

ينفذ خطاف ما قبل إعادة التأسيس (pre-rebase) قبل إعادة تأسيس أي شيء، ويمكنه إيقاف العملية بإنهاء تنفيذه بقيمة خروج غير الصفر. فيمكنك استعماله لمنع إعادة تأسيس أي إيداعات قد دُفعت. البريمج المثال المرفق مع جت يفعل هذا، إلا أنه يفترض بعض الأمور التي قد لا توافق أسلوب عملك.

أما خطاف ما بعد التحرير (post-rewrite) فتنفذ الأوامر التي تبدل الإيداعات، مثل أمر تصحيح الإيداع git commit --amend وأمر إعادة التأسيس git rebase (لكن ليس أمر معالجة الفروع git filter-branch). مُعامله الوحيد هو الأمر الذي سبب التحرير، ويستقبل قائمة بالتحريرات على المدخل المعياري (stdin). معظم استعمالات هذا الخطاف مثل خطافي post-checkout و post-merge التاليين.

بعد نجاح أمر السحب، ينفذ خطاف ما بعد السحب (post-checkout). يمكنك استعماله لإعداد مجلد عملك ليناسب بيئة مشروعك. قد يعني هذا مثلاً أن تنقل إليه ملفات رقمية كبيرة لا تريدها في تاريخ المستودع، أو أن تولد الوثائق، أو شيئاً من هذا القبيل.

أما خطاف ما بعد الدمج (post-merge) فينفذ بعد نجاح أمر الدمج. فيمكنك استعماله لكي تستعيد إلى مجلد العمل البيانات التي لا يسجلها جت، مثل أنونات الملفات. ويمكنه كذلك تحقق وجود ملفات معينة يتجاهلها جت وتريدها عندما يتبدل مجلد العمل.

ويعمل خطاف ما قبل الدفع (pre-push) عند تنفيذ أمر الدفع، بعد تحديث الإشارات البعيدة لكن قبل نقل أي كائن. يستقبل مُعاملين هما اسم البعيد المدفوع إليه، ورابطه، ويستقبل من المدخل المعياري قائمة من الإشارات التي ستُحدث. يمكنك استعماله لتحقيق تحديثات الإشارات قبل إتمام الدفع، فإن إنهاء تنفيذه بقيمة خروج غير الصفر يوقف الدفع.

خلال عمل جت أحياناً يحتاج تنظيف سجلاته الخاصة بالمستودع، فينفذ جت أمر التنظيف الآلي git gc --auto، الذي ينظّم الملفات الداخلية للمستودع ويحذف المهملات. (gc اختصار "garbage collection" «جمع المهملات»). ينفذ خطاف ما قبل التنظيف الآلي (pre-auto-gc) قبيل بدء التنظيف الآلي، ليُخبرك مثلاً بحدوثه، أو ليمنعه إن لم يمكن الوقت مناسباً لذلك.

٨، ٣، ٣، خطاطيف الخادوم

بالإضافة إلى الخطاطيف التي تنفذ عند العميل، يمكنك استخدام الخطاطيف التي تنفذ على الخادوم، إذا كنت مدير أنظمة، حتى تنفذ سياسات مشروعك المختارة. وتعمل هذه البرمجيات قبل الدفع إلى خادومك وبعده. فتستطيع الخطاطيف «القبليّة» رفض الدفع بأن تنهي تنفيذها بقيمة خروج غير الصفر في أي وقت، ثم تطبع رسالة خطأ إلى العميل. ويمكنك إعداد سياسة دفع سهلة أو معقدة كما تشاء.

خطاف ما قبل الاستلام pre-receive

أول برميّج يعمل عند استقبال دفع من عميل هو خطاف ما قبل الاستلام (pre-receive). ويقرأ من المدخل المعياري (stdin) قائمة الإشارات التي تُدفع. فإذا أنهى تنفيذها بقيمة خروج غير الصفر، لم تُقبل أيّ منهم. فيمكنك استعماله مثلاً للتحقق أن الإشارات المحدثة جميعها إيداعات تسريع، أو لتحقيق صلاحيات الوصول للإشارات والملفات التي تعديلها إيداعات هذا الدفع.

خطاف التحديث update

خطاف التحديث (update) يشبه كثيراً خطاف ما قبل الاستلام (pre-receive)، إلا أنه ينفذ مرة لكل فرع يريد تحديثه الذي يدفع. فإن كان يدفع إلى عدة فروع، فإن خطاف ما قبل الاستلام (pre-receive) سينفذ مرة واحدة فقط، أما خطاف التحديث (update) سينفذ مرة لكل فرع يدفع إليه. وبدلاً من القراءة من المدخل المعياري، فإنه يقبل ثلاثة معاملات: اسم الإشارة (الفرع)، وبصمة SHA-1 التي كانت تشير إليها الإشارة قبل الدفع، وبصمة SHA-1 التي يريد المستخدم دفعها. فإذا أنهى البرميّج تنفيذها بقيمة خروج غير الصفر، فسُرفض هذه الإشارة وحدها. أما الإشارات الأخرى فلن تتأثر به وقد تقبل.

خطاف ما بعد الاستلام post-receive

أما خطاف ما بعد الاستلام (post-receive) فيعمل بعد تمام العملية بكاملها، ويُستعمل لتحديث الخدمات الأخرى أو إعلام المستخدمين. ويقرأ المعطيات نفسها من المدخل المعياري مثل خطاف ما قبل الاستلام (pre-receive) تماماً. يمكن استعماله لإرسال بريد شابكي إلى قائمة بريديّة، أو تنبيه خادوم دمج مستمر (CI)، أو تحديث نظام متابعة مسائل. فيمكنك تحليل رسائل الإيداع برمجيّاً لتعلم إن كانت مسألة ما تحتاج إلى الفتح أو التعديل أو الغلق. لن يُوقف هذا البرميّج عملية الدفع، لكن لن يقطع العميل اتصاله إلا عندما ينتهي، فاحذر أن تفعل شيئاً قد يستغرق وقتاً طويلاً.

إذا كنت تكتب برميّج خطاف سيقروّه الآخرون، فعليك بالخيارات الطويلة للأوامر؛ سوف نشكرنا بعد شهور من الآن.



٨، ٣، ٤، البرشامة في الخطاطيف (Cheatsheet)

هذا مرجع موجز لجميع الخطاطيف المشروحة. (من إضافة المترجم).

٨، ٣، ٥، خطاطيف أمر الإيداع git commit

١. خطاف ما قبل الإيداع `pre-commit` :
قبل رسالة الإيداع، لفحص اللقطة المودعة. قد يوقف الإيداع.
٢. خطاف تجهيز رسالة الإيداع `prepare-commit-msg` :
قبل فتح المحرر لكن بعد إنشاء الرسالة المبدئية، لتعديلها.
٣. خطاف رسالة الإيداع `commit-msg` :
بعد الفراغ من تحرير رسالة الإيداع، لمراجعتها. قد يوقف الإيداع.
٤. خطاف ما بعد الإيداع `post-commit` :
بعد إتمام عملية الإيداع بكاملها، للإعلامات وما يشابهها.

٨، ٣، ٦، خطاطيف أمر تطبيق الرقع البريدية git am

١. خطاف رسالة الرقعة `applypatch-msg` :
عند البدء، لمراجعة رسالة الرقعة أو تعديلها. قد يوقف تطبيق الرقعة.
٢. خطاف ما قبل إيداع الرقعة `pre-applypatch` (اسمه الإنجليزي خادع):
بعد تطبيق الرقعة وقبل إيداعها، لفحص اللقطة قبل إيداعها. قد يوقف الإيداع.
٣. خطاف ما بعد إيداع الرقعة `post-applypatch` :
بعد الإيداع، للإعلامات وما يشابهها.

٨، ٣، ٧، خطاطيف العميل الأخرى

- خطاف ما قبل إعادة التأسيس `pre-rebase` :
قبل إعادة تأسيس أي شيء. قد يوقف العملية.
- خطاف ما بعد التحرير `post-rewrite` :
بعد تحرير الإيداعات. يستعمل لتجهيز مجلد العمل.
- خطاف ما بعد السحب `post-checkout` :
بعد نجاح أمر السحب. يستعمل لتجهيز مجلد العمل.
- خطاف ما بعد الدمج `post-merge` :
بعد نجاح أمر الدمج. يستعمل لتجهيز مجلد العمل.
- خطاف ما قبل الدفع `pre-push` :
عند بدء تنفيذ أمر الدفع، بعد تحديث الإشارات وقبل نقل شيء. يستعمل لمراجعة ما سيدفع. قد يوقف الدفع.

- **خطاف ما قبل التنظيف الآلي** pre-auto-gc (تنظيم بيانات جت وجمع المهملات منها):
قُبيل بدء التنظيف الآلي، للإعلام به أو منعه عند الحاجة.

٨، ٣، ٨، خطاطيف الخادوم

١. **خطاف ما قبل الاستلام** pre-receive :

عند الدفع إلى الخادوم، ينفَّذ مرة واحدة للدفع كله، ليراجع جميع الإشارات المدفوعة. قد يوقف الدفع.

٢. **خطاف التحديث** update :

عند الدفع إلى الخادوم، ينفَّذ مرة لكل فرع مدفوع إليه، ليراجع جميع الإشارات المدفوعة إليه. قد يوقف الدفع إلى هذا الفرع وحده.

٣. **خطاف ما بعد الاستلام** post-receive :

بعد نجاح الدفع إلى الخادوم، لتحديث الخدمات أو للإعلامات. يبقى العميل متصلاً حتى ينتهي هذا الخطاف.

٨، ٤، مثال لفرض السياسات في جت

٨، ٥، الخلاصة

تناولنا أهم الطرائق التي تخصص بها عميل جت وخادومه ليناسب أسلوب عملك ومشروعاتك. وتعلمت مختلف إعدادات التهيئة، وسمات الملفات في جت، وخطاطيف الأحداث. ثم بنيت مثالاً لخادوم يفرض سياسات إيداع. تستطيع الآن جعل جت ينسجم مع أي أسلوب عمل تفكر فيه.

الملحق الأول:

جت في البيئات الأخرى

إذا أتممت قراءة الكتاب فقد تعلمت كثيراً عن استعمال جت من سطر الأوامر: تستطيع العمل مع الملفات المحلية وتوصيل مستودعك بمستودعات الآخرين عبر الشبكة والعمل بفعالية مع الآخرين. لكن لا تنتهي الرواية هنا؛ غالباً ما يُستعمل جت ليكون جزءاً من نظام كبير، والطرفية ليست دائماً أمثل وسيلة للعمل معه. فسنرى الآن أنواعاً أخرى من البيئات التي يمكن الانتفاع بجت فيها، وكيف يمكن للتطبيقات الأخرى (وتطبيقاتك) العمل مع جت.

أ، الواجهات الرسومية

أ، جت في Visual Studio

عدة جت مبنية في بيئة التطوير Visual Studio نفسها بدءاً من Visual Studio 2019 النسخة 16.8.

تتيح عدة جت فيه هذه الإمكانيات:

- إنشاء المستودعات واستنساخها.
- عرض تواريخ المستودعات وتصفحها.
- إنشاء الفروع والوسوم وسحبها.
- تخبئة التغييرات وتأهيلها وإيداعها.
- استحضار الإيداعات وجذبها ودفعها ومزامنتها.
- دمج الفروع وإعادة تأسيسها.
- حل نزاعات الدمج.
- رؤية الفروقات.
- ... والمزيد!

اقرأ التوثيق الرسمي [الرابط: https://learn.microsoft.com/en-us/visualstudio/version-control](https://learn.microsoft.com/en-us/visualstudio/version-control) لتعلم المزيد.

أ، جت في Visual Studio Code

تستطيع استخدام جت من داخل Visual Studio Code بغير إضافات. تحتاج النسخة 2.0.0 (أو أحدث) من جت.

أهم الخصائص هي:

- رؤية فروقات الملف الذي تحرره في هامش المحرر ("gutter").
- شريط حالة جت ("Git Status Bar") الذي في أسفل يسار النافذة يظهر لك الفرع الحالي، ومؤشرات التغييرات، والإيداعات الداخلة والخارجة.
- إتاحة عمليات جت الأكثر استخدامًا من داخل المحرر:
 - ابتداء مستودع.
 - استنساخ مستودع.
 - إنشاء فروع ووسوم.
 - تأهيل تغييرات وإيداعات.
 - الدفع إلى فرع بعيد والجذب منه والمزامنة معه.
 - حل نزاعات الدمج.
 - رؤية الفروقات.
- باستخدام إضافة، يمكنك كذلك التعامل مع طلب الجذب على جت هب:

<https://marketplace.visualstudio.com/items?itemName=GitHub.vscode-pull-request-github>

هذا التوثيق الرسمي: <https://code.visualstudio.com/docs/sourcecontrol/overview>

أ، ع، جت في / PhpStorm / WebStorm / PyCharm / IntelliJ / RubyMine

إن IntelliJ IDEA و PyCharm و WebStorm و PhpStorm و RubyMine وغيرهم من بيئات JetBrains — يأتون ومعههم إضافة لدمج إمكانيات جت في بيئة التطوير، التي تتيح نافذة فرعية في بيئة التطوير مخصصة للتعامل مع جت ومع طلبات جذب جت هب.



شكل ١٨٣. Version Control ToolWindow in JetBrains IDEs.

تعتمد هذه الإضافة على برنامج سطر أوامر جت، فتحتاج إلى وجوده مثبتًا على جهازك. التوثيق الرسمي متاح في <https://www.jetbrains.com/help/idea/using-git-integration.html>

أ، ه، جت في Sublime Text

بدءً من النسخة 3.2 فإن Sublime Text يدمج إمكانيات جت في المحرر.

أهم الخصائص:

- يُظهر الشريط الجانبي حالة جت للملفات والمجلدات برموز (شارات).
- الملفات والمجلدات المتجاهلة في جت (أي في ملف `.gitignore` للمشروع) ستجد أسماءها في الشريط الجانبي شفافة جزئياً.
- في شريط الحالة يظهر اسم فرع جت الحالي وعدد التعديلات التي صنعتها.
- كل التعديلات في ملف تظهر لها إشارات في هامش المحرر.
- يمكن استعمال بعضاً من إمكانيات عميل جت: Sublime Merge، من داخل Sublime Text. هذا يحتاج أن يكون Sublime Merge مثبتاً على جهازك. انظر: <https://www.sublimemerge.com>.

التوثيق الرسمي لمحرر Sublime Text متاح هنا: https://www.sublimetext.com/docs/git_integration.html

أ، ه، جت في Bash

إذا كنت تستعمل Bash، فيمكنك أن تستفيد من بعض خصائص صَدَقَتك لجعل تجربتك مع جت ألطف كثيراً. وإن جت يأتيك مُحمّلاً بإضافات لصدقات عديدة، لكنها ليست مفعلة مبدئياً.

أولاً، عليك الحصول على نسخة من ملف الإكمالات من المصدر البرمجي لنسخة جت التي تستعملها. اعرف نسختك بالأمر `git version`، ثم نَقِّد `git checkout tags/vX.Y.Z`، حيث `vX.Y.Z` يشير إلى النسخة التي تستعملها. ثم انسخ الملف `contrib/completion/git-completion.bash` إلى مكان قريب، مثل مجلد المنزل، ثم أضف هذا السطر إلى ملف `~/.bashrc` الخاص بك:

```
. ~/git-completion.bash
```

CONSOLE

وما إن تفعل هذا، انتقل إلى مجلد مستودع جت، واكتب:

```
$ git chec<tab>
```

CONSOLE

...وستكمله لك الصدفة إلى `git checkout`. ويعمل هذا مع كل أوامر جت الفرعية، ومعاملاتها، وأسماء الخواديم البعيدة والإشارات حيثما كان مناسباً.

من النافع كذلك تخصيص مستقبل أوامر الصدفة ليخبرك شيئاً عن مستودع مجلدك الحالي. قد تجعله خفيفاً أو معقداً كما تشاء. ومن أشهر ما يريده أكثر الناس: الفرع الحالي، وحالة مجلد العمل. فلإضافتهما، انسخ ملف `contrib/completion/git-prompt.sh` من مستودع مصدر جت إلى مجلد المنزل، ثم أضف في ملف `~/.bashrc` هذا:

```
CONSOLE
. ~/git-prompt.sh
export GIT_PS1_SHOWDIRTYSTATE=1
export PS1='\w$(__git_ps1 " (%s)")\$ '
```

السلسلة المَحْرِفِيَّة `\w` تعني المجلد الحالي، و `\$` تعني علامة `$` نفسها (خاتمة رسالة مستقبل الأوامر)، أما الأمر `__git_ps1 " (%s)"` فينادي الدالة المَعْرِفَة في ملف `git-prompt.sh` ويعطيها الصيغة المطلوبة. فالآن سيبدو مستقبل صدفتك هكذا في أي مجلد داخل مشروع جت:



شكل ١٨٤. مستقبل Bash مخصص

وستجد مع هذين الملفين توثيقاً نافعاً؛ فانظر في `git-completion.bash` و `git-prompt.sh` لتعرف المزيد.

أ، v، جت في Zsh

تأتي Zsh أيضاً بمكتبة إكمال بزر الجدولة من أجل جت. لاستعمالها، ضع `autoload -Uz compinit && compinit` في ملف `~/.zshrc`. الخاص بك. وإن واجهة Zsh أقوى بعض الشيء من Bash:

```
CONSOLE
$ git che<tab>
check-attr      -- display gitattributes information
check-ref-format -- ensure that a reference name is well formed
checkout        -- checkout branch or paths to working tree
checkout-index  -- copy files from index to working directory
cherry          -- find commits not merged upstream
cherry-pick     -- apply changes introduced by some existing commits
```

فالإكاملات الغامضة لا تُسرد فحسب، بل تجد معها أوصافاً مفيدة، وكذلك يمكنك الانتقال فيها بالضغط المتكرر على زر الجدولة. ويعمل هذا مع أوامر جت، ومعاملاتها، وأسماء الأشياء في المستودع (مثل الإشارات المحلية والمستودعات البعيدة)، وكذلك أسماء الملفات، وكل الأشياء الأخرى التي تعرف Zsh كيف تكملها بزر الجدولة.

ومع Zsh إطار برمجي للاستعلام من أنظمة إدارة النسخ، يسمى `vcs_info`. فلاضافة اسم الفرع الحالي يمين مستقبل الأوامر، أضف هذه الأسطر إلى ملف `~/.zshrc`. الخاص بك:

```
autoload -Uz vcs_info
precmd_vcs_info() { vcs_info }
precmd_functions+=( precmd_vcs_info )
setopt prompt_subst
RPROMPT='${vcs_info_msg_0_}'
# PROMPT='${vcs_info_msg_0_}%# '
zstyle ':vcs_info:git:*' formats '%b'
```

CONSOLE

وسيفيظهر اسم الفرع الحالي في الجهة اليمنى من نافذة الطرفية كلما تكون في داخل مستودع جت. والجانب الأيسر مدعوم أيضاً؛ فقط أزل علامة التعليق من أمام سطر الإسناد إلى `PROMPT`. سيبدو تخصيصنا للجانب الأيمن مثل هذا:

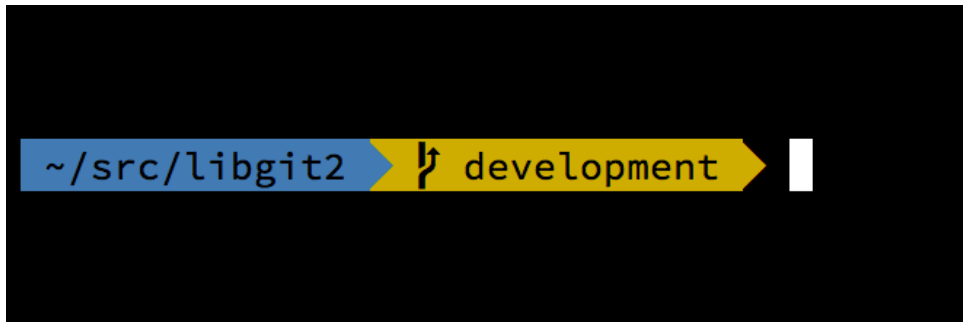


شكل ١٨٥. مستقبل Zsh مخصص

لمعرفة المزيد عن `vcs_info`، انظر توثيقه بالإنجليزية: في صفحة الدليل (1) `zshcontrib` أو <https://zsh.sourceforge.io/Doc/Release/User-Contributions.html#Version-> (الرابط: [Control-Information](https://zsh.sourceforge.io/Doc/Release/User-Contributions.html#Version-)).

وبدلاً من `vcs_info`، قد يناسبك برمج تخييص المستقبل المرفق مع جت، المسمى `git-prompt.sh`؛ انظر [مصدره](https://github.com/git/git/blob/master/contrib/completion/git-prompt.sh) (الرابط: <https://github.com/git/git/blob/master/contrib/completion/git-prompt.sh>). للتفاصيل. وإنه متوافق مع الصدفتين Bash و Zsh.

صدفة Zsh ذات قدرات كبيرة حتى إن لها أطر برمجية خصيصاً لجعلها أفضل. أحدها يسمى “oh-my-zsh”، ويمكن الوصول إليه عبر [مستودعه](https://github.com/ohmyzsh/ohmyzsh) (الرابط: <https://github.com/ohmyzsh/ohmyzsh>). ونظام إضافات oh-my-zsh يتيح إكمالاً قديراً لجت بزر الجدولة، وفيه «أنساق» (“themes”) عديدة للمستقبل، والكثير منها يُظهر معلومات المستودعات. وما ترى في شكل [مثال على أحد أنساق oh-my-zsh](https://github.com/ohmyzsh/ohmyzsh) ليس إلا مثلاً واحداً على ما يمكن عمله به.



شكل ١٨٦. مثال على أحد أنساق oh-my-zsh

أ، ٨، جت في PowerShell

أ، ٩، الخلاصة

لقد تعلمت كيف تسخر قوى جت من داخل الأدوات التي تستعملها في عملك اليومي، وتعلمت كيف تصل إلى مستودعات جت من خلال برامجك الخاصة بك.

الملحق الثاني:

تضمين جت في تطبيقاتك

إذا كان تطبيقك للمبرمجين، فغالباً سينتفع من الاندماج مع نظام إدارة نسخ. حتى تطبيقات غير المبرمجين، مثل محررات المستندات، لعلها تنتفع من ميزات إدارة النسخ. ونموذج جت فعال حقاً في أنواع كثيرة من المواقف.

فإذا أردت دمج جت مع تطبيقك، فعلياً عندك خياران: تشغيل صَدَقَة ونداء برنامج سطر الأوامر git منها، أو تضمين مكتبة جت في تطبيقك. سنتناول هنا دمج جت عبر سطر الأوامر وعبر مجموعة من أشهر مكتبات تضمين جت.

ب، ا، جت في سطر الأوامر

أحد خياري دمج جت مع تطبيقك: أن تشغل صَدَقَة وتستعمل أداة سطر الأوامر git منها لفعل ما تريد. من مزايا هذا الخيار أنه البديهي المعتاد، وأن جميع وظائف جت متاحة فيه. ويصادف أنه كذلك سهل جداً، لأنك تجد في معظم البيئات وسيلة سهلة نسبياً لتشغيل برنامج من سطر الأوامر مع معاملاته. لكن مع ذلك، هذا الأسلوب له عيوب.

أحدها أن كل النواتج نصوص مجردة. فيعني ذلك أنك تحتاج أن تحلل صيغة ناتج جت التي قد تتغير أحياناً، حتى تعرف الحالة والنتيجة. وهذا قد يكون عرضة للخطأ أو غير كفاء.

وآخر هو نقص التعافي من الأخطاء. فإن كان بالمستودع تلف، أو للمستخدم متغير تهيئة معطوب، فعمليات عديدة سيرفض جت إجرائها.

ثم عيب ثالث هو إدارة العمليات (البرامج العاملة). فيحتاج تشغيل جت هكذا إلى صَدَقَة في عملية منفصلة، وهذا قد يضيف تعقيداً غير مرغوب فيه. وقد يصعب فعلاً محاولة التنسيق بين عددٍ من مثل هذه العمليات (خصوصاً إذا كانت بضع عمليات تتعامل مع مستودع واحد).

ب، ا، مكتبة Libgit2

ب، ا، جت في جافا: JGit

ب، ا، جت في لغة غو: go-git

ب، ه، جت بيشون: Dulwich

توجد كذلك نسخة بيثونية خالصة من جت: Dulwich. وهذا المشروع مستضاف في dulwich.io [\(الرابط: https://dulwich.io/\)](#). ومسعاها أن يتيح واجهة لمستودعات جت (المحلية والبعيدة)، لكن بغير برنامج جت نفسه، بل بلغة Python وحدها. لكن فيه امتدادات اختيارية مكتوبة بلغة C، وهي تُزِيد كثيراً من سرعة تنفيذه.

(من المترجم) يتوقف الموقع الرسمي كثيراً. لكنه على أرشيف الإنترنت في: <https://web.archive.org/web/20250114030653/https://www.dulwich.io/>، وكذلك توثيقه ودروسه وحدها في: <https://dulwich.readthedocs.io/en/latest/>.



وفي أبريل ٢٠٢٤م غيّر المشروع امتداداته الاختيارية من لغة C إلى Rust.

يتبع Dulwich تصميم جت فيقسم واجهته البرمجية إلى مستويين: سبابة (أوامر سفلية) وبورسلين (أوامر علوية).

هذا مثال على الواجهة البرمجية السفلى للوصول إلى رسالة الإيداع الأخير:

```
from dulwich.repo import Repo
r = Repo('.')
r.head()
# '57fbe010446356833a6ad1600059d80b1e731e15'

c = r[r.head()]
c
# <Commit 015fc1267258458901a94d228e39f0a378370466>

c.message
# 'Add note about encoding.\n'
```

PYTHON

وهذا لعرض سجل إيداع بالواجهة البرمجية العليا:

```
from dulwich import porcelain
porcelain.log('.', max_entries=1)

#commit: 57fbe010446356833a6ad1600059d80b1e731e15
#Author: Jelmer Vernooij <jelmer@jelmer.uk>
#Date: Sat Apr 29 2017 23:57:34 +0000
```

PYTHON

ب، ه، ا، قراءة المزيد

تجد على موقع Dulwich الرسمي [\(الرابط: https://www.dulwich.io/\)](https://www.dulwich.io/) توثيق الواجهة البرمجية ودروساً وأمثلة كثيرة على فعل أمور معينة به.

الملحق الرابع:

دليل المصطلحات

لأسماء الأوامر، انظر فصل [أسماء أوامر جت](#).

وما وصل إلى [المصطلحات المتفق عليها في معجم يسمو](https://www.noureddin.dev/ysmu/) (الرابط: <https://www.noureddin.dev/ysmu/>) لا نعيده هنا، إلا ما يختص بجت أو بإدارة النسخ مثل "commit" و "repository".

وصول، إذن	access
مستوى الوصول(?)	access control level
مزية (ج: مزايا) (وليس ميزة/مميزات؛ هذه feature؛ انظر الفرق في الملحق الآخر)	advantage
كُنية (ج: كُنَيَات)	alias
تصحيح	amend
وسم معنون (انظر أيضا lightweight tag)	annotated tag
أباتشي	Apache
ملف مضغوط، أو حزمة (ج: حُرْم، والفعل: حَزَم يَحْزِم حَزْمًا، والأداة مَحْزَم (ج: مَحَازِم)؛ من لسان العرب الرابط: https://wiki.dorar-aliraq.net حزم (/lisan-alarab/))	archive, archived file
مُعَامِل (ج: مُعَامِلَات)	argument
سَلَف (ج: أَسْلَاف)	ancestor
إِسْنَاد	assign, assignment
خاصية	attribute
إكمال آلي	autocompletion
تلقائيًا، آليا	automatically

صورة شخصية، صورة تشخيصية	avatar
التوافقية مع الإصدارات السابقة	backward compatibility
مجرد	bare
كتلة (ج: كتل)	blob
فرع، تفريع	branch, branching
علّة (ج: علل)	bug
تعديل	change
باب	chapter (in the book)
مُحرّف (ج: محارف)	character
تحقّق [الشيء] (بغير «من»)، تفقّد [الشيء]	check
سحب	check out
[قيمة] بصمة	checksum
استنساخ	clone
يساهم، مساهمة	collaborate, collaboration
مساهم	collaborator
أمر	command
سطر أوامر	command line
يودع، إيداع، مودع، يصنع إيداعا	commit, commit, committed, do/make a commit
يضغط، ضغط، مضغوط	compress, compression, compressed
حاسوب (ج: حواسيب)	computer
ضبط، تهيئة	configure, configuration

نزاع	conflict
علامات تنازع	conflict markers
دمج مستمر	continuous integration
يساهم، مساهمة	contribute, contribution
مخصص	custom
مخصص، مفضل	customized
عفريت (ج: عفاريت) (انظر ملاحظة المترجم في أول فصل عفريت جت)	daemon
عفرتة	daemonize
صفحة رئيسية	dashboard
مبدئي، مفترض	default
فرق (ج: فروقات)	delta
⌵(⌶)⌵	detached HEAD [state]
تطوير، برمجة	develop, development
مطوّر، مبرمج	developer
مجلد	directory
قرص	disk
موزّع، منوزّع	distributed
تنزيل	download
مصّب (انظر أيضا upstream)	downstream
بريد شابكي (نسبةً إلى الشبكة (الإنترنت) وليس بريد إلكتروني نسبةً إلى الأجهزة الإلكترونية)	email
بيان	entry

بيئة	environment
متغير بيئة (ج: متغيرات بيئة)	environment variable
خطأ، عُرضة للخطأ	error-prone
إنهاء التنفيذ بقيمة خروج غير الصفر	exit non-zero (verb, intransitive)
خبرة	experience (previous knowledge)
تجربة	experience (usage, like user ~ (UX), developer ~ (DX))
تسريع	fast-forward
دمج تسريع	fast-forward merge
خَصِيصَة (ج: خصائص)، ميزة (ج: ميزات) (وليس مزية/مزايا؛ هذه advantage؛ انظر الفرق في الملحق الآخر) (انظر أيضا killer feature)	feature
جدار حماية	firewall
مجلد	folder
اشتق، يشتق، اشتقاق	fork
إطار برمجي (ج: أطر برمجية)	framework
مكتمل الخصائص (انظر أيضا feature)	fully featured
جت	Git
جتهب	GitHub
جت لاب	GitLab
جتوب	GitWeb
أنماط توسيع [المسارات]	glob
واجهة رسومية	graphical user interface, GUI

خارق	1. expert or eager learner: meanings) hacker http://catb.org/esr/jargon/html/H/hacker.html 1-7 in the Jargon File
مخترق	2. cracker: meaning 8 in the the Jargon) hacker http://catb.org/esr/jargon/html/C/cracker.html File or the standalone entry
معالج	handler (Apache)
قرص [صلب]	hard disk
رابط صلب	hardlink
بصمة	hash
فرع الرأس(?)	HEAD branch
ترويسة	header
إشارة الرأس	HEAD [pointer]
شجري	hierarchical
تاريخ	history
خطاف (ج: خطاطيف)	hook
بريمج خطاف	hook script
يستضيف، استضافة	host, hosting
اسم المضيف(?)	hostname
ضم	include
الفهرس	index (in Git)
ابتداءً، يبتدئ، مبتدأ (أو «مُنشأ» في الصفة)	initialize, initialization, initialized
تنشيت	install, installation (1. software)

تركيب	install, installation (2. non-software)
بيئة تطوير [متكاملة]	integrated development environment, IDE
(في البرمجة) دمج، اندماج، ضم (وليس تكامل)	integration
سلامة	integrity
الإنترنت (بهمزة قطع)، الشبكة	Internet
مشكلة، مسألة	issue
ميزة قاتلة للمنافسة (انظر أيضا feature)	killer feature
صفحة استقبال	landing page
رخصة	license
وسم خفيف (انظر أيضا annotated tag)	lightweight tag
تقييد	limit (v)
فاصل سطر (ج: فواصل سطور) (متحد مع newline)	linebreak
قائمة	list (n)
سرد	list (v)
ولوج	login
سجل	log (n)
تطوير، رعاية	maintain, maintenance
القائم على المشروع، مدير المشروع، مشرف	maintainer
إيداع دمج	merge commit
مدموج	merged
دمج [في]	merge [to]
بيانات وصفية	metadata

نسخة مقابلة	mirror [copy]
خادوم مرآة (٢)	mirror [server]
نموذج	model
تعديل	modify
ضم، مضموم	mount, mounted
مساحة أسماء	namespace
فاصل سطر (ج: فواصل سطور) (متحد مع linebreak)	newline
لاخطي (كلمة واحدة، بغير مسافة)	nonlinear
استنظام	normalize, normalization
مصادر مفتوحة	open source
ناتج	output
عُلبَة (ج: عُلْب)	pack
ملف عُلبَة (ج: ملفات عُلبَة)	packfile
[برنامج] عارض	pager
رُقعة (ج: رُقَع، رِقَاع)، ترقيع	patch
قناة [يونكسية]	pipe (n)
منصة	platform
إشارة (متحد مع ref)	pointer
سياسة، سياسات (تُستعمل في العربية غالباً بالإضافة أو بالجمع، تمييزاً لها عن politics = السياسة العامة)	policy
توجيه منفذ	port forwarding
منسَّق (مثل pretty-print أو خيار pretty format في أمر السجل وأمر الإظهار وغيرهما)	pretty

خصوصي (وليس «خاص»)	private
مشروع (ج: مشروعات (الأنصَح)، مشاريع (مقبولة)، وهذا في كل ما على وزن «مفعول» مثل «موضوع»)	project
مستقبل [الأوامر] (ج: مستقبلات [الأوامر])	prompt
رسالة مستقبل [الأوامر] (ج: رسائل مستقبل [الأوامر])	prompt string
احتكاري	proprietary
ميفاق (ج: موافيق)، بروتوكول	protocol
عمومي (وليس عام)	public
جذب	pull
طلب جذب، طلب دمج (انظر ملاحظة المترجم بخصوص المصطلحين في فصل المساهمة في مشروع)	pull request
دفع	push
إذن القراءة (انظر الملحق: إذنا القراءة والتحرير)	read access
إذن التحرير (انظر الملحق: إذنا القراءة والتحرير)	read/write access
إعادة تأسيس [على]	rebase [on]
سجل الإشارات	reflog
إشارة (انظر الملحق: إشارة الرأس وإشارات الكائنات)	ref, reference
إصدار، إصدار	release
[مستودع] بعيد	remote
فرع متعقب لبعيد	remote-tracking branch
إزالة	remove
تغيير اسم	rename
مستودع	repo, repository

إرجاع	reset
استعادة	restore
نقض	revert
مراجعة	revision
التحكم في المراجعات	revision control
تحرير	rewrite
تحرير التاريخ	rewriting history
إعادة(?)	rollback
جذر، جذري	root
مجلد جذر	root directory
بُريمِج (ج: بُريمِجات)	script
برمجة	scripting
قسم	section (in a Git project)
فصل	section (in the book)
قدّم، يقَدِّم، تقديم، يتيح، إتاحة	serve
خادوم (ج: خواديم)	server
تعيين	set [value]
ترتيب (ج: ترتيبات)، تركيب، تكوين	setup (arrangement)
تثبيت	setup (installation)
إعداد	set up, setting
صدّفة، طرفية	shell
وصول صدّفي	shell access

توقيع [شيء]	sign [sth]
نقطة انهيار حاسمة	single point of failure
لقطة	snapshot
يؤهل، تأهيل، مؤهل	stage, staging, staged
منطقة التأهيل	staging area
معيّار	standard
المُدخَل المعياري (اسم مكان من «دخل يدخل»)	stdin
المُخرَج المعياري (اسم مكان من «خرج يخرج»)	stdout
انتقال	switch (v)
نظام (ج: أنظمة)	system
مسافة جدولة	tab (char)
إكمال بزر الجدولة	tab-completion
زر الجدولة	tab (key)
تبويب	tab (UI)
وسم معنون	tag, annotated
وسم خفيف	tag, lightweight
فرقة (ج: فرق) (?)	team
زميل (ج: زملاء)	teammate
طرفية	terminal [emulator]
خَتَم زمني	timestamp
فرع موضوع (ج: فروع مواضيع، وليس موضوعات؛ انظر project للتفصيل)	topic branch

تتبع، متعقب (انظر الملحق: التتبع والمتابعة : tracking)	track, tracking
معاملة	transaction
تراجع	undo
غير متعقب	untracked
رفع	upload
منبع (انظر أيضا downstream)	upstream
توثيق [شيء]، تحقق [شيء] (بغير «من»)	verify [sth]
نسخة	version
إدارة النسخ	version control
نظام إدارة نسخ	version control system, VCS
مراقب [للتغييرات]	versioned
شبكة وهمية [خصوصية]	virtual private network, VPN
وب	Web
أسلوب، أسلوب تطوير، أسلوب عمل	workflow
نسخة عمل	working copy
شجرة العمل	working tree

الملحق الخامس:

المصطلحات والمفاهيم باللغتين

يضم هذا الملحق أوامر جت بأسمائها العربية، ويضم كذلك مفاهيم جت وأنظمة إدارة النسخ المتوزعة، ويتناولها جميعاً بشرح موجز مع توضيح أسمائها باللغتين وأسباب هذه الأسماء، ويضم ملاحظات متفرقة للمترجم.

هـ، ١، الإيداع والسحب

من أهم أغراض أنظمة إدارة النسخ هو حفظ النسخة الحالية من مجلد العمل كما تحفظ الأموال في المصرف، ثم عندما تحتاجها تأخذها منه. ولكنك لا تأخذها للأبد، بل «تستلفها» مؤقتاً مثلما تستلف كتاباً من المكتبة العامة.

فعملية السحب هذه لها اسم واحد شائع في أنظمة إدارة النسخ: check out.

أما عملية الحفظ (الإيداع) فلها اسمان في الأنظمة المختلفة: check in أو commit. ويستعمل جت الاسم الأخير.

انظر أيضاً: <https://stackoverflow.com/q/12510574>

وانظر: <https://www.nouredin.dev/ysmu/link/commit>

هـ، ٢، الدفع والجذب والاستحضار

لأن «سحب» محجوز لـ check out، فكان علينا الإتيان بلفظ آخر ليعني pull.

العملية المرافقة لـ pull هي push، وكلاهما يعرفان بالدفع والجذب، فكان هذان اللفظان مناسبين.

ولكن عملية الجذب pull عمليتان في الحقيقة، أولاهما fetch، لإحضار الكائنات (objects) والإشارات (refs) من المستودع البعيد. فكان اللفظ المناسب لها تنزيل أو إحضار، فاخترت «استحضار» (طلب الحضور) لتمييزها عن الكلمة العامة «إحضار».

هـ، ٣، الإرجاع والاستعادة والنقض

يفرق جت بين الإرجاع reset، والاستعادة restore، والنقض revert، وطبعاً إعادة التأسيس rebase.

وقد حاولت أن أجعل أسماءهم العربية متباعدة، تقليلاً للخلط الأكيد بينهم.

والخلط بينهم وارد حتى إن دليل (“manpage”) جت نفسه يخصص فصلاً للفرق بينهم، ثم يشير إلى هذا الفصل في دليل كل أمر منهم. فنجد في دليل git فصلاً بعنوان “Reset, restore and revert”، هذه ترجمته:

” في جت ثلاثة أوامر بأسماء متشابهة: الإرجاع `git reset`، والاستعادة `git restore`، والنقض `git revert`.

- أمر `git revert` ينقض إيداعاً جديداً ينقض (يعكس) فيه التعديلات التي قدّمتها إيداعات سابقة معينة.
 - أمر `git restore` يستعيد ملفات في شجرة العمل من الفهرس (منطقة التأهيل) أو من إيداع سابق. هذا الأمر لا يحدّث الفرع الحالي. يمكن استعمال هذا الأمر كذلك لاستعادة ملفات في الفهرس من إيداع سابق.
 - أمر `git reset` يحدّث الفرع الحالي بتحريك رأس الفرع ليضيف أو يزيل إيداعات منه. هذه العملية تغيّر تاريخ الإيداعات.
- يمكن استعمال أمر الإرجاع `git reset` لاستعادة الفهرس، وهو استعمال يشترك فيه مع أمر `git restore` لاستعادة.

ثم يذكر هذا في دليل أمر `git revert` :

”لاحظ: يُستعمل أمر `git revert` لتسجيل إيداعات جديدة تنقض (تعكس) تأثير إيداعات سابقة معينة (غالبا إيداعات خاطئة). إن أردت نبذ التعديلات غير المؤهلة جميعا من مجلد العمل، فانظر أمر الإرجاع `git reset`، تحديدا الخيار `--hard`. إذا أردت استخلاص ملفات معينة من إيداع سابق، فانظر أمر الاستعادة `git restore`، تحديدا الخيار `--source`. كن حذرا في استعمالك هذين البديلين، فكلهما يلغي التعديلات غير المؤهلة التي في مجلد عملك.

لم أسمّ أي أمر منهم باسم «إعادة» خشية خلطه على الناس مع «استعادة»، وكذلك لم أسمّ أمراً «تراجع» خشية خلطه مع «إرجاع»، بل أثرت جذراً مختلفاً لكلٍ منهم.

هـ، ٤، أسماء أوامر جت

نسمي أوامر جت، وخصوصا الأوامر العلوية (انظر ~~الأوامر السفلية والعلوية (السبابة والبورسلين)~~)، بأسماء عربية، فمثلا أمر `git commit` اسمه أمر الإيداع، وأمر `git branch` اسمه أمر الفروع، وهكذا.

جميع الأسماء معرّفة، مثل «أمر الإيداع» و«أمر السحب». والأوامر في غالبيتها إما مصادر أفعال («دفع»، «جذب») وإما أسماء أشياء يديرها جت فتُقال بالجمع («فروع»، «وسوم»، «بُعداء»). وبعضها فيه أمر ضمني بالعرض («الحالة»، «السجل»). وبعضها يزيد عن كلمة واحدة (مثل «إعادة التأسيس»)، وتخرج أوامر قليلة عن هذه الأنواع.

وأذكر أسماء الأوامر هنا مع وجودها في الملحق الثالث لسببين: أحدهما أن الملحق الثالث غير منشور لأنه غير مكتمل بعد (فذلك ما في الجدول الأول)، والآخر أن من الأوامر المذكورة في الكتاب ما لم يُذكر في الملحق الثالث (حتى الآن)، مثل الأوامر الجديدة كأمر `git restore` وأمر الانتقال `git switch`، ومثل بعض الأوامر السفلية كأمر سرد الملفات `git ls-files` وأمر استعراض الملف `git cat-file` (وذلك ما في الجدول الآخر).

الإضافة	add
تطبيق الرقع البريدية	am
تطبيق الرقع المجردة	apply
الحَزْم	archive
التفتيش	bisect
العتاب	blame
الفروع	branch
السحب (انظر: الإيداع والسحب)	checkout
الاصطفاء	cherry-pick
التنظيف	clean
الاستنساخ	clone
الإيداع (انظر: الإيداع والسحب)	commit
تهيئة	config
الوصف	describe
الفروق	diff
أداة الفرق	difftool
الاستيراد السريع	fast-import
الاستحضار (انظر: الدفع والجذب والاستحضار)	fetch
معالجة الفروع	filter-branch
تنسيق الرقع	format-patch
فحص نظام الملفات	fsck
جامع المهملات	gc

البحث	grep
المساعدة	help
رفع مسوِّدة البريد	imap-send
الابتداء	init
السجل	log
الدمج	merge
أداة الدمج	mergetool
النقل	mv
الجذب (انظر: الدفع والجذب والاستحضار)	pull
الدفع (انظر: الدفع والجذب والاستحضار)	push
إعادة التأسيس	rebase
سجل الإشارات	reflog
البعداء	remote
طلب الجذب	request-pull
الإرجاع (انظر: الإرجاع والاستعادة والنقض)	reset
النقض (انظر: الإرجاع والاستعادة والنقض)	revert
الإزالة	rm
إرسال البريد	send-email
السجل الموجز	shortlog
الإظهار	show
التخبيئة	stash
الحالة	status

الوحدات الفرعية	submodule
الوسوم	tag

من الأوامر غير المذكورة في الملحق الثالث:

استعراض الملف	cat-file
العفريت	daemon
سرد الملفات	ls-files
استعراض البعيد	ls-remote
استعراض الشجرة	ls-tree
الاستعادة (انظر: الإرجاع والاستعادة والنقض)	restore
الانتقال	switch

هـ، هـ، التعديل والتحرير

يكثر في الكتاب استعمال change أو modify ومشتقاتهما، بالتبادل. ويكثر فيه اتحاد changes مع changeset مع work في المعنى.

أول الأمر استعملت فعل التعديل في هذا المعنى، ثم بدأت باستعمال فعل التغيير بدلاً منه، لأن التعديل يعني التقويم، ولا يعني بالضرورة كل تغيير. ثم وجدت أن التعديل في مواضع كثيرة صالح، وفي مواضع كثيرة أوضح من التغيير. فكنيت وقتئذٍ أبادل بين الفعلين، ولم أستطع إتمام الانتقال إلى فعل التغيير، ورأيت أن استعمال لفظين بالتبادل قد يوهم بعض الناس، فعدلت عن ذلك واقتصرت على التعديل في أكثر المواضع.

وغالباً لا أستعمل التغيير إلا خارج معناه في إدارة النسخ، أي في معناه العام في اللغة.

وأحياناً أترجم work أو changes بالإيداعات، متى كان ذلك أوضح.

أما rewrite وتصريفاتها، فقد ترجمتها أول الأمر بالترجمة الحرفية «إعادة كتابة»، ثم تبين لي بقراءتها في السياق ضعفها. فاخترت عندئذٍ «مراجعة»، ثم لم أجدها مناسبة في بعض الجمل، وبعد تجارب أخرى، هُديت أخيراً إلى «تحرير». وبعد ذلك وجدتني أستعمل معها «تغيير».

و«تحرير» ترادف «تعديل» اليوم عند الكثير من الناس. والفرق بينهما في الكتاب هَيْن. مثلاً تحرير التاريخ هو تغيير الإيداعات نفسها. وتحرير الإيداع هو تغيير فيه. وقد التزمت «تحرير» في مواضع معينة والتزمت «تعديل» في مواضع أخرى. لكن كما ذكرت، كلاهما (أحياناً) «تغيير».

هـ، ٦، إشارة الرأس وإشارات الكائنات

يقول جت أحيانا pointer وأحيانا ref أو reference، لكن خلافاً لبعض لغات البرمجة، هذه جميعاً تعني الشيء نفسه (الرابط: <https://github.com/progit/progit2/issues/1460>)، وهو الشيء الذي «يُشير» إلى كائن أو شيء آخر.

عند التحدث عن الفأرة مثلاً، فكلمة «مؤشر» (pointer أو cursor) صحيحة لأنها اسم الفاعل من الفعل «يؤشر» أي ذلك الشيء الذي «يضع إشارة». أما عند التحدث عن البرمجة، فإن pointer لا تعني «مؤشر»، لأن pointer لا يضع إشارة على شيء، بل يشير إلى شيء، فاللفظ الصحيح هو «مُشير». وهو اللفظ المستخدم في لغة كلمات.

أما كلمة reference، ففي سياق الكتب تعني الكتاب الذي نرجع إليه للبحث عن معلومة، فترجمته إلى «مرجع» عندئذٍ صحيحة. لكن في سياق البرمجة، reference لا يرجع إلى شيء، بل يُرجعنا نحن أو يُحيلنا إلى شيء (refer to)، فترجمته إلى «مُحيل» أفضل.

ولكن المشير والمحيل اسمان لمسمى واحد في جت، فالأفضل توحيد الاسم.

استعملت «مشيراً» في البدء، ثم وجدت أن الأقرب في الاستعمال هو «إشارة»، فهي ما استعملت في الكتاب.

ولمنع اللبس، أقول «إشارة الرأس» غالب الوقت للفظ HEAD.

ولكن جت يفرق بين head وHEAD؛ انظر man gitglossary أو على الشابكة (الرابط: <https://git-scm.com/docs/gitglossary#Documentation/gitglossary.txt-aiddefhashahash>).

هـ، ٧، التعقب والمتابعة: tracking

يُستعمل الفعل tracking في المشروعات البرمجية استعمالين رئيسين، ويترجم بلفظ مختلف حسب استعماله:

١. «المتابعة»، وهي أن يتابع المرء العلل (bugs) والمسائل (issues) والأهداف (milestones) وغير ذلك. ومنها متابع العلل (bug tracker) أو متابع المسائل (issue tracker).

٢. «التعقب»، وهي (١) أن يتعقب جت ملفاً، أي أن يتابع تغييراته ويرصدها ويسجلها، (٢) وأن تجعل جت يتعقب فرعاً بعيداً، أن أي يجعل جت فرعاً محلياً (يسمى «فرعاً متعقباً») أو إشارة محلية (تسمى «فرعاً متعقباً لبعيد») تتابع التغييرات الحادثة في الفرع البعيد. (انظر الفصل التالي لتفصيل هذا الأمر).

هـ، ٨، الفروع البعيدة والفروع المتعقّبة لبعيد والفروع المتعقّبة

يفرق جت وكتاب احترف جت بين الفروع المتعقّبة (tracking branch) والفروع المتعقّبة لبعيد (remote-tracking branch)؛ انظر [تعقب الفروع](#).

لكن لا يبدو أن الكتاب يفرق بين الفروع البعيدة (remote branch) والفروع المتعقّبة لبعيد (remote-tracking branch)، إلا قليلاً، فكلاهما نظرتان للشيء نفسه: الفرع origin/master مثلاً هو الفرع الرئيس في المستودع البعيد، فهو في المستودع المحلي فرع متعقب لبعيد، لكنه يشير إلى الفرع البعيد نفسه. فيجوز قول «الفرع البعيد» للفرع المتعقب لبعيد من باب المجاز المرسل أغلب الوقت. والتبادل بينهما هكذا هو أسلوب الكتاب إلا قليلاً. (من أمثلة هذا القليل: [حذف فروع بعيدة](#)).

هـ، ٨، ا، تفكّر

ربما الأفضل تفريق التسمية بينهما؟ مثلاً:

- الفروع المشيرة = remote-tracking branches
- الفروع المتابعة = tracking branches

فالفرع المشير يشير إلى [حالة] فرع بعيد (remote branch) (الفقرة الثانية في فصل [الفروع البعيدة](#)).
والفرع المتابع يتابع فرعَ منبع (upstream branch) (الفصل الفرعي [تعقب الفروع](#)).

هـ، ٩، إذن القراءة والتحرير

أقترح تعريب “read-only” بـ«القراءة»، و “read/write” بـ«التحرير»، والاسم “access” المرتبط بهما غالباً بـ«إذن» (وجمعه «أذون»).

واخترت هذين الفعلين لأنهما متعديان بغير حرف جر، فلا نحتاج أن نقول «الكتابة على/إلى/في المستودع»، بل نقول «تحرير المستودع» بغير وسيط. ومثله في القراءة.

و«التحرير» أقوى من «الكتابة» المجردة، فهو يعني التعديل والتقويم، وحديثاً يشمل المراجعة والإنشاء. فمعناه واضح فيه أنه “read/write”، فغالباً لا يوجد إذن «كتابة فقط»، فالأولى كلمة تدل على القراءة والكتابة معاً.

وقد استعملت وقتاً قصيراً كلمة «اطلاع» تعريباً لـ “read”، لكنني عدلت عنها إلى «قراءة» لحاجة «اطلاع» إلى حرف جر، وعدم وجود منفعة من «اطلاع» (مثل شمول معنى «تحرير»، فصار أنفع من «كتابة»)، ولشهرة «قراءة».

لكن يستعمل الكتاب كثيراً “write access” للمستودع، فعندئذٍ أترجمها غالباً إلى «إذن الدفع».

هـ، ١٠، تعريب كلمة كود

لهذه الكلمة معانٍ كثيرة في غير البرمجة، منها رمز ورقم ومعيّار وغيرهم.

ومعناها في البرمجة شديد الاتساع ويستعصي على النقل إلى لغة أخرى، فأبى أكثر الناس إلا أن يأخذوا هذه الكلمة بكل معانيها البرمجية. فمنهم من نقلها صوتياً («كود») ومنهم من أتى بكلمة من جذر الكلمة الإنجليزية الأصلية («رمز») في لغتهم، فقال السوريون «رماز» (على وزن «كتاب») وقال الصينيون 代码 (رمز تبديل(?)).

ولا أرى في هذا خيراً كثيراً، فهذا لفظ أعجمي في أصله ومعناه واستعماله، وليس فيه من العربية إلا حرفه.

ففي هذا الكتاب نحاول تعريب هذه الكلمة في مواضعها المختلفة تعريباً يفهمه العربي بغير شرح وبغير معرفة الاستعمال الإنجليزي وبغير معرفة الاصطلاح السوري.

وصعوبة الأمر أن هذه الكلمة شديدة العموم، فنحتاج إلى تخصيصها في كل سياق قبل تعريبها، فغالبا نعتبرها صفة (=«برمجية») لاسم محذوف، ونرد هذا الاسم عند الترجمة.

فهذه أشهر تعريباتها حسب السياق:

- «مصدر برمجي» (= «source code») أو «مصدر» فحسب ←
 - «قاعدة المصدر» = «code base» (أو «مصدر برمجي» أيضاً)
 - «مشروع برمجي» (مثل العبارة: «hosting your code» وما في معناها)
 - «برمجيات» ←
 - «محرر برمجيات» = «code editor» (أو «محرر برمجي»)
 - «تعليمات برمجية» ←
 - «ثغرة تنفيذ تعليمات برمجية عشوائية» = «arbitrary code execution vulnerability»
 - «تعليمات برمجية خبيثة» = «malicious code»
 - «قطعة برمجية» (مثل «ماذا يفعل هذا الكود؟» أو «نسخة كوداً من موقع أجوبة.» أو نحو ذلك، أي أسطر برمجية، أو عبارة برمجية (جملة أو جزء من جملة)، وليست بالضرورة وحدة مستقلة نحويًا (دالة مثلاً) وليست بالضرورة أوامر («تعليمات») أو غير ذلك).
- ولم نفرغ من هذه الكلمة بعد.

وانظر النقاش في [مسائل جتهب](https://github.com/noureddin/ysmu/issues/1) (الرابط: <https://github.com/noureddin/ysmu/issues/1>) وشاركنا فيه.

هـ، ا، المركز والمركزي، الرئيس والرئيسي

الإفراط في الوصف بالنسبة إلى صفة، مثل الرئيسي والأساسي والمركزي والأصلي، من أثر اللغات الأوروبية. فلم نصف الأمر الرئيس نفسه بأنه رئيسي، ونصف الخادوم المركز نفسه بأنه مركزي؟

وكذلك المصدر البرمجي يترجم بـ«كود مصدري» أو «رماز مصدري»، مع أنه ليس «شيئاً له علاقة بالمصدر»، بل هو المصدر نفسه.

إنما الوصف بالنسبة إلى صفة لما يكون «شيء له علاقة بالأشياء الموصوفة بالصفة الأصل» (لاحظ: ليس «الأصلية»). مثلاً «الأنظمة المركزية» صحيحة لأنها تصف الأنظمة التي يميّزها وجود مركز فيها. لكن المركز نفسه نسميه مركزاً وليس مركزياً، فنقول الخادوم المركز.

وقد ذكر معجم الصواب اللغوي إجازة مجمع اللغة العربية بالنسبة إلى الصفة، مع التفريق في المعنى بين الصفة الأصل والصفة المنسوبة، في مادة [رئيسية \(الرابط: https://archive.org/details/99325/page/n388/mode/1up\)](https://archive.org/details/99325/page/n388/mode/1up):

”ولكنّ هناك فرقاً في الدلالة بين الوصف من الرياسة على صيغة «فعيل» «رئيس»، وبين الوصف منها بصيغة النسب «رئيسيّ» فالرئيس هو الشريف وسيد القوم، والرئيسيّ هو المنتمي إلى مفهوم رئيس وكأنه فرد من أفراد، وعلى ذلك فرئيسيّ فصيح والوصف به غير الوصف برئيس، وقد أقره مجمع اللغة المصري بشرط أن يكون المنسوب إليه أمراً من شأنه أن يندرج تحته أفراد متعددة.

فالصفة المنسوبة جائزة لكن دلالتها غير الصفة الأصل: «المركزي» شيء له علاقة بالمركز أو أنه يشبه المركز، لكنه ليس مركزاً هو نفسه. إنما المركز نفسه يوصف بالمركز.

ومن هذا وصف الشيء بأنه «أصليّ» عندما نريد أنه هو الأصل نفسه وليس أنه يشبه الأصل أو منسوب إليه. وكذلك وصف الشيء بأنه «أوليّ» عندما نريد أنه هو الأول.

هذا الأخير مهم لوروده في الكتاب غير مرة: ترجم “initial commit” وما على شاكلتها بـ«أول إيداع» (إذا كانت معرفة) أو بـ«إيداع أول» (إذا كانت نكرة).

هـ، ١٢، إصلاحات لغوية عامة ونصائح أسلوبية

هـ، ١٢، تصويبات

- قل «يستعمل» أغلب الوقت، ولا تقل «يستخدم» إلا للضرورة، فالاستخدام يكون للعاقل. (وتبقى “user” «مستخدم».) ويستثنى من ذلك ما يقمّ خدمة، فنستخدم جت هب، ونستخدم الخواديم، وقد نستخدم جت نفسه تشبيهاً له بالعاقل. ولا بأس من استعمال «استعمال» مع أيّ منهم. ولكن قل من كليهما، لأن استعمال هذا الفعل يكثر في الإنجليزية المعاصرة بغير حاجة. ومثله الفعل «يستطيع» فيه إفراط بغير حاجة، فقل منه.
- لا تقل «أصليّ» إلا لتصف الشيء بأنه مثل الأصل أو يُنسب إليه. وكذلك «مركزيّ»، «رئيسيّ»، «أوليّ»، إلخ. فلا تشتق صفة من صفة إلا للحاجة؛ بل صِف بالصف نفسها: “central” غالباً يكون هو المركز نفسه وليس «مثل المركز» حتى تقول «مركزيّ». فقل «الخادوم المركز»، «الفرع الرئيس»، «النسخة الأصل»، «الإيداع الأول»، بغير ياء النسبة. للتفصيل انظر فصل [المركز والمركزي، الرئيس والرئيسي](#).
- لا تستعمل حرف الجر الكاف إلا للتشبيه، وعلامة ذلك استقامة المعنى وثبوته عندما تبدلها بـ«مثل»، وإلا فغيّر تركيب الجملة واستعمل شيئاً غير الجر بالكاف، كالتمييز والحال والمفعول به.

- قل «يبقي» و«يظل» و«لا يزال» و«ما زال»، لكن لا تقل «لا زال»، فاستعمال «لا» مع الماضي يعني الدعاء، مثل «لا أراك الله مكروها».
- لا تقل «استبدل» لشيوع الخطأ فيه، وقل «أبدل القديم بالجديد»، فلا خلاف فيه، أو «أبدل من القديم الجديد»، أو أنت بفعل من جذر آخر مثل «يحل محل». (الصاحح في مادة بدل: «والأبدال: قوم من الصالحين لا تخلو الدنيا منهم، إذا مات واحد أبدل الله مكانه بآخر».)
- لا تقل «تحقق من [شيء] أو فعل أو أن تفعل»؛ قل «تحقق...» بغير «من»، أو قل «تفقد...».
- لا تقل «بسيط» إلا عندما تعني «غير معقد»، وقل منها عموماً.
- لا تقل «لا داع لـ...»، بل قل «لا داعي إلى...»، بالياء وبـ«إلى».
- لا تبدأ جملة فرعية بـ«مما»، فهي غالباً خاطئة. وعلامة ذلك اختلال المعنى إذا وضعت مكانها «من الذي» أو اسم موصول آخر. أمثلة:
 - «يتيح جت أيضاً خياراً للحالة الموجزة، مما يتيح لك رؤية تعديلاتك بإيجاز» ← «...، لتري...»
 - «إضافة [شيء] تجعل جت [يفعل شيئاً]، مما يتيح لك تخطي مرحلة الإضافة» ← «...، لتتخطي...»
 - «لأن طريقة التفرع في جت خفيفة [...]، مما يجعل إنشاء فرع...» ← «...، فتجعل...»
 - «ثم تستطيع حذف الفرعين [...]، مما يجعل تاريخك...» ← «...، فيصير تاريخك...»
 - «تذكر أنه غير مستوثق، مما يعني أن أي شيء نتيجته...» ← «...، فأأي شيء...»
 - «وقتئذٍ ستبني كل مشروعاتك البرمجية على جهاز واحد، مما يزيد من احتمال فقد البيانات فقد كارثياً.» ← «...، وهذا يزيد احتمال...»
 - «لم تقترب حاله مما صار عليه اليوم» ← صحيحة
 - «فبدلاً مما رأينا،...» ← صحيحة
- «مِيزة»/«مِيزات» تعني الاختلاف (التمييز، وفصل الشيء من الشيء) ولا تعني التفضيل. فإذا أردت معنى الفضل (التمام والكمال)، فقل «مِزِيَّة»/«مِزَايَا». فترجم «advantage» بـ«مِزِيَّة»/«مِزَايَا»، وترجم «feature» بـ«مِيزة»/«مِيزات» أو «خَصِيصة»/«خصائص» (وليس «خاصية»/«خواص»؛ هذه «property»).

هـ، ١٢، ٢، للفصاحة والاتساق

- لا تقل «نفس الشيء» وقل «الشيء نفسه». (أو «شيء واحد» أو «الشيء الواحد» عند إرادة الإبهام، مثل «يمكنكم الآن التعاون في مشروع واحد» أو «ستبني كل مشروعاتك البرمجية على جهاز واحد».)
- لا تقل «بداية»، فهي عامية، وفصيحتها «بِداءة» لكنه غير مألوف، فقل «بِداء» أو «ابتداء» أو «أول». (قال فيها المصباح المنير: «والبِدايَةُ» بالياء مكان الهمز عامي نص عليه ابن بري وجماعة». وقال العباب الزاخز: «وقَوْلُ العامَّةِ: البِدايَةُ - مؤازاةً للبِدايَةِ: لَحْنٌ، ولا تُقَاسُ على الغَدَايا والعِشايا؛ فَإِنَّهَا مَسْمُوعَةٌ بخلاف البِدايَةِ».)
- لا تقل «كل ما عليك هو»؛ إنما قل «ليس عليك سوى/إلا/غير».
- قل من «فقط» واستعمل الاستثناء أو «إنما» متى أمكن.
- قل من «نَفَذَ [الأمر] مجدداً» وقل «كرر [الأمر]».
- قدّم الفعل على فاعله ما لم يسبب ذلك خلطاً على القارئ.

- انظر [موارد معجم يسمو](https://www.noureddin.dev/ysmu/notes/) (الرابط: <https://www.noureddin.dev/ysmu/notes/>) ، وبالأخص كتاب [نحو](#)
[إتقان الكتابة باللغة العربية](https://web.archive.org/web/20100914143217/http://www.reefnet.gov.sy/Arabic_Proficiency/Arabic_Proficiency_Index.htm) (الرابط: https://web.archive.org/web/20100914143217/http://www.reefnet.gov.sy/Arabic_Proficiency/Arabic_Proficiency_Index.htm) .

Version 2.1.449

Last updated 2025-10-24 12:00:00 +0000